

I F T 2 2 2

Computer Architecture and Organisation

Study Manual

Victor Ayodeji

FETCH 1010 0011 1100 1010

DECODE **ADD** R3, [202]

EXEC **|** R3 <- R3 + M[202]

float 0 10000100 1000000... = 64.2

CPU

CACHE

MAIN MEMORY

I/O

Contents

Every line is a link. The page numbers are read back from the rendered book, never typed by hand.

How to use this manual	2
FOUNDATIONS • THE MATHS YOU NEED FIRST	5
F.1 • Number Systems and Positional Notation	6
F.2 • Converting Between Bases	8
MODULE ONE • FUNDAMENTALS OF COMPUTER ARCHITECTURE	13
Unit 1 • Architecture versus Organization	14
Unit 2 • Data Representation	22
Unit 3 • Floating Point Representation	39
Unit 4 • Instruction Set Architecture	48
MODULE TWO • PROCESSOR DESIGN AND PERFORMANCE	60
Unit 1 • Processor Components	61
Unit 2 • CPU Performance Metrics	65
Unit 3 • Pipelining	73
MODULE THREE • COMPUTER MEMORY ORGANIZATION	89
Unit 1 • Computer Memory Overview	90
Unit 2 • Semiconductor Memory	95
Unit 3 • Cache Memory Design	97
MODULE FOUR • BASIC AND ADVANCED PROCESSOR ARCHITECTURES	111
Unit 1 • RISC versus CISC	112
Unit 2 • Parallel Architectures and Interconnection Networks	117
SUPPLEMENT • DIGITAL LOGIC	120
S.1 • Boolean Algebra and K-maps	121
REFERENCE • FORMULAS, NOTATION AND EXAM CRAFT	125
R.1 • Formula and Notation Reference	126
R.2 • Exam Day Strategy	133
R.3 • Trap Index and Cram Sheet	136
PAST PAPERS • SOLVED IN FULL	140
2024/2025 Solved in Full	141
2025/2026 Solved in Full	159
REVISION CLASS • EVERY QUESTION SOLVED	177
Data, Instructions and Addressing	178
Pipelining and Hazards	182
Cache Mapping, Every Case	185
OBJECTIVE SECTION • BOTH TESTS, EVERY OPTION EXPLAINED	188
How to Attack an Objective Question	189
Architecture versus Organization	190
Instruction Set Architecture and Microarchitecture	195
Number Systems and Codes	199
Signed Numbers and Complements	203

Floating Point and IEEE 754	207
Instruction Types and Formats	210
Addressing Modes	213
Memory, Buses and the Processor	216
Performance, RISC and CISC	224
Von Neumann, Harvard and Image Data	227
MOCK EXAMINATION ONE	229
Question Paper	230
Answers	231
MOCK EXAMINATION TWO	236
Question Paper	237
Answers	238
MOCK EXAMINATION THREE	241
Question Paper	242
Answers	244

BEFORE YOU START

How to use this manual

This book takes you from knowing nothing about how a computer is built to solving every past question on this course with a perfect score. Read it with a pen, not just your eyes.

Computer Architecture and Organisation looks frightening because it mixes three things at once: a little binary arithmetic, a handful of diagrams, and a set of formulas for speed and memory. In truth the whole course is about a dozen ideas, each used over and over. This manual finds those ideas, teaches each one from zero, and then makes you use it on the real exam questions straight away. Nothing is assumed. Every number is worked out in full. Every symbol is explained before it appears.

The one habit that makes this work

Reading feels like studying and teaches almost nothing. What moves a fact into your memory is pulling it back out. So all through the book you will meet a **RECALL AND CHECK** box: a question with the answer hidden under a dashed line. Cover the answer, say yours out loud or on paper, then look. When a worked example ends, you will see **Now redo it:** close the book and reproduce the solution on blank paper. If you only ever do that one thing, you will pass.

The boxes, and what each is for

Every idea lives in a labelled box. You can navigate the whole book by the colour and shape of these boxes alone.

TEACH

One idea, in plain language. This is where a concept is explained for the first time. Steel-blue, the colour of the datapaths and buses this course is about.

A real exam question, reproduced **word for word** as the examiner set it. Not summarised, not tidied, not corrected. The part numbering, the casing, the spacing and the mark text such as **9mks** are all the paper's own, and so are its mistakes: you will meet a paper that asks for "Special Locality", one that says "Use 12 bits" above a row holding only 11, and one that prints "in using Single Precision IEEE format". Those stay in, because in the hall you will have to read past them. Where a question carries a defect worth naming, a red note under the quote points it out. Nothing inside this box is ours. No colour, because a photocopy has none.

FROM THE SLIDES

THE LECTURER'S OWN DECK

A question or worked example taken from the lecture slides your lecturer released **this semester**, quoted faithfully and then broken down and answered. It is set apart from an **As printed** box on purpose: that one is a past exam paper, copied letter for letter; this one is the slides, and it matters because the person who writes these slides is the person who sets your exam. When a question here has a model answer on the slide, you are shown both, so you can see exactly what the examiner counts as a full answer. Slate, the colour of a handout, and never the same near-black as the exam paper.

BREAK IT DOWN

BEFORE YOU COMPUTE

The half of the work that a solution normally hides: reading the question. What am I given, what is *actually* being asked, what do the marks say about how much to write, which unit teaches this, and where the trap sits. The instruction verb is the thing to watch, because "determine which of the following is true" is not the same task as "convert". Teal, and it always sits between the paper's words and our answer.

WORKED EXAMPLE

SLIDE / CLASSWORK / PQ

A full solution, one numbered step at a time, with the reason on every line. The chip on the right tells you where the question came from, so a chip that reads **PQ 25/26** is a real exam question you are being shown how to solve, and its wording will be quoted in an **As printed** box just above. Amber, the colour of energy and work being done.

● MUST MEMORISE

MEMORY HOOK

The few things you truly have to know by heart. This violet is **reserved**: it appears nowhere else in the book, so if you see it, commit it to memory. Each one comes with a hook to make it stick.

⚠ TRAP

The exact mistake students make on this topic every year. Read these twice. Losing marks here is the most avoidable way to lose them.

TRACE

A step-by-step table that follows a value, a register, or a pipeline cycle by cycle, so you can see the machine change state one move at a time.

RECALL AND CHECK

What should you do the moment you reach one of these?

ANSWER

Cover this line, answer from memory, then check. Retrieval is the whole point.

FILL IN THE GAP

Short cloze practice: finish the line, then check against the answers underneath.

EXAM QUESTION

PQ 25/26 Q1A

A past question reproduced as it appeared on the paper. Every one in this book is followed by a full worked answer in a green **Answer** box, so no question on this course goes unanswered.

How the book is arranged

A short **Foundations** part comes first: the number sense the rest of the course leans on. Then four modules follow the syllabus exactly: Fundamentals, Processor Design and Performance, Memory Organisation, and Advanced Architectures. As each topic is taught, the real past questions on it are solved right there. Two full **mock examinations** at the end are built in the shape of the real paper, answer question one and any other two, seventy marks in two hours, and both are worked in full.

TEACH

WHAT THE EXAM LOOKS LIKE

The paper is always the same shape. **Question One is compulsory and carries 30 marks.** You then choose **any two of the remaining questions**, 20 marks each, for a total of 70 in 2 hours. Question One is where the numeric work concentrates: number representation, floating point, cache, pipelining and performance. Master Question One and you have already secured nearly half the paper before you choose anything.

FOUNDATIONS

The Maths You Need First

Everything a computer does, it does with two symbols, 0 and 1. Before we can talk about processors, memory or floating point, we need to be completely at home moving between binary, decimal and hexadecimal. This part builds that fluency from the ground up.

1010 0011 = A3 = 163

binary

hexadecimal

decimal

Number Systems and Positional Notation

A number system is just a set of symbols and one rule for reading a row of them. Learn the rule once and every base behaves the same way.

What "base" means

You already use a number system fluently: the decimal system, base 10. It has ten symbols, 0 to 9. When you run out of symbols you do not invent a new one, you start a new column to the left. The genius of the system, invented long before computers, is that **the position of a digit tells you its value**. In 735 the 7 does not mean seven, it means seven hundred, because of where it sits.

TEACH

POSITIONAL NOTATION

Reading right to left, each column is worth the base raised to a higher power: the ones column is base^0 , the next is base^1 , the next base^2 , and so on. The value of the whole number is each digit multiplied by its column, all added up. In base 10:

$10^2 = 100$	$10^1 = 10$	$10^0 = 1$
7	3	5
700	30	5
$700 + 30 + 5 = 735$		

The same rule reads every base. Only the column values change.

Written as a formula, a number whose digits are d in base b has

$$\text{value} = d_{n-1} \times b^{n-1} + \dots + d_1 \times b^1 + d_0 \times b^0$$

WHAT EACH SYMBOL MEANS

b = the base (2 binary, 8 octal, 10 decimal, 16 hexadecimal) · **d_i** = the digit sitting in column i · **i** = the column number, counted from 0 at the rightmost digit · **n** = how many digits the number has

The four bases this course uses

Humans count in base 10. Computers are built from switches that are either on or off, so at heart they count in **base 2, binary**, with just two symbols, 0 and 1. Binary numbers get long quickly, so we also use **base 16, hexadecimal** and occasionally **base 8, octal** as a shorthand: they pack neatly into binary, as F.2 will show. A single binary digit is a **bit**; eight bits make a **byte**.

SYSTEM	BASE	SYMBOLS	SUFFIX WE USE	EXAMPLE
Binary	2	0 1	B, or subscript 2	1011B
Octal	8	0 to 7	subscript 8	13 ₈
Decimal	10	0 to 9	D, or none	735
Hexadecimal	16	0 to 9, A to F	H, or subscript 16	A34EH

● MUST MEMORISE

HEX LETTERS

Hexadecimal needs six extra symbols once it passes 9. Know these cold, both directions:

HEX	A	B	C	D	E	F
DECIMAL	10	11	12	13	14	15

Memory hook: After nine comes ten, and **A** is the first letter, so A is 10. F is the last of the six and the largest single hex digit, 15, one less than the base.

● MUST MEMORISE

POWERS OF TWO

These come up in every cache, memory and address question on the paper. You must know them without counting on your fingers.

N	0	1	2	3	4	5	6	7	8	9	10
2 ⁿ	1	2	4	8	16	32	64	128	256	512	1024

Memory hook: $2^{10} = 1024$ is "1 kilo" (K). After that they climb in steps of ten: 2^{20} is 1 M (mega), 2^{30} is 1 G (giga). So $2^{16} = 2^{10} \times 2^6 = 1024 \times 64 = 65\,536$.

Read the binary number 1011B into decimal.

GIVEN number = 1011 · source base $b = 2$ · target base = 10
 FIND its decimal value
 FORMULA value = $\sum d_i \times b^i$, i counted from 0 at the right

WHAT EACH SYMBOL MEANS

$b = 2$, the base you are reading from · d_i = the bit in column i · $i = 0$ for the rightmost bit, so the columns are worth 1, 2, 4, 8

- 1 Write each bit over its column value, right to left: columns are $2^3, 2^2, 2^1, 2^0$, that is 8, 4, 2, 1.
Bits: 1, 0, 1, 1.
- 2 Keep only the columns with a 1 in them: 8, then 2, then 1.
The 4 column has a 0, so it contributes nothing.
- 3 Add them: $8 + 2 + 1 = 11$.

1011B = $8 + 2 + 1 = 11$ (decimal)

Now redo it: cover the working and convert 10110B. You should get 22.

RECALL AND CHECK

In any base, what is the value of the rightmost column, and why is it the same for every base?

ANSWER

It is always 1, because the rightmost column is base^0 , and any base raised to the power 0 is 1. That is why the last digit is called the units digit in every system.

FOUNDATIONS · F.2

Converting Between Bases

Four conversions cover everything the exam asks: any base to decimal, decimal to any base, and the quick grouping trick between binary and hex or octal.

Any base to decimal: multiply and add

This is just positional notation from F.1 applied in reverse. Multiply each digit by its column value and add. It works for any base, you only change what the columns are worth.

Convert the hexadecimal number A34EH to decimal.

GIVEN number = A34E · source base $b = 16$ · target base = 10
FIND its decimal value
FORMULA value = $\sum d_i \times 16^i$

WHAT EACH SYMBOL MEANS

$b = 16$ · d_i = the digit in column i , with the letters standing for numbers (A = 10 up to F = 15)
· the columns are worth 1, 16, 256, 4096

- 1 Replace each hex letter by its value: A = 10, 3 = 3, 4 = 4, E = 14.
- 2 Write the column values, right to left, as powers of 16: $16^3 = 4096$, $16^2 = 256$, $16^1 = 16$, $16^0 = 1$.
- 3 Multiply digit by column:
 $10 \times 4096 = 40\,960$; $3 \times 256 = 768$; $4 \times 16 = 64$; $14 \times 1 = 14$.
- 4 Add: $40\,960 + 768 + 64 + 14 = 41\,806$.

A34EH = $40\,960 + 768 + 64 + 14 = 41\,806$ (decimal)

Now redo it: reproduce the four column values from memory before you multiply. That is where marks are lost, not in the adding.

Decimal to any base: divide and collect remainders

Go the other way by repeatedly dividing by the base and writing down the remainder each time. The remainders, **read from bottom to top**, are the answer.

Convert decimal 13 to binary.

GIVEN number = 13 · source base = 10 · target base $b = 2$
 FIND its binary form
 FORMULA divide repeatedly by b , keeping each remainder; read the remainders **bottom-up**

WHAT EACH SYMBOL MEANS

$b = 2$, the base you are converting *into*, so it is what you divide by · each **remainder** is one digit of the answer · the remainders come out least significant first, which is why the column is read upwards

DIVIDE	QUOTIENT	REMAINDER
$13 \div 2$	6	1
$6 \div 2$	3	0
$3 \div 2$	1	1
$1 \div 2$	0	1

Read the remainders bottom to top: 1101.

13 = 1101B (check: $8 + 4 + 1 = 13$)

Now redo it: convert 22 to binary. You should get 10110, and the check is $16 + 4 + 2 = 22$.

⚠ TRAP

The remainders come out **least significant first**. If you read them top to bottom you get the number backwards. Always read the column of remainders **from the bottom up**, and sanity-check by converting back.

◆ KEY POINT

A base is a promise about place value, and every conversion in this course is that promise applied in one direction or the other. Multiplying and adding goes from a base into decimal; dividing and collecting remainders goes from decimal into a base. They are the same idea read twice, not two procedures to memorise.

BLUNT VERSION: one rule, run forwards or backwards. If you are memorising two, you have missed it.

Binary to hex, and back: group in fours

Hexadecimal exists to make binary readable, and the reason is exact: **one hex digit is worth exactly four bits** (because $16 = 2^4$). So to turn binary into hex you simply slice the bits into groups of four from the right and convert each group on its own. Octal is the same trick in groups of three, because $8 = 2^3$.

WORKED EXAMPLE

METHOD

Convert 11010110B to hexadecimal.

GIVEN number = 11010110 · source base = 2 · target base = 16
FIND its hexadecimal form
FORMULA group the bits in fours **from the right**; each group of 4 bits is one hex digit

WHAT EACH SYMBOL MEANS

grouping works because $16 = 2^4$, so exactly four bits carry one hex digit · group **from the right**, and pad a short leftmost group with leading zeros

- 1 Slice into groups of four bits, starting from the right:
1101 0110.
- 2 Convert each group using the powers 8, 4, 2, 1:
1101 = $8+4+1 = 13 = D$; 0110 = $4+2 = 6$.
- 3 Write the hex digits in the same order: D6.

11010110B = 1101 0110 = D6H (check: both equal 214)

Now redo it: go backwards. Convert 2FH to binary by expanding each digit to four bits: 2 = 0010, F = 1111, giving 0010 1111.

⚠ TRAP

Group from the **right**, not the left, and pad the leftmost group with leading zeros if it is short. Grouping 110101101011101 from the left would misalign every digit. From the right it is 110 1011 0101 1101, and the short leading group 110 pads to 0110 = 6, giving 6B5DH.

FILL IN THE GAP

1. The rightmost column in base b is always worth _____.
2. One hexadecimal digit is worth exactly _____ bits.
3. To convert decimal to binary you repeatedly divide by 2 and read the _____ from bottom to top.
4. $2^{10} =$ _____.

ANSWERS

1. one ($b^0 = 1$). 2. four. 3. remainders. 4. 1024.

RECALL AND CHECK

Why can binary group into hex in fours but not into decimal in any fixed size?

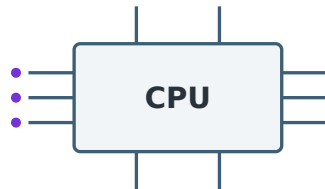
ANSWER

Because 16 is a whole power of 2 ($16 = 2^4$), so four bits map exactly onto one hex digit with none left over. 10 is not a power of 2, so no fixed number of bits ever lines up cleanly with one decimal digit. This is the whole reason hex and octal, not decimal, are used as binary shorthand.

MODULE ONE

Fundamentals of Computer Architecture

What a computer is, from the outside in: the difference between what the programmer sees and how the machine is built, the two great design templates every computer follows, and the fetch, decode, execute rhythm that all the later modules refine.



Architecture versus Organization

Two words that sound like synonyms, mean different things, and are worth an easy few marks on every paper if you can tell them apart cleanly.

By the end of this unit you can define architecture and organization and give examples of each, draw and compare the Von Neumann and Harvard designs, explain the Von Neumann bottleneck and how to reduce it, and answer every past question of this type in full.

TEACH

WHAT A COMPUTER IS, AND ITS PARTS

A **computer** is a programmable device that stores, retrieves and processes data, following a stored set of instructions to turn input into output. Every computer, from a phone to a server, is built from the same handful of parts:

- ▶ **CPU** (processor): the control unit and ALU that carry out instructions.
- ▶ **Memory**: holds the instructions and data the CPU is working on (Module Three).
- ▶ **Storage**: keeps data when the power is off (disk, SSD).
- ▶ **Input/output devices**: move data in and out (keyboard, screen, network).
- ▶ **The motherboard** (system board) and its **buses**: the circuit board that holds the parts and the wires that join them.

The rest of this module looks at how those parts are organised and how the CPU steps through a program.

The one distinction everything hangs on

Think of a car. The **architecture** is what the driver sees and uses: the steering wheel, the pedals, the gear positions, the promise that pressing the accelerator makes the car go faster. The **organization** is what is under the bonnet: the size of the engine, how the fuel is delivered, how the pedal is physically connected to the engine. Two cars can drive identically for the driver (same architecture) while being built completely differently inside (different organization).

TEACH

THE CORE IDEA

Computer architecture is the set of attributes visible to the programmer, the things that affect how a program is written: the instruction set, the number and size of registers, the data types, the addressing modes, the way memory is addressed. It answers "**what does the machine do?**"

Computer organization is how those attributes are actually realised in hardware, the parts hidden from the programmer: the control signals, the buses, the memory technology used, whether an add is done by one circuit or several. It answers "**how does the machine do it?**"

● MUST MEMORISE

THE TWO DEFINITIONS, FORMALLY

Computer architecture refers to the attributes of a system visible to the programmer, those that have a direct impact on the logical execution of a program.

Computer organization refers to the operational units and their interconnections that realize the architectural specifications.

One sentence joins them, and it is the one the objective section keeps asking for: **architecture provides the blueprint and the capabilities, organization determines the efficiency and the performance, and the two are deeply interconnected.**

TEACH

THE DECK'S SMARTPHONE EXAMPLE

The deck works the distinction on a phone, and it is the cleanest example on the course because both halves are things you can feel.

- ▶ **Architecture** of the phone: the instruction set it runs and the data types it supports. That is what decides whether an app will run on it at all.
- ▶ **Organization** of the phone: how fast it feels, how long the battery lasts, and how well it handles several apps at once. Not one of those is in the instruction set, and every one of them is a hardware choice.

That is why the same question comes back every year in different clothes. A change you can name in the instruction set is architecture. A change you can only measure is organization.

TEACH

BINARY COMPATIBILITY, WHERE YOU HAVE ALREADY MET IT

An application is compiled for one instruction set, and it will run only on processors that support that instruction set. This is not theory: it is why an app built for the ARM processors in phones is a different download from the same app for an x86 laptop, and why a program compiled twenty years ago still runs on a modern x86 chip whose insides bear no resemblance to the machine it was built for.

Hold the instruction set steady and you may rebuild everything underneath it. That is the whole commercial reason the architecture and organization split exists, and the reason a manufacturer will keep an awkward old instruction alive for decades.

● MUST MEMORISE

MEMORY HOOK

Architecture = what the programmer sees. Organization = how it is built.

Memory hook: Architecture is **A**bstract (the visible promise); Organization is the **O**peration (the hidden wiring that keeps the promise). Same instruction set, different organization, is exactly how a cheap and an expensive processor can run the identical program.

COMPUTER ARCHITECTURE	COMPUTER ORGANIZATION
Attributes visible to the programmer	Attributes hidden from the programmer
Answers "what is done"	Answers "how it is done"
Instruction set, data types, registers, addressing modes	Control signals, buses, memory technology, circuits
Decided first, changes rarely	Realises the architecture, varies between models
Example: the machine has a "multiply" instruction	Example: multiply is done by repeated addition, or by a dedicated multiplier

AS PRINTED
25/26 Q1(A)(I)

THE EXAMINER'S WORDS, UNEDITED

a. i) Computer Systems are designed using two closely related but distinct concepts which are Computer Architecture and Computer Organization. Differentiate between these concepts.

2mks

Read it again. The stem already names both concepts, so no marks are waiting for you to recall the names; every mark is in the contrast. The verb is **Differentiate**, not "define", and two definitions written side by side with no difference drawn between them answer a question the paper did not set. The mark is printed **2mks** on its own line below the text, which is where this paper puts every mark annotation, and this is part **a. i)** of the compulsory Question One, so it is not one you can skip.

BREAK IT DOWN	BEFORE YOU COMPUTE
GIVEN	only the paper's own claim that the two concepts are "closely related but distinct", and their names. There is nothing to calculate and nothing to look up.
ASKED	one thing: <i>differentiate</i> . That verb wants the axis of difference stated out loud, so the answer must contain the word "whereas" or its equivalent, not two paragraphs that happen to sit next to each other.
MARKS	2mks, so two scoring points. One clean contrast is the first; one concrete example that shows the contrast biting is the second.
TAUGHT IN	this unit, Module One Unit 1.
TRAP	writing "architecture is the design of a computer" and "organization is how a computer is organised". Both are true and neither separates them. Name the viewpoint each belongs to.

- 1 Name the viewpoint of each: architecture is the programmer's view, organization is the hardware's view.
- 2 Give the defining question each answers, and one example, to earn both marks.

Architecture is the set of attributes visible to the programmer (instruction set, registers, addressing modes): it defines WHAT the machine does. Organization is how those attributes are implemented in hardware (control signals, buses, memory technology): it defines HOW the machine does it. Example: architecture says a MULTIPLY instruction exists; organization decides whether it runs on a dedicated multiplier or by repeated addition.

Now redo it: this exact question, worth 2 marks, is on the 24/25 and 25/26 papers. Write the two sentences and one example from memory.

◆ KEY POINT

Architecture is the set of attributes a program can see, so a change to it can break a program that already runs. Organization is everything underneath, so a change to it can only make the same program faster, slower, hotter or cheaper.

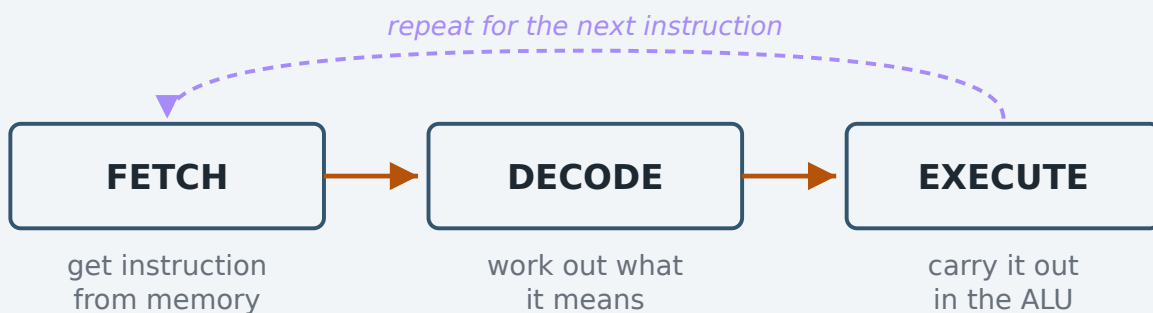
BLUNT VERSION: if you can name it in the instruction set, it is architecture. If you can only measure it, it is organization.

How a computer runs a program: the instruction cycle

Before we compare designs, fix the basic rhythm in your mind, because every later idea (pipelining, cache, performance) is a refinement of it. The processor repeats three steps, over and over, billions of times a second.

TEACH

FETCH, DECODE, EXECUTE



Fetch, decode, execute, then repeat for the next instruction.

Fetch: the control unit reads the next instruction from memory into the processor.

Decode: it works out what the instruction is asking for. **Execute:** the arithmetic and logic unit (ALU) carries it out. Then the cycle starts again on the next instruction.

Everything else in this course is about doing these steps faster or feeding them with data sooner.

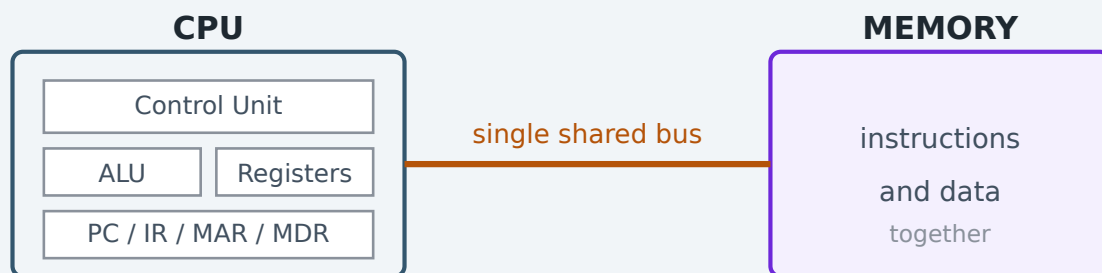
Two ways to wire memory to the processor

A computer must store both the **program** (the instructions) and the **data** those instructions work on. The two great design templates differ in one decision: do instructions and data share one memory and one path, or get separate ones?

TEACH

VON NEUMANN ARCHITECTURE

In the **Von Neumann** design there is **one memory** holding both instructions and data, reached over **one shared bus**. This is elegant and cheap: the same memory stores everything, and a program can even treat its own instructions as data. Almost all general-purpose computers use it.

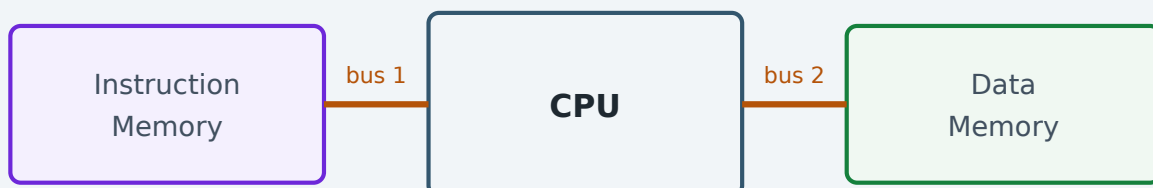


One memory, one bus: instructions and data share the same road.

TEACH

HARVARD ARCHITECTURE

In the **Harvard** design, instructions and data live in **separate memories** reached over **separate buses**. The processor can fetch an instruction and read data **at the same time**, which is faster, but it costs more and is less flexible. It is common in signal processors and microcontrollers, and inside the CPU's own caches.



Two memories, two buses: an instruction and data can move at once.

FEATURE	VON NEUMANN	HARVARD
Memory for code and data	One shared memory	Two separate memories
Buses	One shared bus	Separate instruction and data buses
Fetch and data access	Cannot happen at once	Can happen at the same time
Cost and complexity	Lower, simpler	Higher, more hardware
Typical use	General-purpose computers	Signal processors, microcontrollers, CPU caches

AS PRINTED

25/26 Q1(A)(II)

THE EXAMINER'S WORDS, UNEDITED

ii) Compare Von Neumann Architecture and Harvard Architecture. Include a diagram for each.

3mks

Read it again. "Include a diagram for each" is printed as part of the question, so the drawings are a requirement and not a garnish: a written comparison with no figures cannot reach 3mks. This same question recurs with the wording drifting each year. The 24/25 paper sets it as Q4(a)(i) and the 23/24 paper as Q5(b), and both of those write "Include diagram for each", with no "a". The marks drift too: **3mks** here, **3mk** singular on 24/25, **4mk** on 23/24. Count the marks off the paper in front of you, never off your memory of last year's.

BREAK IT DOWN

BEFORE YOU COMPUTE

GIVEN	two names and nothing else. No data, no numbers, no machine to reason about.
ASKED	two deliverables in one sentence: <i>compare</i> the two designs, and <i>include a diagram for each</i> . Missing either one costs marks even if the other is perfect.
MARKS	3mks. Read it as two comparison points plus the pair of diagrams, so aim for three separate things a marker can tick.
TAUGHT IN	this unit, Module One Unit 1.
TRAP	comparing on one axis only. "One memory against two" is a fact; the mark is in what that fact causes, which is that Harvard can fetch an instruction and read data in the same cycle and Von Neumann cannot.

Full-mark answer

Von Neumann uses ONE memory and ONE bus for both instructions and data, so it is simpler and cheaper but the single bus can carry only one of them at a time. Harvard uses SEPARATE memories and buses for instructions and data, so it can fetch an instruction and access data simultaneously (faster) at higher cost. Draw the two diagrams above: one CPU-to-single-memory, one CPU with two memories.

Three marks means three scoring points: one shared versus two separate memories, the simultaneous-access difference, and the two diagrams. Give all three.

Now redo it: draw both diagrams from memory in under two minutes. The diagrams alone are worth marks and are the fastest to lose under time pressure.

◆ KEY POINT

Von Neumann and Harvard differ in exactly one thing: whether instructions and data share a memory and a bus, or have their own. Every other difference between them, the cost, the flexibility, the bottleneck, follows from that one choice.

BLUNT VERSION: one road or two. Everything else is traffic.

The Von Neumann bottleneck

The Von Neumann design has one famous weakness, and the exam asks about it almost every year, so it is worth full marks to you.

TEACH

THE BOTTLENECK

Because instructions and data share **one bus**, the processor cannot fetch an instruction and read or write data at the same instant. It must take turns. The single shared path between the CPU and memory therefore limits how fast the whole machine can go, however fast the CPU itself is. That limit is the **Von Neumann bottleneck**: the CPU is often left waiting on the one road to memory.

TEACH

HOW TO REDUCE IT

Three standard ways to ease the bottleneck, each worth a mark:

- ▶ **Cache memory:** keep a small, fast copy of the most-used instructions and data inside or beside the CPU, so most accesses never travel the slow bus (Module Three).
- ▶ **Separate caches for instructions and data** (a Harvard-style split inside the chip), so the two can be fetched at once even on a Von Neumann machine.
- ▶ **Wider or faster buses, and registers:** move more bits per trip, and keep frequently used values in fast CPU registers so the bus is used less often.

Modern machines add further tricks in the same spirit: faster **DDR** memory, **prefetching** data into cache before it is asked for, and even **processing-in-memory (PIM)**, doing simple work in the memory chip itself to save a bus trip.

THE EXAMINER'S WORDS, UNEDITED

- a. i) What do you understand by Von Neumann Concept? **1mk**
 ii) Mention any 2 significant aspects of Von Neumann Architecture. **2mks**
 iii) Describe the Von Neumann architecture bottleneck. **2mks**

Read it again. Three parts, three *different* instruction verbs, three different mark values. "What do you understand by" invites one sentence for 1mk. "Mention any 2" wants a bare list for 2mks, and the paper writes the count as the digit **2**. "Describe" wants a mechanism for 2mks, which is a different kind of answer again. The 23/24 paper sets this word for word as Q4(a) with the same three marks, and drops the full stops from the ends of ii) and iii). Note also that the paper capitalises "Von Neumann Concept" as though it were a defined proper noun; it is not one, so say what the concept *is* rather than hunting for a memorised phrase.

BREAK IT DOWN

BEFORE YOU COMPUTE

GIVEN	nothing at all. All three parts are recall, which is why this is one of the cheapest 5 marks on the paper.
ASKED	a definition, then a list of exactly two items, then a description. "any 2" caps you at two: a third aspect earns nothing and costs you time.
MARKS	1 + 2 + 2. The 1mk part gets one sentence. Spending a paragraph on it is the commonest way students run out of time on this question.
TAUGHT IN	this unit, Module One Unit 1.
TRAP	answering (iii) with "one bus is shared" and stopping. That is the cause, not the bottleneck. "Describe" for 2mks wants the consequence as well: the CPU is left waiting on the single path, so memory, not the processor, sets the speed of the whole machine.

WORKED EXAMPLE

PQ 24/25 Q3A · PQ 23/24 Q4A

(i) The Von Neumann concept: a computer stores its program (instructions) and its data together in the SAME memory, and executes instructions one after another by fetching them from that memory. (ii) Two aspects: (a) a single shared memory holds both instructions and data; (b) instructions are executed sequentially, one at a time, using a stored program. (iii) The bottleneck: because one bus is shared for instructions and data, the CPU cannot fetch an instruction and access data simultaneously, so the single CPU-to- memory path limits overall speed.

This same material answers the recurring bottleneck-and-mitigations question on the 20/21 paper: add the three fixes above for the extra marks.

⚠ TRAP

Do not say Harvard is "better" and stop there. The exam wants the **trade-off**: Harvard is faster because of simultaneous access, but Von Neumann is simpler, cheaper and more flexible, which is why general-purpose computers use it. A bare "Harvard is faster" leaves marks on the table.

RECALL AND CHECK

In one sentence each: what is the Von Neumann bottleneck, and what is the single most common cure for it?

ANSWER

The bottleneck is that the single shared bus lets only instructions or data move at a time, so the CPU waits on memory. The most common cure is cache memory: a small fast store that keeps most accesses off the slow bus.

FILL IN THE GAP

1. Architecture describes what the _____ sees; organization describes how it is _____.
2. Von Neumann uses _____ memory for code and data; Harvard uses _____.
3. The Von Neumann bottleneck exists because the _____ is shared between instructions and data.
4. The three steps of the instruction cycle are fetch, _____ and _____.

ANSWERS

1. programmer; built. 2. one (shared); two separate memories. 3. bus. 4. decode; execute.

MODULE ONE · UNIT 2

Data Representation

How whole numbers, positive and negative, are stored in a fixed number of bits. This is the single most examined topic on the paper, and every mark here is a mark you can guarantee with practice.

By the end of this unit you can represent any integer in unsigned, sign-magnitude, one's complement and two's complement form; state the range of each in n bits; add and subtract in binary; recognise overflow; use end-around carry; and size an image in memory. Every past question of these types is worked in full.

Unsigned integers: the starting point

With n bits and no sign, you can store the whole numbers from 0 up to 2^n minus 1. Each pattern of bits is just its plain binary value. An 8-bit byte therefore holds 0 to 255, because 2^8 minus 1 is 255.

MUST MEMORISE

UNSIGNED RANGE

In n bits, unsigned integers run **0 to $2^n - 1$** . That is 2^n different values in total.

WHAT EACH SYMBOL MEANS

n = the number of bits in the register or field (8 for a byte, 12 or 16 if the question says so) ·

2^n = how many different bit patterns those n bits can make · the **$- 1$** is there because one pattern is used up by zero

Memory hook: n bits give 2^n patterns; one of them is zero, so the largest is one less than 2^n .

TEACH

SHIFTING: MULTIPLY AND DIVIDE BY TWO

Because each position is worth twice the one to its right, moving every bit **one place left** doubles a number ($\times 2$) and moving it **one place right** halves it ($\div 2$), in binary or hex alike. To halve **3FC** in hex, shift its bits right by one: **3FC** = 0011 1111 1100, shifted right is 0001 1111 1110 = **1FE**. Check in decimal: **3FC** = 1020, and $1020 \div 2 = 510 = 1FE$; a left shift of **1FE** returns **3FC**. This is why shift instructions are the fastest way a processor multiplies or divides by powers of two.

The problem: how do you store a minus sign?

A memory cell holds only 0s and 1s, with nowhere to write a "-". So we spend one of the bits, the leftmost, to carry the sign, and 0 means positive, 1 means negative. There are three ways to do it, and the exam expects all three.

TEACH

THE THREE SIGNED SCHEMES

- ▶ **Sign-magnitude:** the leftmost bit is the sign, the remaining bits are the plain size (magnitude) of the number. Simple to read, but it has two zeros (+0 and -0) and makes arithmetic awkward.
- ▶ **One's complement:** to make a number negative, **invert every bit** of its positive form. Also has two zeros, but subtraction becomes addition (with a twist called end-around carry, below).
- ▶ **Two's complement:** invert every bit and **add 1**. This is the one real computers use: it has a single zero, and addition and subtraction work with no special cases.

Seeing one small number in all three side by side fixes the idea. Take **-5** in 4 bits. First write +5 as 0101, then apply each rule.

VALUE	SIGN-MAGNITUDE	ONE'S COMPLEMENT	TWO'S COMPLEMENT
+5	0101	0101	0101
-5	1101	1010	1011
How	flip sign bit to 1	invert all bits of +5	invert all bits, then add 1

To negate in two's complement: invert every bit, then add 1. The same recipe converts back, so applying it twice returns the original.

Ranges in n bits:

SCHEME	RANGE	ZEROS
Unsigned	0 to $2^n - 1$	one
Sign-magnitude	$-(2^{n-1} - 1)$ to $+(2^{n-1} - 1)$	two
One's complement	$-(2^{n-1} - 1)$ to $+(2^{n-1} - 1)$	two
Two's complement	-2^{n-1} to $+(2^{n-1} - 1)$	one

WHAT EACH SYMBOL MEANS

n = the total number of bits, *including* the sign bit · $n - 1$ = the bits left for the magnitude once the sign bit is spent · so 2^{n-1} is the largest magnitude the remaining bits can hold

Memory hook: two's complement reaches **one extra negative** value, because it has no wasted second zero. In 8 bits that is -128 to $+127$.

THE EXAMINER'S WORDS, UNEDITED

- b. i) Complete table 1. Show all necessary work done to arrive at the answer. Use 12 bits. **6mks**

Value	Unsigned	S-M	1's Complement	2's Complement
204				
-139				
1101 00110 11				

Read it again. Three things the paper does that a tidy restatement would hide. It sends you to "table 1" and then prints the table with no number and no caption anywhere on the page. It abbreviates sign-magnitude to **S-M** and never expands it. And it instructs **Use 12 bits**, while the third row it asks you to complete, **1101 00110 11**, is only **11** bits, grouped 4, 5, 2 rather than in fours. The contradiction is on the paper and you must resolve it in the hall.

THE EXAMINER'S WORDS, UNEDITED

- b. i) Complete the table below. Show all necessary work done to arrive at the answer. Use 12bits. **6mks**

Value	Unsigned	S-M	1's Complement	2's Complement
204				
-139				
1101 00110 11				

The drift is the point. Same marks, same three rows, same four columns, two differences in the wording: 23/24 says "Complete **the table below**" where 24/25 says "Complete **table 1**", and 23/24 writes **Use 12bits.** closed up with no space. What matters more is what did *not* change: the 11-bit row survives into the next year untouched, so this is a defect the setter copies forward, not a one-off slip. Expect to meet it again, and have your sentence about it ready.

BREAK IT DOWN

BEFORE YOU COMPUTE

- GIVEN** two decimal values, 204 and -139, one bit pattern, 1101 00110 11, and a register width of 12 bits. Four columns are wanted for each row: Unsigned, S-M, one's complement, two's complement.
- ASKED** two things joined by a full stop: *complete the table*, and *show all necessary work done to arrive at the answer*. The second sentence is worth marks on its own, so a page of finished bit patterns with no derivation is a half answer.
- MARKS** 6mks across twelve cells, so no single cell is worth arguing over. The working is what separates a 3 from a 6.
- TAUGHT IN** this unit, Module One Unit 2.
- TRAP** the 11-bit row. Do not silently pad it to 12 and say nothing, and do not stall. Write one line saying the row as printed is 11 bits against an instruction of 12, state the assumption you are making (a leading 0), and carry on. Second trap: 204 is positive, so it is identical in all four columns; only the negative row needs real work.

GIVEN	values = 204 and -139 · $n = 12$ bits · four schemes required
FIND	each value's bit pattern in every scheme
FORMULA	sign-magnitude: sign bit + magnitude · one's complement: invert all n bits · two's complement: invert, then add 1

WHAT EACH SYMBOL MEANS

$n = 12$, the total bits *including* the sign bit · a **positive** value looks identical in all four schemes, so only the negative needs work · "invert all n bits" includes the leading zeros

- 1 Write 204 in 12-bit binary. $204 = 128 + 64 + 8 + 4$, so $204 = 0000\ 1100\ 1100$.
Being positive, 204 looks identical in all four schemes.
- 2 For -139 , first write $+139$ in 12 bits: $139 = 128 + 8 + 2 + 1 = 0000\ 1000\ 1011$.
- 3 Sign-magnitude of -139 : set the sign bit to 1, keep the magnitude: $1000\ 1000\ 1011$.
- 4 One's complement of -139 : invert every bit of $0000\ 1000\ 1011$: $1111\ 0111\ 0100$.
- 5 Two's complement of -139 : add 1 to the one's complement: $1111\ 0111\ 0100 + 1 = 1111\ 0111\ 0101$.

VALUE	UNSIGNED	SIGN-MAGNITUDE	ONE'S COMPLEMENT	TWO'S COMPLEMENT
204	0000 1100 1100	0000 1100 1100	0000 1100 1100	0000 1100 1100
-139	not representable	1000 1000 1011	1111 0111 0100	1111 0111 0101

Now redo it: the third row on the real paper gives a bit pattern and asks its value. Read back $1111\ 0111\ 0101$ as two's complement: the leading 1 says negative, so invert and add 1 ($0000\ 1000\ 1011 = 139$), giving -139 . That is the same number, proving the recipe reverses itself.

THE EXAMINER'S WORDS, UNEDITED

c. A spacecraft telemetry system uses a 16-bit processor to transmit environmental measurements. The following values are to be stored - Temperature = -182 ; Pressure = 487 ; Altitude = -1023 . The processor uses 2's complement representation for signed integers.

i) Determine the range of values representable in a 16-bit two's complement system.

Indicate whether the given numbers fall within the range of value **2mks**

ii) Represent each value in: Binary and Hexadecimal **3mks**

iii) Demonstrate how the processor uses two's complement to compute **2mks**
 $(-182) + 487$

Read it again. The paper spells the scheme two ways inside one question: **2's complement** in the stem, **two's complement** in parts i) and iii). Part i) ends "fall within the range of **value**", singular, with no full stop, and it packs two instructions into 2mks. In part iii) the mark annotation is printed *above* the expression it pays for, so **$(-182) + 487$** hangs on the line beneath **2mks**. That expression uses a true minus sign, while the same value in the stem is written with a plain hyphen, "Temperature = -182 ". Most important of all, iii) says **Demonstrate how**, not "calculate": the number 305 on its own does not answer it.

GIVEN	a 16-bit machine using two's complement, and three values to store: Temperature = -182, Pressure = 487, Altitude = -1023. The spacecraft is scenery; the only numbers that matter are 16, and those three.
ASKED	i) <i>determine</i> the range and then <i>indicate</i> whether the three values sit inside it, which is two instructions on 2mks. ii) <i>represent</i> each value in both binary and hexadecimal, so six answers, not three. iii) <i>demonstrate how</i> the processor does the addition.
MARKS	$2 + 3 + 2 = 7$ of the 30 on the compulsory question, and every one of them is method you already own.
TAUGHT IN	this unit, Module One Unit 2.
TRAP	part iii). "Demonstrate how" is asking for the mechanism, so write out both 16-bit patterns, add them column by column, and say in words that the carry out of the top bit is discarded . That discarded carry is the whole difference between two's complement and one's complement, and it is what the mark is for. Answering "305" earns almost nothing.

GIVEN	$n = 16$ bits · scheme = two's complement · values: Temperature = -182 , Pressure = 487 , Altitude = -1023
FIND	(i) the range and whether each fits, (ii) each in binary and hex, (iii) the sum $(-182) + 487$
FORMULA	range = -2^{n-1} to $+(2^{n-1} - 1)$ · negate by invert-and-add-1 · add the patterns and discard the final carry

WHAT EACH SYMBOL MEANS

$n = 16$, so $n - 1 = 15$ bits of magnitude · $2^{15} = 32768$, which is why the range is -32768 to $+32767$ · the discarded carry is the signature of two's complement (one's complement would add it back)

(i) Range

16-bit two's complement: -2^{15} to $+(2^{15}-1) = -32768$ to $+32767$. All three values (-182 , 487 , -1023) fall inside this range, so all are representable.

(ii) Binary and hex

VALUE	16-BIT TWO'S COMPLEMENT (BINARY)	HEX
-182	1111 1111 0100 1010	FF4A
487	0000 0001 1110 0111	01E7
-1023	1111 1100 0000 0001	FC01

Method for -182 : $+182 = 0000\ 0000\ 1011\ 0110$; invert to $1111\ 1111\ 0100\ 1001$; add 1 to get $1111\ 1111\ 0100\ 1010 = \text{FF4A}$.

(iii) $(-182) + 487$

Add the 16-bit patterns of -182 (FF4A) and 487 (01E7): $1111\ 1111\ 0100\ 1010 + 0000\ 0001\ 1110\ 0111 = 1\ 0000\ 0001\ 0011\ 0001$. The carry out of the 16th bit is discarded in two's complement, leaving $0000\ 0001\ 0011\ 0001 = 0131\text{H} = 305$. Check: $-182 + 487 = 305$. Correct.

Now redo it: this whole question is 25/26 Q1c and is worth easy marks. The trick is that in two's complement you just add the patterns and throw away the final carry.

Seen again: the spacecraft question turned up once more on the lecturer's revision deck, so it is worked a second time in the Revision Class section near the back. It is one to have cold.

◆ KEY POINT

A bit pattern carries no meaning by itself. $1111\ 1111$ is 255 as an unsigned integer, minus 1 in two's complement, minus 127 in sign and magnitude, and a perfectly ordinary character in a text file. The interpretation is a decision made outside the bits, and every question of this type tells you which one to make.

BLUNT VERSION: the bits do not know what they are. Read the question for the code before you read the pattern.

Adding in binary, and when it overflows

Binary addition is column addition with carries, exactly like decimal but with only 0 and 1. The danger is **overflow**: when the true sum needs more bits than the register has, the top bit is lost and the stored answer is wrong.

AS PRINTED

20/21 Q5(B)(II)

THE EXAMINER'S WORDS, UNEDITED

ii) What will be displayed (in binary and decimal) as the sum of 214 and 65 using an 8-bit register? Show details of work? **3mks**

Read it again. The verb is **displayed**, not "is". That one word is the entire question: 214 + 65 is 279, and an 8-bit register has no way to show 279, so what it displays is a different number from what the sum is. Any restatement along the lines of "*add 214 and 65 and comment on overflow*" has already told you the answer and destroyed the question. Note too that "Show details of work?" is an instruction wearing a question mark, a habit this paper repeats, and that the question never says whether the register is signed, so which reading you take is an assumption you must state.

BREAK IT DOWN

BEFORE YOU COMPUTE

GIVEN	two decimal values, 214 and 65, and a register 8 bits wide. Nothing is said about signed or unsigned.
ASKED	what the register <i>displays</i> , given <i>in binary and decimal</i> , with the working shown. That is three deliverables, not one.
MARKS	3mks, which reads naturally as one for the binary pattern, one for its decimal value, one for the working that gets you there.
TAUGHT IN	this unit, Module One Unit 2.
TRAP	writing 279. That is the arithmetic answer, and the question did not ask for it. Give the 8 bits the register actually holds, their decimal value, and one sentence naming what happened to the ninth bit. Second trap: answering in decimal only when the paper asked for both forms.

GIVEN	operands = 214 and 65 · n = 8 bits · unsigned
FIND	the stored result in binary and decimal, and whether it overflows
FORMULA	unsigned capacity = $2^n - 1$ · a register keeps only the low n bits, so the stored value is the true sum mod 2^n

WHAT EACH SYMBOL MEANS

n = 8, so the register holds 0 to 255 · **mod 2^n** means the true sum wraps by exactly 256 ·

overflow = the true answer needed more bits than the register has, signalled by a carry out of the top

- 1 Convert: 214 = 1101 0110, 65 = 0100 0001.
- 2 Add the two bytes: 1101 0110 + 0100 0001 = 1 0001 0111.
The result needs 9 bits.
- 3 An 8-bit register holds only the low 8 bits, 0001 0111 = 23, and sets a carry.

True sum 214 + 65 = 279. In 8 bits the register keeps 0001 0111 = 23 and raises a carry out of the top bit. This is unsigned **overflow**: 279 will not fit in a byte (max 255), so the stored 23 is wrong by exactly $2^8 = 256$.

Now redo it: overflow is simply "the answer needed more bits than the box has". State the true sum, the wrapped value, and that a carry came out.

Subtracting by adding: end-around carry

Computers subtract by adding a negative. In **one's complement**, this needs one extra step: if a carry comes out of the top after the addition, you **add it back** into the lowest bit. That step is the **end-around carry**, and it is asked almost every year.

THE EXAMINER'S WORDS, UNEDITED

- b. i) Describe End-Around Carry in 1's Complement? Illustrate this using 153 – 142 in a 10-bit register. **4mks**

Read it again. "Describe" is a command and the paper closes it with a **question mark**. The second sentence is a separate instruction: **Illustrate this using 153 – 142**, so a description with no worked sum cannot score full marks, and a worked sum with no description cannot either. The 23/24 paper sets this line word for word as Q5(d), same numbers, same 4mks, same stray question mark, so it is a fixture rather than a coincidence.

THE EXAMINER'S WORDS, UNEDITED

ii) Describe End-Around Carry in 1's Complement? Illustrate this using $153 - 142$ in an 10-bit register. **4mks**

The drift is the point. Character for character this is the 24/25 line, with one change: the 20/21 paper says "in **an** 10-bit register" and the later two say "in **a** 10-bit register". Nothing else moved, not the numbers, not the marks, not the question mark after a command. Three papers across four years running the same sentence with the article quietly swapped tells you two useful things: this question is close to guaranteed, and the paper you sit will not be proofread either.

BREAK IT DOWN

BEFORE YOU COMPUTE

GIVEN	the operands 153 and 142, a register 10 bits wide, and the scheme named for you as one's complement.
ASKED	two verbs. <i>Describe</i> what end-around carry is, then <i>illustrate</i> it on these particular numbers. Marks sit behind both.
MARKS	4mks, which splits sensibly as one for the description and three for the illustration, since the illustration is five lines of working.
TAUGHT IN	this unit, Module One Unit 2.
TRAP	reaching for two's complement out of habit and throwing the carry away. In one's complement the carry comes back in at the bottom, and discarding it here leaves you with 10 instead of 11, wrong by exactly one and wrong in a way that looks right. Second trap: writing 153 and 142 in eight bits because they fit. The question says 10, so pad both to 10 before inverting, or the inversion produces the wrong pattern.

GIVEN minuend = 153 · subtrahend = 142 · n = 10 bits · scheme = one's complement
 FIND the difference, as a 10-bit pattern and in decimal
 FORMULA $a - b = a + (-b)$, where $-b$ is the bitwise inversion of b ; then if a carry leaves the top, **add it back** into the lowest bit

WHAT EACH SYMBOL MEANS

n = 10, the register width, so every pattern is written to 10 bits · **end-around carry** = that add-it-back step, and it happens *only* in one's complement · no carry out means no correction, and the result is negative

- 1 153 – 142 becomes $153 + (-142)$. Write 153 in 10 bits: 00 1001 1001.
- 2 Write –142 in one's complement: $+142 = 00\ 1000\ 1110$, invert to 11 0111 0001.
- 3 Add: $00\ 1001\ 1001 + 11\ 0111\ 0001 = 1\ 00\ 0000\ 1010$.
A 1 has carried out of the 10th bit.
- 4 End-around carry: drop that carry and add it into the lowest bit: $00\ 0000\ 1010 + 1 = 00\ 0000\ 1011$.
- 5 Read the result: $00\ 0000\ 1011 = 11$. Positive (leading 0), and $153 - 142 = 11$. Correct.

153 – 142 = 00 0000 1011 (one's complement, 10 bits) = 11.

Now redo it: the signature of one's complement is that a carry out never vanishes, it comes back in at the bottom. If there is no carry out, there is nothing to add and the result is already negative.

Two words the exam likes to separate. **Overflow** is a result too large (too far positive, or a magnitude too big) to fit the available bits. **Underflow** is a result too small to represent: for integers, a value more negative than the smallest storable number; for floating point (Unit 3), a value so close to zero it rounds away to nothing. Example: in 8-bit two's complement (range –128 to +127), $100 + 50 = 150$ overflows; $-100 - 50 = -150$ underflows.

Sizing an image in memory

An image is a grid of dots (pixels). Its size in memory is simply the number of pixels times the bits used per pixel. The number of pixels is the physical size in inches times the resolution in dots per inch (dpi), in each direction.

bits = (width×dpi) × (height×dpi) × bits-per-pixel. Then divide by 8 for bytes, by 1024 for KB, by 1024 again for MB. Unless a colour depth is given, this course assumes a black-and-white image at **1 bit per pixel** (as the lecture example does).

WHAT EACH SYMBOL MEANS

width, height = the physical size of the image in *inches* · **dpi** = dots (pixels) per inch, the resolution · so (width×dpi) is the number of pixels across and (height×dpi) the number down ·

bits-per-pixel = the colour depth (1 for black and white, 8 for 256 greys, 24 for full colour)

AS PRINTED

25/26 Q4(C)

THE EXAMINER'S WORDS, UNEDITED

c. Calculate the memory requirement of an image whose dimension is 17.12 inches by 9.45 inches at resolution 250 dpi. Express your answer in Megabyte **3mks**

Read it again. One sentence, three years, three marks every time. It appears as 24/25 Q3(b)(ii), which adds a full stop after "Megabyte", and as 23/24 Q4(b)(ii), which like this one has none. What the sentence never tells you is the **colour depth**, so bits per pixel is an assumption you have to make and state, and it never tells you whether a **Megabyte** means 1024×1024 bytes or one million bytes, which is why two defensible answers to this part differ by about five per cent. Note also "whose **dimension** is", singular, for two dimensions, and "Megabyte", singular and capitalised.

BREAK IT DOWN

BEFORE YOU COMPUTE

GIVEN	a physical size of 17.12 inches by 9.45 inches and a resolution of 250 dpi. That is everything the paper supplies.
ASKED	<i>calculate</i> the memory requirement, and <i>express your answer in Megabyte</i> . The unit conversion is part of the instruction, so a total left in bits or in bytes has not finished the question.
MARKS	3mks, reading as one for the pixel count, one for turning bits into bytes, one for landing in MB.
TAUGHT IN	this unit, Module One Unit 2.
TRAP	the two silences. Open with a single line naming what you assume for bits per pixel and what you take a Megabyte to be, and no marker can fault the arithmetic that follows. Second trap: 9.45×250 is 2362.5, not a whole number of pixels. Carry it as it is; rounding it early moves the answer.

GIVEN	width = 17.12 in · height = 9.45 in · resolution = 250 dpi · colour depth = 1 bit per pixel (black and white, not stated so assumed)
FIND	the memory required, in megabytes
FORMULA	bits = (w×dpi) × (h×dpi) × bpp, then ÷8 for bytes, ÷1024 for KB, ÷1024 for MB

WHAT EACH SYMBOL MEANS

w, h = the physical size in inches · **dpi** = pixels per inch, so w×dpi is the pixel count across · **bpp** = bits per pixel · the two divisions by 1024 are what turn bytes into MB

- 1 Pixels across: $17.12 \times 250 = 4280$. Pixels down: $9.45 \times 250 = 2362.5$.
The height gives a non-whole 2362.5; carry it as is, the arithmetic is what earns the marks.
- 2 Total pixels: $4280 \times 2362.5 = 10\,111\,500$.
- 3 At 1 bit per pixel that is 10 111 500 bits. Divide by 8: 1 263 937.5 bytes.
- 4 Divide by 1024 twice: $1\,263\,937.5 \div 1\,048\,576 = 1.205$ MB (about 1.21 MB).

10 111 500 pixels × 1 bit = 10 111 500 bits = 1 263 937.5 bytes ≈ **1.21 MB** (dividing by 1024×1024). If the paper divides by one million instead, it is about 1.26 MB: state which you used.

Now redo it: if a colour depth were given (say 24-bit colour), you would multiply the pixel count by 24 before dividing. Always check whether the question gives bits per pixel.

Two ways to store a picture: bitmap and vector

The sizing above assumed a **bitmap**. There are two families of computer graphics, and the point the exam likes is why their sizes differ so much.

TEACH

BITMAP VERSUS VECTOR GRAPHICS

- ▶ **Bitmap (raster)** graphics store the picture as a grid of **pixels**, each pixel's colour held explicitly. More pixels per inch means higher **resolution** and a sharper image, but the size grows with the pixel count (as sized above), and enlarging a bitmap makes the individual pixels show. Photographs are bitmaps.
- ▶ **Vector** graphics store the picture as **objects**, such as lines, curves and shapes, each described by attributes: coordinates, length and breadth, line thickness, colour and fill. The image is redrawn from the maths at any size with no loss, and the file holds a few objects rather than millions of pixels, so line art and logos are far smaller as vectors. A photograph, having no simple shapes, does not reduce to vectors.

The deck states the bitmap side positively, and it is the phrasing to give back: the advantages of bit-mapped graphics, the ones the paint packages use, are that **every single pixel is yours to alter**, and that they look **more realistic for photographs and real-life images**. Vector wins on size and on scaling; bitmap wins on fine control and on realism.

A black-and-white bitmap is 4 inches by 6 inches at 300 dpi. How much memory does it need?

GIVEN	width = 4 in · height = 6 in · resolution = 300 dpi · colour depth = 1 bit per pixel (black and white)
FIND	the memory required, in KB
FORMULA	bits = (w×dpi) × (h×dpi) × bpp, then ÷8 for bytes and ÷1024 for KB

WHAT EACH SYMBOL MEANS

w, **h** = inches · **dpi** = pixels per inch · **bpp** = 1 here, because black-and-white needs only one bit to say which of two colours a pixel is

- 1 Pixels across: $4 \times 300 = 1200$. Pixels down: $6 \times 300 = 1800$.
- 2 Total pixels: $1200 \times 1800 = 2\,160\,000$.
- 3 At 1 bit per pixel (black or white): $2\,160\,000 \text{ bits} \div 8 = 270\,000 \text{ bytes}$.
- 4 In KB: $270\,000 \div 1024 = \mathbf{263.67 \text{ KB}}$.

$2\,160\,000 \text{ pixels} \times 1 \text{ bit} = 2\,160\,000 \text{ bits} = 270\,000 \text{ bytes} \approx \mathbf{263.67 \text{ KB}}$. Same method as the 17.12×9.45 image, different numbers.

Coded decimal: BCD and Excess-3

Pure binary is not always convenient. When decimal digits must stay exact and be read back one digit at a time (tills, meters, calculator displays), each digit is coded on its own.

Binary Coded Decimal (BCD) stores each decimal digit in its own 4 bits, so 47 is **0100 0111**, not the pure-binary 101111. It wastes space (four bits can hold 0 to 15 but only 0 to 9 are used) yet keeps decimal exact. Standard BCD is a **weighted** code: the four positions carry the weights **8 4 2 1**, and a digit is whichever bits sum to it. Other weightings exist; the lectures name **5421** and **4221**, where the same "sum the weights of the 1-bits" rule applies.

Encode **9.2** in 8421 (standard BCD), 5421 and 4221.

GIVEN value = 9.2 · codes = 8421, 5421, 4221 · all are weighted 4-bit codes
FIND the coded form of 9.2 in each
FORMULA code each decimal digit on its own in 4 bits; in a weighted code the digit is the **sum of the weights of the bits set to 1**

WHAT EACH SYMBOL MEANS

the code's name is its weights: **8421** means the four positions are worth 8, 4, 2, 1 · **5421** means 5, 4, 2, 1 · **4221** means 4, 2, 2, 1 · the decimal point simply separates the digit codes, exactly as in decimal

DIGIT	8421	5421	4221
9	1001 (8+1)	1100 (5+4)	1111 (4+2+2+1)
2	0010 (2)	0010 (2)	0010 (2)

9.2 = **1001.0010** in 8421, **1100.0010** in 5421, **1111.0010** in 4221. Each code splits at the point exactly where decimal does.

Excess-3 (XS-3) codes a digit as its 4-bit binary **plus 3**, so 0 is 0011, 5 is 1000, 9 is 1100. It is **non-weighted** but **self-complementing**: the code of (9 – d) is the bit-by-bit complement of the code of d, which makes nine's-complement subtraction easy. For instance XS-3 of 6 is 1001 and XS-3 of 3 is 0110, which are exact complements.

DIGIT	0	1	2	3	4	5	6	7	8	9
XS-3	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100

(i) Encode **24** in Excess-3. (ii) Decode **0100 0110** from Excess-3.

GIVEN	(i) decimal 24 · (ii) the code 0100 0110 · code = Excess-3, 4 bits per decimal digit
FIND	(i) the Excess-3 code of 24, (ii) the decimal the given code stands for
FORMULA	to encode: each digit + 3, written in 4 bits · to decode: read each nibble as a number, then – 3

WHAT EACH SYMBOL MEANS

nibble = one group of 4 bits, holding exactly one decimal digit · the **+3** is the "excess" the code is named for · every valid Excess-3 nibble lies between 0011 (for 0) and 1100 (for 9)

- 1 Encode 24: digit 2 → $2+3 = 5 = 0101$; digit 4 → $4+3 = 7 = 0111$. So 24 = **0101 0111**.
- 2 Decode 0100 0110: 0100 = 4, minus 3 = 1; 0110 = 6, minus 3 = 3. So it reads **13**.

24 in Excess-3 is **0101 0111**; the code 0100 0110 decodes to **13**. Going in, add 3 to each digit; coming out, subtract 3 from each nibble.

Encode 27, 597 and 14.57 in Excess-3. Do all three before you look, and watch what happens at the point.

ANSWER

Add 3 to each decimal digit, then write that sum in four bits. Nothing else changes, and the decimal point stays exactly where it was.

- ▶ **27**: 2 and 7 become 5 and 10, so 0101 1010 = **01011010**.
- ▶ **597**: 5, 9 and 7 become 8, 12 and 10, so 1000 1100 1010 = **100011001010**.
- ▶ **14.57**: 1, 4, 5 and 7 become 4, 7, 8 and 10, so **01000111.10001010**. The point sits where it sat in the decimal, between the digit-4 group and the digit-5 group.

Excess-3 of 14.57 is the one worth practising: a fractional value is where people lose the point, and this code makes no special case of it.

Text is stored by an agreed table mapping each symbol to a number. Historically there was **Morse code** (variable-length dots and dashes, common letters kept short) and **Hollerith code** (punched-card columns, the ancestor of data processing). In computers today:

- ▶ **ASCII** uses 7 bits, so 128 codes (0 to 127). Codes 32 to 126 are printable, code 32 is space and code 127 is DEL; 0 to 31 are control codes. "Extended ASCII" uses the eighth bit for codes 128 to 255 (for example ISO 8859-1, Latin-1).
- ▶ **EBCDIC** uses 8 bits: IBM's mainframe code, descended from punched cards.
- ▶ **Unicode** uses up to 32 bits, one code for every writing system; its first 128 codes are identical to ASCII.

A few ASCII values worth carrying into the exam: 'A' = 65, 'a' = 97, '0' = 48, space = 32.

⚠ TRAP

Three classic slips. (1) In two's complement subtraction you **discard** the final carry; in one's complement you **add it back**. Do not mix them up. (2) Sign-magnitude and one's complement have **two** zeros; two's complement has **one**, which is why its range reaches one more negative value. (3) When negating, invert **all** the bits including leading zeros, not just the ones that are set.

RECALL AND CHECK

In n bits, what is the two's complement range, and why is it not symmetric about zero?

ANSWER

-2^{n-1} to $+(2^{n-1}-1)$. It is asymmetric because there is a single zero on the positive side of the wheel, so the negatives get one extra value. In 8 bits: -128 to $+127$.

FILL IN THE GAP

1. To negate a number in two's complement, invert every bit and then add _____.
2. Unsigned n-bit integers range from 0 to _____.
3. In one's complement subtraction, a carry out of the top is _____ to the lowest bit.
4. Overflow means the result needs _____ bits than the register has.

ANSWERS

1. one. 2. 2^n-1 . 3. added back (end-around carry). 4. more.

Floating Point Representation

How a computer stores numbers with a fractional part, using the IEEE-754 standard. Convert cleanly in both directions and you own a whole exam question, because one appears on every paper.

By the end of this unit you can lay out the IEEE-754 single-precision format, convert a decimal number to its 32-bit hex form, convert a hex pattern back to decimal, and compare stored floating-point values. Every past question of this type is worked in full.

Why floating point, and the idea behind it

Fixed integers cannot store 3.14 or 0.001. Floating point borrows scientific notation: write a number as a sign, a fraction between 1 and 2 (the **mantissa**), and a power of two (the **exponent**), just as 6.02×10^{23} does in base 10. The standard that fixes exactly how those three parts are packed into bits is **IEEE-754**.

ANCHOR

A kitchen scale reads 1.2 kg whether the pan holds 1.19 kg or 1.24 kg. It is not broken. It has a fixed number of digits, so anything falling between two of its steps is pushed to the nearest step it can show. Floating point is that scale, built with 24 binary digits of precision, and anything finer than its smallest step is rounded away the moment it is stored.

⚠ TRAP

WHAT THE ROUNDING ACTUALLY COSTS

The lecture deck opens this topic with two real failures, and they are worth carrying, because they are the answer to "why does a rounding error matter" that the examiner has already read.

Ariane 5, 4 June 1996. An unmanned rocket exploded forty seconds after lift-off, on its first voyage, after a decade of development costing 7 billion dollars. The cause was a software error. A 64-bit floating point number holding the rocket's horizontal velocity with respect to the platform was converted into a 16-bit signed integer. The value was larger than the largest integer a 16-bit signed field can hold, so the conversion failed and the guidance system stopped.

The Vancouver stock exchange, 1982. A new index was started at 1000.000 and re-computed after every transaction. Each recomputation threw away the digits it could not hold instead of rounding them, so every transaction lost a sliver. Twenty two months later the deck records the index at 520. Recomputed correctly it stood at about 1098, so truncation alone had eaten more than half of it.

Both are one fault in two costumes. A stored floating point number is an approximation; convert it into a narrower type, or accumulate it over enough steps, and it becomes a wrong answer that the arithmetic itself never flags.

◆ KEY POINT

Floating point trades exactness for range. It holds numbers far larger and far smaller than an integer field of the same width, and in exchange almost none of them are stored exactly.

BLUNT VERSION: integers are small and exact, floats are huge and approximate. Never test two floats for equality.

TEACH

IEEE-754 SINGLE PRECISION

Single precision uses **32 bits** in three fields:



Sign, then an 8-bit biased exponent, then a 23-bit mantissa.

- ▶ **Sign (1 bit):** 0 for positive, 1 for negative.
- ▶ **Exponent (8 bits):** the power of two, stored with a **bias of 127** added, so a true exponent of 10 is stored as 137. The bias lets negative exponents be stored without a second sign.
- ▶ **Mantissa (23 bits):** the fraction after the leading 1. Because a normalised number always starts "1.", that 1 is not stored: it is the **hidden bit**, giving 24 bits of precision for free.

● MUST MEMORISE

THE RECIPE, DECIMAL TO IEEE

Single precision is **1 sign, 8 exponent, 23 mantissa**, bias **127**. To convert a decimal number:

1. Sign bit: 0 if positive, 1 if negative. Work with the size only from here.
2. Write the number in binary (integer part and fraction part).
3. Normalise to $1.m \times 2^e$ by moving the point.
4. Exponent field = $e + 127$, in 8 bits.
5. Mantissa = the bits of m after the point, filled to 23 bits.

WHAT EACH SYMBOL MEANS

S = the sign bit (0 positive, 1 negative) · **E** = the *stored* exponent field, the number actually written in the bits · **e** = the *true* exponent, the real power of two, so $E = e + \text{bias}$ · **M** = the mantissa field, the fraction bits that follow the hidden leading 1 · **bias** = 127 for single precision (1023 for double)

Memory hook: **S, E, M** is **1, 8, 23**, and the bias is $127 = 2^7 - 1$. Store exponent plus 127; read exponent minus 127.

Represent **−452.25** in IEEE-754 single precision. This one terminates, so it converts exactly.

GIVEN	value = −452.25 · format = IEEE-754 single precision · layout S=1, E=8, M=23 bits · bias = 127
FIND	the 32-bit stored pattern, written in hex
FORMULA	normalise to $1.m \times 2^e$, then $E = e + \text{bias}$, and the stored bits are S E M

WHAT EACH SYMBOL MEANS

S = sign bit · **e** = the true exponent found by normalising · **E** = the stored exponent field, $e + 127$ · **M** = the 23 bits after the hidden leading 1 · **bias** = 127

- 1 Sign: the number is negative, so the sign bit is **1**.
- 2 Binary: $452 = 111000100$, and $0.25 = 0.01$, so $452.25 = 111000100.01$.
- 3 Normalise: move the point 8 places left, $111000100.01 = 1.1100010001 \times 2^8$. So the true exponent is 8.
- 4 Exponent field: $8 + 127 = 135 = 1000\ 0111$.
- 5 Mantissa: the bits after the point, 1100010001 , padded to 23 bits: $1100\ 0100\ 0100\ 0000\ 0000\ 000$.
- 6 Assemble sign, exponent, mantissa and group into hex: $1\ 1000\ 0111\ 1100\ 0100\ 0100\ 0000\ 0000\ 000 = 1100\ 0011\ 1110\ 0010\ 0010\ 0000\ 0000\ 0000$.

$-452.25 = 1\ 10000111\ 110001000100000000000000 = \mathbf{C3E22000H}$ (check: converting C3E22000H back gives −452.25 exactly).

Now redo it: the five steps never change. Sign, binary, normalise, exponent plus 127, mantissa to 23 bits.

Represent **64.2** in IEEE-754 single precision. The previous example terminated. This one does **not**, which is exactly why the deck sets it.

GIVEN	value = 64.2 · format = IEEE-754 single precision · layout S=1, E=8, M=23 bits · bias = 127
FIND	the 32-bit stored pattern, written in hex
FORMULA	normalise to $1.m \times 2^e$, then $E = e + \text{bias}$, and the stored bits are S E M

WHAT EACH SYMBOL MEANS

S = sign bit · **e** = the true exponent found by normalising · **E** = the stored exponent field, $e + 127$ · **M** = the 23 bits after the hidden leading 1 · **bias** = 127

- 1 Sign: the number is positive, so the sign bit is **0**.
- 2 Binary: $64 = 1000000$. For the fraction, double 0.2 again and again and read off the whole part each time: 0.4, 0.8, 1.6, 1.2, and then 0.4 once more. It has come back to where it started, so the bits repeat: $0.2 = 0.0011\ 0011\ 0011\dots$ for ever. So $64.2 = 1000000.0011001100110011\dots$
- 3 Normalise: move the point 6 places left. $64.2 = 1.0000000011001100110011\dots \times 2^6$, so the true exponent is 6.
- 4 Exponent field: $6 + 127 = 133 = 1000\ 0101$.
- 5 Mantissa: take the first 23 bits after the hidden leading 1. That is the six remaining bits of the integer part, then the repeating pattern: 0000 0000 1100 1100 1100 110. The next bit along is 0, so nothing rounds up.
- 6 Assemble sign, exponent, mantissa, then group into fours: 0 1000 0101 0000 0000 1100 1100 1100 110 = 0100 0010 1000 0000 0110 0110 0110 0110.

$64.2 = 0\ 10000101\ 00000000110011001100110 = \mathbf{42806666h}$ (read that pattern back and you get 64.19999694824219, not 64.2, which is the whole point of the example).

Now redo it: and then say out loud what the machine is actually holding. 64.2 cannot be stored exactly in 32 bits, so what goes in is the nearest value that can be.

THE EXAMINER'S WORDS, UNEDITED

- a. Represent the following -1240.56 using Single Precision IEEE format for floating point numbers. **6mks**

Read it again. "Represent **the following**" sets you up for a list and then gives one number, in-line, with the minus sign printed tight against the digits where it is easy to drop while copying onto your script. The line recurs unchanged as 24/25 Q3(c) and 23/24 Q4(c), 6mks every time. What it never says is how many mantissa bits to keep when the fraction refuses to terminate, whether to round or truncate, or whether the answer is wanted as 32 bits or as hex. Contrast the 24/25 paper's own Q5(c), which for a different value does pin it down, "All bits after the first 12 bits of the Mantissa should be 0", and opens with the words "Perform the calculation below **in using** Single Precision IEEE format". Where the paper is silent, say what you chose.

BREAK IT DOWN

BEFORE YOU COMPUTE

GIVEN	one decimal value, -1240.56 , and the format, IEEE single precision. That fixes 1 sign bit, 8 exponent bits, 23 mantissa bits and a bias of 127.
ASKED	a single verb, <i>represent</i> , with no sub-parts. When a paper puts 6 marks behind one verb, the marks are in the steps, not in the final string.
MARKS	6mks and six natural steps: sign, integer part in binary, fraction in binary, normalise, exponent field, mantissa. Show all six and the arithmetic marks itself.
TAUGHT IN	this unit, Module One Unit 3.
TRAP	.56 does not terminate in binary. Keep doubling until you have 23 mantissa bits, then stop, and write one line saying whether you truncated or rounded. Second trap: losing the minus sign, which costs you the sign bit and therefore the whole pattern.

WORKED EXAMPLE

PQ 23/24 Q4C · PQ 24/25 Q3C · PQ 25/26 Q5A

Here the fraction does not terminate, so we truncate the mantissa at 23 bits.

GIVEN	value = -1240.56 · format = IEEE-754 single precision · bias = 127 · mantissa limited to 23 bits
FIND	the 32-bit stored pattern, in hex
FORMULA	normalise to $1.m \times 2^e$, then $E = e + \text{bias}$; the fraction .56 is converted by repeated $\times 2$

WHAT EACH SYMBOL MEANS

e = true exponent · **E** = stored exponent, $e + 127$ · **M** = first 23 fraction bits after the hidden 1 · **repeated $\times 2$** = the method for a decimal fraction: multiply by 2, write down the whole part, keep the rest

- 1 Sign: negative, so 1.
- 2 Binary: $1240 = 100\ 1101\ 1000$. The fraction 0.56 keeps going: $0.56 = 0.10001111010111\dots$ (multiply by 2 repeatedly, taking the whole part each time). So $1240.56 = 10011011000.10001111010111\dots$
- 3 Normalise: $1.001101100010001111010111\dots \times 2^{10}$. True exponent is 10.
- 4 Exponent field: $10 + 127 = 137 = 1000\ 1001$.
- 5 Mantissa: the first 23 bits after the point: 0011 0110 0010 0011 1101 011.

$-1240.56 \approx 1\ 10001001\ 00110110001000111101011 = \text{C49B11EBH}$ (truncated). Rounding the last bit up instead gives C49B11ECH, which is what the hardware actually stores. Either is acceptable if you state your method.

Now redo it: the only new skill is turning a non-terminating fraction into bits by repeated multiplication by 2, stopping once you have 23 mantissa bits.

The reverse: hex pattern back to a decimal value

Going backwards, split the 32 bits into the three fields, undo the bias, restore the hidden 1, and evaluate.

1. Expand the hex to 32 bits and split as 1, 8, 23.
2. Sign bit gives the sign.
3. True exponent $e = (\text{exponent field}) - 127$.
4. Value $= \pm 1.\text{mantissa} \text{ (in binary)} \times 2^e$.

WHAT EACH SYMBOL MEANS

e = the true exponent, recovered as (stored field E) – bias · **1.mantissa** = the hidden leading 1 put back in front of the stored fraction bits · **±** = the sign taken from bit S · **bias** = 127 single, 1023 double

AS PRINTED

25/26 Q5(D)

THE EXAMINER'S WORDS, UNEDITED

- d. By using the reverse of the IEEE Single Precision procedure, determine the value (Base 10) of the following hexadecimal (base 16) value: C4F2E000h **6mks**

Read it again. One sentence, two spellings of the same idea: "(Base 10)" with a capital and "(base 16)" without. The pattern ends in a lower case **h**, which is the paper's mark for hexadecimal and not a digit of the number. The line recurs unchanged as 24/25 Q4(c), also 6mks. The 20/21 paper sets the same task on a different value and rewords it: "Use the reverse of the IEEE Single Precision to determine the value (Base 10) of the following hexadecimal (base 16) value: BE7C2002H", worth 5mks, with a capital H that year. Neither the value, nor the marks, nor the case of the suffix is safe to memorise.

BREAK IT DOWN

BEFORE YOU COMPUTE

GIVEN	one 32-bit pattern written in hex, C4F2E000h, and the format it is stored in, IEEE single precision.
ASKED	the decimal value, and the route is prescribed: <i>by using the reverse of the IEEE Single Precision procedure</i> . The question is about the procedure, so the procedure is what you write down.
MARKS	6mks for a single number, which means almost all of them are in the working: expand to 32 bits, split 1, 8, 23, unbiased the exponent, restore the hidden 1, evaluate.
TAUGHT IN	this unit, Module One Unit 3.
TRAP	the two habitual losses: forgetting the hidden leading 1, so the mantissa is read as 0.1110... instead of 1.1110..., and forgetting to take 127 off the exponent field. Third, smaller trap: the trailing h is notation, not a hex digit, so the pattern is eight digits, not nine.

GIVEN	stored pattern = C4F2E000H (32 bits) · format = IEEE-754 single precision · bias = 127
FIND	the decimal value it represents
FORMULA	value = $\pm 1.M \times 2^{(E - 127)}$

WHAT EACH SYMBOL MEANS

E = the 8-bit stored exponent read straight off the pattern · **M** = the 23 mantissa bits · **1.M** = the hidden leading 1 restored in front of them · the $\pm$ comes from bit S

- 1 Expand to 32 bits: C4F2E000 = 1100 0100 1111 0010 1110 0000 0000 0000.
- 2 Split: sign = 1; exponent = 1000 1001; mantissa = 111 0010 1110 0000... (rest zeros).
- 3 Sign 1 means negative.
- 4 Exponent: 1000 1001 = 137, so $e = 137 - 127 = 10$.
- 5 Restore the hidden 1: $1.11100101110 \times 2^{10}$. Move the point 10 places right: 11110010111.0.
- 6 Evaluate $11110010111 = 1024 + 512 + 256 + 128 + 16 + 4 + 2 + 1 = 1943$.

$$C4F2E000H = -1.1110010111 \times 2^{10} = -11110010111_2 = -1943.$$

Now redo it: the steps that lose marks are forgetting the hidden 1 and forgetting to subtract 127. Do both, every time.

THE EXAMINER'S WORDS, UNEDITED

a. Consider three (3) floating point numbers X, Y and Z stored in registers based on the IEEE 754 Single precision floating point format.

X = C1400000h, Y = 42100000h, Z = 41400000h

Convert X, Y and Z to decimal numbers and determine which of the following is true. **9mks**

- i. $(Y-Z) > 0$
- ii. $X + Z = 0$
- iii. $Z = X + Y$

Read it again. The values carry a lower case **h**, not the capital H this book uses in its own prose. The claims are set unspaced, "(Y-Z)>0" and "Z= X + Y" with nothing after the Z, and they sit below the instruction rather than inside it. The sentence breaks across two lines between "of the following" and "is true", which is the paper's line ending and not a second instruction. And note where this lands: it is the 24/25 Question One (a) reprinted with the same data and the same 9mks, but moved out of the compulsory question into Q4(a) of an optional one, so on this paper you can choose whether to meet it at all.

THE EXAMINER'S WORDS, UNEDITED

c. Consider three (3) floating point numbers X, Y and Z stored in registers based on the IEEE 754 Single precision floating point format.

X = C1400000h, Y = 42100000h, Z = 41400000h

Convert X, Y and Z to decimal numbers and determine which of the following

is true or false **9mks**

- i. $(Y-Z) > 0$
- ii. $X + Z = 0$
- iii. $Z = X + Y$

The drift is the point. Identical hex, identical claims, identical 9mks, and one changed word at the end: 23/24 asks which of them "is **true or false**" and stops without a full stop, where 24/25 and 25/26 ask only which "is **true**." That single word changes the job. "is true or false" demands a verdict on all three; "is true" alone tempts a reader under time pressure to find one, tick it and move on. Two of the three are true here, so the tempting reading loses marks in every year. Answer all three either way.

BREAK IT DOWN

BEFORE YOU COMPUTE

GIVEN	three 32-bit hex words and the statement that they are IEEE 754 <i>single</i> precision. That one word fixes the split at 1 sign bit, 8 exponent bits and 23 mantissa bits, and the bias at 127.
ASKED	two instructions in one sentence: <i>convert</i> all three to decimal, then <i>determine</i> the standing of three claims. The second verb carries the marks. A script that converts perfectly and stops has answered half the question.
MARKS	9mks, the biggest single part anywhere in this unit, split across three conversions and three verdicts. Roughly 2 per conversion and 1 per verdict, so the conversions must be shown rather than asserted.
TAUGHT IN	this unit, Module One Unit 3.
TRAP	letting the year's wording decide how many verdicts you give. Write TRUE or FALSE against every one of the three, every time, and you satisfy both versions of the question. Second trap: comparing the hex patterns directly. They are not ordinary integers; convert first, compare afterwards.

WORKED EXAMPLE

PQ 25/26 Q4A · PQ 24/25 Q1A · PQ 23/24 Q1C

GIVEN $X = \text{C1400000H}$ · $Y = \text{42100000H}$ · $Z = \text{41400000H}$ · all IEEE-754 single precision, bias 127

FIND the decimal value of each, then the truth of the three statements

FORMULA $\text{value} = \pm 1.M \times 2^{(E - 127)}$, then ordinary arithmetic on the decimals

WHAT EACH SYMBOL MEANS

E = stored exponent field of that pattern · **M** = its mantissa bits · convert **first**, compare **second**: the hex patterns cannot be compared directly

Convert each

HEX	SIGN / EXP FIELD / E	1.MANTISSA $\times 2^E$	DECIMAL
C1400000	1 / 130 / 3	$-1.1 \times 2^3 = -1100$	-12
42100000	0 / 132 / 5	$1.0001 \times 2^5 = 100100$	36
41400000	0 / 130 / 3	$1.1 \times 2^3 = 1100$	12

Apply the tests

$X = -12$, $Y = 36$, $Z = 12$. (i) $Y - Z = 36 - 12 = 24 > 0$: **TRUE**. (ii) $X + Z = -12 + 12 = 0$: **TRUE**. (iii) $X + Y = -12 + 36 = 24$, which is not 12, so $Z = X + Y$ is **FALSE**.

Now redo it: convert all three first, then the three tests are one line of ordinary arithmetic each. Do not try to compare the hex directly.

TEACH

DOUBLE PRECISION, AND WHAT IEEE-754 DEFINES

The same layout scaled up gives **double precision**, used wherever single precision's roughly seven significant digits are not enough, as in banking and scientific work.

FORMAT	TOTAL	SIGN	EXPONENT (BIAS)	MANTISSA	APPROX. RANGE
Single	32 bits	1	8 (bias 127)	23	$\pm 10^{38}$
Double	64 bits	1	11 (bias 1023)	52	$\pm 10^{308}$

The method is identical; only the field widths and the bias change, double using **bias 1023 = $2^{10} - 1$** . Beyond the formats, IEEE-754 also fixes the **operations** every conforming machine must provide: add, subtract, multiply, divide, remainder and square root, conversion between decimal and binary, and the handling of exceptions (overflow, underflow, divide-by-zero, and invalid results such as $0 \div 0$).

⚠ TRAP

The three marks students throw away here. (1) The exponent is **biased**: subtract 127 when reading, add 127 when writing. (2) The leading 1 is **hidden**, so a mantissa of 111001... means the value 1.111001..., not 0.111001.... (3) When the decimal fraction does not terminate, keep multiplying by 2 until you have 23 mantissa bits, then stop; do not round the whole number off early.

RECALL AND CHECK

An IEEE-754 single number has exponent field 1000 0001. What is its true exponent, and why?

ANSWER

1000 0001 = 129, and the true exponent is $129 - 127 = 2$. The stored field always carries the bias of 127, so you subtract it to recover the real power of two.

FILL IN THE GAP

1. IEEE-754 single precision splits 32 bits as 1 sign, _____ exponent, _____ mantissa.
2. The exponent bias in single precision is _____.
3. The leading 1 of a normalised mantissa is not stored; it is called the _____ bit.
4. To read the true exponent you _____ 127 from the exponent field.

ANSWERS

1. 8; 23. 2. 127. 3. hidden (implicit). 4. subtract.

MODULE ONE · UNIT 4

Instruction Set Architecture

The instruction set is the vocabulary the processor understands, and the boundary between hardware and software. Two exam staples live here: writing an expression in the 0, 1, 2 and 3 address formats, and computing an effective address.

By the end of this unit you can list the parts of an instruction set architecture, lay out an instruction format, translate an arithmetic expression into the four address organisations and count their bytes, and compute the effective address and operand for every addressing mode.

What the ISA is, and its parts

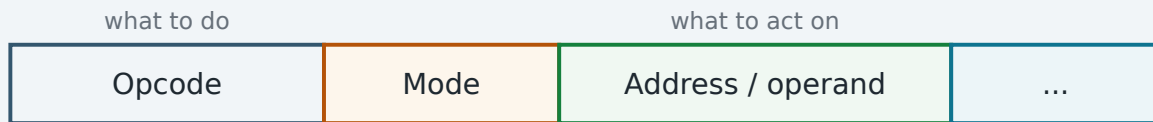
The **Instruction Set Architecture (ISA)** is the complete set of instructions a processor can execute, together with everything a programmer needs to use them. It is exactly the "architecture" of Unit 1: the programmer-visible design. Its components:

TEACH

COMPONENTS OF AN ISA

- ▶ **Instruction set:** the operations available (add, load, jump, and so on).
- ▶ **Registers:** the small fast stores inside the CPU. General-purpose registers hold working values; special ones include the **PC** (program counter, address of the next instruction), **IR** (instruction register, the current instruction), **MAR** (memory address register) and **MDR** (memory data register).
- ▶ **Data types:** the kinds of value handled (integers, floating point, characters, and Booleans, meaning true/false).
- ▶ **Addressing modes:** the ways an instruction can name where its operand is.
- ▶ **Instruction format:** how the bits of an instruction are laid out.

An instruction is a row of bits split into fields: an **opcode** saying what to do, zero or more **operand or address** fields saying what to do it to, and sometimes a **mode** field saying how to read the address.



Opcode plus operands. The number of address fields names the format.

WORKED EXAMPLE

SLIDE: DECODE THE BITS

An instruction is just a number, so with the format you can read one back. Suppose a 16-bit machine uses **opcode 4 bits, register 4 bits, address 8 bits**, and its opcode table has 1010 = ADD.

GIVEN	instruction = 1010 0011 11001010 · word = 16 bits · fields = opcode 4, register 4, address 8 · opcode 1010 = ADD
FIND	the assembly instruction the bits stand for
FORMULA	chop the bits into the given field widths, left to right, then translate each field

WHAT EACH SYMBOL MEANS

opcode field = which operation · **register field** = which register, read as a plain binary number · **address field** = the memory address, also plain binary · the field *widths* must be given or derived; they are not guessable from the bits alone

- 1 Decode **1010 0011 11001010**. Split 4 / 4 / 8: opcode 1010, register 0011, address 11001010.
- 2 Opcode 1010 = ADD; register 0011 = R3; address 11001010 = 128+64+8+2 = **202**.
- 3 So the instruction reads **ADD R3, [202]**: add the contents of location 202 to R3.

A 32-bit three-address machine just widens the fields. With **opcode 6 bits then three 5-bit register fields**, the bits **000001 00001 00010 00011** split as opcode 000001, then R1, R2, R3, giving **ADD R1, R2, R3**.

Reading an instruction is the reverse of building one: know the field widths, chop the bits, translate each field. 1010 0011 11001010 = ADD R3, [202].

THE LECTURER'S OWN EXAMPLES, NAMED PROCESSOR AND ALL

The deck does not leave the instruction types abstract. It names a real processor family and prints the actual mnemonics, and the revision class repeats them unchanged, which on this course is the strongest signal there is that they are worth knowing.

INSTRUCTION TYPE	WHAT IT DOES	THE DECK'S EXAMPLES
Load and store	move a value between memory and a register without changing it	LDA, STA, LDB, STB, LDD, STD
Arithmetic and logic	combine two operands and produce a result	ADDB, SUBA, SUBD, ANDA, EORA

The mnemonics decode themselves once you see the pattern. **LD** is load and **ST** is store, and the letter after it names the register: LDA loads accumulator A, STB stores accumulator B, LDD loads the double accumulator D. On the arithmetic side **ADDB** adds into accumulator B, **SUBA** subtracts from accumulator A, **ANDA** ands into accumulator A, and **EORA** is exclusive-OR into accumulator A. Each of these two-operand instructions requires one of its operands to already be in an accumulator, which is the one-address style this unit is about to formalise.

Note. The slides write the load and store family as MC68009 and the arithmetic family as MC6809. The processor family is the Motorola 6809; the extra zero is a slip on the slide. It is recorded here rather than tidied away, because a question quoting the slide will quote the slip too.

Zero, one, two and three address formats

Processors differ in how many operands one instruction may name. Fewer addresses means shorter instructions but more of them. Learn the four by translating one expression. Take **X** = **(A + B) × (C + D)**.

3-ADDRESS OP D,S1,S2	2-ADDRESS OP D,S	1-ADDRESS ACCUMULATOR	0-ADDRESS STACK
ADD T1,A,B	MOV T1,A	LOAD A	PUSH A
ADD T2,C,D	ADD T1,B	ADD B	PUSH B
MUL X,T1,T2	MOV T2,C	STORE T1	ADD
	ADD T2,D	LOAD C	PUSH C
	MUL T1,T2	ADD D	PUSH D
	MOV X,T1	MUL T1	ADD
		STORE X	MUL
			POP X
3 instructions	6 instructions	7 instructions	8 instructions

The pattern: three addresses name both sources and the destination, so each operation is one instruction. Two addresses reuse the destination as a source. One address keeps a running value in the **accumulator** and names only the other operand. Zero addresses take everything from a **stack**, so the expression is written in postfix (reverse Polish) order first.

● MUST MEMORISE

THE FOUR FORMATS

Fewer address fields, more instructions:

- ▶ **3-address:** op dest, src1, src2. Shortest programs.
- ▶ **2-address:** op dest, src (dest is also a source).
- ▶ **1-address:** LOAD, STORE, and ops against the single **accumulator**.
- ▶ **0-address:** PUSH and POP a **stack**; operators take the top items. Write the expression in **postfix** first.

Memory hook: 3, 2, 1, 0 addresses map to register, register, accumulator, stack machines. Postfix of A op B is "A B op".

AS PRINTED

25/26 Q4(B)

THE EXAMINER'S WORDS, UNEDITED

- b. Use a Zero, One, Two and Three address instruction formats to write a Pentium 4 assembly language program segment for the expression below. For each format determine the number of bytes required.

$X = ((A * B) + C/D)/(F * G + J * K)$ **8mks**

Read it again. The sentence opens ungrammatically, "Use a Zero, One, Two and Three address instruction **formats**", and the expression is spaced three different ways in one line: "(A * B)" spaced out, "+C/D" and "F * G + J * K" closed up. The paper names Pentium 4 and then wants generic accumulator and stack listings. Most important, it demands "the number of bytes required" while giving you **no opcode size and no address size**, so the count is impossible until you supply those numbers yourself. One more thing this printed sheet carries: that byte-count sentence is **struck through in pen**, with a short handwritten word beside it, so on the day the requirement was withdrawn. The 24/25 paper sets the identical sentence as Q1(d), nothing struck out, with the 8mks printed before the expression rather than after it.

GIVEN	one arithmetic expression and four instruction formats to express it in. No machine parameters at all.
ASKED	two deliverables per format: the <i>program segment</i> , and the <i>number of bytes required</i> . Eight answers in total.
MARKS	8mks, which lands neatly as one mark per listing and one per byte count. That tells you the listings must be complete, not sketched.
TAUGHT IN	this unit, Module One Unit 4.
TRAP	starting the byte counts before declaring your assumptions. Open with one line, for example 1 byte of opcode and 2 bytes per memory address, and every figure after it is defensible whatever the marker expected. Second trap: the zero-address listing. Convert the expression to postfix first; writing the pushes in infix order gives a stack sequence that does not evaluate.

The paper supplies no opcode or address size, so take **1 byte per opcode and 2 bytes per memory address** and say so before you count.

GIVEN	expression $X = ((A \times B) + C/D) / (F \times G + J \times K)$ · opcode = 1 byte · each memory address = 2 bytes
FIND	the program in all four formats, and the byte total of each
FORMULA	bytes per instruction = opcode bytes + (address fields × bytes per address); total = instruction count × that

Instruction counts

THE BYTE-COUNT FORMULA, SYMBOL BY SYMBOL

bytes per instruction = opcode bytes + (number of address fields × bytes per address). Here **opcode bytes** = 1 (given) · **number of address fields** = 3, 2, 1 or 0, which is exactly what names the format · **bytes per address** = 2 (given) · then **total bytes = instruction count × bytes per instruction**. A stack machine's PUSH and POP still carry one address each, while its bare operators (ADD, MUL) carry none, which is why the 0-address total is counted in two parts.

FORMAT	INSTRUCTIONS	BYTES (OPCODE 1, ADDRESS 2)
3-address	7	$7 \times (1 + 3 \times 2) = 7 \times 7 = 49$
2-address	12	$12 \times (1 + 2 \times 2) = 12 \times 5 = 60$
1-address	17	$17 \times (1 + 1 \times 2) = 17 \times 3 = 51$
0-address	16	$8 \text{ pushes} \times 3 + 1 \text{ pop} \times 3 + 7 \text{ ops} \times 1 = 34$

3-address program (the shortest)

```

MUL  T1, A, B      ; T1 = A*B
DIV   T2, C, D      ; T2 = C/D
ADD   T1, T1, T2    ; T1 = A*B + C/D  (numerator)
MUL   T2, F, G      ; T2 = F*G
MUL   T3, J, K      ; T3 = J*K
ADD   T2, T2, T3    ; T2 = F*G + J*K  (denominator)
DIV   X,  T1, T2    ; X = numerator / denominator

```

0-address program (postfix A B * C D / + F G * J K * + /)

```

PUSH A  PUSH B  MUL      ; A*B on stack
PUSH C  PUSH D  DIV  ADD  ; + C/D -> numerator
PUSH F  PUSH G  MUL      ; F*G
PUSH J  PUSH K  MUL  ADD  ; + J*K -> denominator
DIV      POP X      ; divide, store X

```

Now redo it: the marks are in the counts and one clean listing. Note that fewer addresses always trade shorter instructions for more of them, and the byte totals here are close precisely because of that trade.

Seen again: the same expression is on the lecturer's revision deck, worked once more in the Revision Class section near the back (the revision drops the byte-count part, which is the part struck out by hand on the exam paper).

◆ KEY POINT

The number of addresses an instruction carries trades instruction WIDTH against instruction COUNT. Three addresses name everything at once, so each instruction is long and few are needed; zero addresses name nothing, so each instruction is tiny and many are needed. The total work is conserved.

BLUNT VERSION: pay in instruction width or pay in instruction count. You do not get to skip both.

Addressing modes: where the operand really is

An address field does not always hold the operand directly. The **addressing mode** says how to turn that field into the **effective address (EA)**, the actual location of the operand.

MODE	EFFECTIVE ADDRESS	OPERAND IS
Immediate	none	the address field itself (a constant)
Direct	$EA = \text{address field}$	$M[\text{address field}]$
Indirect	$EA = M[\text{address field}]$	$M[M[\text{address field}]]$
Register (direct)	none	contents of the named register
Register indirect	$EA = \text{contents of the register}$	$M[\text{register}]$
Relative	$EA = PC + \text{address field}$	$M[PC + \text{address field}]$
Indexed	$EA = \text{index register} + \text{address field}$	$M[\text{index} + \text{address field}]$

THE EXAMINER'S WORDS, UNEDITED

e. A processor contains the following information: PC = 7000; R1 = 1500;

Memory Contents

Address	Value
7000	1200
1200	4500
1500	6000
3200	8100
4500	999
6000	200
8100	400

Assume an instruction contains address field 1200.

Determine the Effective Address (EA) and operand for the following addressing modes:

- i) Immediate addressing mode **1mk**
- ii) Direct addressing mode **1mk**
- iii) Indirect addressing mode **1mk**
- iv) Register addressing mode **1mk**
- v) Register Indirect (R1) addressing mode **1mk**
- vi) Relative addressing mode **1mk**

Read it again. Two things in that table repay a second look. The row **3200** holding **8100** is never reached by any of the six modes: it is there to be ignored, and following it wastes time you do not have. And part vi), Relative, gives $EA = PC + 1200 = \mathbf{8200}$, an address the table *does not list*, so the paper asks for an operand it never supplies. Notice as well that part v) names its register, "Register Indirect (**R1**)", while part iv) does not, and that every value in the table is decimal even though addresses usually arrive in hex. The single sentence "Assume an instruction contains address field 1200" is the input to all six parts.

BREAK IT DOWN

BEFORE YOU COMPUTE

GIVEN	PC = 7000, R1 = 1500, a seven-row memory table, and one address field, 1200, stated once and used by all six parts.
ASKED	for each mode, <i>both</i> the effective address and the operand. Six modes, so twelve answers, not six.
MARKS	six parts at 1mk each. One mark buys one line, so answer in the form "EA = ..., operand = ..." and move on.
TAUGHT IN	this unit, Module One Unit 4.
TRAP	inventing an effective address for immediate and for register mode. Neither has one; write "none" and say why in four words. Second trap: part vi). When the table does not hold M[8200], say plainly that the operand is outside the data given. Reporting the gap scores; making a number up does not.

GIVEN	PC = 7000 · R1 = 1500 · address field = 1200 · memory contents as tabulated below
FIND	the effective address and the operand for each addressing mode
FORMULA	immediate: operand = the field · direct: EA = field · indirect: EA = M[field] · register indirect: EA = register · relative: EA = PC + field

WHAT EACH SYMBOL MEANS

EA = effective address, the place the operand actually lives · **M[x]** = "the contents of memory location x" · **address field** = the number written in the instruction, here 1200 · **PC** = program counter

ADDRESS	7000	1200	1500	3200	4500	6000	8100
VALUE	1200	4500	6000	8100	999	200	400

MODE	EFFECTIVE ADDRESS	OPERAND
Immediate	none	1200 (the field is the value)
Direct	1200	M[1200] = 4500
Indirect	M[1200] = 4500	M[4500] = 999
Register (R1)	none	contents of R1 = 1500
Register indirect (R1)	R1 = 1500	M[1500] = 6000
Relative	PC + 1200 = 8200	M[8200] (outside the given table)

Now redo it: immediate uses the field as the value; direct reads memory once; indirect reads it twice; register indirect uses the register as a pointer; relative adds the PC. That single sentence answers the whole question.

Seen again: the lecturer put this same PC = 7000 table on his revision deck, worked once more in the Revision Class section near the back.

Indirect mode works for writing and read-modify, not only for reading. The address field points to a location whose **contents** are the effective address.

GIVEN	ADD R1,(150) with $R1 = 10$, $M[150] = 250$, $M[250] = 5$ · STORE R3,(500) with $R3 = 30$, $M[500] = 800$ · mode = indirect
FIND	the value left in R1, and the location and value written by the STORE
FORMULA	$EA = M[\text{address field}]$; then read $M[EA]$ for an ADD, or write to $M[EA]$ for a STORE

WHAT EACH SYMBOL MEANS

(150) = the brackets mark the mode as indirect, so 150 is *not* the operand's address, it holds it · **EA** = effective address · the direction of the transfer is set by the opcode: ADD reads $M[EA]$, STORE writes it

- 1 ADD R1, (150)** with $R1 = 10$, $M[150] = 250$, $M[250] = 5$. $EA = M[150] = 250$; operand = $M[250] = 5$; so $R1 = 10 + 5 = 15$.
- 2 STORE R3, (500)** with $R3 = 30$, $M[500] = 800$. $EA = M[500] = 800$, so the value 30 is written there: $M[800] = 30$.

ADD R1,(150) leaves $R1 = 15$; STORE R3,(500) sets $M[800] = 30$. Indirect always takes the address from memory first, then reads or writes the operand at it.

TEACH

CATEGORIES OF INSTRUCTIONS

Instructions fall into three families, and naming them with examples is a common short question:

- ▶ **Data transfer:** copy data, leaving the source unchanged; only the destination changes. **LOAD** copies from memory into a register, **STORE** copies a register back to memory. MOVE, PUSH, POP move it around too.
- ▶ **Arithmetic and logic:** compute. ADD, SUB, MUL, DIV, AND, OR, NOT, shifts.
- ▶ **Control (test and branch):** change what runs next. JUMP, BEQ (branch if equal), CALL, RETURN.

TEACH

CONDITION CODES AND CONDITIONAL BRANCHES

Test-and-branch instructions decide what runs next by reading the **condition code register** (also called the flags or status register). An arithmetic or logic operation sets flags as a side effect: **Z** (the result was zero), **N** (negative), **C** (a carry out), **V** (signed overflow). A conditional branch then tests a flag: **BEQ** branches if equal (Z set), **BNE** if not equal, **BMI** if minus (N set), **BCC** if carry clear, and **BRA** branches always.

Branches are usually **relative**: the target is the PC plus a signed offset, so a **negative offset branches backwards**, which is how loops are built. For example **BNE -40** with $PC = 1000$ gives $EA = 1000 - 40 = 960$, jumping back 40 locations.

THE EXAMINER'S WORDS, UNEDITED

- c. Provide an assembly language program to add two numbers and store the result.
4mks

Read it again. Twelve words and 4mks, which is a lot of marks for a very short instruction, and the gap between the two is the question. The paper names no machine, no register set and no addressing mode, so it never tells you whether it wants an accumulator machine's three lines or a three-address machine's one. That silence is what you are being paid to notice: the marks are for stating the machine model you assume, for the listing itself, and for a comment on each line. "Provide" also asks for a program, not a description of one, so prose alone will not do.

BREAK IT DOWN

BEFORE YOU COMPUTE

GIVEN	nothing at all beyond the task itself: add two numbers, store the result.
ASKED	a program, in assembly language. A listing on the page, with instruction mnemonics.
MARKS	4mks for what is three instructions. Read that as an invitation to write more than the three: the assumption, the listing, and a comment per line.
TAUGHT IN	this unit, Module One Unit 4.
TRAP	writing high-level pseudocode, or writing a correct listing without ever saying which machine it is for. Name the model in one line first, then LOAD, ADD, STORE, then the comments.

WORKED EXAMPLE

PQ 24/25 Q2C

On a one-address (accumulator) machine, load the first, add the second, store the sum:

```
LOAD  A      ; accumulator = A
ADD   B      ; accumulator = A + B
STORE C      ; C = A + B
```

On a three-address machine it is a single instruction: `ADD C, A, B`. Either is a full-mark answer; state which machine you assume.

⚠ TRAP

Do not confuse **indirect** with **register indirect**. Plain indirect goes to memory twice (the field points to a location that holds the real address). Register indirect uses a **register** as the pointer, so it touches memory once. And **immediate** has no effective address at all: the operand is the number in the instruction.

RECALL AND CHECK

In direct mode the address field is 1200 and $M[1200] = 4500$. What are the effective address and the operand? What changes in indirect mode?

ANSWER

Direct: $EA = 1200$, operand = $M[1200] = 4500$. Indirect: $EA = M[1200] = 4500$, and the operand is $M[4500]$, one further memory read.

FILL IN THE GAP

1. A 0-address machine evaluates expressions using a _____, so the expression is first written in _____ order.
2. The register holding the address of the next instruction is the _____.
3. In immediate mode the operand is _____.
4. In relative mode the effective address is _____ plus the address field.

ANSWERS

1. stack; postfix (reverse Polish). 2. program counter (PC). 3. the address field itself (a constant). 4. the PC.

MODULE TWO

Processor Design and Performance

Inside the CPU: the parts that fetch, decode and execute, and then the arithmetic of speed. How fast a processor runs is a number you can calculate, and this module turns clock cycles, instruction mixes and pipelines into marks you can bank.

$$\text{CPU time} = \text{IC} \times \text{CPI} \times T$$

instructions	cycles/instr	seconds/cycle
--------------	--------------	---------------

speed is arithmetic

Processor Components

The parts inside the CPU that carry out the fetch, decode, execute cycle, and the buses that connect them to memory. A few of these carry number questions about address size and memory capacity.

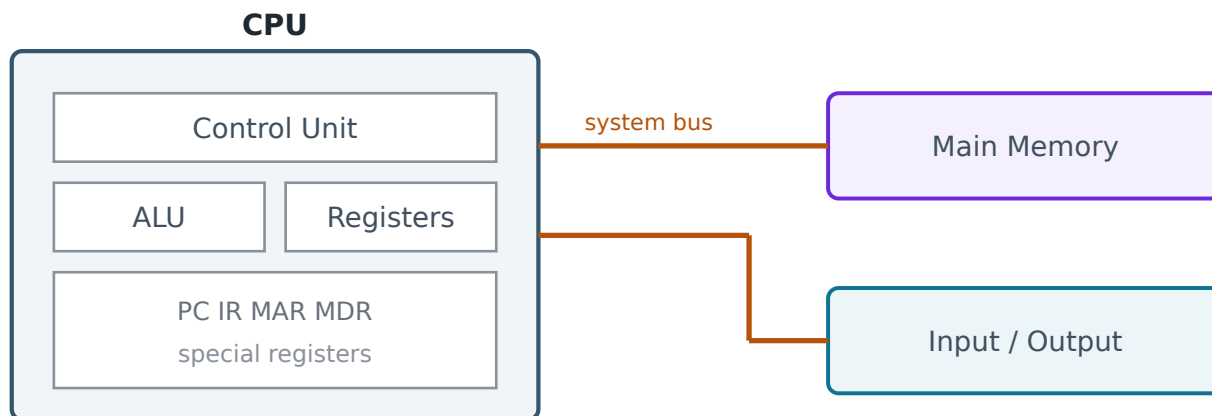
By the end of this unit you can name the components of a CPU and say what each does, describe the three system buses, and calculate address space, memory capacity and address width.

What is inside the processor

TEACH

THE CPU'S PARTS

- ▶ **Control Unit (CU):** the conductor. It fetches each instruction, decodes it, and sends the control signals that make every other part act at the right moment.
- ▶ **Arithmetic and Logic Unit (ALU):** the calculator. It performs arithmetic (add, subtract) and logic (AND, OR, NOT, compare, shift).
- ▶ **Registers:** the CPU's own tiny, fastest memory. General-purpose registers hold working values; special ones are the PC, IR, MAR and MDR met in Module One, and the **stack pointer (SP)**, which holds the address of the top of the stack.
- ▶ **Internal buses:** the wires that carry data between the registers, the ALU and the control unit.



The CPU, memory and I/O joined by the system bus.

TEACH

THE THREE SYSTEM BUSES

The **system bus** that joins CPU, memory and I/O is really three sets of wires:

- ▶ **Address bus:** carries the address the CPU wants to read or write. Its width sets how many locations can be addressed: **k address lines reach 2^k locations**. One direction only (CPU to memory).
- ▶ **Data bus:** carries the actual data, both directions. Its width sets how many bits move per transfer.
- ▶ **Control bus:** carries the signals that say read, write, or wait.

The third partner on the system bus is **input/output (I/O)**: the devices that move data between the computer and the outside world (keyboard, screen, disk, network). The CPU manages I/O in one of three ways:

- ▶ **Programmed I/O (polling)**: the CPU repeatedly checks whether a device is ready. Simple, but it wastes CPU time waiting.
- ▶ **Interrupt-driven I/O**: the device raises an **interrupt** when it is ready and the CPU stops to service it, so it need not poll.
- ▶ **Direct memory access (DMA)**: a DMA controller moves a whole block between a device and memory without the CPU, interrupting it only once the transfer is done. Fastest for large transfers.

● MUST MEMORISE

ADDRESSING CAPACITY

k address lines can address 2^k locations. Turn it around: to address N locations you need **$k = \log_2 N$** address lines. The total capacity is $2^k \times (\text{bytes per location})$.

WHAT EACH SYMBOL MEANS

k = the number of address lines, that is the width of the address bus in bits (here it means address lines; the same letter means pipeline *stages* in Module Two Unit 3 and *ways* in a set-associative cache) · **N** = how many separate locations you need to reach · **bytes per location** = 1 if the memory is byte-addressable, or the word size in bytes if it is word-addressable

Memory hook: every extra address line **doubles** the number of locations you can reach.

AS PRINTED

20/21 Q1(A)

THE EXAMINER'S WORDS, UNEDITED

- a. Minicomputers in the eighties had 18 address signals and 18-bit address bus. Answer the following questions: **3mks**
- i. What was the address space of these computers?
 - ii. What may have been the largest possible memory of these computers in bytes if the memory location is 1 Byte?
 - iii) How long should the program counter (PC) of these computers be?

Read it again. The paper states the same fact twice, "18 address signals and 18-bit address bus", and then numbers its three sub-items **i., ii., iii)**, switching from a full stop to a closing bracket at the third. It never states the word size as given data: the only mention of it is the condition buried inside (ii), "if the memory location is 1 Byte". Everything is in the past tense, "was", "may have been", which is flavour and changes nothing.

BREAK IT DOWN

BEFORE YOU COMPUTE

GIVEN	18 address signals, which is the same thing as an 18-bit address bus, and (from the wording of part ii only) one byte per memory location.
ASKED	three separate quantities. (i) the <i>address space</i> , which is a count of locations, not a size in bytes. (ii) the largest memory <i>in bytes</i> . (iii) "How long should the program counter (PC) of these computers be?", meaning its width in bits.
MARKS	3mks for all three sub-items together, so one mark each. That is one line of working per item, not an essay.
TAUGHT IN	this unit, Module Two Unit 1.
TRAP	(i) and (ii) look like the same question and are not. The address space is a count of locations; the memory size is that count times the bytes in a location. They agree here only because the paper quietly sets a location to 1 byte, inside part (ii).

WORKED EXAMPLE

PQ 20/21 Q1A

GIVEN	$k = 18$ address lines $\cdot$ 1 byte per location
FIND	(i) the address space, (ii) the maximum memory in bytes, (iii) the width of the PC
FORMULA	address space = $2^k \cdot$ capacity = $2^k \times$ bytes per location $\cdot$ PC width = k

WHAT EACH SYMBOL MEANS

k = the number of address lines, that is the address-bus width in bits $\cdot$ 2^k = how many distinct locations those lines can name $\cdot$ the **PC** must be able to hold any address, so it is exactly k bits wide

- 1 Address space = $2^{\text{number of lines}} = 2^{18} = 262\,144$ locations.
 $2^{18} = 2^{10} \times 2^8 = 1024 \times 256$.
- 2 At one byte per location that is $262\,144$ bytes = 256 KB.
- 3 The PC must hold any address, so it is as wide as the address bus: 18 bits.

(i) $2^{18} = 262\,144$ locations. (ii) $262\,144$ bytes = 256 KB. (iii) PC length = 18 bits.

Now redo it: address space is always 2 to the power of the number of address lines, and the PC is always as wide as an address.

AS PRINTED

20/21 Q5(A)(I)

THE EXAMINER'S WORDS, UNEDITED

- a. i) A processor can support a maximum memory of 4GB where the memory is word addressable (a word = 2 bytes). Determine the number of bits required to uniquely address the entire memory. **2mks**

Read it again. Only part (i) is worked here. The capacity is printed "4GB", closed up, and the whole question turns on two phrases: "word addressable" and the bracket that follows it, "(a word = 2 bytes)". Part (ii) of the same lettered part, not quoted here, is an unrelated end-around-carry question, and it reads "in an 10-bit register" where the 23/24 and 24/25 papers print "in a 10-bit register".

BREAK IT DOWN

BEFORE YOU COMPUTE

GIVEN	a maximum memory of 4GB, word addressable, with a word of 2 bytes.
ASKED	"the number of bits required to <i>uniquely</i> address the entire memory". The word uniquely is the instruction to count addressable units, and the addressable unit here is the word.
MARKS	2mks, so two steps: turn the capacity into a count of words, then take the base 2 logarithm.
TAUGHT IN	this unit, Module Two Unit 1.
TRAP	answering 32. That is the count of <i>bytes</i> . Word addressable means each address names a 2-byte word, so divide before you take the log.

WORKED EXAMPLE

PQ 20/21 Q5A

GIVEN	memory = 4 GB · word size = 2 bytes · addressing = word-addressable (each address names a word, not a byte)
FIND	k, the number of address bits needed
FORMULA	$N = \text{total bytes} / \text{bytes per addressable unit}$, then $k = \log_2 N$

WHAT EACH SYMBOL MEANS

N = the number of addressable units, here *words* not bytes · **k** = address bits · the phrase **word-addressable** is the whole question: divide the byte total by the word size *before* taking the log

- 1 4 GB = 4×2^{30} bytes = 2^{32} bytes.
- 2 Word-addressable means each address names a word, not a byte. Number of words = $2^{32} \text{ bytes} \div 2 \text{ bytes} = 2^{31}$ words.
- 3 To address 2^{31} words needs **31** bits (because 2^{31} words need 31 address lines).

4 GB $\div$ 2-byte word = 2^{31} words, so **31 address bits** are needed.

Now redo it: the trap is word-addressable versus byte-addressable. Divide the total bytes by the bytes per addressable unit first, then take the log base 2.

RECALL AND CHECK

A machine has 20 address lines and a 1-byte cell. How much memory can it address?

ANSWER

$2^{20} = 1\,048\,576$ locations = 1 MB. Every address line doubles the reach, and 2^{20} is exactly one mega.

◆ KEY POINT

Every component inside the processor exists to serve one repeating loop: fetch the next instruction, decode what it asks for, execute it. The program counter says where, the instruction register holds what, the control unit turns it into signals, and the arithmetic and logic unit does the work.

BLUNT VERSION: a CPU fetches, decodes and executes. Forever. It is the only thing it ever does.

MODULE TWO · UNIT 2

CPU Performance Metrics

How fast a processor runs is not a mystery, it is a calculation. This is the most heavily weighted numeric topic on the recent papers, so every formula here is worth learning to reflex.

By the end of this unit you can compute CPU execution time, average cycles per instruction from a mix, MIPS, the effect of an optimisation, and overall speedup by Amdahl's law. The favourite exam questions of this type are worked in full.

The one performance equation

A program takes time equal to how many instructions it runs, times how many clock cycles each takes on average, times how long one clock cycle lasts. That is the whole of it.

● MUST MEMORISE

THE PERFORMANCE FORMULAS

CPU time = **IC** × **CPI** × **T**, where **IC** is the instruction count, **CPI** is the average cycles per instruction, and **T** is the clock cycle time (seconds per cycle). Since $T = 1 / \text{clock rate}$:

- ▶ **CPU time** = **(IC × CPI) / clock rate**
- ▶ **Average CPI** = $\Sigma (\text{fraction of type } i \times \text{CPI of type } i)$
- ▶ **MIPS** = **clock rate** / **(CPI × 10⁶)** (millions of instructions per second)
- ▶ **Speedup** = **time(old) / time(new)**; **improvement** = **(old – new) / old**

WHAT EACH SYMBOL MEANS

IC = instruction count, how many instructions the program actually executes (not how many lines of source) · **CPI** = average clock cycles per instruction · **T** = clock cycle time, the seconds one cycle takes, and $T = 1 / \text{clock rate}$ · **clock rate** = cycles per second in Hz (a 2 GHz clock is 2×10^9 Hz) · **fraction of type i** = the share of the instruction mix that is of type i, so the fractions add to 1 · **CPI of type i** = cycles one instruction of that type costs

Memory hook: "I see PT" for **IC** × **CPI** × **T**. Fewer instructions, fewer cycles each, or a shorter cycle: any of the three cuts the time.

THE EXAMINER'S WORDS, UNEDITED

- d. A processor executes one million(1M) instructions distributed as follows.

Addressing Mode	Percentage	Cycles
Register	40%	1
Immediate	20%	1
Direct	15%	2
Indirect	10%	4
Indexed	10%	3
Relative	5%	2

- i) Compute the average CPI **2mks**
 ii) Determine total clock cycles. **1mk**
 iii) If indirect addressing is optimized to 2 cycles, compute the new CPI. **1mk**
 iv) Evaluate the performance improvement. **2mks**

Read it again. The paper writes "one million(1M)" with no space before the bracket, and it never uses the letters CPI in the stem: the third column of the table is headed simply "Cycles". Part (iii) says "optimized", and it changes one row of that table and nothing else.

BREAK IT DOWN

BEFORE YOU COMPUTE

GIVEN	one million instructions and a six-row table pairing each addressing mode with a share of the mix and a cycle cost. The shares total 100%, which is worth checking before you weight anything.
ASKED	four things in order: "Compute", "Determine", "compute the new CPI", "Evaluate". The last verb is the soft one, and it wants a number and a sentence saying what the number means.
MARKS	$2 + 1 + 1 + 2 = 6\text{mks}$. The evaluation in (iv) is priced the same as the average CPI in (i), so it is not a throwaway line.
TAUGHT IN	this unit, Module Two Unit 2.
TRAP	part (iii) changes the indirect row only. Recompute the whole weighted sum, or faster, subtract that row's old contribution and add its new one. Do not rescale the other five rows.

GIVEN	IC = 1 000 000 instructions · the mix and the cycle cost of each mode, tabulated below · for (iii), indirect falls from 4 cycles to 2
FIND	(i) average CPI, (ii) total cycles, (iii) the new average CPI, (iv) the percentage improvement
FORMULA	average CPI = $\sum (\text{fraction}_i \times \text{CPI}_i)$ · total cycles = IC × CPI · improvement = (old – new) / old

WHAT EACH SYMBOL MEANS

IC = instruction count · **fraction_i** = the share of the mix that is mode i, as a decimal, and the shares must total 1 · **CPI_i** = cycles one instruction of that mode costs · improvement is measured against the **old** CPI, not the new one

MODE	REGISTER	IMMEDIATE	DIRECT	INDIRECT	INDEXED	RELATIVE
SHARE	40%	20%	15%	10%	10%	5%
CYCLES	1	1	2	4	3	2

- 1 Average CPI = weighted sum: $0.40(1) + 0.20(1) + 0.15(2) + 0.10(4) + 0.10(3) + 0.05(2)$.
 $= 0.40 + 0.20 + 0.30 + 0.40 + 0.30 + 0.10$.
- 2 Average CPI = **1.70**.
- 3 Total clock cycles = IC × CPI = 1 000 000 × 1.70 = 1 700 000 cycles.
- 4 Optimise indirect from 4 to 2 cycles: its contribution falls from $0.10(4) = 0.40$ to $0.10(2) = 0.20$, a drop of 0.20. New CPI = $1.70 - 0.20 = \mathbf{1.50}$.
- 5 Improvement = $(1.70 - 1.50) / 1.70 = 0.20 / 1.70 = \mathbf{11.76\%}$ faster (equivalently a speedup of $1.70 / 1.50 = 1.133$ times).

(i) CPI = 1.70. (ii) 1 700 000 cycles. (iii) New CPI = 1.50. (iv) Improvement = 11.76% (speedup 1.13 times).

Now redo it: average CPI is just a weighted average. To measure an optimisation, recompute CPI with the one changed cost and compare. This is the cleanest marks on the paper.

When different instructions take different numbers of cycles

The same weighted-average idea handles a pipeline where some instruction types add penalty cycles. Start from an ideal CPI of 1 (one instruction finished per cycle) and add the average extra cycles each type contributes. A pipelined processor is normally split into five stages, and the papers refer to them by initials throughout: instruction fetch (IF), instruction decode (ID), execute (EX), memory access (MEM) and write-back (WB). Module Two Unit 3 takes the five apart properly; here you need only their times.

THE EXAMINER'S WORDS, UNEDITED

b. A computer manufacturer is evaluating whether to replace a non-pipelined processor with a five-stage pipelined processor. The processor executes the following instruction mix: Arithmetic – 40%, Memory Access – 35%, Branch – 20%, Floating Point – 5%. The Non-pipelined processor clock cycle is 25ns. Pipelined processor stage times are: IF – 5ns, ID – 4ns, EX – 6ns, MEM – 5ns and WB – 4ns. Furthermore, Pipeline register overhead is 1 ns. Branch instructions incur a two-cycle penalty, Floating-point instructions has three cycles for EX stage, Memory accesses experience cache misses 10% of the time, adding four extra cycles. The program contains 20 million instructions.

- i) Determine the pipelined clock cycle. **1mk**
- ii) Calculate the actual CPI. **2mks**
- iii) Determine the actual execution time. **1mk**
- iv) Compute the speedup over the non-pipelined processor. **1mk**

Read it again. The stem uses spaced en dashes to mean "equals" all the way through the instruction mix and the stage times, so a dash on this paper is not a minus sign. It prints "Floating-point instructions has three cycles", a plural subject with a singular verb. And it never says whether the branch penalty, the floating point cycles and the cache miss cycles are *extra* cycles or *total* cycles. That ambiguity is on the page and is not resolved anywhere, which is why the solution below states its reading before it computes anything. Part (v), not quoted here, asks why the speedup falls short of the theoretical maximum.

BREAK IT DOWN

BEFORE YOU COMPUTE

GIVEN	five stage times, a pipeline register overhead, a four-way instruction mix, a non-pipelined clock period and an instruction count. Every number in that paragraph is used and none of it is spare.
ASKED	"Determine", "Calculate", "Determine", "Compute". Four numbers, in an order that feeds forward: the clock period is needed for the execution time, and the execution time is needed for the speedup.
MARKS	1 + 2 + 1 + 1. The actual CPI carries 2mks because it is the only part with a judgement call in it, so show how you read each penalty.
TAUGHT IN	this unit, Module Two Unit 2, with the pipeline background in Module Two Unit 3.
TRAP	the 1 ns register overhead is added to the <i>slowest stage</i> , not to the sum of the stages. And the word "actual" in "actual CPI" is the signal that an ideal CPI of 1 is only where you start.

GIVEN	stages = 5 (IF, ID, EX, MEM, WB) · stage times = 5, 4, 6, 5, 4 ns · register overhead = 1 ns · IC = 20 000 000 · mix: arithmetic 40%, memory 35%, branch 20%, floating point 5% · non-pipelined clock = 25 ns
FIND	the pipelined clock period, the actual CPI, the execution time, and the speedup over the non-pipelined machine
FORMULA	clock = slowest stage + overhead · actual CPI = ideal CPI + $\sum (\text{fraction}_i \times \text{penalty}_i)$ · time = IC × CPI × T · speedup = time(non-pipelined) / time(pipelined)

WHAT EACH SYMBOL MEANS

slowest stage = 6 ns (EX), because a pipeline can go no faster than its slowest stage · **overhead** = the latch time added to every cycle · **ideal CPI** = 1, one instruction finishing per cycle once the pipe is full · **penalty_i** = the extra cycles that class of instruction costs · **T** = the clock period just computed

Reading the penalties (state your assumptions, the wording is dense): ideal CPI 1; branch adds 2 extra cycles (EX takes 3); a memory access adds a 2-cycle penalty plus, on a 10% cache miss, 4 more (so $2 + 0.10 \times 4 = 2.4$ average); floating point adds 4 extra; arithmetic adds none.

- 1 Pipelined clock = slowest stage + overhead = $6 + 1 = 7$ ns.
A pipeline runs at the speed of its slowest stage.
- 2 Actual CPI = $1 + 0.20(2) + 0.35(2.4) + 0.05(4) + 0.40(0) = 1 + 0.40 + 0.84 + 0.20 = 2.44$.
- 3 Execution time = IC × CPI × T = $20\,000\,000 \times 2.44 \times 7$ ns = **341.6 ms** (0.3416 s).
- 4 Non-pipelined time = $20\,000\,000 \times 25$ ns = 500 ms (0.5 s).
- 5 Speedup = $0.5 / 0.3416 = 1.46$ times.

Clock = 7 ns; CPI = 2.44; execution time = 0.3416 s; speedup ≈ 1.46 (under the stated reading of the penalties).

Now redo it: the method never changes even if you read a penalty differently: clock = slowest stage + overhead, CPI = 1 + weighted extra cycles, time = IC × CPI × T. State how you read each penalty and you keep the method marks.

The lecturer's deck compares two designs of the same machine. The instruction mix is **40% ALU, 20% branch, 40% memory**. **Design 1** is non-pipelined with a 10 ns clock, where ALU and branch instructions take 4 cycles and memory instructions take 5.

Design 2 pipelines it, so one instruction finishes per cycle, but clock skew and setup add 1 ns of overhead to the cycle. Find the speedup.

GIVEN	mix = 40% ALU, 20% branch, 40% memory · Design 1 clock = 10 ns, cycles = 4 (ALU), 4 (branch), 5 (memory) · Design 2 pipelined, 1 ns overhead
FIND	the speedup of Design 2 over Design 1
FORMULA	Design 1 time = clock × average CPI · average CPI = $\sum(\text{fraction}_i \times \text{cycles}_i)$ · Design 2 time = clock + overhead (one instruction per cycle) · speedup = Design 1 / Design 2

WHAT EACH SYMBOL MEANS

fraction_i = each type's share of the mix as a decimal, which must total 1 · **cycles_i** = cycles that type costs on the non-pipelined design · **average CPI** = the weighted average of those cycle costs · **overhead** = the 1 ns of latch and clock-skew time the pipeline adds to every cycle · at steady state the pipeline finishes one instruction per cycle, so its instruction time is just the cycle

- 1 Average CPI = $0.40(4) + 0.20(4) + 0.40(5) = 1.6 + 0.8 + 2.0 = 4.4$.
- 2 Design 1 time per instruction = clock × CPI = $10 \times 4.4 = 44$ ns.
- 3 Design 2 time per instruction = clock + overhead = $10 + 1 = 11$ ns.
- 4 Speedup = $44 / 11 = 4$.

Design 1 = 44 ns/instruction, Design 2 = 11 ns/instruction, speedup = 4. The pipeline turns an average of 4.4 cycles per instruction into one, at the cost of 1 ns of overhead. The pipeline mechanics themselves are in Module Two Unit 3.

Now redo it: the non-pipelined machine pays its full weighted CPI on every instruction; the pipelined one pays a single cycle plus overhead. The whole speedup is 44 over 11.

◆ KEY POINT

CPU time = $IC \times CPI \times T$, where IC is the instruction count, CPI is the average cycles per instruction and T is the seconds per clock cycle. Every claim that something made a processor faster must show which of those three it changed, and a change that improves one at the expense of another may be no improvement at all.

BLUNT VERSION: three numbers decide the whole thing. If an answer does not trace a speedup to one of those three, it is not an answer.

Amdahl's law: the limit on any speedup

If you speed up only part of a task, the untouched part limits the gain. Amdahl's law puts a number on it, and the exam asks it most years.

THE LAW, STATED

Amdahl's law. The gain in overall performance from improving one part of a system is limited by the fraction of the total time that part accounts for.

Formally: let f be the fraction of the execution time spent in the part being improved, where $0 \leq f \leq 1$, and let that part be made s times faster while the rest of the work is unchanged. Then the overall speedup S is

$$S = 1 / ((1 - f) + f / s)$$

In the limit, as s grows without bound, f/s tends to 0 and so S tends to $1 / (1 - f)$.

Consequence: however much faster the improved part becomes, the whole task can never be sped up by more than $1 / (1 - f)$. With $f = 0.85$ that ceiling is $1 / 0.15 = 6.67$, so even an infinitely fast cache could not make the machine more than 6.67 times faster.

WHAT EACH SYMBOL MEANS

f = the fraction of the total work that the improvement actually speeds up, written as a decimal between 0 and 1 (85% is $f = 0.85$) · s = how many times faster that fraction becomes · $(1 - f)$ = the untouched part, which still takes its original time · f / s = the sped-up part's new, shorter time

Memory hook: the part you did not speed up, $(1 - f)$, never goes away, so it caps the total gain no matter how large s is.

THE EXAMINER'S WORDS, UNEDITED

- c. i) State Amdahl's law' **2mks**
 ii) Assume in hypothetical system cache is 10 times faster than main memory and CPU accesses the cache 85% of the time. Using Amdahl's law, calculate the overall speed up of the system.

2mks

Read it again. "State Amdahl's law'" carries a stray closing apostrophe after the word law. It is not a scanning artefact: the 23/24 paper below prints the same stray mark, so it has been copied forward between years. The stem of (ii) is ungrammatical too, "Assume in hypothetical system", with the article missing, and it writes "speed up" as two words. The 2mks for (ii) is printed on the line below the sentence it belongs to.

THE EXAMINER'S WORDS, UNEDITED

- b. i) State Amdahl's law' **2½mks**
 ii) Assume in hypothetical system cache is 10 times faster than main memory and CPU accesses the cache 85% of the time. Using Amdahl's law, calculate the overall speed up of the system. **2mks**

And the same question two years earlier. Word for word identical in (ii), with two differences that change how you plan the answer: it is part **b** of Question One rather than part **c** of Question Three, and stating the law is worth **2½mks** here against 2mks in 25/26. Two and a half marks for a statement means the one-line definition is not enough: give the formula and say what it limits.

BREAK IT DOWN

BEFORE YOU COMPUTE

GIVEN	a cache 10 times faster than main memory, and a CPU that finds what it needs in the cache 85% of the time.
ASKED	two things. First "State" the law, which is a statement in words plus the formula, not the formula alone. Then "calculate the overall speed up of the system".
MARKS	2mks (2½mks in 23/24) for the statement and 2mks for the number, so the words are worth as much as the arithmetic. Write the law out.
TAUGHT IN	this unit, the Must memorise box directly above.
TRAP	reading "10 times faster" as the answer. That is s, and it applies only to the fraction $f = 0.85$. The 15% still going to main memory at full cost is what holds the whole machine down to about 4.26.

WORKED EXAMPLE

PQ 25/26 Q3C · PQ 23/24 Q1B

GIVEN	$f = 0.85$ (the 85% of accesses served by cache) · $s = 10$ (cache is ten times faster than main memory)
FIND	the overall speedup of the machine
FORMULA	$\text{speedup} = 1 / ((1 - f) + f/s)$

WHAT EACH SYMBOL MEANS

f = the fraction of the work the improvement touches, here the cache hits · **s** = how many times faster that fraction runs · **(1 - f)** = the 15% still going to main memory at full cost, which is what caps the answer well below 10

- The sped-up fraction is $f = 0.85$ (the cache accesses), and the speedup factor is $s = 10$.
- $\text{Speedup} = 1 / ((1 - 0.85) + 0.85 / 10) = 1 / (0.15 + 0.085)$.
- $= 1 / 0.235 = \mathbf{4.26}$.

$\text{Speedup} = 1 / (0.15 + 0.085) = 1 / 0.235 \approx \mathbf{4.26 \text{ times}}$.

Now redo it: notice the answer is far below 10, because the 15% of slow accesses dominate. That is exactly the point Amdahl's law makes.

⚠ TRAP

Two slips. (1) Improvement percentage is measured against the **old** time, $(\text{old} - \text{new}) / \text{old}$, not against the new one. (2) In Amdahl's law, **f** is the fraction of work that is sped up, and you divide only that part by **s**; the rest, $(1 - f)$, stays at full cost.

RECALL AND CHECK

A program runs 5 million instructions at an average CPI of 2 on a 1 GHz clock. How long does it take?

ANSWER

CPU time = IC $\times$ CPI / clock rate = $(5\,000\,000 \times 2) / 10^9 = 10\,000\,000 / 10^9 = 0.01\text{ s} = 10\text{ ms}$.

FILL IN THE GAP

1. CPU time = instruction count $\times$ _____ $\times$ clock cycle time.
2. Average CPI is the _____ average of each type's cycle cost.
3. A pipeline's clock cycle equals its _____ stage time plus the register overhead.
4. In Amdahl's law the term $(1 - f)$ represents the part that is _____.

ANSWERS

1. CPI. 2. weighted. 3. slowest (longest). 4. not sped up.

MODULE TWO • UNIT 3

Pipelining

Overlapping instructions so several are in progress at once, the way a laundry overlaps washing, drying and folding. It speeds a processor up, and it comes with hazards you must handle. Both the speedup sums and the hazard diagrams are exam favourites.

By the end of this unit you can compute a pipeline's cycle time, its time for n tasks, its speedup and throughput; identify structural, data and control hazards; and draw a timing diagram showing stalls and forwarding.

The idea: overlap the stages

Split each instruction into k stages. Without a pipeline, one instruction finishes before the next begins. With a pipeline, as soon as an instruction leaves stage 1 the next enters it, so after the pipeline fills, **one instruction completes every clock cycle**. The clock cycle is set by the slowest stage plus the small latch delay between stages.

The deck states it formally, and the wording is worth having, because a definition question wants this exact shape: **pipelining is a technique of decomposing a sequential process into sub-operations, with each sub-process executed in a special dedicated segment that operates concurrently with all other segments.**

What a pipeline exploits is the parallelism *among instructions*, found by overlapping them, and that has a name: **Instruction Level Parallelism (ILP)**. It is one level among several. The deck lists where a typical system finds its speedup:

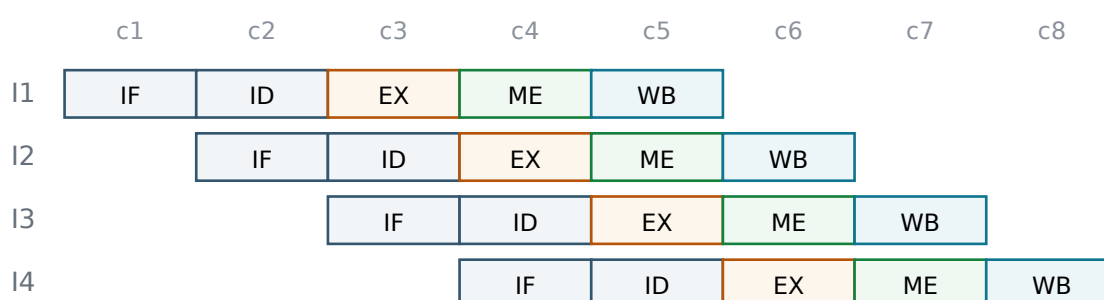
- ▶ **Multi-user:** one machine serving many people at once.
- ▶ **Multitasking:** one user running several programs at once.
- ▶ **Multi-processing:** several processors working at once.
- ▶ **Multi-programming:** several programs resident in memory together, the CPU switching between them.
- ▶ **Multi-threading:** several threads of one program running at once.
- ▶ **Compiler optimizations:** the compiler rearranging code so the hardware can overlap more of it.

Pipelining is the hardware end of that list, and Instruction Level Parallelism is what it buys.

◆ KEY POINT

Pipelining does not shorten one instruction. It raises throughput, the number of instructions completed per unit time, while the latency of any single instruction stays the same, or slightly worsens because of the latches between stages.

BLUNT VERSION: no single load of washing dries any faster. You just get more loads through the evening.



Four instructions overlapped in a 5-stage pipeline. After it fills, one finishes each cycle.

- ▶ **Cycle time** = slowest stage time + latch delay.
- ▶ **Time for n tasks** = $(k + n - 1) \times \text{cycle time}$, for k stages.
- ▶ **Non-pipelined time** = $n \times (\text{sum of all stage times})$.
- ▶ **Speedup** = non-pipelined time / pipelined time. Its ceiling is k.
- ▶ **Throughput** = $n / \text{pipelined time}$ (tasks per second); in the steady state this is $1 / \text{cycle time}$.

WHAT EACH SYMBOL MEANS

k = the number of pipeline *stages* (here k means stages, not the address lines of Module Two Unit 1 nor the ways of a set-associative cache) · **n** = the number of tasks or instructions pushed through the pipeline (here n means tasks, not the bit-width n of Module One) · **cycle time** = the clock period the pipeline runs at, set by the slowest stage plus the latch delay · **latch delay** = the small extra time to write each stage's result into the register between stages

Memory hook: "**k + n - 1**" is k cycles to fill the pipe with the first task, then one more cycle for each of the remaining $n - 1$ tasks.

AS PRINTED

25/26 Q5(B)

THE EXAMINER'S WORDS, UNEDITED

- b. Consider a pipeline having 4 phases with duration 60, 50, 90 and 80 ns. Given latch delay is 10 ns. Calculate-
- i. Pipeline cycle time **1mk**
 - ii. Non-pipeline execution time **1mk**
 - iii. Speed up ratio **1mk**
 - iv. Pipeline time for 500 tasks **1mk**
 - v. Sequential time for 500 tasks **1mk**

Read it again. The instruction ends "Calculate-" with a hyphen where a colon belongs, and the list underneath is the whole of the question. On the physical sheet the five 1mk annotations are set one line lower than the items they price, so count the items rather than trusting the eye.

AS PRINTED

24/25 Q5(B)

THE EXAMINER'S WORDS, UNEDITED

- b. Consider a pipeline having 4 phases with duration 60, 50, 90 and 80 ns. Given latch delay is 10 ns. Calculate-
- i. Pipeline cycle time **1mk**
 - ii. Non-pipeline execution time **1mk**
 - iii. Speed up ratio **1mk**
 - iv. Pipeline time for 500 tasks **1mk**
 - v. Sequential time for 500 tasks **1mk**
 - vi. Throughput **1mk**

And the year before. The same question, the same durations, the same "Calculate-", with one difference: 24/25 carries a sixth item, **vi. Throughput**, which the 25/26 paper drops. Answer throughput anyway. It is one line, it is on the older paper, and it is the number that says what the pipeline is actually delivering.

BREAK IT DOWN

BEFORE YOU COMPUTE

GIVEN	four phase durations and a latch delay. Nothing says the phases are equal, and they are not, which is the point of the question.
ASKED	six short quantities in 24/25 and five in 25/26, each priced at 1mk. "Speed up ratio" with no task count attached means the ideal ratio, the one for a full pipe.
MARKS	1mk each, six marks or five. Each answer is one line of arithmetic and a unit, so there is nothing to gain by writing prose.
TAUGHT IN	this unit, Module Two Unit 3.
TRAP	the cycle time is the <i>slowest</i> phase plus the latch delay, $90 + 10$. It is not the average and it is not the sum. And "Non-pipeline execution time" with no task count means one task, 280 ns, while item (v) asks for 500 tasks separately.

WORKED EXAMPLE

PQ 25/26 Q5B · PQ 24/25 Q5B

GIVEN	$k = 4$ stages · stage times = 60, 50, 90, 80 ns · latch delay = 10 ns · $n = 500$ tasks for parts (iv) and (v)
FIND	cycle time, non-pipelined time for one task, speedup, pipelined and sequential time for 500 tasks, and throughput
FORMULA	cycle = slowest stage + latch · pipelined = $(k + n - 1) \times$ cycle · sequential = $n \times \Sigma(\text{stage times})$ · speedup = sequential / pipelined · throughput = $n /$ pipelined time

WHAT EACH SYMBOL MEANS

k = 4, the number of *stages* (not address lines, not cache ways) · **n** = 500, the number of *tasks* (not a bit width) · **slowest stage** = 90 ns, which alone sets the clock · **latch delay** = the 10 ns added to every cycle to store each stage's result

- 1 Cycle time = slowest phase + latch = $90 + 10 = 100$ ns.
- 2 Non-pipeline time for one task = $60 + 50 + 90 + 80 = 280$ ns.
- 3 Speedup ratio (ideal) = $280 / 100 = 2.8$.
- 4 Pipeline time for 500 tasks = $(k + n - 1) \times \text{cycle} = (4 + 499) \times 100 = 503 \times 100 = 50\,300$ ns.
- 5 Sequential time for 500 tasks = $500 \times 280 = 140\,000$ ns.
- 6 Throughput = $500 / 50\,300 \text{ ns} \approx 9.94$ million tasks per second (in the steady state, $1 / 100 \text{ ns} = 10$ million per second). Actual speedup for 500 tasks = $140\,000 / 50\,300 = 2.78$.

Cycle 100 ns; non-pipe 280 ns; ideal speedup 2.8; pipe(500) = 50 300 ns; seq(500) = 140 000 ns; throughput ≈ 9.94 M tasks/s.

Now redo it: cycle = slowest + latch, pipe = $(k + n - 1) \times$ cycle, and everything else falls out of those two.

Seen again: this same 60, 50, 90, 80 ns pipeline is on the lecturer's revision deck too, worked once more in the Revision Class section near the back.

THE EXAMINER'S WORDS, UNEDITED

- b. i) What is pipelining? **2mks**
 ii) Calculate how long it will take to execute on a regular non-pipelined and pipelined architecture. There are 4 operations with 5 stages each: IF, ID, EX, M and WB; stage1 = 8ns, stage2 = 9ns, stage3 = 11ns, stage4 = 13ns, stage5 = 16ns. Provide the Gantt Charts for both pipeline and non-pipeline operations. Also calculate the Speedup of the Pipeline system **6mks**

Read it again. The paper's spacing around its own equals signs is inconsistent, "stage1 = 8ns", "stage2 = 9ns", "stage5 = 16ns", and it names the fourth stage **M** rather than MEM. It asks for "the Gantt Charts for both pipeline and non-pipeline operations", so a chart is part of the answer and not an optional extra. The sentence never ends: it runs straight into its 6mks with no full stop.

BREAK IT DOWN

BEFORE YOU COMPUTE

GIVEN	four operations, five stages, five stage times, and no latch delay mentioned at all, so take the latch as zero and say that you have.
ASKED	three deliverables packed into one sentence: the two execution times, the two charts, and the speedup. A student who computes the numbers and draws nothing has left marks that were never about arithmetic.
MARKS	2mks for the definition in (i) and 6mks for (ii), which is the largest single part of that question.
TAUGHT IN	this unit, Module Two Unit 3. Part (i), the definition of pipelining, is the opening paragraph of this unit.
TRAP	the non-pipelined time is n times the <i>sum</i> of the stages, 4 times 57. The pipelined time uses only the <i>slowest</i> stage, 16 ns, as its clock. Mixing the two is the usual loss here.

GIVEN	$k = 5$ stages · stage times = 8, 9, 11, 13, 16 ns · $n = 4$ operations · no latch delay stated, so take it as 0
FIND	the non-pipelined time, the pipelined time, and the speedup
FORMULA	cycle = slowest stage · non-pipelined = $n \times \Sigma(\text{stage times})$ · pipelined = $(k + n - 1) \times \text{cycle}$ · speedup = non-pipelined / pipelined

WHAT EACH SYMBOL MEANS

k = 5 stages · **n** = 4 operations · **$\Sigma(\text{stage times})$** = $8+9+11+13+16 = 57$ ns, the full journey one operation makes without a pipeline · **cycle** = 16 ns, the slowest stage, because every stage must be given that long

- 1 Non-pipelined: each operation passes through all stages, $8+9+11+13+16 = 57$ ns each, so $4 \times 57 = \mathbf{228 \text{ ns}}$.
- 2 Pipelined cycle time = slowest stage = 16 ns.
- 3 Pipelined time = $(k + n - 1) \times \text{cycle} = (5 + 4 - 1) \times 16 = 8 \times 16 = \mathbf{128 \text{ ns}}$.
- 4 Speedup = $228 / 128 = \mathbf{1.78}$.

Non-pipelined 228 ns, pipelined 128 ns, speedup ≈ 1.78 . The speedup is below 5 because only 4 tasks run, so the fill time still dominates.

Efficiency, throughput, and the lecturer's speedup setups

The deck the lecturer released frames the same arithmetic three more ways, and adds two metrics the papers can ask for by name: **efficiency** and **throughput**. The intuition is the laundry from the top of this unit: four loads that take **8 hours** done one after another finish in **3.5 hours** overlapped, a speedup of $8 / 3.5 = \mathbf{2.29}$ (the slide rounds this to about 2.1). The deck sets it out in full, and the numbers are worth having because they are the lecturer's own: **Ann, Brian, Cathy and Dave** each have one load to **wash, dry, stash and fold**, and the washer, the dryer, the folder and the stasher each take **30 minutes**. So one load is $4 \times 30 = \mathbf{120 \text{ minutes}}$, four loads run one after another are $4 \times 120 = \mathbf{480 \text{ minutes}}$, which is the 8 hours, and overlapped they take $(4 + 4 - 1) \times 30 = \mathbf{210 \text{ minutes}}$, which is the 3.5 hours. The speedup is short of the ideal 4 because, with only four loads, the time to fill and drain the pipe still counts. Push many loads through and it climbs toward 4.

- ▶ **Efficiency** $E = n / (k + n - 1)$: how fully the stages are kept busy, the ideal speedup k scaled down by the fill and drain. As n grows without bound, E tends to **1**.
- ▶ **Throughput** $= E / \text{cycle time} = n / [(k + n - 1) \times \text{cycle time}]$: results finished per second. In the steady state, once the pipe is full, that is simply **1 / cycle time**.
- ▶ **Speedup with stalls** $= k / (1 + \text{stalls per instruction})$: every stall cycle added per instruction eats into the ideal k -fold speedup.

WHAT EACH SYMBOL MEANS

k = the number of pipeline stages · **n** = the number of tasks pushed through · **cycle time** = the clock period, the slowest stage plus any latch delay · **k + n - 1** = the total cycles for n tasks, fill included · **stalls per instruction** = the average number of bubble cycles each instruction adds because of hazards

Memory hook: efficiency is the fraction of the ideal you actually get, so it is always the real speedup divided by k , and it only reaches 1 when the fill time becomes negligible against a very long run.

WORKED EXAMPLE

SLIDE: EQUAL-STAGE SPEEDUP

A 4-stage pipeline, each stage 20 ns, running 100 tasks. Find the non-pipelined and pipelined times, the speedup, the efficiency and the throughput.

GIVEN $k = 4$ stages · cycle time = 20 ns (all stages equal) · $n = 100$ tasks
 FIND non-pipelined time, pipelined time, speedup, efficiency, throughput
 FORMULA one task = $k \times \text{stage}$ · non-pipelined = $n \times k \times \text{stage}$ · pipelined = $(k + n - 1) \times \text{cycle}$ · speedup = non-pipelined / pipelined · $E = n / (k + n - 1)$

WHAT EACH SYMBOL MEANS

k = 4 stages · **stage** = 20 ns, and because the stages are equal the cycle time is also 20 ns · **n** = 100 tasks · **k + n - 1** = 103, the cycles for 100 tasks with the fill counted

- 1 One task, non-pipelined = $k \times \text{stage} = 4 \times 20 = \mathbf{80 \text{ ns}}$.
- 2 All 100, non-pipelined = $100 \times 80 = \mathbf{8000 \text{ ns}}$.
- 3 Pipelined = $(k + n - 1) \times \text{cycle} = (4 + 99) \times 20 = 103 \times 20 = \mathbf{2060 \text{ ns}}$.
- 4 Speedup = $8000 / 2060 = \mathbf{3.88}$.
- 5 Efficiency = $n / (k + n - 1) = 100 / 103 = \mathbf{0.97}$, so the pipe runs at 97% of its ideal fourfold speedup.
- 6 Throughput = $100 / 2060 \text{ ns} \approx \mathbf{48.5 \text{ million tasks per second}}$ (steady state 1 / 20 ns = 50 million per second).

Non-pipelined 8000 ns; pipelined 2060 ns; speedup 3.88; efficiency 0.97; throughput $\approx 48.5 \text{ M tasks/s}$.

Now redo it: equal stages make the cycle time the stage time, so the only sums are $n \times k \times \text{stage}$ and $(k + n - 1) \times \text{stage}$. Efficiency is just the speedup over k .

A pipeline's stages take 10, 8, 10, 10 and 7 ns, with a 1 ns overhead per stage. Find the speedup of the data path.

GIVEN	stage times = 10, 8, 10, 10, 7 ns · overhead = 1 ns per stage · steady-state comparison (many instructions)
FIND	the pipelined instruction time, the non-pipelined instruction time, and the speedup
FORMULA	pipelined cycle = slowest stage + overhead · non-pipelined = $\Sigma(\text{stage times})$ · speedup = non-pipelined / pipelined

WHAT EACH SYMBOL MEANS

slowest stage = 10 ns, because every stage is given the longest stage's time · **overhead** = 1 ns of latch and clock-skew time added to every cycle · **$\Sigma(\text{stage times})$** = $10 + 8 + 10 + 10 + 7 = 45$ ns, one instruction's whole journey without a pipeline

- 1 Pipelined cycle = $\max(10, 8, 10, 10, 7) + \text{overhead} = 10 + 1 = \mathbf{11 \text{ ns}}$. At steady state one instruction finishes every cycle, so this is its average time.
- 2 Non-pipelined = $10 + 8 + 10 + 10 + 7 = \mathbf{45 \text{ ns}}$ per instruction.
- 3 Speedup = $45 / 11 = \mathbf{4.09}$ (the slide rounds to 4.1).

Pipelined 11 ns/instruction, non-pipelined 45 ns/instruction, speedup ≈ 4.09 . The unequal stages waste time: the slowest stage sets a cycle every other stage must wait out.

Now redo it: the pipeline runs at the slowest stage plus overhead, so its instruction time is 11 ns however fast the other stages are. That waste is the reason designers balance stages.

Hazards: when overlap goes wrong

Overlapping instructions can clash. There are three kinds of hazard, and the exam expects you to name them and say how each is handled.

- ▶ **Structural hazard:** two instructions need the same hardware at once (for example one memory for both a fetch and a data access). Fixed by duplicating the resource, such as separate instruction and data caches.
- ▶ **Data hazard:** an instruction needs a result the previous one has not written back yet (a read-after-write, RAW). Reduced by **forwarding** (feeding the result straight from where it is produced to where it is needed) and, when that is not enough, by **stalling** one or more cycles.
- ▶ **Control hazard:** a branch changes what runs next, but the pipeline has already fetched the following instructions. Handled by resolving the branch early, predicting it, or flushing the wrongly fetched instructions.

The one data hazard forwarding cannot fully hide is **load-use**: a value loaded from memory is not ready until the memory stage, so an instruction that uses it the very next cycle must wait one cycle even with forwarding. That single unavoidable stall appears in the exam's RISC (reduced instruction set computer) pipeline question.

The lecturer's slides sort data hazards into three named cases, by the *order* in which two instructions touch the same register. This is the vocabulary the three drills below and the papers use:

- ▶ **RAW, read after write:** a later instruction reads a register an earlier one has not finished writing. The true dependency, and the common one, handled by forwarding.
- ▶ **WAR, write after read:** a later instruction writes a register an earlier one has still to read. A name dependency, harmless in an in-order pipeline because reads happen in decode, before writes in write-back.
- ▶ **WAW, write after write:** two instructions write the same register, so their order must be preserved. Also a name dependency.

To classify a pair, ignore the operation and look only for the **one register that appears on both lines**, then read off the order it is written and read.

THE LECTURER'S OWN QUESTION, LEFT UNANSWERED ON THE SLIDE

Describe each of the following categories of Data Hazards: RAW, WAR, WAW. Using (i) to (iii) below state which is RAW, WAR or WAW and indicate how each occurs using an arrow.

- (i) $R3 \leftarrow R1 \text{ op } R2$ (ii) $R3 \leftarrow R1 \text{ op } R2$ (iii) $R3 \leftarrow R1 \text{ op } R2$
 $R5 \leftarrow R3 \text{ op } R4$ $R1 \leftarrow R4 \text{ op } R5$ $R3 \leftarrow R6 \text{ op } R7$

The slide leaves this to you, so here it is worked. Ignore "op" and the right-hand source registers; the hazard is decided by the single register on both lines and the order it is written then read. The arrow runs from the earlier access to the later one.

CASE	REGISTER ON BOTH LINES	ORDER (ARROW)	HAZARD
(i) $R3 \leftarrow R1 \text{ op } R2 / R5 \leftarrow R3 \text{ op } R4$	R3	written in (1), then read in (2)	RAW
(ii) $R3 \leftarrow R1 \text{ op } R2 / R1 \leftarrow R4 \text{ op } R5$	R1	read in (1), then written in (2)	WAR
(iii) $R3 \leftarrow R1 \text{ op } R2 / R3 \leftarrow R6 \text{ op } R7$	R3	written in (1), then written in (2)	WAW

(i) RAW (read after write), (ii) WAR (write after read), (iii) WAW (write after write). Describe each in one line as above, and draw the arrow from the first access to the second.

Seen again: the lecturer set this exact question a second time in his revision class, so it is worked once more in the Revision Class section near the back. When a question is asked twice, expect it in the hall.

THE LECTURER'S OWN WORKED EXAMPLE, WITH THE SLIDE'S ANSWER

Identify hazards in the following code:

I1: ADD R1, R2, R3
 I2: SUB R4, R1, R5
 I3: MUL R1, R6, R7
 I4: DIV R8, R1, R9

The slide's answer: I1 → I2 is **RAW** (I2 reads R1 after I1 writes it), I1 → I3 is **WAW** (both write R1), and I3 → I4 is **RAW** (I4 reads R1 after I3 writes it). Every clash here is on **R1**, which is why writing down what each instruction reads and writes finds them all.

I1 → I2 : RAW · I1 → I3 : WAW · I3 → I4 : RAW. One line more than the slide gives: I2 → I3 is also a **WAR** (I2 reads R1, I3 writes it), which an in-order pipeline ignores because it reads in decode before it writes in write-back. Name it and say so.

THE LECTURER'S OWN WORKED EXAMPLE, WITH THE SLIDE'S ANSWER

Given the instruction sequence:

I1: LOAD R1, 0(R2)

I2: ADD R3, R1, R4

I3: SUB R5, R6, R7

I4: MUL R8, R3, R9

Rearrange instructions to minimize stalls.

Why it stalls, and the fix. I2 needs R1 the cycle after I1 loads it, which is the one load-use stall forwarding cannot remove. I3 (SUB) depends on nothing above it, so slot it into the gap: the load is still in flight while SUB runs, and by the time I2 needs R1 it is ready.

Reordered: **LOAD, SUB, ADD, MUL** (I1, I3, I2, I4). The independent SUB fills the load-delay slot, so the load-use stall disappears and MUL still gets its R3 from the ADD just before it.

THE EXAMINER'S WORDS, UNEDITED

b. Consider the following code segment executed on a classic five-stage RISC processor.

I1: LW R1,0(R2)

I2: ADD R3,R1,R4

I3: SUB R5,R3,R6

I4: SW R5,8(R2)

I5: BEQ R5,R0,LABEL

I6: MUL R7,R5,R8

Assume:

- forwarding is available
- branch resolved in EX stage
- load-use hazards require one stall
- multiplier occupies EX for three cycles
- separate instruction and data caches

i) Identify and explain every pipeline hazard. **5mks**

ii) Draw the pipeline timing diagram **2mks**

iii) Determine the total execution cycles **1mk**

iv) Suggest any two hardware improvements that would reduce stalls. **1mk**

Read it again. The five lines under "Assume:" are the question. Three of them decide the cycle count (forwarding, the one-cycle load-use stall, the three-cycle multiplier) and the other two are there to tell you which hazards are already handled (branch resolved in EX, separate caches). The operand lists are printed closed up, "R1,0(R2)", with no space after the comma, and items (ii) and (iii) end without a full stop while (i) and (iv) have one.

GIVEN	six instructions of a classic five-stage RISC (reduced instruction set computer) pipeline, and five stated assumptions. The dependencies are not listed anywhere: you read them off the register names yourself.
ASKED	"Identify and explain", "Draw", "Determine", "Suggest". Four different verbs and only one of them is arithmetic.
MARKS	$5 + 2 + 1 + 1 = 9$ mks. Identifying and explaining the hazards carries more than half of it, and the total cycle count, the part that looks hardest, is worth 1mk.
TAUGHT IN	this unit, Module Two Unit 3.
TRAP	"every pipeline hazard" means every one, including the ones forwarding hides. Name them and say they are covered, rather than leaving them out because they cost no cycles.

GIVEN $k = 5$ stages (IF, ID, EX, MEM, WB) · $n = 6$ instructions · forwarding available · load-use costs 1 stall · MUL holds EX for 3 cycles

FIND the hazards present, the timing diagram, and the total cycle count

FORMULA ideal = $(k + n - 1)$ cycles, then **add** every stall: 1 per load-use, and $(\text{EX cycles} - 1)$ for each multi-cycle execute

WHAT EACH SYMBOL MEANS

k = 5 stages · **n** = 6 instructions · **forwarding** = a result is passed straight from one stage to the next instruction, removing most data stalls · **load-use** = the one hazard forwarding cannot remove, because the value is still being fetched · a **3-cycle EX** adds $3 - 1 = 2$ extra cycles

```

I1: LW   R1, 0(R2)    ; load R1 from memory
I2: ADD  R3, R1, R4    ; needs R1 -> load-use hazard with I1
I3: SUB  R5, R3, R6    ; needs R3 (forwarded from I2)
I4: SW   R5, 8(R2)    ; needs R5 (forwarded from I3)
I5: BEQ  R5, R0, LABEL ; branch, resolved in EX
I6: MUL  R7, R5, R8    ; multiply, EX takes 3 cycles

```

Hazards

I1 to I2 is a **load-use data hazard**: R1 is not ready until I1's MEM stage, so I2 stalls one cycle. I2 to I3, I3 to I4, I3 to I5 and I3 to I6 are ordinary data hazards **hidden by forwarding**. I6's **multiply holds EX for three cycles**, adding two extra cycles at the end (assuming the branch is not taken).

Timing diagram (each cell is the stage that instruction is in)

CYCLE	1	2	3	4	5	6	7	8	9	10	11	12	13
I1 LW	IF	ID	EX	ME	WB								
I2 ADD		IF	ID	--	EX	ME	WB						
I3 SUB			IF	--	ID	EX	ME	WB					
I4 SW					IF	ID	EX	ME	WB				
I5 BEQ						IF	ID	EX	ME	WB			
I6 MUL							IF	ID	EX	EX	EX	ME	WB

One load-use stall (the -- in cycle 4) plus two extra EX cycles for the multiply give a total of **13 cycles** (a stall-free run of 6 instructions would take $5 + 6 - 1 = 10$).

Two hardware improvements that reduce stalls

Add **forwarding paths** (already assumed here) and a **pipelined or duplicated multiplier** so a long multiply does not block EX; separate instruction and data memories remove structural stalls.

Now redo it: mark the one load-use stall, let forwarding cover the rest, and remember a 3-cycle EX adds two cycles. Count the last WB: that cycle number is the total.

Multi-cycle execute, with and without forwarding

When some instructions hold the EX stage for several cycles, they stall the ones behind them, and a dependent instruction must wait for its operand. The next question asks you to schedule exactly this, both with forwarding and without.

AS PRINTED

25/26 Q3(B)

THE EXAMINER'S WORDS, UNEDITED

b. A 5-stage pipelined processor has instruction fetch (IF), Instruction decode (ID), Operand Fetch (OF), Execute (Ex) and Write Back (WB) stages. The IF, ID, OF and WB stages take 1 clock cycle each for any instruction. The PO stage take 1 cycle for ADD and SUB instructions, 3 clock cycles for MUL instruction and 6 clock cycles for DIV instruction respectively. Operand forwarding is used in the pipeline.

Draw the cycle diagram and determine the number of cycles needed to execute the following sequence of instructions:

- i. When Operand forwarding is used **4½mks**
- ii. When Operand forwarding is not used **5½mks**

```
I1  MUL R2, R0, R1    ; R2 ← R0 X R1
I2  DIV R5, R3, R4    ; R5 ← R3/R4
I3  ADD R2, R5, R2    ; R2 ← R5 + R2
I4  SUB R5, R2, R6    ; R5 ← R2 - R6
```

Read it again. The sentence names its five stages as IF, ID, OF, **Ex** and WB, and then says "The **PO** stage take 1 cycle for ADD and SUB". PO is not one of the five names it has just given. On the physical sheet the printed word PO is struck through in pen with **EX** written beneath it, and the B of WB earlier in the same sentence is overwritten in pen. The 23/24 paper set this question and printed "The EX stage takes 1 cycle", so PO is a corruption introduced for this sitting. Note also "take" for takes, "Ex" for EX, and a comment column that uses **X** for multiply, a slash for divide and an en dash for subtract across four consecutive lines.

BREAK IT DOWN

BEFORE YOU COMPUTE

GIVEN	five stages at one clock cycle each except the execute stage, whose length depends on the opcode: 1 cycle for ADD and SUB, 3 for MUL, 6 for DIV. Four instructions whose dependencies you must read off the register names.
ASKED	"Draw the cycle diagram and determine the number of cycles", then the same thing twice over, once with operand forwarding and once without. Two diagrams and two totals, not one.
MARKS	4½mks with forwarding and 5½mks without, 10 marks in all, the largest part of that question. Most of it is the diagram.
TAUGHT IN	this unit, Module Two Unit 3.
TRAP	the stem ends "Operand forwarding is used in the pipeline" and then part (ii) asks for the case where it is not used. The question contradicts its own sub-item. Answer both cases and label each diagram with the assumption it is drawn under. Note too that the $k + n - 1$ shortcut does not apply once stages take unequal time.

GIVEN	$k = 5$ stages (IF, ID, OF, EX, WB) · $n = 4$ instructions · 1 cycle per stage, except EX = 1 (ADD, SUB), 3 (MUL), 6 (DIV) · dependencies: I3 needs R5 from I2 and R2 from I1; I4 needs R2 from I3
FIND	the schedule and the total cycles, both with and without operand forwarding
FORMULA	no single formula fits a multi-cycle EX: build the cycle diagram row by row under the stated assumptions, then read the total off the last write-back

WHAT EACH SYMBOL MEANS

k = 5 stages · **n** = 4 instructions · **OF** = operand fetch · a **multi-cycle EX** means the stage is occupied for that many cycles, so the instruction behind it cannot enter EX · **forwarding** lets a result reach the next instruction's EX without waiting for write-back · the $(k + n - 1)$ shortcut does **not** apply once stages take unequal time

I1: MUL R2, R0, R1 I2: DIV R5, R3, R4
I3: ADD R2, R5, R2 I4: SUB R5, R2, R6

I3 needs R5 from I2 and R2 from I1; I4 needs R2 from I3.

Stated assumptions

One execution unit, in order: an instruction enters EX only when EX is free, so a long EX stalls those behind it (a stalled instruction repeats its stage below). Write-back is one cycle after EX. **With forwarding**, a result reaches a dependent instruction's EX the cycle after it is produced; **without forwarding**, the dependent must wait until the producer has written back.

(i) With operand forwarding

CYCLE	I1 MUL	I2 DIV	I3 ADD	I4 SUB
1	IF			
2	ID	IF		
3	OF	ID	IF	
4	EX	OF	ID	IF
5	EX	OF	ID	IF
6	EX	OF	ID	IF
7	WB	EX	OF	ID
8		EX	OF	ID
9		EX	OF	ID
10		EX	OF	ID
11		EX	OF	ID
12		EX	OF	ID
13		WB	EX	OF
14			WB	EX
15				WB

The DIV holds EX for cycles 7 to 12; I3 forwards R5 straight from it into EX at cycle 13, and I4 forwards R2 from I3 at cycle 14. Total = **15 cycles**.

(ii) Without operand forwarding

Now I3 cannot read R5 until I2 has written back (cycle 13), so its EX slips to cycle 14; and I4 must wait for I3's write-back (cycle 15), so its EX slips to cycle 16.

CYCLE	I1 MUL	I2 DIV	I3 ADD	I4 SUB
7	WB	EX	OF	ID
12		EX	OF	ID
13		WB	OF	ID
14			EX	OF
15			WB	OF
16				EX
17				WB

I1 to I2 run exactly as above (cycles 1 to 13). Then I3 waits for I2's write-back and executes at cycle 14, and I4 waits for I3's and executes at cycle 16. Total = **17 cycles**. Forwarding saved 2 cycles here; it saves more when short instructions depend on each other back to back.

Now redo it: lay out the long DIV first (it fixes everything behind it), then place each dependent instruction's EX one cycle after its operand is ready: after the producer's EX with forwarding, after its write-back without.

⚠ TRAP

Forwarding removes **most** data-hazard stalls but not the **load-use** one: a loaded value simply is not available in time for the very next instruction, so exactly one stall remains. And do not forget that a multi-cycle execute (multiply, divide) holds the EX stage and delays everything behind it.

RECALL AND CHECK

A 5-stage pipeline runs 100 independent instructions with no hazards. How many cycles, and what is the speedup over the non-pipelined version?

ANSWER

Cycles = $k + n - 1 = 5 + 100 - 1 = 104$. The non-pipelined version takes $100 \times 5 = 500$ stage-times, so the speedup is $500 / 104 \approx 4.8$, close to the ceiling of 5.

FILL IN THE GAP

1. A k-stage pipeline finishes n tasks in _____ cycles.
2. The pipeline cycle time is set by the _____ stage plus the latch delay.
3. A read-after-write clash is a _____ hazard, eased by _____.
4. The one stall forwarding cannot remove is the _____ hazard.

ANSWERS

1. $k + n - 1$. 2. slowest (longest). 3. data (RAW); forwarding. 4. load-use.

MODULE THREE

Computer Memory Organization

Memory is a hierarchy: a little fast storage close to the processor, a lot of slow storage far away, and the art of keeping the data you need in the fast part. Cache is where this module earns its marks, and cache is all over the recent papers.



Computer Memory Overview

Why memory is built as a hierarchy, how memory is accessed, and the principle of locality that makes the whole scheme work.

By the end of this unit you can describe the memory hierarchy and the four access methods, and explain locality of reference, including temporal and spatial locality. The recurring exam questions of this type are answered here.

ANCHOR

Think of where you keep things while working. The two or three pages actually under your hand are on the desk: instantly reachable, and there is room for almost nothing. The rest of the file is in the drawer, a second away. The other files are on the shelf across the room, and the archive is in a warehouse across town. Nobody designed that as a hierarchy. It happened because fast access and large capacity cost each other, and a computer's memory is organised the same way for the same reason.

The memory hierarchy

No single memory is both fast and large and cheap. So computers use several, arranged in a hierarchy: fast, small, expensive memory near the CPU, backed by slower, larger, cheaper memory further away. Data moves up the hierarchy when it is needed and down when it is not.

● MUST MEMORISE

THE HIERARCHY

From fastest and smallest to slowest and largest: **registers, cache (L1, L2, L3), main memory (RAM), secondary storage (disk, SSD), tertiary (tape).**

Memory hook: as you go down, everything grows except speed and cost per byte: **bigger, slower, cheaper.** As you go up: **smaller, faster, dearer.**

TEACH

ACCESS METHODS

- ▶ **Sequential:** read from the start in order until you reach the item (magnetic tape). Slow, access time depends on position.
- ▶ **Direct:** jump near the item, then search a small region (disk: seek the track, then read round it).
- ▶ **Random:** any location takes the same time to reach, by its address (RAM).
- ▶ **Associative:** located by content, not address; all locations checked at once (used inside cache to find a tag). Fastest, most expensive.

Memory is described by four **characteristics**: its **location** (registers, cache, main, external), its **access method** (above), its **performance** (access time and transfer rate), and its **capacity**. The units of capacity, smallest first:

- ▶ **Bit**, one 0 or 1. A **nibble** is 4 bits; a **byte** is 8 bits.
- ▶ **Cell**, one addressable location (often a byte); an **address** names one cell.
- ▶ **Word**, the natural unit the CPU works on (say 32 or 64 bits).
- ▶ **Block**, the unit moved between levels (several words), the key idea for cache.

If a memory holds 2^n words with K words per block, it has $M = 2^n / K$ blocks, and needs an **n-bit address**.

WHAT EACH SYMBOL MEANS

n = the number of address bits, so 2^n is the total number of addressable words · **K** = how many words are in one block (the unit moved between levels) · **M** = the resulting number of blocks in main memory (this M is a block count; the M of Module One Unit 3 was the IEEE mantissa)

WORKED EXAMPLE

SLIDE: SIZING A MEMORY

How many bits does a **4 MB** byte-addressable memory hold, and how many address lines does it need?

GIVEN	capacity = 4 MB · addressing = byte-addressable, so one cell is 1 byte = 8 bits
FIND	the total number of bits, and the number of address lines
FORMULA	bytes = MB × 1024 × 1024 · bits = bytes × 8 · address lines $k = \log_2(\text{number of cells})$

WHAT EACH SYMBOL MEANS

MB = 1024 × 1024 bytes, not a round million · **byte-addressable** = each address names one byte, so cells = bytes · **k** = address lines, and because $4\,194\,304 = 2^{22}$, k is just that exponent

- 1 4 MB = $4 \times 1024 \times 1024 = 4\,194\,304$ bytes.
- 2 Bits: $4\,194\,304 \times 8 = 33\,554\,432$ bits $\approx$ **33.5 million bits**.
- 3 Address lines: $4\,194\,304 = 2^{22}$ cells, so **22** address lines.

A 4 MB memory = 4 194 304 bytes = 33 554 432 bits ($\approx$ 33.5 Mbit), addressed by 22 lines ($2^{22} = 4\,194\,304$).

Cache works because real programs do not touch memory randomly.

THE PRINCIPLE, STATED

The principle of locality of reference. Over any short interval of its execution, a program references only a small and predictable subset of its address space, rather than spreading its accesses evenly across the whole of it.

It takes two forms, and a full answer names both: **temporal locality**, that a location referenced now is likely to be referenced again soon; and **spatial locality**, that locations near one just referenced are likely to be referenced soon.

Consequence: a small fast memory holding recently used blocks will satisfy the large majority of accesses. Locality is the assumption the entire memory hierarchy rests on: without it, caching would give no benefit at all.

Taking the two forms in turn:

- ▶ **Temporal locality:** a location just used is likely to be used again soon (a loop counter, a variable). So keep recently used data close.
- ▶ **Spatial locality:** a location near one just used is likely to be used soon (the next array element, the next instruction). So fetch neighbours together, a whole block at a time.

These are exactly why cache stores **recently used blocks** (temporal) and stores data in **blocks of neighbours** (spatial).

AS PRINTED

25/26 Q5(C)

THE EXAMINER'S WORDS, UNEDITED

- c. Explain the following terms **3mks**
- i. Locality of reference
 - ii. Temporal locality

Read it again. This year's version lists only **two** terms and prices them at 3mks, and it drops the colon after "Explain the following terms". The two older papers below list a third term, and both of them misspell it.

AS PRINTED

24/25 Q2(B)(I)

THE EXAMINER'S WORDS, UNEDITED

- b. i) Explain the following terms: **4½mks**
1. Locality of reference
 2. Temporal locality
 3. Special Locality

The year before, three terms. The third is printed "**Special Locality**". The term meant is spatial locality, and special is a different word with a different meaning. Part (ii) of the same lettered part, not quoted here, asks what happens when there is a cache miss.

THE EXAMINER'S WORDS, UNEDITED

- c. Explain the following **4½mks**
- i) Locality of reference
 - ii) Temporal locality
 - iii) Spacial Locality

And the year before that, the same three terms misspelled differently. Here it is "**Spacial Locality**". So the same word has come out wrong two ways in two years and neither of them is "spatial". This version also drops the word "terms" and the colon, leaving the bare "Explain the following". Write **spatial** in your answer and explain it: the mark is for the idea, not the spelling.

BREAK IT DOWN

BEFORE YOU ANSWER

GIVEN	nothing but the terms. This is a definition question, not a calculation.
ASKED	"Explain", which is neither "define" nor "list". An explanation names the idea, says why it holds, and says what the machine does about it.
MARKS	3mks for two terms in 25/26 and 4½mks for three in the two older papers, so about 1½ marks a term. That is a sentence of definition plus a sentence of consequence, each.
TAUGHT IN	this unit, the Teach box directly above.
TRAP	the misspelling. Do not lose time in the hall wondering whether "Special Locality" or "Spacial Locality" is a fourth concept you have never met. It is spatial locality both times.

WORKED EXAMPLE

PQ 25/26 Q5C · PQ 24/25 Q2B · PQ 23/24 Q3C

Locality of reference is the tendency of a program to access a small, predictable set of memory locations during any short period. Temporal locality: a location accessed now is likely to be accessed again soon (so cache keeps recently used items). Spatial locality: locations near one just accessed are likely to be accessed soon (so cache fetches a whole block of neighbouring words at once).

A two-level store has a fast level (1000 words, access 0.01 μs) and a slow level (100 000 words, access 0.1 μs). If 95% of accesses are found in the fast level, what is the average access time?

GIVEN	fast level: 1000 words at $T_1 = 0.01 \mu\text{s}$ · slow level: 100 000 words at $T_2 = 0.1 \mu\text{s}$ · $H = 0.95$
FIND	the average access time of the pair
FORMULA	$\text{average} = H \times T_1 + (1 - H) \times (T_1 + T_2)$

THE FORMULA, SYMBOL BY SYMBOL

average = $H \times T_1 + (1 - H) \times (T_1 + T_2)$. Here **H** = the hit ratio at the fast level, 0.95 · **T₁** = the fast level's access time, 0.01 μs · **T₂** = the slow level's access time, 0.1 μs · the miss term adds **both** times because you look in the fast level first and only then go to the slow one · the capacities (1000 and 100 000 words) do not enter the arithmetic at all, they only explain why the hit ratio is what it is

- 1 Hit (95%): the word is in the fast level, costing 0.01 μs .
- 2 Miss (5%): pay the fast level, then the slow level: 0.01 + 0.1 = 0.11 μs .
- 3 Average = $0.95 \times 0.01 + 0.05 \times 0.11 = 0.0095 + 0.0055 = \mathbf{0.015 \mu\text{s}}$.

Average access time = $0.95(0.01) + 0.05(0.11) = \mathbf{0.015 \mu\text{s}}$, close to the fast level's 0.01 μs and far below the slow level's 0.1 μs . That gap, bought by a high hit rate, is the whole reason for the hierarchy. The same formula scales to any two levels (see average memory access time, AMAT, in Unit 3).

RECALL AND CHECK

A loop repeatedly reads the same counter and steps through an array. Which kind of locality does each show?

ANSWER

The repeated counter is **temporal** locality (same location, used again and again). Stepping through the array is **spatial** locality (successive nearby locations).

◆ KEY POINT

Memory technologies trade speed, capacity and cost against each other, and no technology wins on all three at once. The hierarchy exists so that the small fast expensive store holds what is being used now, and the large slow cheap store holds everything else, giving the average speed of the fast one at close to the price of the slow one.

BLUNT VERSION: fast, big, cheap: pick two. The hierarchy is how a machine fakes all three.

Semiconductor Memory

The chips that make up main memory: read-write RAM and read-only ROM, and the two kinds of RAM the exam always compares, DRAM and SRAM.

By the end of this unit you can distinguish RAM from ROM and give four differences between DRAM and SRAM, the exact question that recurs on the paper.

TEACH

RAM AND ROM

RAM (Random Access Memory) is read-write and **volatile**: it loses its contents when power is removed. It holds the running program and its data. **ROM (Read Only Memory)** is read-mostly and **non-volatile**: it keeps its contents without power, and holds firmware such as the boot code. Both are random access: any location takes the same time to reach.

TEACH

THE ROM FAMILY

ROM is not one chip but a family, and its members differ in one thing only: how, and how often, the contents can be written.

- ▶ **ROM (mask ROM):** written once, at manufacture, by the mask that makes the chip. It can never be changed. Cheapest per bit in very large quantities.
- ▶ **PROM (Programmable ROM):** blank when made, written once by the customer on a device that blows fusible links. After that it is fixed for good.
- ▶ **EPROM (Erasable PROM):** written electrically, and erased as a whole by shining ultraviolet light through a quartz window in the package. Rewritable, but only out of circuit and only all at once.
- ▶ **EEPROM (Electrically Erasable PROM):** written and erased electrically, in place, a byte at a time, with no ultraviolet light and no removal from the board. Writing is far slower than reading.
- ▶ **Flash memory:** an EEPROM that erases in blocks rather than in bytes, which makes it much denser and much faster to write in bulk. Solid-state disks and memory cards are flash.

All five are non-volatile. Read down that list and the trend is the one this whole course keeps meeting: each step buys flexibility and pays for it in money, in speed or in density.

◆ KEY POINT

The distinction that earns the mark is volatility against writability. RAM is volatile and freely writable; every member of the ROM family is non-volatile, and they differ from each other only in how much trouble a rewrite is.

BLUNT VERSION: RAM forgets when the power goes. ROM does not. The rest is just how hard it is to change your mind.

TEACH

DRAM AND SRAM

RAM comes in two technologies. **Dynamic RAM (DRAM)** stores each bit as a charge on a tiny capacitor, which leaks, so it must be **refreshed** thousands of times a second. **Static RAM (SRAM)** stores each bit in a latch of several transistors, which holds as long as power is on, so it needs no refresh. SRAM is faster and simpler to use but larger and dearer per bit, so it is used for cache; DRAM is denser and cheaper, so it is used for main memory.

AS PRINTED

20/21 Q5(B)(I)

THE EXAMINER'S WORDS, UNEDITED

- b. i) List four (4) differences between DRAM and SRAM? **4mks**

Read it again. It is a command, "List four (4) differences", punctuated with a question mark. The count is written twice, in words and in digits, so four is exactly what is wanted. The 4mks over four differences prices each at one mark. The second half of that lettered part, not quoted here, is an unrelated 8-bit addition worth 3mks, so do not spend the whole part here.

BREAK IT DOWN

BEFORE YOU ANSWER

- GIVEN** nothing. This is straight recall, and it is the cheapest mark on the paper if you have the table memorised.
- ASKED** "List", so a list. Each line must be a *difference*, which means both sides named on one property.
- MARKS** 4mks for four differences, one mark each. A fifth earns nothing, a third loses one.
- TAUGHT IN** this unit, the Teach box directly above.
- TRAP** writing four statements about DRAM alone. A difference needs DRAM and SRAM contrasted in the same line, so set it out as pairs.

WORKED EXAMPLE

PQ 20/21 Q5B

DYNAMIC RAM (DRAM)	STATIC RAM (SRAM)
Stores a bit as charge on a capacitor	Stores a bit in a transistor latch
Must be refreshed periodically	Needs no refresh while powered
Slower access	Faster access
Higher density, cheaper per bit	Lower density, dearer per bit
Used for main memory	Used for cache memory

Now redo it: the four you must have are refresh, speed, density and cost. The memory hook: **DRAM** is **D**ense, **D**ynamic (needs refresh) and cheap; **SRAM** is **S**tatic (no refresh), **S**peedy and costly.

RECALL AND CHECK

Why is SRAM used for cache and DRAM for main memory?

ANSWER

SRAM is faster and needs no refresh, which suits the small, speed-critical cache; DRAM is denser and cheaper per bit, which suits large main memory where cost per bit matters more than raw speed.

MODULE THREE • UNIT 3

Cache Memory Design

A small fast memory that holds copies of the data most likely to be needed, so the CPU rarely waits on slow main memory. Splitting an address into cache fields and finding a block are the most examined calculations on the whole course.

By the end of this unit you can define hit, miss and hit ratio; compute average memory access time; break an address into tag, index and offset for direct, associative and set-associative mapping; and find which cache block an address maps to. Every recent cache question is worked here.

Why cache works, and the vocabulary

Because of locality (Unit 1), a small store of recently used blocks satisfies most accesses. When the CPU needs data, it checks cache first. A **hit** means it is there; a **miss** means it must be fetched from main memory (and the whole **block** around it is brought in, for spatial locality).

● MUST MEMORISE

HIT RATIO AND ACCESS TIME

Hit ratio h = hits / total accesses; miss ratio = $1 - h$.

Average memory access time depends on how cache and memory are arranged:

- ▶ **Hierarchical (look-through):** always check cache first, go to memory only on a miss. $AMAT = T_{\text{cache}} + (1 - h) \times T_{\text{memory}}$.
- ▶ **Simultaneous (look-aside):** start cache and memory together, use whichever is right. $AMAT = h \times T_{\text{cache}} + (1 - h) \times T_{\text{memory}}$.

WHAT EACH SYMBOL MEANS

h = the hit ratio, the fraction of accesses found in cache, a decimal from 0 to 1 (96% is $h = 0.96$) · **$1 - h$** = the miss ratio · **T_{cache}** (also written T_c) = the time for one cache access · **T_{memory}** (T_m) = the time for one main-memory access · both times in the same unit, usually nanoseconds · note this **T** is an access time, not the clock cycle time T of the CPU-performance formula

Memory hook: a high hit ratio pulls the average toward the fast cache time; even a small miss rate hurts, because a miss pays the full slow memory cost.

ANCHOR

A coat-check ticket answers three questions with one number: which rail your coat is on, which hook on that rail, and, if several coats hang on that hook, which one is yours. A memory address does the same job in the same order. The offset says which byte inside the block, the index says which slot of the cache the block may occupy, and the tag says which of the many blocks that share that slot is actually sitting there. Read the split in that order and it stops being three formulas.

Splitting the address: tag, index, offset

Main memory is divided into blocks; cache is divided into lines the same size as a block. A memory address is split into fields that say which block, which line, and where inside the block.

TEACH

THE THREE MAPPING FUNCTIONS

Direct	Tag	Line (index)	Offset
Set-assoc	Tag	Set	Offset
Assoc	Tag		Offset

All three carry an offset. Direct has a line field, set-associative a set field, fully associative neither.

- ▶ **Direct mapping:** each block has exactly one line it may sit in, $\text{line} = (\text{block number}) \bmod (\text{number of lines})$. Simple and fast, but two blocks that map to the same line keep evicting each other (conflict misses).
- ▶ **Fully associative:** a block may sit in any line. No conflict misses, but every line's tag must be searched, which is expensive.
- ▶ **Set associative (k-way):** the compromise. Lines are grouped into sets of k ; a block maps to one set, $\text{set} = (\text{block number}) \bmod (\text{number of sets})$, and may sit in any of the k lines in it.

Work from the block and cache sizes:

- ▶ **Offset bits** = $\log_2(\text{bytes, or words, per block})$.
- ▶ **Number of lines** = cache size / block size.
- ▶ **Index bits** (direct) = $\log_2(\text{number of lines})$; **set bits** (k-way) = $\log_2(\text{number of sets})$, where sets = lines / k.
- ▶ **Tag bits** = address bits – index (or set) bits – offset bits.

WHAT EACH SYMBOL MEANS

address bits = the width of a memory address, from the main-memory size (1 GB = 2^{30} , so 30 bits) · **block size** = bytes (or words) in one block, which equals one cache line · **number of lines** = how many blocks the cache can hold at once · **k** = the associativity, the number of ways or lines per set (here k means ways, not pipeline stages or address lines) · **sets** = lines / k · **$\log_2$** of a power of two is just the exponent, so $\log_2 1024 = 10$

Memory hook: offset picks the byte in the block, index/set picks the slot, tag is everything left over to confirm the right block is there.

Each cache line stores a **tag** beside its data, to prove which block it currently holds. The storage spent on tags, the **tag directory**, is **number of lines × tag bits per line**. It grows as the mapping gets more associative, because dropping the index (or shrinking the set field) lengthens the tag on every line.

WHAT EACH SYMBOL MEANS

number of lines = cache size / block size, and it does *not* change with the mapping · **tag bits per line** = the tag width you worked out for that mapping · the product is in **bits**, so divide by 8 for bytes

THE EXAMINER'S WORDS, UNEDITED

- a. Assume a computer has 32-bit addresses. Each block stores 16 words. A direct-mapped cache has 256 blocks. In which block (line) of the cache (decimal value) would you look for each of the following addresses? Addresses are given in hexadecimal for convenience. **4mks**
- i. CA2DFC12
 - ii. F24F25FF
 - iii. 24BBABCD
 - iv. C1D9EEF2

Read it again. The stem asks for the block "(decimal value)", so an answer left in hexadecimal is incomplete however right the working. Sentences are separated by double spaces, and the closing sentence about hexadecimal is set as a plain sentence here where the 24/25 twin of this question puts it in brackets and asks about **24-bit** addresses instead of 32.

BREAK IT DOWN

BEFORE YOU COMPUTE

- GIVEN** a 32-bit address, 16 words to a block, and a direct-mapped cache of 256 blocks, then four addresses in hexadecimal.
- ASKED** "In which block (line) of the cache ... would you look", in decimal. One number per address, four numbers in all.
- MARKS** 4mks, one an address. The field widths earn nothing on their own, so work them out once and then answer four times.
- TAUGHT IN** this unit, Module Three Unit 3.
- TRAP** 256 blocks needs 8 index bits and 16 words needs a 4-bit offset, which is exactly one hexadecimal digit. Drop the last digit, read the next two, convert. Reading the *first* two digits gives the tag, which is the wrong answer to this question.

WORKED EXAMPLE

PQ 25/26 Q2A

- GIVEN** address = 32 bits · block = 16 words · cache = 256 blocks (lines) · mapping = direct · four hex addresses to place
- FIND** the cache line each address maps to
- FORMULA** offset = $\log_2(\text{words per block})$ · index = $\log_2(\text{lines})$ · tag = address bits – index – offset · the index field **is** the line number

WHAT EACH SYMBOL MEANS

offset = 4 bits, picking one of the 16 words inside the block · **index** = 8 bits, naming one of the 256 lines · **tag** = the remaining 20 bits · because 4 bits is exactly one hex digit, the last hex digit is the offset and the next two are the line, which is why this can be read straight off the hex

- Field widths: offset = $\log_2 16 = 4$ bits; index = $\log_2 256 = 8$ bits; tag = $32 - 8 - 4 = 20$ bits.
- The shortcut in hex: the **last hex digit** is the 4-bit offset, and the **next two hex digits** are the 8-bit line (index). Read them straight off.

ADDRESS	OFFSET (LAST DIGIT)	LINE (NEXT TWO DIGITS)	LINE IN DECIMAL
CA2DFC12	2	C1	193
F24F25FF	F	5F	95
24BBABCD	D	BC	188
C1D9EEF2	2	EF	239

Lines: CA2DFC12 → 193, F24F25FF → 95, 24BBABCD → 188, C1D9EEF2 → 239.

Now redo it: once you know offset is 4 bits and index is 8 bits, the hex reads straight off: drop one digit, take the next two. Confirm the method on the tutorial address 1A2BC012, which gives line 1.

THE EXAMINER'S WORDS, UNEDITED

- a. A computer system has the following characteristics:
- Main memory = 64 MB
 - Cache memory = 128 KB
 - Block size = 64 bytes
- i) Determine the number of cache lines **1½mks**
- ii) For Direct Mapping, determine: Number of index bits; Number of tag bits; Number of off-set bits **1½mks**
- iii) For a 4-way Set Associative Cache, determine: Number of sets; Number of set index bits; Number of tag bits **1½mks**

Read it again. The three characteristics are printed as bullets with equals signs, so they are given data and not a sentence to unpick. Items (ii) and (iii) each pack three separate quantities into one line separated by semicolons, and each of those lines is worth 1½mks *in total*, not 1½ apiece. Item (i) ends with no full stop where (iv) has one. Part (iv), not quoted here, is a discussion of fully associative mapping for a further 1½mks.

BREAK IT DOWN

BEFORE YOU COMPUTE

GIVEN	main memory 64 MB, cache 128 KB, block 64 bytes. The address width is not given: you derive it from the memory size.
ASKED	"Determine" three times. Every quantity named is a count or a bit width, so every answer is a number.
MARKS	1½mks an item. Item (ii) is three bit widths for 1½mks, half a mark each, so show the split and move on rather than proving it.
TAUGHT IN	this unit, Module Three Unit 3.
TRAP	the number of lines does not change when the mapping changes. Only the split of the address changes, and every bit taken off the index lands in the tag.

GIVEN	main memory = 64 MB · cache = 128 KB · block = 64 bytes · mappings = direct, and 4-way set associative ($k = 4$)
FIND	the number of lines, and the tag / index (or set) / offset widths for both mappings
FORMULA	address bits = $\log_2(\text{memory})$ · lines = cache / block · offset = $\log_2(\text{block})$ · sets = lines / k · tag = address – index (or set) – offset

WHAT EACH SYMBOL MEANS

k = 4, the associativity or ways per set · **lines** = how many blocks the cache holds, unchanged by the mapping · **sets** = groups of k lines, so a 4-way cache has a quarter as many sets as lines · every bit taken off the index lands in the **tag**

- 1 Address bits: 64 MB = 2^{26} bytes, so **26-bit** addresses.
- 2 Number of lines = cache / block = 128 KB / 64 B = $2^{17} / 2^6 = 2^{11} = \mathbf{2048 \text{ lines}}$.
- 3 Offset = $\log_2 64 = 6$ bits.
- 4 Direct: index = $\log_2 2048 = 11$ bits, so tag = $26 - 11 - 6 = \mathbf{9 \text{ bits}}$.
- 5 4-way: sets = $2048 / 4 = 512$, so set bits = $\log_2 512 = 9$, and tag = $26 - 9 - 6 = \mathbf{11 \text{ bits}}$.

2048 lines. Direct: tag 9, index 11, offset 6. 4-way: tag 11, set 9, offset 6.
(Grouping lines into sets moves bits from the index into the tag.)

A cache is **16 KB**, block **256 bytes**, main memory **128 KB**. Find the address format and the tag-directory size for direct, fully associative, and 2-way set-associative mapping.

GIVEN	cache = 16 KB · block = 256 bytes · main memory = 128 KB · mappings = direct, fully associative, 2-way (k = 2)
FIND	the address format and the tag-directory size under each mapping
FORMULA	lines = cache / block · offset = $\log_2(\text{block})$ · tag = address – index (or set) – offset · tag directory = lines × tag bits

WHAT EACH SYMBOL MEANS

lines = 64, and this is the same for all three mappings · **fully associative** has no index at all, so its tag is the widest · **tag directory** comes out in **bits**, so divide by 8 to answer in bytes · the directory grows with associativity even though the data capacity does not change

- 1 Address: 128 KB = 2^{17} , so **17-bit** physical addresses. Offset = $\log_2 256 = 8$ **bits**. Lines = 16 KB / 256 B = **64**.
- 2 **Direct:** index = $\log_2 64 = 6$, tag = $17 - 6 - 8 = 3$. Tag directory = $64 \times 3 = 192$ bits = **24 bytes**.
- 3 **Fully associative:** no index, tag = $17 - 8 = 9$. Tag directory = $64 \times 9 = 576$ bits = **72 bytes**.
- 4 **2-way set associative:** sets = $64 / 2 = 32$, set bits = $\log_2 32 = 5$, tag = $17 - 5 - 8 = 4$. Tag directory = $64 \times 4 = 256$ bits = **32 bytes**.

Same 64 lines throughout, but the tag directory grows from **24 bytes** (direct) to **32 bytes** (2-way) to **72 bytes** (fully associative), because less indexing leaves a longer tag on every line.

FROM THE SLIDES

THE LECTURER'S PRACTICE CACHES

THE LECTURER'S OWN PRACTICE SET, GIVEN WITHOUT ANSWERS

Direct Mapping Example 2: Main Memory = 128 KB, Cache = 4 KB, Block = 32 B
 Example 3: Main Memory = 1 MB, Cache = 8 KB, Block = 64 B
 Fully Associative Example 2: Main Memory = 256 KB, Cache = 4 KB, Block = 32 B
 Example 3: Main Memory = 2 MB, Cache = 16 KB, Block = 64 B
 Set Associative Example 2 (4-Way): Main Memory = 512 KB, Cache = 8 KB, Block = 32 B
 Example 3 (8-Way): Main Memory = 4 MB, Cache = 32 KB, Block = 64 B
 Find: Offset, Index/Set bits, Tag bits, Cache lines, Tag Directory Size.

The slide sets these six but leaves them blank. Example 1 of each mapping is the 64 KB / 1 KB / 16 B cache, which is the 24/25 Q2(a) paper solved in the Past Papers section (direct tag 6, fully tag 12, 2-way tag 7). The six below are solved in full underneath.

Every one uses the same method, so it is worked once in steps and then tabulated for all six.

GIVEN	six cache configurations (memory, cache and block sizes, with the associativity for the set-associative pair)
FIND	offset bits, index or set bits, tag bits, the number of lines, and the tag-directory size for each
FORMULA	$\text{address} = \log_2(\text{memory}) \cdot \text{offset} = \log_2(\text{block}) \cdot \text{lines} = \text{cache} / \text{block} \cdot \text{sets} = \text{lines} / \text{ways} \cdot \text{tag} = \text{address} - \text{index (or set)} - \text{offset} \cdot \text{tag directory} = \text{lines} \times \text{tag bits}$

WHAT EACH SYMBOL MEANS

address bits = $\log_2$ of the main-memory size · **offset** = $\log_2$ of the block size in bytes · **lines** = cache size / block size, the same whatever the mapping · **fully associative** has no index, so its tag is the widest · **ways** = the associativity (4 or 8 here), and sets = lines / ways · **tag directory** comes out in bits, so divide by 8 for bytes

- 1 Worked for Direct Example 2 (128 KB / 4 KB / 32 B): address = $\log_2 128 \text{ KB} = 17$ **bits**; offset = $\log_2 32 = 5$ **bits**; lines = 4 KB / 32 B = **128**, so index = $\log_2 128 = 7$ **bits**.
- 2 Tag = $17 - 7 - 5 = 5$ **bits**. Tag directory = $128 \times 5 = 640$ **bits (80 bytes)**.
- 3 The other five follow the same three steps, changing only the sizes:

PRACTICE	MM / CACHE / BLOCK	ADDR	OFFSET	INDEX/SET	TAG	LINES	TAG DIRECTORY
Direct Ex 2	128 KB / 4 KB / 32 B	17	5	index 7	5	128	640 bits (80 bytes)
Direct Ex 3	1 MB / 8 KB / 64 B	20	6	index 7	7	128	896 bits (112 bytes)
Fully Ex 2	256 KB / 4 KB / 32 B	18	5	none	13	128	1664 bits (208 bytes)
Fully Ex 3	2 MB / 16 KB / 64 B	21	6	none	15	256	3840 bits (480 bytes)
Set Ex 2 (4-way)	512 KB / 8 KB / 32 B	19	5	set 6	8	256	2048 bits (256 bytes)
Set Ex 3 (8-way)	4 MB / 32 KB / 64 B	22	6	set 6	10	512	5120 bits (640 bytes)

Tag directories: direct 80 and 112 bytes; fully associative 208 and 480 bytes; set associative 256 and 640 bytes. The fully associative caches carry the widest tags because they keep no index at all.

Now redo it: cover the table and rebuild one row: address from the memory size, offset from the block, lines from cache over block, then tag is what is left, and the directory is lines times tag.

Seen again: this whole cache set is on the lecturer's revision deck, so all nine cases are gathered and worked once more in the Revision Class section near the back. The cache question is the one the lecturer drills hardest.

AS PRINTED

25/26 Q2(C)

THE EXAMINER'S WORDS, UNEDITED

c. A processor has the following configuration: Main memory is 1 GB; Cache is 512 KB; Block size is 128 bytes and it is a 4-way Set Associative cache.

i) Determine the total number of cache blocks. **1mk**

ii) Determine the number of sets. **1mk**

iii) Construct the complete address format. **1mk**

iv) If the hit rate is 96%, cache access time is 2 ns, and main memory access time is 80 ns, calculate the Average Memory Access Time (AMAT). **2mks**

v) Discuss how increasing associativity beyond four ways might affect performance. **2mks**

Read it again. The configuration is one sentence held together by semicolons, and the associativity is its last clause, which is easy to read past. Item (v) is the only part on the page whose marks are set inline after a single space, "performance. 2mks", where every other item has its marks tabbed to the right margin. It is worth 2mks, as much as the calculation above it.

BREAK IT DOWN

BEFORE YOU COMPUTE

GIVEN	1 GB of main memory, a 512 KB cache, 128-byte blocks, 4-way set associative, and for (iv) a hit rate, a cache access time and a memory access time.
ASKED	"Determine", "Determine", "Construct", "calculate", "Discuss". The last is prose and it carries 2mks.
MARKS	$1 + 1 + 1 + 2 + 2 = 7\text{mks}$. Two of the seven are for a discussion with no arithmetic in it, so leave time to reach it.
TAUGHT IN	this unit, which covers both the address split and the average memory access time formula.
TRAP	"the complete address format" means all three fields with their widths, tag and set and offset, and they must add up to the full address width. Give the widths and check the sum in front of the examiner.

GIVEN main memory = 1 GB · cache = 512 KB · block = 128 bytes · k = 4 (4-way) · h = 0.96 · $T_c = 2 \text{ ns}$ · $T_m = 80 \text{ ns}$

FIND (i) blocks, (ii) sets, (iii) the address format, (iv) the AMAT

FORMULA blocks = cache / block · sets = blocks / k · tag = address – set – offset · AMAT (hierarchical) = $T_c + (1 - h) \times T_m$

WHAT EACH SYMBOL MEANS

h = 0.96, so the miss ratio $(1 - h) = 0.04$ · **T_c** = cache access time · **T_m** = main-memory access time · **hierarchical** means cache is searched first, so its 2 ns is paid on *every* access, hit or miss · answer carries the unit of T, here ns

- 1 Blocks (lines) = cache / block = 512 KB / 128 B = $2^{19} / 2^7 = \mathbf{4096 \text{ blocks}}$.
- 2 Sets = blocks / 4 = 4096 / 4 = **1024 sets**.
- 3 Address = 1 GB = 2^{30} so 30 bits. Offset = $\log_2 128 = 7$, set = $\log_2 1024 = 10$, tag = $30 - 10 - 7 = 13$. Format: **tag 13, set 10, offset 7**.
- 4 AMAT (hierarchical) = $T_{\text{cache}} + (1 - h) \times T_{\text{memory}} = 2 + 0.04 \times 80 = 2 + 3.2 = \mathbf{5.2 \text{ ns}}$.

4096 blocks; 1024 sets; address tag 13 / set 10 / offset 7; AMAT = 5.2 ns. Higher associativity cuts conflict misses but needs more tag comparators, so beyond a point it adds cost and delay for little gain.

THE EXAMINER'S WORDS, UNEDITED

- d. If an 8-way set association cache is made up of 32 bits words, 4 words per line and 4096 sets.
- i) Size of each line of the cache **1mk**
 - ii) Size of each set of the cache **1mk**
 - iii) Size of the cache itself **1mk**

Read it again. The paper prints "8-way set **association** cache", not associative, and "32 bits words". More seriously the stem is a sentence fragment: it opens with "If" and never completes, so there is no question anywhere in it. The sub-items have to supply one, and they are phrased as nouns rather than commands. Part (iv), not quoted here, asks you to sketch the structure for 1½mks.

BREAK IT DOWN

BEFORE YOU COMPUTE

GIVEN	8 ways to a set, a 32-bit word, 4 words to a line, and 4096 sets.
ASKED	three sizes, each written as a noun phrase, "Size of each line of the cache". A number with a unit is the whole of the answer.
MARKS	1mk each. There is no working mark on offer, so give the number and one line showing where it came from.
TAUGHT IN	this unit, Module Three Unit 3.
TRAP	32 bits is 4 bytes. Answering in bits where the natural unit is bytes, or multiplying by 32 where you meant 4, is where this one is lost. Build up in order: line, then set, then cache.

WORKED EXAMPLE

PQ 23/24 Q1D

GIVEN	$k = 8$ (8-way) · word = 32 bits = 4 bytes · 4 words per line · 4096 sets
FIND	the size of a line, of a set, and of the whole cache
FORMULA	line = words per line × word size · set = $k \times$ line size · cache = sets × set size

WHAT EACH SYMBOL MEANS

k = 8, the lines per set · a **32-bit word** is 4 bytes, and mixing the two units is the classic slip here · build it up in order, line then set then cache, and the arithmetic stays trivial

- 1 Line size = 4 words × 32 bits = 4 × 4 bytes = **16 bytes**.
- 2 Set size = 8 lines × 16 bytes = **128 bytes**.
- 3 Cache size = 4096 sets × 128 bytes = 524 288 bytes = **512 KB**.

Line 16 bytes, set 128 bytes, cache 512 KB.

AS PRINTED

24/25 Q1(B)

THE EXAMINER'S WORDS, UNEDITED

- b. Determine the average memory access time for a machine with a cache hit rate of 80% and cache access time of 5 ns and main memory access time of 100 ns when:
- i. Simultaneous access memory organization is used. **2mks**
 - ii. Hierarchical access memory organization is used. **2mks**
 - iii. Draw the diagrams representing the two scenarios **3mks**

Read it again. The stem ends with a colon, "100 ns when:", which is what the list underneath needs. Item (iii) is printed "iii. Draw", with no space after the full stop.

THE EXAMINER'S WORDS, UNEDITED

- a. Determine the average memory access time for a machine with a cache hit rate of 80% and cache access time of 5 ns and main memory access time of 100 ns when.
- Simultaneous access memory organization is used. **2½mks**
 - Hierarchical access memory organization is used. **2½mks**
 - Draw the diagrams representing the two scenarios **3mks**

The same question a year earlier, and the wording drifts twice. This version ends its stem with a **full stop**, "100 ns when.", where the list plainly needs a colon, and each of the first two items is worth **2½mks** against 2mks in 24/25. The "iii.Draw" spacing defect is in both years. Same data, same method, different price.

BREAK IT DOWN

BEFORE YOU COMPUTE

GIVEN	a hit rate of 80%, a cache access time of 5 ns, and a main memory access time of 100 ns.
ASKED	the average access time under two <i>named</i> organisations, then the diagrams. The two names, "Simultaneous" and "Hierarchical", are the whole question: they choose which of the two formulas applies.
MARKS	2 + 2 + 3 in 24/25, and 2½ + 2½ + 3 in 23/24. Either way the diagrams are worth more than either calculation, so draw them.
TAUGHT IN	this unit, Module Three Unit 3.
TRAP	the two formulas differ in one place only: whether the cache time is weighted by the hit rate or paid on every access. Simultaneous starts both lookups at once; hierarchical searches the cache first. Give both numbers and label which is which.

WORKED EXAMPLE

PQ 24/25 Q1B · PQ 23/24 Q3A

GIVEN	$h = 0.80 \cdot T_c = 5 \text{ ns} \cdot T_m = 100 \text{ ns} \cdot$ both organisations required
FIND	the AMAT under (i) simultaneous and (ii) hierarchical organisation
FORMULA	simultaneous = $h \times T_c + (1-h) \times T_m$ · hierarchical = $T_c + (1-h) \times T_m$

WHAT EACH SYMBOL MEANS

h = hit ratio · **T_c**, **T_m** = cache and main-memory access times · the *only* difference between the two formulas is whether the cache time is weighted by h (simultaneous, both started at once) or paid in full every time (hierarchical, cache searched first)

- 1 Simultaneous: $\text{AMAT} = h \times T_c + (1-h) \times T_m = 0.8 \times 5 + 0.2 \times 100 = 4 + 20 = \mathbf{24 \text{ ns.}}$
- 2 Hierarchical: $\text{AMAT} = T_c + (1-h) \times T_m = 5 + 0.2 \times 100 = 5 + 20 = \mathbf{25 \text{ ns.}}$

Simultaneous 24 ns, hierarchical 25 ns. Hierarchical pays the cache time on every access (even misses), so it is slightly slower here.

Now redo it: the difference is only whether the cache time is always paid (hierarchical) or only on hits (simultaneous). Learn both formulas.

◆ KEY POINT

The three fields of an address are not three separate calculations. The offset is fixed by the block size, the index is fixed by the number of slots, and the tag is whatever is left of the address. Work them out in that order and the tag never needs its own formula.

BLUNT VERSION: which byte, which slot, which block. Offset and index first, tag is the remainder.

Elements of cache design

Beyond mapping, a few design decisions round out the topic and appear as short questions.

TEACH

THE CACHE READ OPERATION

On every read the CPU puts out an address and the cache checks it. **On a hit**, the cache returns the word straight to the CPU and **no traffic goes onto the system bus** at all, which is why a hit is fast. **On a miss**, the address is driven onto the system bus, main memory returns the whole block, and it is delivered to the cache and to the CPU together (the fetch and the fill happen in parallel), so the waited-for word arrives as soon as it can.

TEACH

PHYSICAL AND LOGICAL (VIRTUAL) ADDRESSES

Programs use **logical (virtual) addresses**; the hardware stores data at **physical addresses**. A **memory management unit (MMU)** translates one to the other through a page table, which is what lets each program behave as if it owns all of memory (**virtual memory**). A cache can be indexed by either address:

- ▶ A **physical cache** waits for the MMU to translate the address, then looks up the physical address. Simple and always correct.
- ▶ A **logical (virtual) cache** looks up the virtual address directly, in parallel with translation, so it is faster. The cost: different programs reuse the same virtual addresses, so the cache must be **flushed on every context switch**, or tagged with an address-space identifier, to avoid handing one program another's data.

TEACH

CHOOSING THE CACHE SIZE

Cache size is a balance. Too small and the hit rate falls and the CPU stalls; too large and it costs more, uses scarce chip or board area, and the extra addressing logic makes each lookup slightly slower. So cache is sized large enough for a good hit rate yet small enough to stay fast and affordable, which is exactly why machines use several small-to-large levels (L1, L2, L3) rather than one big cache.

⚠ TRAP

Two common errors. (1) Watch the **units** of the block: "16 words" with a 4-byte word is 64 bytes, so the offset counts words or bytes depending on how the address is given. (2) For set-associative, divide the lines by the associativity to get the **sets** before taking the log for the set bits; the extra bits move into the **tag**, not away.

RECALL AND CHECK

A direct-mapped cache has 1024 lines and 32-byte blocks, on a 32-bit machine. How many tag, index and offset bits?

ANSWER

Offset = $\log_2 32 = 5$, index = $\log_2 1024 = 10$, tag = $32 - 10 - 5 = 17$ bits.

FILL IN THE GAP

1. The number of cache lines equals cache size divided by _____.
2. In direct mapping a block maps to line number equal to block number _____ the number of lines.
3. Offset bits = log base 2 of the _____ size.
4. Going from direct to k-way set associative moves bits from the index into the _____.

ANSWERS

1. block size. 2. modulo (mod). 3. block. 4. tag.

MODULE FOUR

Basic and Advanced Processor Architectures

Two design philosophies, the reduced instruction set (RISC) and the complex instruction set (CISC), and the ways many processors are joined to work in parallel. Mostly a describing-and-comparing module, which means fast, reliable marks if you know the contrasts cleanly.

RISC

vs

CISC

RISC versus CISC

Two philosophies of instruction-set design: keep instructions few and simple, or make them many and powerful. Naming the differences and example processors is worth easy marks every year.

By the end of this unit you can give at least four differences between RISC and CISC and name example processors of each.

TEACH

THE TWO PHILOSOPHIES

CISC (Complex Instruction Set Computer) provides many powerful instructions, some doing a lot of work in one instruction (including direct memory-to-memory operations). This makes programs short but the hardware complex, and instructions take varying numbers of cycles.

RISC (Reduced Instruction Set Computer) provides few, simple, fixed-length instructions, each doing a little in (ideally) one cycle, with memory touched only by dedicated load and store instructions (a **load-store** architecture). Programs are longer but the hardware is simple and fast, and it pipelines well.

TEACH

WHY CISC CAME FIRST, THEN RISC

The order is not an accident, it follows the cost of memory. In the 1960s and 1970s **memory was expensive and slow while processor hardware was relatively cheap**, so the sensible trade was to make each instruction do as much as possible and keep programs short. That is the **CISC** idea, and it also made the compiler's job easy, since one powerful instruction could stand in for a whole sequence.

By the early 1980s two measurements changed the trade. Researchers found that **most programs use only a small subset of the available instructions**, and that **the complex instructions took many cycles** while simple ones could run in one. If the fancy instructions are rarely used and slow when they are, why pay for them? Stripping them out gives the **RISC** idea: a few fast instructions, and let the compiler build the rest. RISC grew out of that early-1980s work: the **IBM 801**, **Stanford MIPS** and **Berkeley RISC 1 and 2** were all designed on the philosophy that came to be called RISC.

FROM THE SLIDES

Q & A ON THE DECK

THE LECTURER'S OWN QUESTION, AND THE ANSWER THE SLIDE GIVES

Why were CISC architectures initially popular?

The slide's answer. Because memory was expensive, and CISC instructions could perform complex operations using fewer instructions, thereby reducing program size. In other words the whole point of CISC was to spend cheap processor complexity to save on costly memory, which is exactly the trade the paragraph above describes. A one line question like this is worth 2 marks on a describing question, so learn the sentence.

The clearest way to feel the difference is to watch each philosophy multiply two numbers held in memory. CISC offers one instruction that reaches into memory, multiplies and writes back. RISC forbids arithmetic on memory: you must **load** both operands into registers, multiply **register to register**, then **store** the result. RISC needs four instructions where CISC needs one, yet each RISC instruction is simple, fixed-length and one cycle, so it pipelines cleanly.

```
; CISC: one instruction, memory to memory
MUL C, A, B           ; C ← A × B, all in one

; RISC: load, work in registers, store
LOAD R1, A           ; bring A into a register
LOAD R2, B           ; bring B into a register
MUL R3, R1, R2       ; multiply, register to register
STORE C, R3          ; write the result back to memory
```

This is the trade in miniature: CISC hides the loads and stores inside the multiply and pays for it with a slow, many-cycle instruction; RISC exposes them and pays with a longer program of fast instructions. The compiler, not the hardware, arranges the four RISC lines, which is what "complexity pushed to the compiler" means in the table below.

A CISC instruction is too complex to run directly, so inside the processor it is first translated into a string of tiny internal steps called **microinstructions**, and those are what actually execute. A RISC instruction is already simple enough to run directly in hardware (**hardwired control**), with no translation layer. That extra step is a large part of why CISC instructions take more cycles and are harder to pipeline.



RISC executes the instruction straight away; CISC first turns it into microinstructions. The extra layer is flexible but costs cycles.

When the paper asks for "clear differences", these concrete figures turn a vague contrast into a marked one:

- ▶ **Instruction count:** CISC typically has a large set, on the order of **100 to 250** instructions; RISC keeps it small and simple.
- ▶ **Addressing modes:** CISC offers many, roughly **5 to 20**; RISC has only a few, because nearly every instruction uses simple register addressing.
- ▶ **Instruction length:** CISC instructions are **variable** in size; RISC are a **fixed** size, aligned on word boundaries, which is what makes them easy to decode and to pipeline.
- ▶ **Control:** CISC uses **microcode**; RISC uses **hardwired** control.
- ▶ **Registers:** RISC provides **many** general-purpose registers, CISC relatively **few**.

RISC	CISC
Few, simple instructions	Many, complex instructions
Fixed-length instructions	Variable-length instructions
Mostly one cycle per instruction	Instructions take several cycles
Load-store: only load/store touch memory	Many instructions access memory directly
Many general-purpose registers	Fewer registers
Simple hardware, easy to pipeline	Complex hardware, harder to pipeline
Complexity handled in the compiler	Complexity handled in the hardware

◆ KEY POINT

RISC and CISC do not disagree about what a program must do. They disagree about WHERE the complexity should live. CISC puts it in the hardware, in rich instructions and a microprogrammed control unit. RISC puts it in the compiler, building complex operations out of simple fast instructions.

BLUNT VERSION: somebody has to do the hard work. CISC makes the chip do it, RISC makes the compiler do it.

The slides ask you to weigh each side, so keep two points ready for each:

- ▶ **CISC advantages:** fewer instructions do a given task, which saves on the slow, expensive memory CISC was built for; and microprogramming the control unit is easier and cheaper than hardwiring it, so a rich instruction set is affordable.
- ▶ **CISC disadvantages:** the complex instructions are rarely used yet the hardware still has to carry them, the many memory operations are slow, and the whole design grows more complex, and harder to pipeline, with every generation.
- ▶ **RISC advantages:** often **2 to 4 times** the performance of a comparable CISC at the same clock, simpler hardware that uses less chip area, a shorter design cycle, and one instruction finished per cycle, which pipelines cleanly.
- ▶ **RISC disadvantages:** performance leans heavily on the compiler, so poor instruction scheduling leaves the processor stalling; and because each instruction does little, programs expand (code expansion) and need a fast memory system, usually a large on-chip cache, to keep the pipeline fed.

THE EXAMINER'S WORDS, UNEDITED

ii) Mention any four (4) clear differences between RISC and CISC. Give two (2) examples of processors in each category. **4mks**

Read it again. The count is spelled and then repeated in digits, "any four (4)", and so is the number of examples, "two (2)". The verb is "Mention", the mildest of the three years. Part (i) of the same lettered part, not quoted here, asks you to compare Von Neumann with Harvard architecture for "3mk", printed singular.

THE EXAMINER'S WORDS, UNEDITED

b. i) State Four (4) clear differences between RISC and CISC. Give two examples of processors in each category. **4mks**

Four years earlier, a firmer verb. "State Four (4) clear differences", with the count capitalised mid-sentence, and the examples asked for without a digit, "two examples". Same 4mks.

THE EXAMINER'S WORDS, UNEDITED

c. Mention any 3 clear differences between RISC and CISC. Give two examples of processors in each category. **3mks**

And the year in between, a different count. Here it drops to **3**, written as a bare digit, "Mention any 3 clear differences", for 3mks rather than 4. The number of differences the examiner wants has changed between sittings, so count what the paper asked for before you count what you wrote: a fourth difference on the 23/24 paper earns nothing.

BREAK IT DOWN

BEFORE YOU ANSWER

GIVEN	nothing. Pure recall, and the fastest marks on any of these three papers.
ASKED	a fixed number of "clear differences", then two example processors of each family. Both halves are marked and the examples are the half students forget.
MARKS	4mks in 20/21 and 24/25, 3mks in 23/24. The wording tells you how many differences buy them.
TAUGHT IN	this unit, the comparison table directly above.
TRAP	the word "clear". A difference has to contrast both sides on one property, so write each as a pair rather than as two separate descriptions.

WORKED EXAMPLE

PQ 24/25 Q4A · PQ 20/21 Q4B · PQ 23/24 Q5C

Four differences (any four): RISC has few simple fixed-length one-cycle instructions and only load/store touch memory, with many registers and simple pipelined hardware; CISC has many complex variable-length multi-cycle instructions, many of which access memory directly, with fewer registers and complex hardware. Examples: RISC, ARM and MIPS (also SPARC, PowerPC); CISC, Intel x86 (8086, Pentium) and the Motorola 68000.

Now redo it: pick your four contrasts (instruction count, length, cycles, memory access) and memorise two examples each: ARM/MIPS for RISC, x86/68000 for CISC.

● MUST MEMORISE

EXAMPLE PROCESSORS

RISC: ARM, MIPS (R2000, R4000), SPARC, PowerPC (601), Intel i860, Motorola 88000, IBM RS/6000. **CISC:** Intel x86 (8086, 80286, 80386, Pentium), Motorola 68000, VAX 11/780, IBM 370, PDP-11.

The lecturer's slide lists its own favourites, so two safe ones to quote are **MIPS and SPARC for RISC** and **Intel x86 and VAX for CISC**. Any two from each row score; the paper only ever asks for two.

Memory hook: the phone in your pocket is **ARM (RISC)**; the older desktop is **x86 (CISC)**.

FROM THE SLIDES

Q & A ON THE DECK

THE LECTURER'S OWN QUESTION, AND THE ANSWER THE SLIDE GIVES

Why is pipelining more difficult in CISC processors?

The slide's answer. Because instructions have varying lengths and execution times, making it harder to process instructions uniformly through the pipeline stages. This is the flip side of everything above: a pipeline wants every instruction to be the same shape so the stages line up, and CISC instructions are not, whereas RISC's fixed-length one-cycle instructions are. Expect this as a short 2 to 3 mark question, and answer it in exactly this shape.

RECALL AND CHECK

Why does RISC pipeline more easily than CISC?

ANSWER

RISC instructions are fixed-length and mostly take one cycle, so the stages line up uniformly and hazards are easier to manage. CISC instructions vary in length and cycle count, which makes overlapping them much harder.

MODULE FOUR · UNIT 2

Parallel Architectures and Interconnection Networks

When one processor is not enough, many are joined to work together. This unit classifies parallel machines and the networks that connect them, and completes the data-hazard picture from pipelining.

By the end of this unit you can state Flynn's taxonomy, describe common interconnection networks, and name the three types of data hazard.

TEACH

FLYNN'S TAXONOMY

Parallel machines are classified by how many instruction streams and data streams they run at once:

- ▶ **SISD:** Single Instruction, Single Data. The classic single processor.
- ▶ **SIMD:** Single Instruction, Multiple Data. One instruction acts on many data items at once (vector and graphics processors).
- ▶ **MISD:** Multiple Instruction, Single Data. Rare; several operations on one data stream (some fault-tolerant designs).
- ▶ **MIMD:** Multiple Instruction, Multiple Data. Many processors running different instructions on different data (multicore CPUs, clusters).

TEACH

INTERCONNECTION NETWORKS

The processors and memories must be wired together. Common schemes: **shared bus** (simple, but one at a time), **crossbar** (every processor to every memory, fast but costly), **mesh** (each node to its neighbours, scales well), and **hypercube** and **ring** topologies. The trade-off is always cost and wiring against how many transfers can happen at once.

Completing the pipelining picture (Module Two), data hazards come in three types, named by the order of the clashing accesses:

- ▶ **RAW (read after write):** an instruction reads a value before the earlier instruction has written it. The true dependency, and the common one.
- ▶ **WAR (write after read):** an instruction writes a value before an earlier one has read it.
- ▶ **WAW (write after write):** two instructions write the same location in the wrong order.

THE EXAMINER'S WORDS, UNEDITED

- c. i) Describe the three types of data hazards. **1½mks**
 ii) What type of data hazards do the following instructions belong? **1½ mks**
1. MULTD F0, F2, F4
DIVD F10, F0, F6
 2. DIVD F10, F0, F6
ADDD F6, F8, F2
 3. MULTD F0, F2, F4
SD F8, 50 (R5)

Read it again. Part (i)'s verb is "Describe", not name and not list. Part (ii) is missing a preposition, "What type of data hazards do the following instructions belong?", and sets its "1½ mks" in-line where (i) tabs it to the margin. **And look hard at pair 3.** Pairs 1 and 2 share a register, so each carries a hazard. Pair 3 does not: MULTD writes F0, while SD reads F8 and R5 and writes memory. No register is common to both, so as printed there is no data hazard between them. Sitting third after a RAW and a WAR, it is plainly meant to be the WAW case, which would need the store to name F0. The paper is quoted here as it stands, not as it was meant.

GIVEN	for (i) nothing, it is recall. For (ii) three pairs of floating-point instructions, each pair to be classified.
ASKED	(i) "Describe the three types". The three names on their own are not a description: each needs the order of the two clashing accesses spelled out. (ii) classify each pair, which is a different skill from reciting the list.
MARKS	1½mks for three types, half a mark each, so one short sentence apiece; then 1½ mks for three classifications, again half a mark each.
TAUGHT IN	this unit, the Teach box directly above.
TRAP	classifying by instruction name instead of by register. The mnemonic tells you nothing; only the registers do. Write down what each instruction <i>writes</i> and what it <i>reads</i> , then look for the same name on both sides. That method is also the only thing that shows you pair 3 has no overlap at all.

(i)

Read after write (RAW), write after read (WAR), and write after write (WAW). RAW is a true dependency; WAR and WAW are name dependencies that arise when instructions are reordered.

(ii) Take each pair and list what it writes and what it reads. The register, not the mnemonic, decides the answer.

PAIR	WRITES	READS	REGISTER IN COMMON	HAZARD
1. MULTD F0, F2, F4 DIVD F10, F0, F6	F0 F10	F2, F4 F0, F6	F0: written first, read second	RAW
2. DIVD F10, F0, F6 ADDD F6, F8, F2	F10 F6	F0, F6 F8, F2	F6: read first, written second	WAR
3. MULTD F0, F2, F4 SD F8, 50 (R5)	F0 memory at R5+50	F2, F4 F8, R5	none	none as printed

1 is **RAW**, a true dependency: DIVD needs the F0 that MULTD has not finished writing. 2 is **WAR**, an anti-dependency: ADDD would overwrite the F6 that DIVD still has to read. 3, **as the paper prints it, is no hazard at all**: the two instructions touch no register in common.

What to write in the hall. Answer pair 3 the way the page is printed, then add one line: "as printed these two share no register, so there is no data hazard; had the store referenced F0 it would be WAW, write after write." That earns the mark for the analysis and shows the examiner you read the registers rather than pattern matching on the position in the list.

RECALL AND CHECK

A graphics processor applies the same operation to thousands of pixels at once. Which Flynn category is it?

ANSWER

SIMD: a single instruction acting on multiple data items simultaneously.

S U P P L E M E N T

Digital Logic and Boolean Algebra

The gates and algebra beneath every processor. This material is not a unit of the current course outline, but older past papers set questions on it, so it is taught here to a full pass in case your paper does.

$$F = AB + AC + BC \quad = \text{majority}(A,B,C)$$

Boolean Algebra and K-maps

Simplifying logic expressions, by algebra and by Karnaugh map, and turning them into gate circuits.

⚠ NOT IN THE CURRENT SYLLABUS

The 2024/25 and 2025/26 papers do **not** ask Boolean algebra or K-maps. Older papers (2020/21, 2023/24) do. Learn this only after the graded topics are secure; it is here so no past question is left unanswerable.

The operations and the laws

TEACH

BOOLEAN BASICS

Three operations: **AND** (written as a product, AB , true only if both), **OR** (written as a sum, $A+B$, true if either), and **NOT** (written A' or A with a bar, the opposite). The useful laws for simplifying:

- ▶ **Identity:** $A + 0 = A$, $A \cdot 1 = A$. **Null:** $A + 1 = 1$, $A \cdot 0 = 0$.
- ▶ **Complement:** $A + A' = 1$, $A \cdot A' = 0$.
- ▶ **Absorption:** $A + AB = A$, and $A + A'B = A + B$.
- ▶ **De Morgan:** $(AB)' = A' + B'$, and $(A + B)' = A'B'$.

Each of those is a law, and a paper may ask you to **state** one rather than use it. Stated in words:

THE LAWS, STATED

De Morgan's laws. The complement of a product is the sum of the complements, and the complement of a sum is the product of the complements. Formally, for any Boolean variables A and B : $(A \cdot B)' = A' + B'$ and $(A + B)' = A' \cdot B'$. In words: breaking a bar over a group flips the operator underneath it, AND becoming OR and OR becoming AND.

The complement law. A variable ORed with its own complement is always 1, and ANDed with its own complement is always 0: $A + A' = 1$, $A \cdot A' = 0$. This is the law that does the work in most simplifications, because it lets a variable and its opposite cancel out of a pair of terms.

The identity and null laws. ORing with 0 and ANDing with 1 leave a variable unchanged ($A + 0 = A$, $A \cdot 1 = A$); ORing with 1 forces 1 and ANDing with 0 forces 0 ($A + 1 = 1$, $A \cdot 0 = 0$).

The absorption law. A term already implied by a simpler one adds nothing: $A + AB = A$. A related form removes a redundant complement: $A + A'B = A + B$.

Simplify $F = A'BC + AB'C + ABC' + ABC$ by algebra.

GIVEN $F = A'BC + AB'C + ABC' + ABC$ · three variables A, B, C
FIND the simplest equivalent expression
FORMULA $X + X' = 1$ · $X \times 1 = X$ · $X + X = X$ (a term may be reused), then factor common pairs

WHAT EACH SYMBOL MEANS

A' = NOT A, the complement · writing two letters together (**AB**) means AND · $+$ means OR · $X + X' = 1$ is the law doing all the work: a variable and its complement together cover both cases, so the variable drops out

- 1 Group the last two by AB: $ABC' + ABC = AB(C' + C) = AB$.
- 2 Take ABC again (it can be reused) with $AB'C$ by AC: $AB'C + ABC = AC(B' + B) = AC$.
- 3 Take ABC again with $A'BC$ by BC: $A'BC + ABC = BC(A' + A) = BC$.
- 4 So $F = AB + AC + BC$.

$F = AB + AC + BC$. This is the **majority function**: it is 1 when at least two of A, B, C are 1.

Now redo it: the trick is that a minterm may be reused in more than one group. That is exactly what a K-map makes visual, next.

The Karnaugh map

A K-map arranges the truth table so that cells differing in one variable are neighbours. Circle groups of 1s in powers of two (1, 2, 4, 8), as large as possible; each group drops the variable that changes across it. The circled groups, ORed together, are the simplified expression.

WORKED EXAMPLE

METHOD

Simplify the same F on a 3-variable K-map (rows A , columns BC in Gray-code order). $F = 1$ at minterms 3, 5, 6, 7.

GIVEN	the same F · 3 variables · $F = 1$ at minterms 3, 5, 6, 7 · map = rows A , columns BC in Gray-code order
FIND	the simplified expression, by grouping
FORMULA	circle the largest groups of adjacent 1s in powers of two (1, 2, 4, 8); each group drops the variable that changes across it; OR the surviving groups together

WHAT EACH SYMBOL MEANS

minterm = the decimal number of one input combination, so minterm 3 is $A=0, B=1, C=1$ · **Gray-code order** = columns run 00, 01, 11, 10 so that neighbours differ by one bit, which is what makes adjacency mean simplification · groups **may overlap**, and a variable that is 0 in part of a group and 1 in the rest is the one that cancels

BC	00	01	11	10
A=0	0	0	1	0
A=1	0	1	1	1

Three pairs: BC (amber, both rows), AC (green), AB (teal). $F = AB + AC + BC$.

The three circled pairs give BC , AC and AB , so $F = AB + AC + BC$, the same answer as the algebra, reached by eye.

TEACH

FROM EXPRESSION TO CIRCUIT

To draw the logic circuit for $F = AB + AC + BC$: one **AND** gate per product term (AB , AC , BC), then one **OR** gate combining the three AND outputs into F . A sum-of-products expression always becomes a layer of AND gates feeding one OR gate. **Don't-care** outputs (marked d or X) may be treated as 0 or 1, whichever makes a larger group, so use them to simplify further.

RECALL AND CHECK

On a K-map, what does a group of two adjacent 1s tell you, and why circle the largest groups?

ANSWER

A group of two drops the one variable that changes across it, leaving a product of the rest. Larger groups drop more variables, giving fewer and simpler terms, so always circle the biggest power-of-two groups you can.

◆ KEY POINT

A Karnaugh map is a truth table with its rows rearranged so that any two cells sharing an edge differ in exactly one variable. That is the whole trick: adjacency becomes algebraic cancellation, so grouping neighbouring ones is the same operation as applying the law that a variable and its complement sum to one.

BLUNT VERSION: it is a truth table folded so that neighbours differ by one bit. Grouping is cancelling.

REFERENCE

Formulas, Notation and Exam Craft

Everything the modules taught, compressed into what you actually carry into the hall: every formula with every symbol spelled out, the letters this course reuses for different jobs, a plan for the two hours, and the mistakes that cost marks. Read this part last, and read it again the night before.

$\text{CPU time} = \text{IC} \times \text{CPI} \times T$	instructions, cycles each, seconds per cyc
$\text{AMAT} = T_c + (1 - h) \times T_m$	hit ratio, cache time, memory time
$\text{time} = (k + n - 1) \times \text{cycle}$	stages, tasks, clock period
$\text{tag} = \text{addr} - \text{index} - \text{offset}$	address width, slot field, byte in block
$\text{speedup} = 1 / ((1-f) + f/s)$	fraction sped up, and by how much

not one letter left unexplained

Formula and Notation Reference

Every formula this course uses, in one place, with what each letter in it stands for. The modules teach these where they are needed; this section is for revision and for the moment in the exam when you know the method but want the symbols confirmed.

By the end of this section you can state any formula on the syllabus from memory, say what each of its symbols means and in what units, tell apart the letters this course reuses for different jobs, and pick the right formula from the wording of a question.

What every symbol means

Read these three tables once, slowly. Most lost marks in the numeric questions come from substituting a number into the right formula in the wrong place.

Numbers, bits and floating point

SYMBOL	WHAT IT MEANS	UNITS OR FORM	USED IN
b	the base of a number system	2, 8, 10 or 16	Number systems
d_i	the digit sitting in column i, counting from 0 at the right	a digit	Positional notation
n	number of bits in a register or field	bits	Integer ranges
S	the sign bit, 0 positive and 1 negative	1 bit	IEEE-754
E	the stored exponent field, the bits actually written	8 or 11 bits	IEEE-754
e	the true exponent, the real power of two	a signed integer	IEEE-754
M	the mantissa field, the fraction after the hidden 1	23 or 52 bits	IEEE-754
bias	the constant added to the exponent before storing	127 single, 1023 double	IEEE-754

Processors, performance and pipelines

SYMBOL	WHAT IT MEANS	UNITS OR FORM	USED IN
k	number of address lines, the width of the address bus	bits	Address space
N	how many separate locations must be reachable	a count	Address space
IC	instruction count, instructions actually executed	a count	CPU performance
CPI	average clock cycles per instruction	cycles per instruction	CPU performance
T	clock cycle time, the length of one cycle	seconds (often ns)	CPU performance
clock rate	cycles per second, $T = 1 / \text{clock rate}$	Hz (GHz = 10^9)	CPU performance
f	fraction of the work that the improvement speeds up	0 to 1	Amdahl's law
s	how many times faster that fraction becomes	a multiplier	Amdahl's law
k	number of pipeline stages	a count	Pipelining
n	number of tasks or instructions fed through a pipeline	a count	Pipelining

Memory, cache and images

SYMBOL	WHAT IT MEANS	UNITS OR FORM	USED IN
k	associativity, the ways or lines per set	a count	Cache
h (or H)	hit ratio, the share of accesses found in cache	0 to 1	Cache
T_c	time for one cache access	ns	Cache
T_m	time for one main-memory access	ns	Cache
M	the number of blocks main memory holds	a count	Memory, cache
K	the number of words in one block	words	Memory, cache
EA	effective address, where the operand really is	an address	Addressing modes
dpi	dots (pixels) per inch, the resolution	pixels per inch	Image sizing
bits-per-pixel	colour depth: 1 black and white, 24 full colour	bits	Image sizing

⚠ TRAP

This course reuses four letters for completely different jobs, and the formulas sit only a few pages apart. Fix these now:

- ▶ **k** is **address lines** in address space (2^k locations), **pipeline stages** in $(k + n - 1)$, and **ways per set** in a set-associative cache. Three meanings, three topics.
- ▶ **n** is a **bit width** in every integer-range formula, but the **number of tasks** in the pipeline formula. In $(k + n - 1)$, **n** is not a bit count.
- ▶ **M** is the **mantissa** in IEEE-754 but the **number of memory blocks** in the cache formulas.
- ▶ **T** alone is the **clock cycle time** in CPU performance, but T_c and T_m are **memory access times** in cache. Different quantities, both in the same units.
- ▶ **e** is the **true** exponent and **E** the **stored** one. They differ by the bias, which is the whole point of the field.

When a question hands you a letter, name the topic first, then the meaning.

Number systems and integer representation

FORMULA	WHAT EACH SYMBOL IS	USE IT WHEN
$\text{value} = \sum d_i \times b^i$	b the base, d_i the digit in column i , i counted from 0 at the right	Converting any base into decimal
divide by b , read remainders bottom-up	b the target base; the remainders are the digits	Converting decimal into any base
group bits in fours from the right	each group of 4 bits is one hex digit; pad the leftmost group	Binary to or from hexadecimal
left shift = $\times 2$, right shift = $\div 2$	one place per doubling or halving	Fast multiply or divide by a power of two
unsigned: 0 to $2^n - 1$	n the total bits available	No sign bit is used
sign-magnitude and one's complement: $-(2^{n-1}-1)$ to $+(2^{n-1}-1)$	n the total bits including the sign bit; both have two zeros	Stating the range of these schemes
two's complement: -2^{n-1} to $+(2^{n-1}-1)$	n total bits; one zero, so one extra negative value	Stating the range, or spotting overflow
negate: invert every bit, add 1	applies to all n bits, leading zeros included	Making a two's complement negative
end-around carry	a carry out of the top is added back into the lowest bit	One's complement subtraction only
BCD: 4 bits per digit, weights 8 4 2 1 Excess-3 = digit + 3	each decimal digit coded separately, not the whole number	Coded-decimal questions
$\text{bits} = (w \times \text{dpi}) \times (h \times \text{dpi}) \times \text{bpp}$	w , h in inches; dpi the resolution; bpp the colour depth	Sizing an image, then $\div 8$, $\div 1024$, $\div 1024$

Floating point (IEEE-754)

FORMULA	WHAT EACH SYMBOL IS	USE IT WHEN
single = 1 + 8 + 23 bits, bias 127 double = 1 + 11 + 52 bits, bias 1023	sign S, stored exponent E, mantissa M	Laying out the format
$E = e + \text{bias}$	e the true exponent you found by normalising; E what you write	Going decimal to IEEE
$\text{value} = \pm 1.M \times 2^{(E - \text{bias})}$	the leading 1 is hidden and must be put back	Going IEEE to decimal

Instruction sets and addressing

FORMULA	WHAT EACH SYMBOL IS	USE IT WHEN
bytes per instruction = op-code bytes + (address fields $\times$ bytes per address)	address fields is 3, 2, 1 or 0, which names the format	Counting program size across formats
immediate: operand = the address field	no EA at all	The value is in the instruction
direct: EA = address field	one memory read	The field is the address
indirect: EA = M[address field]	two memory reads	The field points to the address
register indirect: EA = contents of register	the register is the pointer	One memory read
relative: EA = PC + address field	a negative offset branches backwards	Branches and loops
indexed: EA = index register + address field	stepping through an array	Indexed access

Processor performance

FORMULA	WHAT EACH SYMBOL IS	USE IT WHEN
$\text{CPU time} = \text{IC} \times \text{CPI} \times T$	IC instructions executed, CPI cycles each, T seconds per cycle	Any "how long does it take" question
$\text{CPU time} = (\text{IC} \times \text{CPI}) / \text{clock rate}$	same, when the clock is given as a rate not a period	The clock is in Hz or GHz
$\text{average CPI} = \sum (\text{fraction}_i \times \text{CPI}_i)$	fraction_i the share of the mix (they sum to 1)	An instruction mix table is given
$\text{MIPS} = \text{clock rate} / (\text{CPI} \times 10^6)$	millions of instructions per second	Asked for MIPS
$\text{speedup} = \text{time}(\text{old}) / \text{time}(\text{new})$ $\text{improvement} = (\text{old} - \text{new}) / \text{old}$	improvement is measured against the old time	"How much faster", "percentage improvement"
Amdahl: $\text{speedup} = 1 / ((1-f) + f/s)$	f the fraction sped up, s the factor it is sped up by	Only part of the program is improved
$\text{address space} = 2^k, k = \log_2 N$	k address lines, N locations; PC width equals k	Address lines, memory capacity, PC width

Pipelining

FORMULA	WHAT EACH SYMBOL IS	USE IT WHEN
$\text{cycle time} = \text{slowest stage} + \text{latch delay}$	the pipeline runs at the pace of its slowest stage	Before any other pipeline sum
$\text{pipelined time} = (k + n - 1) \times \text{cycle time}$	k stages, n tasks; k cycles to fill, then one per remaining task	Time for n instructions
$\text{non-pipelined time} = n \times (\text{sum of stage times})$	every task pays every stage in full	The comparison baseline
$\text{speedup} = \text{non-pipelined} / \text{pipelined}$	its ceiling is k, the number of stages	"How much does pipelining gain"
$\text{throughput} = n / \text{pipelined time}$	in the steady state this is 1 / cycle time	Tasks per second

Memory and cache

FORMULA	WHAT EACH SYMBOL IS	USE IT WHEN
$M = 2^n / K$	n address bits, K words per block, M resulting blocks	Counting blocks in main memory
miss ratio = $1 - h$	h the hit ratio as a decimal	Always, before an AMAT
AMAT (hierarchical) = $T_c + (1-h) \times T_m$	cache is checked first, so its time is always paid	Look-through, the usual default
AMAT (simultaneous) = $h \times T_c + (1-h) \times T_m$	cache and memory start together	Look-aside, when stated
two-level = $H \times T_1 + (1-H) \times (T_1 + T_2)$	a miss pays both levels	A fast and a slow level are given
lines = cache size / block size	a line holds exactly one block	First step of every mapping question
offset bits = $\log_2(\text{block size})$	picks the word or byte inside the block	Every mapping
index bits = $\log_2(\text{lines})$ sets = lines / k, set bits = $\log_2(\text{sets})$	k the associativity; fully associative has neither field	Direct and set-associative
tag bits = address bits – index (or set) – offset	whatever is left over confirms the block	Every mapping
direct: line = block number mod lines set-assoc: set = block number mod sets	which slot a block is allowed to occupy	"Which line does address X map to"
tag directory = lines $\times$ tag bits	answer in bits; divide by 8 for bytes	Cost of the tags themselves

The named laws, stated

A paper may ask you to **state** a law rather than use it, and a formula on its own does not answer that. Each of these is stated as a law should be: the claim in words, the conditions it holds under, the formula, and what it implies.

Amdahl's law. The gain in overall performance from improving one part of a system is limited by the fraction of the total time that part accounts for. If a fraction f of the execution time is made s times faster and the rest is unchanged, the overall speedup is $S = 1 / ((1 - f) + f/s)$; as s grows without bound, S tends to $1 / (1 - f)$, a ceiling no improvement to that part can pass.

The principle of locality of reference. Over any short interval of its execution, a program references only a small and predictable subset of its address space. It has two forms: **temporal** (a location used now is likely to be used again soon) and **spatial** (locations near one just used are likely to be used soon). It is the assumption the whole memory hierarchy rests on.

De Morgan's laws. The complement of a product is the sum of the complements, and the complement of a sum is the product of the complements: $(A \cdot B)' = A' + B'$ and $(A + B)' = A' \cdot B'$. Breaking a bar over a group flips the operator underneath it.

The complement law. A variable ORed with its complement is always 1, and ANDed with its complement is always 0: $A + A' = 1$, $A \cdot A' = 0$.

TEACH

WHICH FORMULA DOES THIS QUESTION WANT?

Exam wording maps onto formulas fairly reliably. Learn the tells:

IF THE QUESTION SAYS	REACH FOR
"address lines", "address space", "how wide is the PC"	2^k , $k = \log_2 N$
an instruction mix table with percentages	average CPI = $\sum f_i \text{CPI}_i$, then CPU time
"percentage improvement", "how much faster"	speedup and $(\text{old} - \text{new})/\text{old}$
"a fraction of the program", "enhanced portion"	Amdahl's law
"n instructions", "k stages", "throughput"	$(k + n - 1) \times \text{cycle time}$
"which block or line does this address map to"	split the address: tag / index / offset
"hit ratio", "average access time"	AMAT, and say which organisation you assumed
"represent $-X$ ", "state the range"	the signed-scheme ranges
a hex pattern and "IEEE-754"	bias and the hidden bit
inches and dpi	$(w \times \text{dpi})(h \times \text{dpi}) \times \text{bpp}$

RECALL AND CHECK

A question gives you $k = 4$ and $n = 100$. What are k and n , and which formula is this?

ANSWER

It depends on the topic, which is exactly the trap. If the question is about a **pipeline**, k is 4 **stages** and n is 100 **tasks**, so the time is $(4 + 100 - 1) = 103$ cycle times. If it is about a **cache**, $k = 4$ would be the **associativity** (4-way). If it is about the **address bus**, $k = 4$ lines would reach only $2^4 = 16$ locations. Name the topic before you name the letter.

REFERENCE · R.2

Exam Day Strategy

Knowing the material and scoring the material are different skills. This section is the second one: how to spend the two hours, which questions to take, and how to write an answer so that every mark it has earned is actually given.

By the end of this section you can budget the paper by the minute, choose your optional questions on evidence rather than instinct, structure an answer so partial credit is unavoidable, and recognise the time sinks that cost students the most.

The shape of the paper

The paper runs **2 hours** for **70 marks**. **Question 1 is compulsory and carries 30 marks**. You then **answer any two** of the remaining questions, each worth **20 marks**. Question 1 is the broad one, usually several short parts spread across the whole syllabus; the optional questions go deeper on fewer topics.

70 marks in 120 minutes is about **1.7 minutes per mark**. Do not spend it evenly; spend it deliberately:

MINUTES	SPEND IT ON	WHY
0 to 5	Read the whole paper. Mark every part you can definitely do.	You cannot choose your best two questions without seeing all of them.
5 to 55	Question 1 (30 marks, about 50 minutes).	Compulsory and broad, so it is where marks are most spread out and most certain.
55 to 85	First optional (20 marks, 30 minutes).	Take the stronger of your two choices first, while you are fresh.
85 to 115	Second optional (20 marks, 30 minutes).	Keep to the clock even if unfinished; a started part scores, a skipped one cannot.
115 to 120	Check: units, carries, and that you answered exactly two optionals.	The cheapest marks on the paper are the ones you already earned and lost.

If a part is costing more than double its mark value in minutes, leave a gap and move on. Come back only if time remains.

Choosing your two optional questions

Students pick the question whose *topic* feels familiar, then discover part (c) is the one they cannot do. Choose like this instead:

- ▶ For each candidate question, read **every part** and tally the marks you are **confident** of. Not the marks you might get: the ones you are sure of.
- ▶ Take the two questions with the highest confident tallies, even if the topics feel less comfortable.
- ▶ Prefer questions whose parts are **numeric with a stated method** (cache fields, CPI, IEEE, pipeline timing). These are self-checking, so you know when you are right.
- ▶ Beware a question with one huge part: a single 12-mark part you stall on is far more dangerous than four 5-mark parts.

Answer exactly two. Writing a third wastes minutes you needed elsewhere, and you cannot rely on the best two being the ones marked.

Writing an answer that collects its marks

● MUST MEMORISE

THE FOUR-LINE ANSWER

For **every** numeric part, write these four things, in this order. Each is worth marks on its own, even when the final number is wrong:

1. **The formula**, stated in symbols before any number goes into it.
2. **The substitution**, showing each number in the place of its symbol.
3. **The arithmetic**, in steps a marker can follow.
4. **The answer with its units**, and a sentence saying what it means.

Memory hook: formula, substitute, work, units. A wrong final number with the first three shown usually keeps most of the marks; a bare correct number with none of them can still lose some.

TEACH

MARKS ARE SCORING POINTS, SO COUNT THEM

The mark allocation tells you how many distinct things to say. Treat it as a checklist, not a suggestion:

- ▶ **2 marks, "differentiate X and Y"**: one sentence on each, ideally with the question each answers. Two points, two marks.
- ▶ **3 marks, "compare, include a diagram"**: two differences **and** the diagram. The diagram is a scoring point; students skip it under time pressure and lose it.
- ▶ **"State two aspects"**: write exactly two, clearly numbered. A vague paragraph containing both may be read as one.
- ▶ **"Explain" versus "state"**: "state" wants the fact, "explain" wants the fact **and** the reason. If the verb is "explain", write a "because".

When a part is worth more marks than you have points, you are missing something: look for an example, a diagram, or a consequence to add.

TEACH

WHEN YOU ARE STUCK

- ▶ **State your assumption and continue.** If a question is ambiguous (which AMAT organisation, whether the block is in words or bytes), write "assuming look-through" and solve it. A clearly stated assumption is usually accepted; a blank is never.
- ▶ **Do the parts out of order.** Nothing requires part (a) first. Bank (c) if (c) is the one you can do.
- ▶ **If a later part needs an earlier answer you could not get**, invent a plausible value, say "taking $X = \dots$ from (a)", and carry it through. Method marks still apply.
- ▶ **Convert everything to one unit first.** Most stalls in cache and performance questions are unit confusion, not ignorance: KB against bytes, ns against s, words against bytes.

⚠ TRAP

The four most expensive habits in this paper. (1) Answering **three** optional questions and running out of time on all of them. (2) Doing arithmetic in your head and writing only the final number, so a slip costs the whole part instead of one mark. (3) Skipping the **diagram** when a question says "include a diagram", which is a guaranteed mark left behind. (4) Giving a number with **no units**: 5.2 is not an answer, 5.2 ns is.

RECALL AND CHECK

You have 12 minutes left, one 8-mark part untouched, and a 4-mark part you rushed. What do you do?

ANSWER

Start the 8-mark part. Write its formula, substitute, and take the arithmetic as far as you can: an untouched part scores zero, while a started one collects method marks almost immediately. Only if it collapses should you return to polish the 4-mark part.

REFERENCE • R.3

Trap Index and Cram Sheet

Every trap in this manual, gathered in one place, and then the shortest possible list of things to be able to reproduce cold. This is the last thing to read before the exam.

By the end of this section you can name the classic error in each topic and the correction for it, and reproduce the whole formula spine from memory.

The trap index

Each of these cost marks on a real paper. The fix is in the right-hand column.

TOPIC	THE TRAP	THE FIX
Base conversion	Reading the remainders top-down	Read the remainder column bottom-up , then convert back to check
Binary to hex	Grouping the bits from the left	Group in fours from the right , pad the leftmost group with zeros
Signed numbers	Mixing up the two carry rules	Two's complement discards the final carry; one's complement adds it back (end-around)
Signed numbers	Forgetting the second zero	Sign-magnitude and one's complement have two zeros; two's complement has one , so it reaches one more negative
Negating	Inverting only the set bits	Invert all n bits, leading zeros included, then add 1
IEEE-754	Forgetting the bias, or the hidden 1	Subtract 127 when reading, add 127 when writing; a mantissa of 111... means 1.111..., not 0.111...
IEEE-754	Rounding a non-terminating fraction early	Keep multiplying by 2 until you have 23 mantissa bits, then stop
Addressing modes	Confusing indirect with register indirect	Indirect touches memory twice ; register indirect once ; immediate has no EA at all
Performance	Measuring improvement against the new time	Improvement is (old – new) / old
Amdahl	Dividing the whole program by s	Only f is divided by s; (1 – f) stays at full cost and caps the gain
Pipelining	Expecting forwarding to fix everything	Forwarding cannot fix the load-use hazard: exactly one stall remains. A multi-cycle execute also holds the EX stage
Cache	Ignoring the units of the block	"16 words" with a 4-byte word is 64 bytes ; check whether the address counts words or bytes
Cache	Taking the log of the lines for a set-associative cache	Divide lines by k to get sets first; the freed bits move into the tag
Architecture	Saying only "Harvard is faster"	Give the trade-off : Harvard is faster, Von Neumann is simpler, cheaper and more flexible
Notation	Assuming a letter means what it meant last topic	k , n , M and T each have two or three jobs: name the topic first (R.1)

The cram sheet

If you can reproduce this page from memory, you can attempt every numeric question on any past paper. Cover the right-hand side and recite.

ASK YOURSELF	THE ANSWER
Unsigned range in n bits	0 to $2^n - 1$
Two's complement range	-2^{n-1} to $+(2^{n-1} - 1)$
How to negate	Invert every bit, add 1
One's complement subtraction	Add, then end-around carry
IEEE single layout and bias	1, 8, 23 and bias 127 (double: 1, 11, 52, bias 1023)
IEEE decode	$\pm 1.M \times 2^{(E - 127)}$
Address space	2^k locations from k lines; PC is k bits wide
CPU time	$IC \times CPI \times T$
Average CPI	$\Sigma (\text{fraction}_i \times CPI_i)$
Improvement	$(\text{old} - \text{new}) / \text{old}$
Amdahl	$1 / ((1 - f) + f/s)$
Pipeline cycle time	Slowest stage + latch delay
Pipeline time for n tasks	$(k + n - 1) \times \text{cycle time}$
Cache lines	Cache size / block size
Address split	tag index (or set) offset, offset = $\log_2(\text{block})$
Tag bits	address bits – index (or set) – offset
AMAT, look-through	$T_c + (1 - h) \times T_m$
Tag directory	lines $\times$ tag bits (in bits)
Image size	$(w \times \text{dpi})(h \times \text{dpi}) \times \text{bpp}$ bits

THE LETTERS, ONE LAST TIME

n bits (but tasks in the pipeline row) · **k** address lines (but stages in the pipeline row, and ways in a cache) · **IC** instruction count · **CPI** cycles per instruction · **T** clock cycle time · **f** fraction sped up · **s** its speedup factor · **h** hit ratio · **T_c**, **T_m** cache and memory access times · **E** stored exponent, **M** mantissa · **w**, **h** inches and **bpp** bits per pixel in the image row

Memory hook: four families, in order: **represent** a number, **decode** a float, **time** a processor, **place** a block in cache. Every numeric question on the paper is one of those four.

FILL IN THE GAP

1. In $(k + n - 1)$, k is the number of _____ and n is the number of _____.
2. Improvement is measured against the _____ time.
3. Tag bits = address bits minus _____ minus _____.
4. The IEEE single-precision bias is _____, and the leading 1 is _____.
5. For a set-associative cache you divide the lines by _____ before taking the log.

ANSWERS

1. pipeline stages; tasks (instructions). 2. old. 3. index (or set); offset. 4. 127; hidden (implicit). 5. the associativity k .

PAST PAPERS

The Two Favourite Papers, Solved in Full

The 2024/25 and 2025/26 papers, start to finish, in the order you would meet them in the hall. Every part is answered, every number is recomputed, and every solution names the unit that teaches it. Sit each paper closed-book first, then mark yourself against these.

2024 / 2025

2025 / 2026

Q1 compulsory, 30 marks

any two more, 20 each

70 marks, 2 hours

every part answered

2024/2025 Solved in Full

The whole 2024/25 paper, question by question. Question One is compulsory and carries 30 marks; you then answer any two of Questions Two to Five, 20 marks each, for 70 marks in 2 hours.

Work each question yourself before reading the solution. Where a part is taught in full elsewhere in this manual, the unit is named so you can go straight back to it.

AS PRINTED

24/25 Q1(A)

THE EXAMINER'S WORDS, UNEDITED

a. Consider three (3) floating point numbers X, Y and Z stored in registers based on the IEEE 754 Single precision floating point format. $X = C1400000h$, $Y = 42100000h$, $Z = 41400000h$

Convert X, Y and Z to decimal numbers and determine which of the following is true. **9mks**

- i. $(Y-Z) > 0$
- ii. $X + Z = 0$
- iii. $Z = X + Y$

Read it again. The paper writes the values with a lower case **h** for hex, not **H**, and marks the part **9mks**, not "(9 marks)". The three claims are numbered **i. ii. iii.** and sit under the instruction, not inside it. None of that changes the arithmetic; all of it is what the page in front of you will actually look like.

BREAK IT DOWN

BEFORE YOU COMPUTE

- GIVEN** three 32-bit hex words, and the statement that they are IEEE 754 *single* precision. That fixes the split as 1 sign bit, 8 exponent bits, 23 mantissa bits, and the bias as 127.
- ASKED** two things, not one. First *convert* all three to decimal. Then *determine which of the following is true*. The second verb is the one that earns the marks: a script that converts correctly and stops has answered half the question.
- MARKS** 9 for the part, and there are three conversions plus three tests. Read that as roughly 2 marks per conversion and 1 per verdict, so the conversions carry the weight and must be shown, not asserted.
- TAUGHT IN** Module One Unit 3.
- TRAP** the question says "which of the following is true", singular in feel, and a reader in a hurry stops at the first true claim. Two of the three are true here. Test all three and say so for each.

- GIVEN** $X = C1400000H$, $Y = 42100000H$, $Z = 41400000H$ · IEEE-754 single precision · bias 127
- FIND** the decimal value of each, then the truth of the three statements
- FORMULA** $\text{value} = \pm 1.M \times 2^{(E - 127)}$

HEX	SIGN / E / E	$1.M \times 2^E$	DECIMAL
C1400000	1 / 130 / 3	$-1.1 \times 2^3 = -1100$	-12
42100000	0 / 132 / 5	$1.0001 \times 2^5 = 100100$	36
41400000	0 / 130 / 3	$1.1 \times 2^3 = 1100$	12

$X = -12$, $Y = 36$, $Z = 12$. (i) $Y - Z = 36 - 12 = 24 > 0$, so **TRUE**. (ii) $X + Z = -12 + 12 = 0$, so **TRUE**. (iii) $X + Y = -12 + 36 = 24$, and $Z = 12$, so **FALSE**.

THE EXAMINER'S WORDS, UNEDITED

- b. Determine the average memory access time for a machine with a cache hit rate of 80% and cache access time of 5 ns and main memory access time of 100 ns when:
- Simultaneous access memory organization is used. **2mks**
 - Hierarchical access memory organization is used. **2mks**
 - Draw the diagrams representing the two scenarios **3mks**

Read it again. The paper sets **iii.Draw** with no space after the numeral, and it splits the 7 marks 2, 2 and 3, so the two drawings are worth more than either calculation. It also names the two schemes "Simultaneous access memory organization" and "Hierarchical access memory organization", not parallel and serial, so answer in the paper's own words.

BREAK IT DOWN

BEFORE YOU COMPUTE

GIVEN	a cache hit rate of 80%, a cache access time of 5 ns and a main memory access time of 100 ns. Nothing else, so the miss rate 20% is yours to derive.
ASKED	<i>determine</i> the average access time twice, once under each organisation, and then <i>draw</i> both. Three separate deliverables under one part.
MARKS	2 + 2 + 3 = 7. The 3 sits on the diagrams, which is the largest single share, so a part answered with numbers only throws away nearly half of it.
TAUGHT IN	Module Three Unit 3, Cache Memory Design.
TRAP	using one formula for both. Simultaneous access starts cache and main memory together, so the cache time is not paid twice; hierarchical pays the cache time on every access and the memory time again on a miss. The answers differ by 1 ns, and a copied formula scores zero on one of the two.

GIVEN	$h = 0.80 \cdot T_c = 5 \text{ ns} \cdot T_m = 100 \text{ ns}$
FIND	AMAT under both organisations
FORMULA	simultaneous = $h \times T_c + (1-h) \times T_m$ · hierarchical = $T_c + (1-h) \times T_m$

Simultaneous = $0.8(5) + 0.2(100) = 4 + 20 = \mathbf{24 \text{ ns}}$. Hierarchical = $5 + 0.2(100) = 5 + 20 = \mathbf{25 \text{ ns}}$. Diagrams: simultaneous shows the CPU addressing cache and main memory in parallel; hierarchical shows the CPU reaching main memory only through the cache. Hierarchical is 1 ns slower here because the cache time is paid on every access, hit or miss.

THE EXAMINER'S WORDS, UNEDITED

c. Assume a computer has 24-bit addresses. Each block stores 16 words. A direct-mapped cache has 256 blocks. In which block (line) of the cache would you look for each of the following addresses? (Addresses are given in hexadecimal for convenience). **6mks**

- a. 1A2BC0
- b. FFFF00
- c. 123456
- d. C109D5

Read it again. The four addresses are labelled **a. b. c. d.**, which collide with Question One's own top-level parts a. b. c. d.: two different things on the page are both called "c.". The paper also closes with "(Addresses are given in hexadecimal for convenience)." and puts the full stop outside the bracket, and it never says whether it wants the line number in decimal or in hex.

BREAK IT DOWN

BEFORE YOU COMPUTE

- GIVEN** a 24-bit address, 16 words per block, a direct-mapped cache of 256 blocks, and four addresses written in hexadecimal.
- ASKED** in which block (line) each address lands. One number per address, four numbers in all.
- MARKS** 6 for four addresses, so roughly $1\frac{1}{2}$ each with the remainder for showing the field split once. Do the split once, then apply it four times.
- TAUGHT IN** Module Three Unit 3, Cache Memory Design.
- TRAP** counting bits when you could count hex digits. 4 offset bits are exactly one hex digit and 8 index bits are exactly two, so the line is the two digits before the last. Also give the answer in decimal and say so, since the paper leaves the base of the answer open.

- GIVEN** address = 24 bits · block = 16 words · cache = 256 blocks · mapping = direct
- FIND** the cache block (line) for each address
- FORMULA** offset = $\log_2 16 = 4$ bits · index = $\log_2 256 = 8$ bits · tag = $24 - 8 - 4 = 12$ bits · the index field is the line number

READING IT OFF THE HEX

4 offset bits are exactly **one hex digit**, and 8 index bits are exactly **two**. So in a 6-digit address, drop the last digit and the next two digits are the line.

ADDRESS	OFFSET (LAST DIGIT)	LINE (NEXT TWO DIGITS)	LINE IN DECIMAL
1A2BC0	0	BC	188
FFFF00	0	F0	240
123456	6	45	69
C109D5	5	9D	157

Blocks: 1A2BC0 → **188**, FFFF00 → **240**, 123456 → **69**, C109D5 → **157**. Address format: tag 12, line 8, offset 4.

AS PRINTED

24/25 Q1(D)

THE EXAMINER'S WORDS, UNEDITED

d. Use a Zero, One, Two and Three address instruction formats to write a Pentium 4 assembly language program segment for the expression below. For each format determine the number of bytes required. **8mks**

$X = ((A * B) + C/D)/(F * G + J * K)$

Read it again. The expression is printed $X = ((A * B) + C/D)/(F * G + J * K)$, with a star for multiply and no space after the plus. The paper names the machine, a Pentium 4, so the answer is x86 style assembly, not pseudocode. The same question reappears as 25/26 Q4(b), there with its byte-count sentence struck through in pen.

BREAK IT DOWN

BEFORE YOU COMPUTE

- GIVEN** one arithmetic expression and four instruction formats to write it in. The paper does not state the opcode or address field sizes, so you state them yourself: 1 byte per opcode and 2 bytes per address is the usual convention and the one used here.
- ASKED** two things joined by "For each format": *write* the program segment, and *determine* the byte count. Four programs and four totals.
- MARKS** 8, which is 2 per format. A format with a listing and no byte count is half answered.
- TAUGHT IN** Module One Unit 4, Instruction Set Architecture.
- TRAP** assuming fewer address fields always means fewer bytes. It does not: 1-address needs 51 bytes against 2-address at 60 but 3-address at only 49, because shortening each instruction lengthens the program. Count instructions first, then multiply.

- GIVEN** the expression · opcode = 1 byte · each address = 2 bytes
- FIND** the four programs and their byte totals
- FORMULA** bytes per instruction = opcode bytes + (address fields × bytes per address); total = count × that

FORMAT	INSTRUCTIONS	BYTES
3-address	7	$7 \times (1 + 3 \times 2) = 7 \times 7 = 49$
2-address	12	$12 \times (1 + 2 \times 2) = 12 \times 5 = 60$
1-address	17	$17 \times (1 + 1 \times 2) = 17 \times 3 = 51$
0-address	16	$8 \text{ pushes} \times 3 + 1 \text{ pop} \times 3 + 7 \text{ ops} \times 1 = 34$

The full listings are worked in Module One Unit 4. Totals: **49, 60, 51 and 34 bytes** for 3, 2, 1 and 0 address respectively. Fewer address fields shortens each instruction but needs more of them, which is why the totals stay close.

AS PRINTED

24/25 Q2(A)

THE EXAMINER'S WORDS, UNEDITED

a. i) Describe how cache memory improves the performance of a computer system. **2mks**

ii) Using the specifications below, design a cache system for each of Direct, Fully Associative and 2-way Set Associative Mapping.

iii) State one advantage and one disadvantage of each method. **8mks**
specifications:

- Cache size: 1 KB = 1024 bytes
- Block size: 16 bytes
- Main memory size: 64 KB = 65,536 bytes
- Address size: 16 bits

Read it again. Only two mark annotations are printed, **2mks** against (i) and **8mks** against (iii), so the 8 has to cover (ii) and (iii) together and the design work is not separately priced. The specifications are printed *below* all three instructions, so a reader working top to bottom meets "design a cache system" before meeting a single number.

BREAK IT DOWN

BEFORE YOU COMPUTE

GIVEN	cache 1 KB, block 16 bytes, main memory 64 KB, address 16 bits, and three mapping schemes to apply them to.
ASKED	three verbs. <i>Describe</i> how cache improves performance, <i>design</i> the cache under each of three mappings, and <i>state</i> one advantage and one disadvantage of each. Nine separate statements in the last part alone.
MARKS	2 + 8 = 10, half of Question Two. The design and the trade offs share 8, so budget roughly 5 for the three address formats and 3 for the six verdicts.
TAUGHT IN	Module Three Unit 3, Cache Memory Design.
TRAP	recomputing the line count for each mapping. It is 1 KB / 16 B = 64 lines under all three; only the way those lines are addressed changes. The other trap is answering "design" with prose: the mark is for a tag, index and offset split in bits.

GIVEN	cache = 1 KB · block = 16 bytes · main memory = 64 KB · address = 16 bits
FIND	the address format under each of the three mappings, plus one advantage and one disadvantage each
FORMULA	lines = cache / block · offset = $\log_2(\text{block})$ · sets = lines / k · tag = address – index (or set) – offset · tag directory = lines × tag bits

WHAT EACH SYMBOL MEANS

lines = 1 KB / 16 B = **64**, the same under all three mappings · **offset** = $\log_2 16$ = **4** bits · **k** = the ways per set, 2 for the 2-way case · 64 KB = 2^{16} , which is why the address is 16 bits

MAPPING	TAG	INDEX / SET	OFFSET	TAG DIRECTORY
Direct	6	6 (index, 64 lines)	4	$64 \times 6 = 384 \text{ bits} = 48 \text{ bytes}$
Fully associative	12	none	4	$64 \times 12 = 768 \text{ bits} = 96 \text{ bytes}$
2-way set assoc.	7	5 (set, 32 sets)	4	$64 \times 7 = 448 \text{ bits} = 56 \text{ bytes}$

Direct: simplest and fastest lookup (one line to check, one comparator), but two blocks mapping to the same line evict each other, causing conflict misses. **Fully associative:** no conflict misses, any block may sit anywhere, but every line's tag must be compared at once, which is expensive and slow, and the tag directory doubles to 96 bytes. **2-way:** the compromise, most conflict misses removed for only two comparators and a modest tag growth to 56 bytes.

AS PRINTED

24/25 Q2(B)

THE EXAMINER'S WORDS, UNEDITED

- b. i) Explain the following terms: **4½mks**
1. Locality of reference
 2. Temporal locality
 3. Special Locality
- ii) What happens when there is a Cache Miss? **1½ mks**

Read it again. Term 3 is printed **Special Locality**. The word the examiner wants is *spatial*, and the same setter misspelled it differently on the 23/24 paper, so this is a habit and not a one off. Note also that the marks are set inconsistently: **4½mks** closed up, **1½ mks** with a space.

BREAK IT DOWN

BEFORE YOU COMPUTE

GIVEN	nothing numeric. This is a definitions part, four terms in all if you count the cache miss.
ASKED	<i>explain</i> , not merely name, three terms, then say what <i>happens</i> on a cache miss. The second verb wants a sequence of events, not a definition.
MARKS	$4\frac{1}{2} + 1\frac{1}{2} = 6$, so $1\frac{1}{2}$ per term in (i). That is two or three sentences each, with an example, not one line.
TAUGHT IN	Module Three Unit 1, Computer Memory Overview.
TRAP	correcting the paper. Answer the third term as spatial locality, but keep the examiner's numbering and do not waste a line pointing out the spelling. The other trap is defining temporal and spatial without first defining locality of reference, which is the term the other two are special cases of.

(i) Locality of reference is the tendency of a program to access a small, predictable set of memory locations during any short period. **Temporal:** a location used now is likely to be used again soon, so cache keeps recently used items. **Spatial:** locations near one just used are likely to be used soon, so cache fetches a whole block of neighbours at once. (ii) On a miss the word is not in cache, so the address goes onto the system bus, main memory returns the **whole block** containing it, that block is loaded into a cache line (evicting one if necessary), and the required word is delivered to the CPU at the same time.

THE EXAMINER'S WORDS, UNEDITED

- c. Provide an assembly language program to add two numbers and store the result. **4mks**

Read it again. Fourteen words for 4 marks, and no machine is named. The paper does not say one address, two address or three address, and it does not name a processor, so any consistent instruction set earns the marks provided you say which one you are writing for.

BREAK IT DOWN

BEFORE YOU COMPUTE

GIVEN	nothing at all. Two numbers exist only as names you choose.
ASKED	<i>provide</i> a program: three operations, load, add, store. "store the result" is a separate demand from "add", so an answer that ends at ADD is incomplete.
MARKS	4, which for three instructions means roughly one per instruction plus one for the assumption you state.
TAUGHT IN	Module One Unit 4, Instruction Set Architecture.
TRAP	leaving the machine unstated. Write the accumulator assumption on the line above the code and the same three lines become unambiguous. Comment each line: comments cost nothing and show the marker you know what the accumulator holds.

On a one-address (accumulator) machine:

LOAD A ; accumulator = A

ADD B ; accumulator = A + B

STORE C ; C = A + B

On a three-address machine it is the single instruction ADD C, A, B. Either scores full marks provided you state which machine you assume.

AS PRINTED

24/25 Q3(A)

THE EXAMINER'S WORDS, UNEDITED

- a. i) What do you understand by Von Neumann Concept? **1mk**
 ii) Mention any 2 significant aspects of Von Neumann Architecture. **2mks**
 iii) Describe the Von Neumann architecture bottleneck. **2mks**

Read it again. Part (i) is annotated **1mk**, singular, where the rest of the paper writes "mks". The paper capitalises "Von Neumann Concept" as a proper noun in (i) but writes "Von Neumann architecture bottleneck" in lower case in (iii), and it asks for "any 2" aspects with a numeral rather than a word.

BREAK IT DOWN

BEFORE YOU COMPUTE

GIVEN	nothing but the name of the architecture. This is a recall part from end to end.
ASKED	three different verbs on the same topic: what you <i>understand</i> by the concept, <i>mention</i> two aspects, and <i>describe</i> the bottleneck. Mention wants a list, describe wants a mechanism.
MARKS	1 + 2 + 2 = 5. The single mark on (i) tells you one sentence is enough there; the two marks on (iii) tell you a bare phrase is not.
TAUGHT IN	Module One Unit 1, Architecture versus Organization.
TRAP	describing the bottleneck as "the CPU is faster than memory". That is a different problem. The bottleneck named here is the single shared path that instructions and data must both travel, so no instruction fetch and data access can happen in the same instant.

The Von Neumann concept: a computer stores its program (instructions) and its data together in the **same** memory, and executes instructions one after another by fetching them from that memory. Two aspects: (a) a single shared memory holds both instructions and data; (b) instructions execute sequentially, one at a time, under the stored-program idea. The bottleneck: because one bus is shared between instructions and data, the CPU cannot fetch an instruction and access data at the same instant, so that single path limits overall speed however fast the CPU is.

THE EXAMINER'S WORDS, UNEDITED

- b. i) Complete table 1. Show all necessary work done to arrive at the answer. Use 12 bits. **6mks**

Value	Unsigned	S-M	1's Complement	2's Complement
204				
-139				
1101 00110 11				

- ii) Calculate the memory requirement of an image whose dimension is 17.12 inches by 9.45 inches at resolution 250 dpi. Express your answer in Megabyte. **3mks**

Read it again. The instruction says **Use 12 bits**, but the third row of the table, **1101 00110 11**, is only 11 bits long, and it is grouped 4, 5, 2 rather than in fours. The paper also calls it "table 1" although no caption prints anywhere on the page, and it never says which scheme the third row is already written in, so the row wants reading four ways. Decide in the hall and write your decision down.

BREAK IT DOWN

BEFORE YOU COMPUTE

- GIVEN** two decimal values, 204 and -139, one bit pattern, four representation schemes and a width of 12 bits.
- ASKED** *complete* the table and "Show all necessary work done to arrive at the answer", which is an explicit instruction that a filled grid with no working will not score. Part (ii) then switches subject entirely to an image size calculation.
- MARKS** $6 + 3 = 9$. The 6 covers eight or more cells plus the working, so a cell is worth well under a mark: speed matters more than elegance here.
- TAUGHT IN** Module One Unit 2, Data Representation.
- TRAP** writing "not representable" and moving on. Unsigned genuinely cannot hold -139, and saying so earns the mark, but every other cell must still be filled. The second trap is the 11-bit row: pad it to 12 on the left, state that you have done so, and read it in all four schemes.

- GIVEN** values 204 and $-139 \cdot n = 12 \text{ bits} \cdot \text{four schemes}$
- FIND** each value in each scheme
- FORMULA** sign-magnitude = sign bit + magnitude · one's complement = invert all n bits · two's complement = invert then add 1

VALUE	UNSIGNED	SIGN-MAGNITUDE	ONE'S COMPLEMENT	TWO'S COMPLEMENT
204	0000 1100 1100	0000 1100 1100	0000 1100 1100	0000 1100 1100
-139	not representable	1000 1000 1011	1111 0111 0100	1111 0111 0101

Reading 1111 0111 0101 back as two's complement: the leading 1 says negative, so invert and add 1, giving 0000 1000 1011 = 139, so the value is **-139**. The recipe

reverses itself, which is the check to write down.

AS PRINTED

24/25 Q3(B)(II)

THE EXAMINER'S WORDS, UNEDITED

ii) Calculate the memory requirement of an image whose dimension is 17.12 inches by 9.45 inches at resolution 250 dpi. Express your answer in Megabyte. **3mks**

Read it again. This half of part (b) has nothing to do with the table above it: a data representation question turns into an image size calculation mid part. The paper says "Express your answer in Megabyte", singular, and gives no bit depth, so the pixel size is yours to assume and to declare. It reappears untouched as 25/26 Q4(c).

BREAK IT DOWN

BEFORE YOU COMPUTE

GIVEN	17.12 inches by 9.45 inches at 250 dots per inch. No colour depth, which is the number you need and are not given.
ASKED	<i>calculate</i> the memory requirement and <i>express</i> it in megabytes. Two instructions, and the second one is a unit conversion that carries its own mark.
MARKS	3. One for the pixel count, one for the conversion to bytes, one for reaching megabytes.
TAUGHT IN	Module One Unit 2, Data Representation.
TRAP	the silent choice of 1 MB. Dividing by 1024 twice gives about 1.21 MB; dividing by one million gives about 1.26 MB. Both are defensible, neither is safe unstated, so name your convention on the page.

GIVEN	17.12 in × 9.45 in · 250 dpi · 1 bit per pixel (black and white, not stated so assumed)
FIND	the memory in MB
FORMULA	bits = (w×dpi)(h×dpi)×bpp, then ÷8, ÷1024, ÷1024

$4280 \times 2362.5 = 10\,111\,500$ pixels × 1 bit = 10 111 500 bits = 1 263 937.5 bytes ≈ **1.21 MB** (dividing by 1024 twice). State your convention: dividing by one million instead gives about 1.26 MB.

AS PRINTED

24/25 Q3(C)

THE EXAMINER'S WORDS, UNEDITED

c. Represent the following -1240.56 using Single Precision IEEE format for floating point numbers. **6mks**

Read it again. The paper writes "Represent the following -1240.56" with the sign closed up against the digits and no colon between the instruction and the number. Unlike Q5(c) on the same paper it gives *no* truncation rule, so the mantissa runs to its full 23 bits here. It is reprinted word for word as 25/26 Q5(a).

GIVEN	one decimal value, -1240.56, and the target format, IEEE single precision.
ASKED	<i>represent</i> it. The verb wants the 32-bit pattern, and by convention the hexadecimal equivalent alongside it.
MARKS	6, which is the whole of a five part question's third part. Expect a mark each for the sign, the integer part in binary, the fraction in binary, the normalisation, the biased exponent and the assembled pattern.
TAUGHT IN	Module One Unit 3, Floating Point Representation.
TRAP	the fraction. 0.56 never terminates in binary, so it must be expanded by repeated doubling until the mantissa is full, and then rounded or truncated. Say which you did: the last hex digit changes.

Sign 1. 1240 = 100 1101 1000 and 0.56 = 0.10001111010111... Normalising gives $1.001101100010001111010111... \times 2^{10}$, so $E = 10 + 127 = 137 = 1000\ 1001$, and the first 23 mantissa bits are 0011 0110 0010 0011 1101 011. Pattern 1 10001001 00110110001000111101011 = **C49B11EBH** (truncated). Rounding the last bit instead gives C49B11ECH, which is what hardware stores; either is accepted if you state the method.

AS PRINTED

24/25 Q4(A)

THE EXAMINER'S WORDS, UNEDITED

- a. i) Compare Von Neumann Architecture and Harvard Architecture. Include diagram for each. **3mk**
- ii) Mention any four (4) clear differences between RISC and CISC. Give two (2) examples of processors in each category. **4mks**

Read it again. Part (i) is annotated **3mk**, singular, the only such annotation on the paper. It also says "Include diagram for each" with no article. Part (ii) asks for "any four (4) *clear* differences", so four vague ones will not do, and the two example processors per family are a separate demand inside the same 4 marks.

BREAK IT DOWN

BEFORE YOU COMPUTE

- GIVEN** four architecture names and nothing numeric.
- ASKED** *compare* two architectures with diagrams, then *mention* four differences and *give* four example processors. Three verbs, and the diagrams are compulsory, not decoration.
- MARKS** $3 + 4 = 7$. In part (ii) that is four differences plus four processors for 4 marks, so half a mark a line: write in a two column table, not prose.
- TAUGHT IN** Module One Unit 1 for the architectures, Module Four Unit 1 for RISC and CISC.
- TRAP** forgetting the diagrams. They are explicitly asked for and carry their own marks, and a compare answered as prose alone cannot score full marks however good the prose is. The second trap is naming processors nobody can check: ARM and MIPS against x86 and 68000 are safe.

(i) Von Neumann uses **one** memory and **one** bus for instructions and data, so it is simpler and cheaper but the shared bus carries only one at a time. Harvard uses **separate** memories and buses, so an instruction fetch and a data access happen simultaneously: faster, but more hardware and less flexible. Draw both diagrams; they carry marks of their own.

RISC	CISC
Few, simple, fixed-length instructions	Many, complex, variable-length instructions
Mostly one cycle per instruction	An instruction may take many cycles
Load/store: only these touch memory	Arithmetic may operate directly on memory
Many registers, hardwired control, easy to pipeline	Fewer registers, microcoded control, harder to pipeline

Examples: RISC, ARM and MIPS (also SPARC, RISC-V). CISC, Intel x86 and Motorola 68000 (also VAX).

THE EXAMINER'S WORDS, UNEDITED

- b. i) Describe End-Around Carry in 1's Complement? Illustrate this using $153 - 142$ in a 10-bit register. **4mks**
 ii) What do you understand by Overflow and Underflow in data representation? Give one example for each. **3mks**

Read it again. Part (i) is written as a command and then ends in a **question mark**. The dash between the two numbers is an en dash on the page, measurably longer than the hyphen in "10-bit" on the same line, and the paper writes "1's Complement" with the digit and the apostrophe.

BREAK IT DOWN

BEFORE YOU COMPUTE

GIVEN	153 and 142, a 10-bit register, and one's complement arithmetic. Part (ii) is given nothing at all.
ASKED	<i>describe</i> end-around carry and then <i>illustrate</i> it, so a worked subtraction with no explanation, or an explanation with no subtraction, each lose half. Part (ii) wants two definitions and one example each, four items.
MARKS	$4 + 3 = 7$. The example in (ii) is worth as much as the definition, so never skip it.
TAUGHT IN	Module One Unit 2, Data Representation.
TRAP	discarding the carry. In one's complement the carry out of the top bit is added back into the lowest bit, and that end-around step is the entire point of the question; in two's complement the same carry is thrown away. Mixing the two rules gives an answer one short.

GIVEN	$153 - 142 \cdot n = 10 \text{ bits} \cdot \text{one's complement}$
FIND	the difference as a pattern and in decimal
FORMULA	$a - b = a + (-b)$ with $-b$ the bitwise inversion; a carry out of the top is added back into the lowest bit

$153 = 00\ 1001\ 1001$; $+142 = 00\ 1000\ 1110$ so $-142 = 11\ 0111\ 0001$. Adding: $00\ 1001\ 1001 + 11\ 0111\ 0001 = 1\ 00\ 0000\ 1010$, with a carry out. End-around: $00\ 0000\ 1010 + 1 = 00\ 0000\ 1011 = 11$, and $153 - 142 = 11$. Correct. (ii) **Overflow** is a result too large for the available bits (in 8-bit two's complement, $100 + 50 = 150$ overflows the $+127$ limit). **Underflow** is a result too small to represent: for integers more negative than the smallest storable value ($-100 - 50 = -150$ below -128), and for floating point a value so close to zero it rounds away to nothing.

THE EXAMINER'S WORDS, UNEDITED

- c. By using the reverse of the IEEE Single Precision procedure, determine the value (Base 10) of the following hexadecimal (base 16) value: C4F2E000h **6mks**

Read it again. The value is printed **C4F2E000h** with a lower case h. The phrase "By using the reverse of the IEEE Single Precision procedure" names the method you must show, so a correct number with no procedure is not a full answer. It is reprinted word for word as 25/26 Q5(d).

BREAK IT DOWN

BEFORE YOU COMPUTE

GIVEN	one 32-bit hexadecimal word and the statement that it is IEEE single precision.
ASKED	<i>determine</i> the base 10 value, by running the conversion procedure backwards. The words "Base 10" and "base 16" are spelled out so that no answer in binary or hex will do.
MARKS	6. Roughly one each for the sign, the exponent field, the unbiased exponent, restoring the hidden 1, the shifted binary value and the decimal total.
TAUGHT IN	Module One Unit 3, Floating Point Representation.
TRAP	forgetting the hidden bit. The stored mantissa is the fraction after an implied leading 1, so 1.11100101110 is what you shift, not 0.11100101110. Miss it and the answer halves.

C4F2E000 = 1100 0100 1111 0010 1110 0000 0000 0000. Sign 1, so negative. E = 1000 1001 = 137, so e = 137 - 127 = 10. Restoring the hidden 1 gives 1.11100101110 × 2¹⁰ = 11110010111₂ = 1024+512+256+128+16+4+2+1 = 1943. So the value is **-1943**.

AS PRINTED

24/25 Q5(A)

THE EXAMINER'S WORDS, UNEDITED

- a. i) Describe three (3) categories of Instructions that we have. Give two (2) examples for each. **4½mks**
- ii) Discuss the characteristics of computer memory based on Access Methods. Give one example for each method. **4mks**

Read it again. Part (i) capitalises "Instructions" mid sentence and asks for "three (3) categories ... that we have", the setter's phrasing rather than a textbook one. Part (ii) says "based on Access Methods", which pins the classification: not by technology, not by position in the hierarchy, by how a location is reached.

BREAK IT DOWN

BEFORE YOU COMPUTE

GIVEN	nothing numeric. Two recall parts.
ASKED	<i>describe</i> three instruction categories and <i>give</i> two examples of each, six examples in all, then <i>discuss</i> memory by access method with <i>one</i> example each.
MARKS	$4\frac{1}{2} + 4 = 8\frac{1}{2}$, the largest recall block on the paper. That is $1\frac{1}{2}$ per instruction category, so a name with two examples and a sentence of description.
TAUGHT IN	Module One Unit 4 for the instruction categories, Module Three Unit 1 for the access methods.
TRAP	answering (ii) with RAM, ROM and cache. Those are technologies. The access methods are sequential, direct, random and associative, and each wants its own example device.

(i) **Data transfer:** LOAD, STORE (also MOVE, PUSH, POP). **Arithmetic and logic:** ADD, AND (also SUB, MUL, OR, NOT, shifts). **Control (test and branch):** JUMP, BEQ (also CALL, RETURN). (ii) **Sequential:** read in order from the start, access time depends on position, as in magnetic tape. **Direct:** jump near the item then search a small region, as on a disk. **Random:** every location takes the same time via its address, as in RAM. **Associative:** located by content rather than address, all locations compared at once, as in a cache tag search.

THE EXAMINER'S WORDS, UNEDITED

- b. Consider a pipeline having 4 phases with duration 60, 50, 90 and 80 ns. Given latch delay is 10 ns. Calculate-
- Pipeline cycle time **1mk**
 - Non-pipeline execution time **1mk**
 - Speed up ratio **1mk**
 - Pipeline time for 500 tasks **1mk**
 - Sequential time for 500 tasks **1mk**
 - Throughput **1mk**

Read it again. The stem ends "Calculate-" with a trailing hyphen and no colon, and every one of the six items is annotated **1mk**, singular. Six equal marks means no single item is worth a long derivation. The same question reappears as 25/26 Q5(b) with item vi. Throughput dropped.

BREAK IT DOWN

BEFORE YOU COMPUTE

- GIVEN** four stage durations, 60, 50, 90 and 80 ns, a latch delay of 10 ns, and 500 tasks.
- ASKED** six numbers, in the paper's order. Nothing here asks for explanation, so every mark is a figure with its unit.
- MARKS** 6, one per item. Answer all six even if one is guessed: an omitted line cannot score, a wrong line costs only itself.
- TAUGHT IN** Module Two Unit 3, Pipelining.
- TRAP** leaving the latch out of the cycle time. The cycle is the slowest stage *plus* the latch, $90 + 10 = 100$ ns, and every later item depends on it, so one omission poisons five answers. The second trap is the $k + n - 1$ formula: 500 tasks through 4 stages take 503 cycles, not 504.

- GIVEN** $k = 4$ stages · 60, 50, 90, 80 ns · latch = 10 ns · $n = 500$ tasks
- FIND** cycle time, one-task non-pipelined time, speedup, 500-task pipelined and sequential times, throughput
- FORMULA** cycle = slowest + latch · pipelined = $(k + n - 1) \times$ cycle · sequential = $n \times \Sigma \text{stages}$ · throughput = $n / \text{pipelined time}$

Cycle = $90 + 10 = 100$ ns. Non-pipelined one task = $60+50+90+80 = 280$ ns. Ideal speedup = $280/100 = 2.8$. For 500 tasks: pipelined = $(4 + 499) \times 100 = 50\ 300$ ns; sequential = $500 \times 280 = 140\ 000$ ns. Throughput = $500 / 50\ 300$ ns, which in the steady state is one task per cycle, that is $1/100$ ns = 10 million tasks per second.

THE EXAMINER'S WORDS, UNEDITED

- c. Perform the calculation below in using Single Precision IEEE format. All bits after the first 12 bits of the Mantissa should be 0. -421.55 (all operations must be done in binary). **5½mks**

Read it again. The stem reads "Perform the calculation below **in using** Single Precision IEEE format", which is the paper's own wording, and there is no calculation to perform: it is a conversion. The instruction that matters is the one about the mantissa, and the bracket "(all operations must be done in binary)" rules out working the fraction in decimal and converting at the end.

BREAK IT DOWN

BEFORE YOU COMPUTE

GIVEN	the value -421.55, the single precision format, and a truncation rule: everything after the first 12 mantissa bits is zero.
ASKED	the 32-bit pattern, in binary, with the mantissa truncated as instructed. The truncation is not a simplification you are being offered, it is a requirement you are being marked on.
MARKS	5½. Sign, integer part, fraction, normalisation, exponent field and the truncated assembly, so about a mark each with a half for the unit of the answer.
TAUGHT IN	Module One Unit 3, Floating Point Representation.
TRAP	truncating at the wrong place. "the first 12 bits of the Mantissa" means the 23-bit stored fraction, not the 24 bits including the hidden 1, so you keep 12 fraction bits and set the remaining 11 to zero. Say what the truncated pattern now reads back as: that is what the question is testing.

GIVEN	value = -421.55 · IEEE-754 single precision · keep only the first 12 mantissa bits, rest zero
FIND	the full 32-bit pattern in binary
FORMULA	normalise to $1.m \times 2^e$, $E = e + 127$, then truncate the mantissa as instructed

- 1** Sign: negative, so **1**.
- 2** $421 = 1\ 1010\ 0101$, and $0.55 = 0.100011001100\dots$ so $421.55 = 110100101.100011001100\dots$
- 3** Normalise: $1.10100101100011001100110 \times 2^8$, so $e = 8$.
- 4** Exponent field: $8 + 127 = 135 = \mathbf{1000\ 0111}$.
- 5** Mantissa, first 12 bits: $1010\ 0101\ 1000$, then eleven zeros: **1010 0101 1000 0000 0000 000**.

$-421.55 \approx \mathbf{1\ 10000111\ 101001011000000000000000} = \mathbf{C3D2C000H}$. Truncating at 12 mantissa bits loses precision: this pattern reads back as -421.5 exactly, against the untruncated C3D2C666H. Say so, because the question is testing that you know truncation costs accuracy.

2025/2026 Solved in Full

The newest and richest paper, question by question. Same shape: Question One compulsory for 30 marks, then any two of Questions Two to Five at 20 marks each.

This paper leans harder on performance, pipelining and cache than any before it, and Question One alone spans five topics. Treat it as the best single prediction of what you will meet.

AS PRINTED

25/26 Q1(A)

THE EXAMINER'S WORDS, UNEDITED

a. i) Computer Systems are designed using two closely related but distinct concepts which are Computer Architecture and Computer Organization. Differentiate between these concepts.

2mks

ii) Compare Von Neumann Architecture and Harvard Architecture. Include a diagram for each.

3mks

Read it again. The mark annotations are set on their own line under each part rather than at the end of it, so the 2mks belongs to (i) and the 3mks to (ii). Part (ii) is 24/25 Q4(a)(i) again, this time reading "a diagram for each" where the older paper read "diagram for each". The rest is identical.

BREAK IT DOWN

BEFORE YOU COMPUTE

GIVEN	nothing numeric. The stem itself tells you the two concepts are "closely related but distinct", which is the shape your answer must take.
ASKED	<i>differentiate</i> , then <i>compare</i> with diagrams. Differentiate wants the contrast made explicit, so a paragraph on each with no comparison drawn between them does not answer it.
MARKS	$2 + 3 = 5$. The 3 on (ii) covers two diagrams and the comparison, so the drawings are about a mark of it.
TAUGHT IN	Module One Unit 1, Architecture versus Organization.
TRAP	answering with definitions and no example. One concrete pair, such as the instruction set saying a multiply exists while the hardware decides whether a multiplier or repeated addition performs it, is worth more than two abstract sentences.

(i) Architecture is the set of attributes visible to the programmer (instruction set, registers, addressing modes): it defines **what** the machine does. Organization is how those attributes are implemented in hardware (control signals, buses, memory technology): it defines **how**. Example: architecture says a MULTIPLY instruction exists; organization decides whether a dedicated multiplier or repeated addition performs it. (ii) Von Neumann: one memory, one bus, simpler and cheaper, bus is a bottleneck. Harvard: separate memories and buses, simultaneous fetch and data access, faster but costlier. Both diagrams are worth marks; draw them.

THE EXAMINER'S WORDS, UNEDITED

b. A computer manufacturer is evaluating whether to replace a non-pipelined processor with a five-stage pipelined processor. The processor executes the following instruction mix: Arithmetic – 40%, Memory Access – 35%, Branch – 20%, Floating Point – 5%. The Non-pipelined processor clock cycle is 25ns. Pipelined processor stage times are: IF – 5ns, ID – 4ns, EX – 6ns, MEM – 5ns and WB – 4ns. Furthermore, Pipeline register overhead is 1 ns. Branch instructions incur a two-cycle penalty, Floating-point instructions has three cycles for EX stage, Memory accesses experience cache misses 10% of the time, adding four extra cycles. The program contains 20 million instructions.

- i) Determine the pipelined clock cycle. **1mk**
- ii) Calculate the actual CPI. **2mks**
- iii) Determine the actual execution time. **1mk**
- iv) Compute the speedup over the non-pipelined processor. **1mk**
- v) Explain why the speedup differs from the theoretical maximum. **1mk**

Read it again. The paper uses a spaced dash to mean "equals" throughout the instruction mix and the stage times, so the dash after "Arithmetic" is not a minus sign. "Floating-point instructions has three cycles" is the paper's grammar. Every penalty is buried in a prose sentence *after* the stage times rather than tabulated, which is where a rushed reader loses them.

BREAK IT DOWN

BEFORE YOU COMPUTE

GIVEN	five stage times, a register overhead, an instruction mix in four classes, penalties for three of those classes, a non-pipelined clock and an instruction count. Nine numbers, and the mix percentages sum to 100.
ASKED	five things, four numeric and one written. Part (v) asks you to <i>explain</i> , so it cannot be answered with a figure.
MARKS	$1 + 2 + 1 + 1 + 1 = 6$. The CPI carries 2, twice any other item, because every later answer depends on it.
TAUGHT IN	Module Two Unit 2, CPU Performance Metrics.
TRAP	taking the clock from the sum of the stages. The pipelined clock is the <i>slowest</i> stage plus the overhead, $6 + 1 = 7$ ns, not $24 + 1$. The second trap is the memory penalty: only 10% of memory accesses miss, so the four extra cycles are weighted by 0.10 before they are weighted by the 35%.

GIVEN	stages 5, 4, 6, 5, 4 ns · overhead 1 ns · IC = 20 000 000 · the mix above · non-pipelined clock 25 ns
FIND	clock period, actual CPI, execution time, speedup
FORMULA	clock = slowest stage + overhead · actual CPI = ideal + $\Sigma(\text{fraction} \times \text{penalty})$ · time = IC × CPI × T

Clock = $6 + 1 = 7$ ns. Reading the stated penalties (ideal CPI 1; branch +2; memory +2 plus 4 more on a 10% miss, so 2.4 average; floating point +4): $\text{CPI} = 1 + 0.20(2) + 0.35(2 + 0.10 \times 4) + 0.05(4) = 2.44$. Non-pipelined time = $20\,000\,000 \times 25$ ns = **0.5 s**. The pipelined time is $20\,000\,000 \times 2.44 \times 7$ ns = 0.3416 s, a speedup of about 1.46. It falls short of the ideal 5 because stalls raise CPI above 1 and the latch overhead lengthens every cycle.

THE EXAMINER'S WORDS, UNEDITED

- c. A spacecraft telemetry system uses a 16-bit processor to transmit environmental measurements. The following values are to be stored - Temperature = -182; Pressure = 487; Altitude = -1023. The processor uses 2's complement representation for signed integers.
- Determine the range of values representable in a 16-bit two's complement system. Indicate whether the given numbers fall within the range of value **2mks**
 - Represent each value in: Binary and Hexadecimal **3mks**
 - Demonstrate how the processor uses two's complement to compute **2mks**
 $(-182) + 487$

Read it again. Part (i) ends "fall within the range of value" with no full stop and a singular "value". Part (iii) breaks across the mark annotation: the expression $(-182) + 487$ is printed *after* the 2mks, on its own line, and its sign is a true minus sign where "Temperature = -182" three lines above uses a plain hyphen.

BREAK IT DOWN

BEFORE YOU COMPUTE

- GIVEN** a 16-bit two's complement processor and three values, one positive and two negative.
- ASKED** *determine* a range and judge three values against it, *represent* each in two bases, then *demonstrate* an addition. Demonstrate means show the machine doing it in binary, not compute 305 in decimal.
- MARKS** $2 + 3 + 2 = 7$. Part (ii) at 3 covers six answers, three binary and three hexadecimal, so half a mark each.
- TAUGHT IN** Module One Unit 2, Data Representation.
- TRAP** the asymmetry of the range. It is -32768 to +32767, not -32767 to +32767, because zero sits on the positive side. The other trap is the carry out of bit 16 in part (iii): in two's complement it is discarded, and keeping it turns 305 into a 17-bit number.

- GIVEN** $n = 16$ · two's complement · the three values
- FIND** range and fit, binary and hex, and the sum
- FORMULA** range = -2^{n-1} to $+(2^{n-1}-1)$ · negate by invert-and-add-1 · add and **discard** the final carry

VALUE	16-BIT TWO'S COMPLEMENT	HEX
-182	1111 1111 0100 1010	FF4A
487	0000 0001 1110 0111	01E7
-1023	1111 1100 0000 0001	FC01

- (i) Range -32768 to +32767; all three values fall inside it, so all are representable. (iii) $FF4A + 01E7 = 1\ 0000\ 0001\ 0011\ 0001$; discard the carry out of bit 16, leaving $0000\ 0001\ 0011\ 0001 = 0131H = 305$, and indeed $-182 + 487 = 305$.

THE EXAMINER'S WORDS, UNEDITED

- d. A processor executes one million(1M) instructions distributed as follows.

Addressing Mode	Percentage	Cycles
Register	40%	1
Immediate	20%	1
Direct	15%	2
Indirect	10%	4
Indexed	10%	3
Relative	5%	2

- i) Compute the average CPI **2mks**
 ii) Determine total clock cycles. **1mk**
 iii) If indirect addressing is optimized to 2 cycles, compute the new CPI. **1mk**
 iv) Evaluate the performance improvement. **2mks**

Read it again. "one million(1M)" is set with no space before the bracket. Six addressing modes are tabulated with percentages and cycle counts; the paper never says the percentages total 100, so check that yourself before you weight anything. Part (iv) says "Evaluate", which is a different demand from "compute".

BREAK IT DOWN

BEFORE YOU COMPUTE

- GIVEN** an instruction count of one million and a six row table of modes, percentages and cycles per instruction.
- ASKED** three computations and one evaluation. Part (iii) changes one row of the table and asks for the new average, so it is the same sum with one term altered.
- MARKS** $2 + 1 + 1 + 2 = 6$. Parts (i) and (iv) carry 2 each, so the average CPI wants its six products written out and the evaluation wants a percentage with a sentence, not a bare number.
- TAUGHT IN** Module Two Unit 2, CPU Performance Metrics.
- TRAP** recomputing the whole sum in part (iii). Only the indirect term changes, so subtract its old contribution and add the new one. The trap in (iv) is quoting the CPI drop as the improvement: performance improvement is measured against the old value, which is where the 11.76% comes from.

- GIVEN** $IC = 1\,000\,000$ · register 40% at 1, immediate 20% at 1, direct 15% at 2, indirect 10% at 4, indexed 10% at 3, relative 5% at 2
- FIND** average CPI, total cycles, new CPI, percentage improvement
- FORMULA** $CPI = \sum(\text{fraction} \times CPI_i) \cdot \text{cycles} = IC \times CPI \cdot \text{improvement} = (\text{old} - \text{new})/\text{old}$

(i) $CPI = 0.40(1) + 0.20(1) + 0.15(2) + 0.10(4) + 0.10(3) + 0.05(2) = 0.40 + 0.20 + 0.30 + 0.40 + 0.30 + 0.10 = \mathbf{1.70}$. (ii) Total cycles = $1\,000\,000 \times 1.70 = \mathbf{1\,700\,000}$. (iii) With indirect at 2: $CPI = 1.70 - 0.10(4) + 0.10(2) = \mathbf{1.50}$. (iv) Improvement = $(1.70 - 1.50)/1.70 = \mathbf{11.76\%}$.

THE EXAMINER'S WORDS, UNEDITED

e. A processor contains the following information: PC = 7000; R1 = 1500;

Memory Contents

Address	Value
7000	1200
1200	4500
1500	6000
3200	8100
4500	999
6000	200
8100	400

Assume an instruction contains address field 1200.

Determine the Effective Address (EA) and operand for the following addressing modes:

- i) Immediate addressing mode **1mk**
- ii) Direct addressing mode **1mk**
- iii) Indirect addressing mode **1mk**
- iv) Register addressing mode **1mk**
- v) Register Indirect (R1) addressing mode **1mk**
- vi) Relative addressing mode **1mk**

Read it again. The memory table lists address 3200 holding 8100, which no mode in the list ever reaches: it is there to be ignored. Part (iv) asks for the effective address under **register** addressing, where there is no effective address at all, and saying so is the mark. The stem gives R1 = 1500 but never names a register for part (iv).

BREAK IT DOWN

BEFORE YOU COMPUTE

GIVEN	PC = 7000, R1 = 1500, an address field of 1200, and a seven row memory table.
ASKED	<i>determine</i> two things for each of six modes, the effective address and the operand. Twelve answers, not six.
MARKS	6, one per mode, so each mark buys both the address and the operand. Tabulate.
TAUGHT IN	Module One Unit 4, Instruction Set Architecture.
TRAP	one dereference too many or too few. Direct reads memory once, indirect reads it twice, immediate reads it not at all and the operand is the field 1200 itself. Register indirect uses R1 as the pointer, so the effective address is 1500 and the operand is what sits there.

MODE	EFFECTIVE ADDRESS	OPERAND
Immediate	none	1200, the field itself
Direct	1200	M[1200] = 4500
Indirect	M[1200] = 4500	M[4500] = 999
Register indirect (R1)	R1 = 1500	M[1500] = 6000
Relative	PC + 1200 = 8200	M[8200]

One sentence answers the whole part: immediate uses the field as the value, direct reads memory once, indirect reads it twice, register indirect uses the register as a pointer, and relative adds the PC.

AS PRINTED

25/26 Q2(A)

THE EXAMINER'S WORDS, UNEDITED

a. Assume a computer has 32-bit addresses. Each block stores 16 words. A direct-mapped cache has 256 blocks. In which block (line) of the cache (decimal value) would you look for each of the following addresses? Addresses are given in hexadecimal for convenience. **4mks**

- i. CA2DFC12
- ii. F24F25FF
- iii. 24BBABCD
- iv. C1D9EEF2

Read it again. This is 24/25 Q1(c) with the address width changed from 24 to 32 bits, four new addresses, the words "(decimal value)" added so the base of the answer is now fixed, and the marks cut from 6 to 4. The sub-items are numbered **i. ii. iii. iv.** here where the older paper used a. b. c. d.

BREAK IT DOWN

BEFORE YOU COMPUTE

GIVEN	a 32-bit address, 16 words per block, 256 cache blocks, four hexadecimal addresses.
ASKED	the block (line) number for each, "(decimal value)", which the older version of this question left open and this one closes.
MARKS	4 for four addresses, one each. There is no mark for the field split on its own, so do it once, briefly, and spend the time on the four answers.
TAUGHT IN	Module Three Unit 3, Cache Memory Design.
TRAP	letting the wider address change the answer. Offset and index depend only on the block size and the number of blocks, so they are still 4 and 8 bits; only the tag grows, from 12 bits to 20. The line is still the two hex digits before the last.

Offset 4 bits, index 8 bits, tag 20 bits. Reading the hex: drop the last digit, take the next two. CA2DFC12 → C1 = **193**; F24F25FF → 5F = **95**; 24BBABCD → BC = **188**; C1D9EEF2 → EF = **239**.

THE EXAMINER'S WORDS, UNEDITED

b. Consider the following code segment executed on a classic five-stage RISC processor.

```
I1: LW  R1,0(R2)
I2: ADD R3,R1,R4
I3: SUB R5,R3,R6
I4: SW  R5,8(R2)
I5: BEQ R5,R0,LABEL
I6: MUL R7,R5,R8
```

Assume:

- forwarding is available
- branch resolved in EX stage
- load-use hazards require one stall
- multiplier occupies EX for three cycles
- separate instruction and data caches

- Identify and explain every pipeline hazard. **5mks**
- Draw the pipeline timing diagram **2mks**
- Determine the total execution cycles **1mk**
- Suggest any two hardware improvements that would reduce stalls. **1mk**

Read it again. The five assumptions are printed as bullets *after* the code, and they are what the whole question turns on: "separate instruction and data caches" rules structural memory hazards out by fiat, and "branch resolved in EX stage" fixes the control penalty. Part (ii) "Draw the pipeline timing diagram" ends with no full stop, and neither does (iii), where (i) and (iv) both do.

BREAK IT DOWN

BEFORE YOU COMPUTE

GIVEN	six instructions with real register operands, and five assumptions that between them fix every penalty in the question.
ASKED	<i>identify and explain</i> every hazard, <i>draw</i> the diagram, <i>determine</i> the cycle count, <i>suggest</i> two improvements. Four verbs, and "every" in the first one means an incomplete list is marked as wrong, not short.
MARKS	$5 + 2 + 1 + 1 = 9$, and 5 of the 9 sit on naming and explaining the hazards. Spend the time there, not on the drawing.
TAUGHT IN	Module Two Unit 3, Pipelining.
TRAP	calling the I2 to I3 dependency a stall. Forwarding covers it. The only stall forwarding cannot cover is the load-use hazard from I1 to I2, because the value is not out of memory until the end of MEM. Miss that distinction and the cycle count is wrong by exactly one.

Hazards: a **load-use data hazard** between I1 and I2 (forwarding cannot fix it, so exactly one stall); **data hazards** I2 to I3 and I3 to I4, both covered by forwarding; a **control hazard** at I5, the branch resolved in EX; and a **structural delay** at I6, whose 3-cycle EX holds the stage for two extra cycles. Ideal $(k + n - 1) = 5 + 6 - 1 = 10$, plus 1 load-use stall and 2 for the long multiply, giving **13 cycles**. Two improvements: add a **dedicated multiplier** so a long multiply does not block EX, and use **separate instruction and data memories** (or split caches) to remove structural stalls.

THE EXAMINER'S WORDS, UNEDITED

- c. A processor has the following configuration: Main memory is 1 GB; Cache is 512 KB; Block size is 128 bytes and it is a 4-way Set Associative cache.
- Determine the total number of cache blocks. **1mk**
 - Determine the number of sets. **1mk**
 - Construct the complete address format. **1mk**
 - If the hit rate is 96%, cache access time is 2 ns, and main memory access time is 80 ns, calculate the Average Memory Access Time (AMAT). **2mks**
 - Discuss how increasing associativity beyond four ways might affect performance. **2mks**

Read it again. Part (v) carries its **2mks** inline on the same line as the sentence, where every other item on the page sets the marks out at the margin. The address width is never stated: it follows from a main memory of 1 GB, which the paper leaves you to notice.

BREAK IT DOWN

BEFORE YOU COMPUTE

GIVEN	main memory 1 GB, cache 512 KB, block 128 bytes, 4-way set associative, and later a hit rate, a cache time and a memory time.
ASKED	three structural numbers, then one address format, then one AMAT, then a discussion. The last part asks you to <i>discuss</i> , so it wants a trade off argued, not a number.
MARKS	$1 + 1 + 1 + 2 + 2 = 7$. The discussion is worth as much as the AMAT calculation, which is the part most students leave blank.
TAUGHT IN	Module Three Unit 3, Cache Memory Design.
TRAP	the set count. Blocks are $512 \text{ KB} / 128 \text{ B} = 4096$, but a 4-way cache has $4096 / 4 = 1024$ sets, and it is the set count, not the block count, that sizes the index field. The second trap is which AMAT formula: with a stated cache access time and a stated memory time this is the hierarchical form.

Blocks = $512 \text{ KB} / 128 \text{ B} = 4096$. Sets = $4096 / 4 = 1024$. Address = 30 bits, offset = $\log_2 128 = 7$, set = $\log_2 1024 = 10$, tag = $30 - 10 - 7 = 13$, so the format is **tag 13 / set 10 / offset 7**. AMAT (hierarchical) = $2 + 0.04(80) = 2 + 3.2 = 5.2 \text{ ns}$. Higher associativity cuts conflict misses but needs more comparators, so past a point it costs more than it saves.

AS PRINTED

25/26 Q3(A)

THE EXAMINER'S WORDS, UNEDITED

- a. A computer system has the following characteristics:
- Main memory = 64 MB
 - Cache memory = 128 KB
 - Block size = 64 bytes
- i) Determine the number of cache lines **1½mks**
- ii) For Direct Mapping, determine: Number of index bits; Number of tag bits; Number of offset bits **1½mks**
- iii) For a 4-way Set Associative Cache, determine: Number of sets; Number of set index bits; Number of tag bits **1½mks**
- iv) Explain why fully associative mapping eliminates conflict misses but introduces another performance challenge. **1½mks**

Read it again. All four parts are annotated **1½mks**, so the part is worth 6 and every sub-part the same. That is generous on (i), which is one division, and tight on (ii) and (iii), which each ask for three separate numbers. The address width is not given; it follows from a 64 MB main memory.

BREAK IT DOWN

BEFORE YOU COMPUTE

GIVEN	main memory 64 MB, cache 128 KB, block 64 bytes. Three numbers, and everything else is derived.
ASKED	a line count, then two full field splits, then an explanation. Part (iv) is the only one that wants prose, and it asks for a challenge <i>introduced</i> , so naming the benefit is only half of it.
MARKS	1½ four times, 6 in all. Parts (ii) and (iii) buy three numbers each for 1½, so half a mark a number: tabulate and move on.
TAUGHT IN	Module Three Unit 3, Cache Memory Design.
TRAP	using the main memory size as the address width without converting. 64 MB is 2^{26} , so addresses are 26 bits, and every tag figure on the page depends on that one step. The second trap in (iv) is stopping at "no conflict misses": the challenge asked for is the comparator per line that full associativity costs.

64 MB = 2^{26} , so **26-bit** addresses. Lines = 128 KB / 64 B = **2048**. Offset = $\log_2 64 = 6$. **Direct:** index = $\log_2 2048 = 11$, tag = $26 - 11 - 6 = 9$. **4-way:** sets = $2048/4 = 512$, set bits = 9, tag = $26 - 9 - 6 = 11$. Fully associative lets a block sit in any line, so no two blocks are forced to collide and conflict misses vanish; the challenge is that every line's tag must then be compared simultaneously, needing one comparator per line, which is expensive in area and power and slows the lookup.

THE EXAMINER'S WORDS, UNEDITED

b. A 5-stage pipelined processor has instruction fetch (IF), Instruction decode (ID), Operand Fetch (OF), Execute (Ex) and Write Back (WB) stages. The IF, ID, OF and WB stages take 1 clock cycle each for any instruction. The PO stage take 1 cycle for ADD and SUB instructions, 3 clock cycles for MUL instruction and 6 clock cycles for DIV instruction respectively. Operand forwarding is used in the pipeline.

Draw the cycle diagram and determine the number of cycles needed to execute the following sequence of instructions:

- i. When Operand forwarding is used **4½mks**
- ii. When Operand forwarding is not used **5½mks**

I1 MUL R2, R0, R1 ; R2 ← R0 X R1
 I2 DIV R5, R3, R4 ; R5 ← R3/R4
 I3 ADD R2, R5, R2 ; R2 ← R5 + R2
 I4 SUB R5, R2, R6 ; R5 ← R2 - R6

Read it again. The stem names five stages, IF, ID, OF, Ex and WB, and then says "The **PO** stage take 1 cycle", which is not one of them. On the physical paper PO is struck through in pen with EX written beneath it, and the B of WB earlier in the same sentence is overwritten in pen, so the room was told to read PO as EX. The stem also states that operand forwarding is used and then part (ii) asks for the case where it is not.

BREAK IT DOWN

BEFORE YOU COMPUTE

GIVEN	five stages, a cycle cost per stage, three different EX costs by instruction type, and four instructions with real dependencies.
ASKED	<i>draw</i> the cycle diagram and <i>determine</i> the number of cycles, twice, once with forwarding and once without. The diagram is asked for first, so it is the working, not an afterthought.
MARKS	4½ + 5½ = 10, half of Question Three. The no-forwarding case is worth more because it stalls more, so do not run out of time before it.
TAUGHT IN	Module Two Unit 3, Pipelining.
TRAP	answering without stating your model. One execution unit or several, write-back one cycle after EX or in the same cycle, forwarding into EX or into OF: each choice changes the total. Write the assumptions down first, then the marks follow the reasoning even if your number differs from the marking scheme.

Under the stated assumptions (one in-order execution unit, so an instruction enters EX only when EX is free; write-back one cycle after EX; forwarding delivers a result to the next instruction's EX in the following cycle), the schedule gives **15 cycles with forwarding** and **17 cycles without**. The full cycle diagram for both cases is worked in Module Two Unit 3. State your assumptions: this question is ambiguous without them, and the marks follow the reasoning.

THE EXAMINER'S WORDS, UNEDITED

- c. i) State Amdahl's law' **2mks**
 ii) Assume in hypothetical system cache is 10 times faster than main memory and CPU accesses the cache 85% of the time. Using Amdahl's law, calculate the overall speed up of the system.
2mks

Read it again. Part (i) reads "State Amdahl's law'" with a stray closing apostrophe after "law". Part (ii) says "Assume in hypothetical system" with the article missing, and sets its **2mks** on the line below. Two marks for stating the law means the statement is worth exactly as much as the calculation.

BREAK IT DOWN

BEFORE YOU COMPUTE

- GIVEN** for (ii): a cache 10 times faster than main memory, and 85% of accesses served by it. Part (i) is given nothing.
- ASKED** state the law, then use it. State means give the law and its formula, and the limiting case is what makes it a law worth stating.
- MARKS** 2 + 2 = 4. Half the marks are for a statement you can write from memory in twenty seconds, so never skip part (i) to buy time for part (ii).
- TAUGHT IN** Module Two Unit 2, CPU Performance Metrics.
- TRAP** reading "10 times faster" as the overall speedup. It is the speedup of the improved fraction only, and the 15% that still goes to main memory at full cost caps the whole system at about 4.26. That gap between 10 and 4.26 is the law.

GIVEN $f = 0.85 \cdot s = 10$
FIND the overall speedup
FORMULA $\text{speedup} = 1 / ((1 - f) + f/s)$

$\text{Speedup} = 1 / (0.15 + 0.085) = 1 / 0.235 = 4.26$. Note it is far short of 10: the 15% still going to main memory at full cost caps the gain, which is the point of Amdahl's law.

AS PRINTED

25/26 Q4(A)

THE EXAMINER'S WORDS, UNEDITED

a. Consider three (3) floating point numbers X, Y and Z stored in registers based on the IEEE 754 Single precision floating point format.

$X = C1400000h$, $Y = 42100000h$, $Z = 41400000h$

Convert X, Y and Z to decimal numbers and determine which of the following is true. **9mks**

- i. $(Y-Z) > 0$
- ii. $X + Z = 0$
- iii. $Z = X + Y$

Read it again. This is 24/25 Question One (a) reprinted word for word: the same three hexadecimal values, the same three claims, the same **9mks**. Only the layout moved, all three values now sitting on one line. The question has now appeared on three papers, which makes it the single most repeated item in the course.

BREAK IT DOWN

BEFORE YOU COMPUTE

GIVEN	three 32-bit hexadecimal words and the statement that they are IEEE 754 single precision, which fixes 1 sign bit, 8 exponent bits, 23 mantissa bits and a bias of 127.
ASKED	<i>convert</i> all three, then <i>determine which</i> of the three claims are true. The second verb carries the part.
MARKS	9, the largest single part on the paper. Three conversions and three verdicts, so show every conversion.
TAUGHT IN	Module One Unit 3, Floating Point Representation.
TRAP	stopping at the first true claim. Two of the three are true. Test all three and write a verdict for each.

Identical to 2024/25 Q1(a): $X = -12$, $Y = 36$, $Z = 12$; (i) TRUE, (ii) TRUE, (iii) FALSE. This question has now appeared on three papers (23/24 Q1c, 24/25 Q1a, 25/26 Q4a). Learn it cold.

AS PRINTED

25/26 Q4(B)

THE EXAMINER'S WORDS, UNEDITED

b. Use a Zero, One, Two and Three address instruction formats to write a Pentium 4 assembly language program segment for the expression below. For each format determine the number of bytes required.

$X = ((A * B) + C/D)/(F * G + J * K)$ **8mks**

Read it again. This is 24/25 Q1(d) reprinted, with the **8mks** moved to the end of the expression line. On the physical paper the sentence "For each format determine the number of bytes required." is struck through in pen and a short handwritten word stands beside it, not crisply legible but reading most nearly as "Dont", so the byte count was cancelled in the hall. The printed sentence is quoted here because that is what the paper prints.

BREAK IT DOWN

BEFORE YOU COMPUTE

GIVEN	the same expression as the earlier paper, and four instruction formats. Opcode and address field sizes are again unstated, so 1 byte and 2 bytes are assumed and declared.
ASKED	as printed, <i>write</i> the four program segments and <i>determine</i> four byte counts. If the strike-out is repeated in your hall, only the listings are wanted, but write the totals anyway: they cost four lines and cannot lose marks.
MARKS	8, which is 2 per format.
TAUGHT IN	Module One Unit 4, Instruction Set Architecture.
TRAP	the stack format. Zero address means every operand is pushed and popped, so the instruction count rises even as each instruction shrinks, and the byte total lands between the 3-address and 2-address cases rather than below both.

As 2024/25 Q1(d): **49, 60, 51 and 34 bytes** for 3, 2, 1 and 0 address respectively, from 7, 12, 17 and 16 instructions. Full listings in Module One Unit 4.

AS PRINTED

25/26 Q4(C)

THE EXAMINER'S WORDS, UNEDITED

c. Calculate the memory requirement of an image whose dimension is 17.12 inches by 9.45 inches at resolution 250 dpi. Express your answer in Megabyte **3mks**

Read it again. This is 24/25 Q3(b)(ii) reprinted, minus the full stop after "Megabyte". There it was buried as the second half of a data representation part; here it stands on its own as a whole part of Question Four. Same numbers, same 3mks, same missing bit depth.

BREAK IT DOWN

BEFORE YOU COMPUTE

GIVEN	17.12 inches by 9.45 inches at 250 dots per inch, and no colour depth.
ASKED	<i>calculate</i> the requirement and <i>express</i> it in megabytes. The unit is part of the instruction, so an answer in bits is not finished.
MARKS	3: pixel count, byte count, megabyte conversion.
TAUGHT IN	Module One Unit 2, Data Representation.
TRAP	the megabyte convention. Dividing twice by 1024 gives about 1.21 MB and dividing by one million gives about 1.26 MB, so name the convention you used on the page and the marker cannot mark you down for the other one.

10 111 500 pixels × 1 bit = 1 263 937.5 bytes ≈ **1.21 MB**.

AS PRINTED

25/26 Q5(A)

THE EXAMINER'S WORDS, UNEDITED

- a. Represent the following -1240.56 using Single Precision IEEE format for floating point numbers. **6mks**

Read it again. Identical to 24/25 Q3(c), down to "Represent the following" and the **6mks**. Note what is *not* here: no truncation rule, unlike 24/25 Q5(c), so the mantissa runs to its full 23 bits.

BREAK IT DOWN

BEFORE YOU COMPUTE

- GIVEN** the value -1240.56 and the single precision format.
- ASKED** *represent* it, which means the 32-bit pattern, and the hex equivalent is the natural way to present it.
- MARKS** 6, of the 20 for Question Five.
- TAUGHT IN** Module One Unit 3, Floating Point Representation.
- TRAP** the recurring fraction. 0.56 does not terminate in binary, so the mantissa has to be filled to 23 bits and then rounded or truncated, and the last hex digit differs between the two choices. State which you used.

1 10001001 00110110001000111101011 = **C49B11EBH** truncated (C49B11ECH if rounded). Working as 2024/25 Q3(c).

AS PRINTED

25/26 Q5(B)

THE EXAMINER'S WORDS, UNEDITED

- b. Consider a pipeline having 4 phases with duration 60, 50, 90 and 80 ns. Given latch delay is 10 ns. Calculate-
- Pipeline cycle time **1mk**
 - Non-pipeline execution time **1mk**
 - Speed up ratio **1mk**
 - Pipeline time for 500 tasks **1mk**
 - Sequential time for 500 tasks **1mk**

Read it again. This is 24/25 Q5(b) with item **vi. Throughput** dropped, so five items and 5 marks where the older paper had six and 6. The five 1mk annotations are printed one line lower than the items they belong to, which makes the page look as though the first item carries no marks.

BREAK IT DOWN

BEFORE YOU COMPUTE

- GIVEN** stage durations 60, 50, 90 and 80 ns, a latch delay of 10 ns, and 500 tasks. The same numbers as the earlier paper.
- ASKED** five figures, in order. No explanation is asked for anywhere, so every mark is a number with a unit.
- MARKS** 5, one per item. Throughput is not asked for here, so do not spend time on it.
- TAUGHT IN** Module Two Unit 3, Pipelining.
- TRAP** the same one as on the older paper: the cycle time is the slowest stage plus the latch, 100 ns and not 90, and four of the five answers are built on it. For 500 tasks the count is $k + n - 1 = 503$ cycles, not 504.

Cycle **100 ns**; non-pipelined one task **280 ns**; speedup **2.8**; 500 tasks pipelined **50 300 ns** against sequential **140 000 ns**.

AS PRINTED

25/26 Q5(C)

THE EXAMINER'S WORDS, UNEDITED

- c. Explain the following terms **3mks**
- Locality of reference
 - Temporal locality

Read it again. The stem is "Explain the following terms" with no colon, and it lists only **two** terms where the matching 24/25 part listed three. The third term there was the misspelled "Special Locality", and it has quietly gone. Three marks for two terms means 1½ each, the same rate as the older paper.

BREAK IT DOWN

BEFORE YOU COMPUTE

- GIVEN** nothing numeric. A pure definitions part.
- ASKED** *explain*, not name. At 1½ marks a term that is a definition, a reason and an example.
- MARKS** 3 for two terms.
- TAUGHT IN** Module Three Unit 1, Computer Memory Overview.
- TRAP** defining temporal locality without first defining locality of reference, which is the general principle temporal locality is one case of. Define the parent term, then the special case, then say what the cache does about it. Spatial locality is not asked for here, but one sentence naming it as the other case shows you know the pair.

Locality of reference is the tendency of a program to concentrate its accesses in a small, predictable set of locations over any short period. **Temporal locality** is the form where a location just accessed is likely to be accessed again soon, as a loop counter is, which is why a cache keeps recently used data rather than discarding it.

THE EXAMINER'S WORDS, UNEDITED

- d. By using the reverse of the IEEE Single Precision procedure, determine the value (Base 10) of the following hexadecimal (base 16) value: C4F2E000h **6mks**

Read it again. This is 24/25 Q4(c) reprinted exactly: the same hexadecimal value, the same lower case h, the same **6mks**, the same instruction to use "the reverse of the IEEE Single Precision procedure". Paired with Q5(a) on the same page, Question Five asks you to run the conversion in both directions.

BREAK IT DOWN

BEFORE YOU COMPUTE

GIVEN	one 32-bit hexadecimal word, stated to be IEEE single precision.
ASKED	<i>determine</i> the base 10 value by reversing the procedure, so the working is part of the answer.
MARKS	6.
TAUGHT IN	Module One Unit 3, Floating Point Representation.
TRAP	the hidden bit and the bias, in that order. Subtract 127 from the stored exponent field to get the true exponent, then restore the implied leading 1 before shifting. Skipping either step is the difference between -1943 and a number nowhere near it.

Sign 1; E = 137 so e = 10; restoring the hidden 1 gives $1.11100101110 \times 2^{10} = 11110010111_2 = 1943$. Value = **-1943**.

RECALL AND CHECK

Across both papers, which single question type appeared most often, and what does that tell you about where to spend revision time?

ANSWER

IEEE-754 conversion, in both directions, appears on every paper and more than once on some (24/25 Q1a, Q3c, Q4c, Q5c; 25/26 Q4a, Q5a, Q5d). Cache field-splitting and CPI are close behind. If you own floating point, cache addressing and CPI, you have secured the large majority of the numeric marks on any paper of this course.

REVISION CLASS

The Revision Deck, Solved A to Z

The lecturer's revision class, every question in it worked out and explained in the plainest words possible. Read it last, the night before, with a pen. Nearly every question here is one you have already met in this book, which is the whole point: the revision is the exam, and you have drilled it.

A

start



Z

exam

Data, Instructions and Addressing

The first block of the revision deck: signed numbers, instruction formats, and addressing modes. Every question here is straight off the 2025/2026 paper, so treat this as the paper itself.

By the end you can, from memory, place any signed number in a 16-bit register in binary and hex, write an expression in all four instruction formats, and read an operand out of memory under any addressing mode.

TEACH

WHAT THIS DECK ACTUALLY IS

Before the questions, the one thing worth knowing about this revision deck: **almost every worked example in it is a question you have already met.** The data representation, instruction format, addressing, 4-phase pipeline, hazard classification and the whole cache section are the 2025/2026 paper and the pipelining and cache slides, which this manual has already drilled. Only **one** question, the 6-stage pipeline, is new. Each one below is labelled with where else it lives, so you can see for yourself that if you can do this section, you have done the exam.

FROM THE SLIDES

REVISION A

THE LECTURER'S REVISION DECK

A spacecraft telemetry system uses a 16-bit processor to transmit environmental measurements. The following values are to be stored: Temperature = -182 ; Pressure = 487 ; Altitude = -1023 . The processor uses 2's complement representation for signed integers.

(i) Determine the range of values representable in a 16-bit two's complement system.

Indicate whether the given numbers fall within the range.

(ii) Represent each value in binary and hexadecimal.

(iii) Demonstrate how the processor uses two's complement to compute $(-182) + 487$.

You have met this. It is **PQ 2025/2026 Q1(c)**, word for word, spacecraft and all, solved in full in the Past Papers section. Here it is again, as simply as it goes.

GIVEN	a 16-bit machine, 2's complement · Temperature = -182, Pressure = 487, Altitude = -1023
FIND	(i) the range and whether each fits, (ii) each in binary and hex, (iii) the sum (-182) + 487 in binary
FORMULA	range = -2^{15} to $2^{15} - 1$ · negative in 2's complement = invert the positive and add 1 · group the 16 bits in fours to read the hex

- 1 Range.** A 16-bit 2's complement register holds -2^{15} to $2^{15} - 1$, that is **-32768 to 32767**. All three of -182, 487 and -1023 sit comfortably inside it.
- 2 487** is positive, so just write it in binary: 0000 0001 1110 0111.
Group in fours: 0000=0, 0001=1, 1110=E, 0111=7, so hex = 01E7.
- 3 -182.** First +182 = 0000 0000 1011 0110. Invert every bit: 1111 1111 0100 1001. Add 1: 1111 1111 0100 1010.
Hex: 1111=F, 1111=F, 0100=4, 1010=A, so -182 = FF4A.
- 4 -1023.** First +1023 = 0000 0011 1111 1111. Invert: 1111 1100 0000 0000. Add 1: 1111 1100 0000 0001.
Hex: 1111=F, 1100=C, 0000=0, 0001=1, so -1023 = FC01.
- 5 (-182) + 487.** Add the two 16-bit patterns and throw away any carry out of the top:
1111 1111 0100 1010 + 0000 0001 1110 0111 = 0000 0001 0011 0001.
That pattern is positive and equals $256 + 32 + 16 + 1 = \mathbf{305}$, which is exactly $-182 + 487$. The machine got the right answer with nothing but addition.

Range -32768 to 32767 (all three fit). 487 = 01E7; -182 = FF4A; -1023 = FC01.
 $(-182) + 487 = 0000\ 0001\ 0011\ 0001 = 305$.

THE LECTURER'S REVISION DECK

Use a Zero, One, Two and Three address instruction format to write a Pentium 4 assembly language program segment for the expression:

$$X = ((A * B) + C/D) / (F * G + J * K)$$

You have met this. Same expression as **PQ 2025/2026 Q4(b)**, solved in full in Module One Unit 4. The revision drops the "number of bytes required" part, which is the very part struck out by hand on the exam paper, so here you only write the four programs. The byte counts (49, 60, 51, 34 for 3, 2, 1, 0-address) are in Module One Unit 4 if you want them.

GIVEN	$X = ((A \times B) + C/D) / (F \times G + J \times K)$
FIND	the program in each of the four formats
FORMULA	3-address names both sources and the destination · 2-address lets the destination double as a source · 1-address works through one accumulator (LOAD, op, STORE) · 0-address pushes operands on a stack and each operation pops two

Do the numerator ($A \times B + C/D$) and the denominator ($F \times G + J \times K$) first, then divide.

3-address (shortest)

```

MUL R1, A, B      ; R1 = A*B
DIV R2, C, D      ; R2 = C/D
ADD R1, R1, R2    ; R1 = A*B + C/D      (numerator)
MUL R2, F, G      ; R2 = F*G
MUL R3, J, K      ; R3 = J*K
ADD R2, R2, R3    ; R2 = F*G + J*K      (denominator)
DIV X, R1, R2     ; X = numerator / denominator

```

The revision slide prints the last line as "DIV X, R2, R3", which is a slip: that would divide the denominator by J*K. The correct last line is **DIV X, R1, R2**, numerator over denominator.

2-address (destination is also a source)

```

MOV R1, A         ; R1 = A
MUL R1, B         ; R1 = A*B
MOV R2, C
DIV R2, D         ; R2 = C/D
ADD R1, R2        ; R1 = numerator
MOV R3, F
MUL R3, G         ; R3 = F*G
MOV R2, J
MUL R2, K         ; R2 = J*K
ADD R2, R3        ; R2 = denominator
DIV R1, R2        ; R1 = X
MOV X, R1

```

1-address (one accumulator, AC)

```

LOAD F           ; AC = F
MUL G            ; AC = F*G
STORE T          ; T = F*G
LOAD J
MUL K            ; AC = J*K
ADD T            ; AC = F*G + J*K
STORE T          ; T = denominator
LOAD A
MUL B            ; AC = A*B
STORE R          ; R = A*B
LOAD C
DIV D            ; AC = C/D
ADD R            ; AC = numerator
DIV T            ; AC = numerator / denominator
STORE X          ; X = AC

```

0-address (stack, postfix $A B * C D / + F G * J K * + /$)

PUSH A	PUSH B	MUL		; A*B
PUSH C	PUSH D	DIV	ADD	; + C/D -> numerator
PUSH F	PUSH G	MUL		; F*G
PUSH J	PUSH K	MUL	ADD	; + J*K -> denominator
DIV	POP X			; numerator / denominator, store X

Fewer address fields means shorter instructions but more of them: 3-address is 7 instructions, 0-address is 16. Each program computes the same X.

FROM THE SLIDES

REVISION C

THE LECTURER'S REVISION DECK

A processor contains: PC = 7000; R1 = 1500. Memory contents:

Address	Value
7000	1200
1200	4500
1500	6000
3200	8100
4500	999
6000	200
8100	400

An instruction contains address field 1200. Determine the Effective Address (EA) and operand for:

(i) Immediate (ii) Direct (iii) Indirect (iv) Register (v) Register Indirect (R1) (vi) Relative

You have met this. It is **PQ 2025/2026 Q1(e)**, the same table and the same field 1200, solved in Module One Unit 4.

GIVEN PC = 7000, R1 = 1500, the memory table above, and address field = 1200
FIND the EA and the operand under each of the six modes
FORMULA the mode tells you how to turn the address field into the EA, and the operand is what sits at the EA. M[x] means "the value stored at address x".

MODE	EFFECTIVE ADDRESS	OPERAND
(i) Immediate	none	the field itself = 1200
(ii) Direct	EA = 1200	M[1200] = 4500
(iii) Indirect	EA = M[1200] = 4500	M[4500] = 999
(iv) Register	none	contents of R1 = 1500
(v) Register indirect	EA = R1 = 1500	M[1500] = 6000
(vi) Relative	EA = PC + 1200 = 7000 + 1200 = 8200	M[8200] = not in the table

Immediate 1200; Direct 4500; Indirect 999; Register 1500; Register indirect 6000; Relative EA = 8200 (its operand is not listed, so say so). The trick is to follow the arrows: each mode is one more hop through memory than the last.

REVISION CLASS

Pipelining and Hazards

Two pipeline speedup questions and the data-hazard classification. The second pipeline question and the hazard one you have already met; the first is the deck's one new example, so learn it fresh.

THE LECTURER'S REVISION DECK

A non-pipelined processor requires 60 ns per instruction. The processor is redesigned into a 6-stage pipeline with a cycle time of 12 ns. Calculate:

- (a) Time required for 50 instructions without pipelining
- (b) Time required for 50 instructions with pipelining
- (c) Speedup obtained

This one is new. It is not on any past paper, so it is the only question in the whole revision deck you have not already drilled. It is an easy one, worth learning cold.

GIVEN	non-pipelined time = 60 ns per instruction · pipeline stages $k = 6$ · pipeline cycle time = 12 ns · $n = 50$ instructions
FIND	(a) non-pipelined time, (b) pipelined time, (c) speedup
FORMULA	non-pipelined = $n \times \text{time per instruction}$ · pipelined = $(k + n - 1) \times \text{cycle time}$ · speedup = non-pipelined / pipelined

- 1 **(a) Without pipelining.** Each instruction takes the full 60 ns and they run one after another: $50 \times 60 = \mathbf{3000 \text{ ns}}$.
- 2 **(b) With pipelining.** The first instruction must pass through all 6 stages, which takes $6 \times 12 = 72$ ns to fill the pipe. After that, one instruction finishes every cycle, so the remaining 49 add $49 \times 12 = 588$ ns. Total = $72 + 588 = \mathbf{660 \text{ ns}}$.
Same thing in one line: $(k + n - 1) \times \text{cycle} = (6 + 50 - 1) \times 12 = 55 \times 12 = 660 \text{ ns}$.
- 3 **(c) Speedup.** = $3000 / 660 = \mathbf{4.55}$ (about four and a half times faster).

Without pipelining 3000 ns; with pipelining 660 ns; speedup $3000 / 660 \approx 4.55$.

THE LECTURER'S REVISION DECK

Consider a pipeline having 4 phases with duration 60, 50, 90 and 80 ns. Given latch delay is 10 ns. Calculate:

- (i) Pipeline cycle time
- (ii) Non-pipeline execution time
- (iii) Speed up ratio
- (iv) Pipeline time for 500 tasks
- (v) Sequential time for 500 tasks

You have met this. It is **PQ 2025/2026 Q5(b)** and **PQ 2024/2025 Q5(b)**, exactly, and it is worked in Module Two Unit 3 and again in the Past Papers section. (The 2024/2025 version adds a sixth part, "Throughput", which this one drops.)

WORKED EXAMPLE

REVISION E

GIVEN	four phases of 60, 50, 90, 80 ns · latch delay = 10 ns · n = 500 tasks for parts (iv) and (v)
FIND	cycle time, non-pipeline time for one task, speedup, and the pipelined and sequential times for 500 tasks
FORMULA	cycle = slowest phase + latch · pipelined(n) = (k + n - 1) × cycle · sequential(n) = n × sum of phases · speedup = sum of phases / cycle

- 1 **(i) Cycle time** = slowest phase + latch = max(60, 50, 90, 80) + 10 = 90 + 10 = **100 ns**. The slowest phase sets the clock, not the average.
- 2 **(ii) Non-pipeline time** for one task = 60 + 50 + 90 + 80 = **280 ns**.
- 3 **(iii) Speed up ratio** = 280 / 100 = **2.8**.
- 4 **(iv) Pipeline time for 500 tasks** = (k + n - 1) × cycle = (4 + 499) × 100 = 503 × 100 = **50 300 ns**.
- 5 **(v) Sequential time for 500 tasks** = 500 × 280 = **140 000 ns**.

Cycle 100 ns; one task 280 ns; speedup 2.8; 500 tasks pipelined 50 300 ns against sequential 140 000 ns.

FROM THE SLIDES

REVISION F

THE LECTURER'S REVISION DECK

Describe each of the following categories of Data Hazards: RAW, WAR, WAW. Using (i) to (iii) below state which is RAW, WAR or WAW and indicate how each occurs using an arrow.

- | | | |
|-------------------|--------------------|---------------------|
| (i) R3 ← R1 op R2 | (ii) R3 ← R1 op R2 | (iii) R3 ← R1 op R2 |
| R5 ← R3 op R4 | R1 ← R4 op R5 | R3 ← R6 op R7 |

You have met this. It is the exact classification question already worked in Module Two Unit 3 (it was on the pipelining slides too). Here is the short answer.

GIVEN three pairs of instructions, each writing and reading registers
FIND which data hazard each pair is, with an arrow from the first access to the second
FORMULA find the one register on both lines, then read its order: write then read is RAW, read then write is WAR, write then write is WAW

PAIR	REGISTER ON BOTH LINES	ORDER	HAZARD
(i) R3 written, then R3 read	R3	write → read	RAW (read after write)
(ii) R1 read, then R1 written	R1	read → write	WAR (write after read)
(iii) R3 written, then R3 written	R3	write → write	WAW (write after write)

(i) RAW, (ii) WAR, (iii) WAW. Only RAW is a real data dependency; WAR and WAW are naming clashes that an ordinary in-order pipeline handles by reading in decode and writing in write-back.

REVISION CLASS

Cache Mapping, Every Case

The last third of the deck is one long cache exercise: direct, fully associative and set associative, each with three sizes. It is the same cache slide set already worked in Module Three, so here every case is laid out in one place for revision.

THE LECTURER'S REVISION DECK

For each cache, find the Offset, Index or Set bits, Tag bits, number of Cache lines, and the Tag Directory size.

Direct mapping Ex 1: MM 64 KB, Cache 1 KB, Block 16 B

Ex 2: MM 128 KB, Cache 4 KB, Block 32 B

Ex 3: MM 1 MB, Cache 8 KB, Block 64 B

Fully associative Ex 1: MM 64 KB, Cache 1 KB, Block 16 B

Ex 2: MM 256 KB, Cache 4 KB, Block 32 B

Ex 3: MM 2 MB, Cache 16 KB, Block 64 B

Set associative Ex 1 (2-way): MM 64 KB, Cache 1 KB, Block 16 B

Ex 2 (4-way): MM 512 KB, Cache 8 KB, Block 32 B

Ex 3 (8-way): MM 4 MB, Cache 32 KB, Block 64 B

You have met all of these. This is the cache slide set worked in Module Three Unit 3. **Example 1 of each mapping** (64 KB / 1 KB / 16 B) is also **PQ 2024/2025 Q2(a)**. Examples 2 and 3 are the practice bank in Module Three. All nine are solved below.

GIVEN	nine caches, each with a memory size, a cache size and a block size (and a way count for the set-associative three)
FIND	offset, index or set bits, tag bits, lines, and the tag-directory size for each
FORMULA	$\text{address} = \log_2(\text{memory}) \cdot \text{offset} = \log_2(\text{block}) \cdot \text{lines} = \text{cache} / \text{block} \cdot \text{sets} = \text{lines} / \text{ways} \cdot \text{tag} = \text{address} - \text{index (or set)} - \text{offset} \cdot \text{tag directory} = \text{lines} \times \text{tag bits}$ (divide by 8 for bytes)

The method never changes: work the address width from the memory size, the offset from the block, the lines from cache over block, and whatever is left after offset and index (or set) is the tag. Fully associative has no index at all, so its tag is the widest. The three tables below apply exactly this to all nine caches.

Direct mapping

CACHE (MM / CACHE / BLOCK)	ADDR	OFFSET	INDEX	TAG	LINES	TAG DIRECTORY
Ex 1: 64 KB / 1 KB / 16 B	16	4	6	6	64	384 bits (48 bytes)
Ex 2: 128 KB / 4 KB / 32 B	17	5	7	5	128	640 bits (80 bytes)
Ex 3: 1 MB / 8 KB / 64 B	20	6	7	7	128	896 bits (112 bytes)

Fully associative (no index field, so the whole non-offset part is tag)

CACHE (MM / CACHE / BLOCK)	ADDR	OFFSET	TAG	LINES	TAG DIRECTORY
Ex 1: 64 KB / 1 KB / 16 B	16	4	12	64	768 bits (96 bytes)
Ex 2: 256 KB / 4 KB / 32 B	18	5	13	128	1664 bits (208 bytes)
Ex 3: 2 MB / 16 KB / 64 B	21	6	15	256	3840 bits (480 bytes)

Set associative (sets = lines / ways; the set field replaces the index)

CACHE (MM / CACHE / BLOCK, WAYS)	ADDR	OFFSET	SET	TAG	LINES / SETS	TAG DIRECTORY
Ex 1: 64 KB / 1 KB / 16 B, 2-way	16	4	5	7	64 / 32	448 bits (56 bytes)
Ex 2: 512 KB / 8 KB / 32 B, 4-way	19	5	6	8	256 / 64	2048 bits (256 bytes)
Ex 3: 4 MB / 32 KB / 64 B, 8-way	22	6	6	10	512 / 64	5120 bits (640 bytes)

The lines are set by cache over block and never change with the mapping; the tag directory grows as the mapping gets more associative, because dropping the index lengthens the tag on every line. Every value above is checked against an independent recompute by the numeric gate.

Now redo it: pick any one row, cover the answer, and rebuild it from the four sizes. If you can do the 64 KB / 1 KB / 16 B case in all three mappings, you can do the exam's cache question in your

sleep.

TEACH

HOW THE DECK ENDS

The revision deck closes with two instructions to you: **read the slides on RISC versus CISC**, and **read every other slide provided**, then **prepare for the examination**.

Everything it points you to is already in this book: RISC versus CISC with its history, the multiply example and the microcode diagram is Module Four Unit 1; the pipelining, cache, data representation and instruction-set material fills Modules Two, Three and One. So the honest revision plan is short: work this section, then sit the three mock papers at the back under exam conditions.

OBJECTIVE SECTION

Both Tests, Every Option Explained

All 120 questions from the two computer-based tests, grouped by topic and answered in full. Not only why the answer is the answer: why each of the other three is not. Sit a topic, then read every line of its answer block.

Which analogy best represents computer organization?

~~the owner of the building~~

~~the cost estimate~~

☒ the construction process implementing the design

~~the building design~~

KILL THE WRONG ONES

How to Attack an Objective Question

Thirty of these decide the first third of the paper, and they are answered under more time pressure than anything in the theory half. This part is where you practise them.

TEACH

An objective question is not a smaller version of a theory question. It is a different task: you are not producing an answer, you are eliminating three wrong ones. That changes the method, and the method is worth marks on its own.

Every question below came from one of the two computer-based tests actually set on this course. Both are reproduced whole, 60 questions each, grouped by topic rather than by test so that a run of questions drills one skill at a time. The stems and options were read off the test screenshots by eye. That is a weaker guarantee than the four written papers in this book carry, since those are held character for character against a transcript by a gate, and it is said here rather than left for you to assume.

◆ KEY POINT

In a four-option question, three options are wrong for a statable reason. Finding the reason is faster and safer than recognising the right answer, because recognition is exactly what a well-written distractor attacks.

BLUNT VERSION: do not look for the right one. Kill the wrong ones.

TEACH

The four moves, in order, on every question:

- 1 Read the stem twice and underline the instruction word.** NOT, BEST, MOST DIRECTLY, PRIMARILY and EXCEPT each flip or narrow the question, and each one appears on these tests. A question containing NOT wants the odd one out, and reading it as a normal question makes every wrong answer look right.
- 2 Answer it before you read the options where you can.** If you know that organization is the construction and architecture is the blueprint, decide that first. Then the options are a lookup, not a suggestion.
- 3 Strike the impossible.** Two of the four usually die immediately: an option that contradicts a definition, names the wrong layer, or is true but answers a different question. Cross them out physically.
- 4 Choose between the last two on the exact wording.** This is where the mark is won or lost, and it is nearly always one word: only, always, all, determines, eliminates. A distractor that says "eliminates hardware complexity entirely" is wrong because of "entirely", not because of hardware complexity.

⚠ TRAP

The longest option is right more often than chance on these two tests, because a fully qualified statement takes more words than a slogan. Do not turn that into a rule. It is a tiebreaker for a question you cannot otherwise split, and nothing more. Several questions here punish it deliberately.

⚠ TRAP

One question on Test 1 asks how a floating point number is stored and its marked answer is Sign, Mantissa, Exponent. The true IEEE 754 field order, the one Module One Unit 3 teaches and the one every other question in this book uses, is Sign, Exponent, Mantissa. Learn the true order, and know that this particular question was marked the other way. Both facts are worth carrying: one is correct, the other is what the marking scheme did.

HOW TO USE THIS PART

Sit one topic at a time, timed at 45 seconds a question, with the answers covered. Then read the answer block for that topic in full, including the explanations for the questions you got right: on an objective paper, a right answer for a wrong reason is a mark you will not get twice.

OBJECTIVE SECTION

Architecture versus Organization

12 questions. Taught in Module One, Unit 1. Cover the answers before you start.

1. **TEST 1 Q16** An engineer describes computer architecture as the "blueprint" of a system. Which analogy best represents computer organization? (a) The owner of the building (b) The cost estimate (c) The construction process implementing the design (d) The building design
2. **TEST 1 Q24** Which of the following is NOT an example of a computer architecture component? (a) Memory technology used (b) Addressing modes (c) Register organization (d) Instruction set
3. **TEST 1 Q33** A cloud computing provider improves application response time by introducing multiple levels of cache memory. This improvement is mainly related to: (a) Data representation (b) Character encoding (c) ISA design (d) Memory organization
4. **TEST 1 Q34** A laptop and a desktop both use x86 processors. The desktop performs significantly better due to more CPU cores and larger caches. This performance difference arises mainly from: (a) Organizational differences (b) Different programming languages (c) Different operating systems (d) Architectural incompatibility
5. **TEST 1 Q35** A bank upgrades its servers to processors with larger caches and faster RAM while retaining the same x86 instruction set. What has primarily changed? (a) Architecture only (b) Organization only (c) Both architecture and organization (d) Neither architecture nor organization
6. **TEST 1 Q41** A computer scientist states that architecture and organization must be studied together because: (a) Organization alone determines functionality. (b)

Architecture alone determines performance. (c) Both concepts refer to the same thing. (d) Architecture defines capabilities, while organization determines efficiency and performance.

7. **TEST 2 Q26** A national supercomputing center is evaluating two competing processor designs. Processor X achieves higher benchmark performance but consumes significantly more power. Processor Y delivers slightly lower performance while offering substantially better energy efficiency and scalability. Considering contemporary computer architecture principles for high-performance computing, which evaluation is the most appropriate? (a) Processor Y may represent the superior architectural choice because modern processor evaluation considers performance/scalability, power efficiency, thermal constraints, and total cost of ownership rather than peak speed alone. (b) Both processors are architecturally equivalent. (c) Processor X is always preferable because peak performance is the sole design objective. (d) Energy efficiency has no relevance in processor architecture.
8. **TEST 2 Q27** A research laboratory designs a custom processor specifically optimized for matrix multiplication in artificial intelligence workloads. This design illustrates which architectural principle? (a) General-purpose optimization always outperforms specialization. (b) Matrix operations require no architectural optimization. (c) Specialized architectures eliminate software dependence. (d) Domain-specific architectures can significantly improve efficiency for targeted applications.
9. **TEST 2 Q31** Which statement best characterizes the distinction between computer architecture and computer engineering? (a) Computer engineering excludes hardware design. (b) Both disciplines are identical. (c) Computer architecture concerns only software. (d) Computer architecture focuses primarily on the conceptual design and operational behavior of computing systems, whereas computer engineering encompasses the broader design, implementation, and integration of hardware systems.
10. **TEST 2 Q38** Two processors implement exactly the same Instruction Set Architecture but exhibit significantly different execution times when running identical benchmark applications. Which explanation best accounts for this observation? (a) Different ISAs always produce identical performance. (b) The compiler determines all performance differences regardless of hardware. (c) Identical ISAs require identical transistor layouts. (d) Variations in clock frequency, cache hierarchy, execution units, and pipeline organization influence performance despite identical ISAs.
11. **TEST 2 Q46** Which statement most accurately reflects the role of abstraction in computer architecture? (a) Abstraction eliminates hardware complexity entirely. (b) Abstraction reduces processor speed. (c) Abstraction only applies to programming languages. (d) Abstraction enables designers to separate concerns across different layers, allowing software developers and hardware engineers to work independently while maintaining compatibility.
12. **TEST 2 Q54** A processor supports backward compatibility with software written twenty years ago despite significant improvements in hardware implementation. Which architectural philosophy is primarily responsible for this capability? (a) Eliminating compiler optimizations. (b) Maintaining a stable ISA while evolving the underlying organization and microarchitecture. (c) Replacing the ISA every processor generation. (d) Increasing clock frequency only.

1. The answer is **The construction process implementing the design**. Architecture is what the machine looks like to the programmer, the blueprint. Organization is how that blueprint is actually built and wired up, the construction process.

- ✗ **The owner of the building.** Wrong. The owner commissions the building. That is neither the design nor the act of building, so it maps to nothing in this pair.
- ✗ **The cost estimate.** Wrong. A cost estimate is a budget. Organization is about how the design is realised in hardware, not what it costs.
- ✓ **The construction process implementing the design.** Correct. Organization is the construction: the buses, the cache, the control paths that turn the blueprint into a working machine.
- ✗ **The building design.** Wrong. The building design IS the blueprint, and the stem has already given the blueprint to architecture. Choosing the building design says architecture and organization are the same thing.

2. The answer is **Memory technology used**. Architecture is the programmer-visible contract: instruction set, addressing modes, registers. Which memory technology is fitted is an implementation choice, so it belongs to organization.

- ✓ **Memory technology used.** Correct, and note the question asks which is NOT. Whether the machine uses SRAM, DRAM or flash cannot be seen from any program, so it is an organization decision.
- ✗ **Addressing modes.** Wrong choice, because addressing modes ARE architecture. They are the ways an instruction may name its operand, written into the ISA.
- ✗ **Register organization.** Wrong choice, because register organization IS architecture. The programmer-visible registers are named directly in instructions.
- ✗ **Instruction set.** Wrong choice, because the instruction set IS the architecture, the very heart of it.

3. The answer is **Memory organization**. Adding cache levels changes nothing a programmer can see in the instruction set. It changes how memory is arranged behind the scenes.

- ✗ **Data representation.** Wrong. Data representation is how numbers are encoded in bits. No number changed here.
- ✗ **Character encoding.** Wrong. Character encoding is ASCII and Unicode, which have nothing to do with cache levels.
- ✗ **ISA design.** Wrong. The instruction set is untouched. Programs run unmodified, they just run faster.
- ✓ **Memory organization.** Correct. Levels of cache are an arrangement of memory, which is precisely what memory organization means.

4. The answer is **Organizational differences**. Both run the same x86 instruction set, so the architecture is identical. Cores and cache size are organization, and organization is where performance comes from.

- ✓ **Organizational differences.** Correct. The stem names cores and caches, and both are organization.
- ✗ **Different programming languages.** Wrong. No language is mentioned, and the same program is assumed on both machines.
- ✗ **Different operating systems.** Wrong. No operating system is mentioned, and an OS would not account for a hardware gap of this kind.
- ✗ **Architectural incompatibility.** Wrong, and impossible. Both are x86, so the architectures are identical, not incompatible.

5. The answer is **Organization only**. The instruction set is untouched, so the architecture is untouched. Bigger caches and faster RAM are pure organization. The giveaway

phrase is "retaining the same instruction set".

- ✗ **Architecture only.** Wrong, and backwards. The one thing explicitly UNCHANGED is the instruction set, which is the architecture.
- ✓ **Organization only.** Correct. Cache size and RAM speed are implementation, so only the organization moved.
- ✗ **Both architecture and organization.** Wrong. It would need the ISA to have changed too, and the stem rules that out.
- ✗ **Neither architecture nor organization.** Wrong. Something did change: the caches and the memory. Only its category is in question.

6. The answer is Architecture defines capabilities, while organization determines efficiency and performance. The one-line rule for this whole topic: architecture says WHAT the machine can do, organization says HOW WELL it does it.

- ✗ **Organization alone determines functionality.** Wrong, the two are swapped. Functionality, what the machine can do at all, comes from the architecture.
- ✗ **Architecture alone determines performance.** Wrong, swapped the other way. Performance comes from the organization, which is why two x86 chips differ in speed.
- ✗ **Both concepts refer to the same thing.** Wrong. If they were the same thing there would be nothing to study together, and the whole distinction would collapse.
- ✓ **Architecture defines capabilities, while organization determines efficiency and performance.** Correct, and this is the sentence to memorise for every question in this topic.

7. The answer is Processor Y may represent the superior architectural choice because modern processor evaluation considers performance/scalability, power efficiency, thermal constraints, and total cost of ownership rather than peak speed alone. Modern evaluation weighs performance per watt, thermal limits, scalability and total cost of ownership, not peak speed alone. On this paper, any option containing an absolute such as "always", "sole" or "no relevance" is the wrong one.

- ✓ **Processor Y may represent the superior architectural choice because modern processor evaluation considers performance/scalability, power efficiency, thermal constraints, and total cost of ownership rather than peak speed alone.** Correct, and note the hedged verb "may represent". It weighs several criteria instead of crowning one.
- ✗ **Both processors are architecturally equivalent.** Wrong. The stem itself lists differences in performance, power and scalability, so they cannot be equivalent.
- ✗ **Processor X is always preferable because peak performance is the sole design objective.** Wrong on two counts: "always" admits no trade-off, and peak performance is not the sole objective in a facility that must be powered and cooled.
- ✗ **Energy efficiency has no relevance in processor architecture.** Wrong, and the strongest claim of the four. Power is a first-order constraint in high-performance computing, since a machine that cannot be cooled cannot be run.

8. The answer is Domain-specific architectures can significantly improve efficiency for targeted applications. Building the silicon around one workload is the definition of a domain-specific architecture, and the payoff is efficiency on that workload.

- ✗ **General-purpose optimization always outperforms specialization.** Wrong, and contradicted by the stem. If general-purpose always won, nobody would build the custom chip described.
- ✗ **Matrix operations require no architectural optimization.** Wrong. The entire scenario exists because matrix work rewards architectural optimisation.
- ✗ **Specialized architectures eliminate software dependence.** Wrong. A specialised chip still needs drivers, libraries and compilers. Hardware never removes the software.
- ✓ **Domain-specific architectures can significantly improve efficiency for targeted applications.** Correct. Narrow the workload, and you may spend the transistors where that workload actually needs them.

9. The answer is Computer architecture focuses primarily on the conceptual design and operational behavior of computing systems, whereas computer engineering encompasses the broader design, implementation, and integration of hardware systems. Architecture is the conceptual and operational design.

Engineering is the wider job of designing, implementing and integrating real hardware.

✗ **Computer engineering excludes hardware design.** Wrong, and the reverse of the truth.

Hardware design is the core of computer engineering.

✗ **Both disciplines are identical.** Wrong. Architecture is one concern inside the broader engineering discipline, not a synonym for it.

✗ **Computer architecture concerns only software.** Wrong. Architecture is the interface BETWEEN hardware and software, so it can never be software only.

✓ **Computer architecture focuses primarily on the conceptual design and operational behavior of computing systems, whereas computer engineering encompasses the broader design, implementation, and integration of hardware systems.** Correct. Conceptual and operational design on one side, the broader build and integration of hardware on the other.

10. The answer is Variations in clock frequency, cache hierarchy, execution units, and pipeline organization influence performance despite identical ISAs.

A shared ISA guarantees the same instructions, never the same speed. Clock, cache, execution units and pipeline depth are microarchitecture, and they are free to differ.

✗ **Different ISAs always produce identical performance.** Wrong, and it answers a question that was not asked. The stem says the ISAs are the SAME, and the claim itself is false anyway.

✗ **The compiler determines all performance differences regardless of hardware.** Wrong. The benchmark is identical on both machines, so the compiler is a constant here, not a variable.

✗ **Identical ISAs require identical transistor layouts.** Wrong, and it is the exact misconception being tested. An ISA fixes the interface, never the transistors under it.

✓ **Variations in clock frequency, cache hierarchy, execution units, and pipeline organization influence performance despite identical ISAs.** Correct. This is the microarchitecture, and it is where identical-ISA machines diverge.

11. The answer is Abstraction enables designers to separate concerns across different layers, allowing software developers and hardware engineers to work independently while maintaining compatibility. Abstraction hides complexity behind a stable interface, it does not delete it. That separation of layers is what lets the two sides of the industry work independently and still interoperate.

✗ **Abstraction eliminates hardware complexity entirely.** Wrong. The complexity is still there, it is merely hidden behind the interface. "Entirely" gives this away.

✗ **Abstraction reduces processor speed.** Wrong. Abstraction is a design principle, not a runtime cost imposed on the processor.

✗ **Abstraction only applies to programming languages.** Wrong. The layering runs from transistors up through logic, microarchitecture, ISA, operating system and applications.

✓ **Abstraction enables designers to separate concerns across different layers, allowing software developers and hardware engineers to work independently while maintaining compatibility.** Correct. Separation of concerns across layers, with compatibility preserved at each interface.

12. The answer is Maintaining a stable ISA while evolving the underlying organization and microarchitecture. Old binaries keep running because the programmer-visible contract, the ISA, is frozen. Everything under it is free to be rebuilt generation after generation.

✗ **Eliminating compiler optimizations.** Wrong. Twenty-year-old binaries were compiled long ago, so nothing the compiler does now affects whether they run.

- ✓ **Maintaining a stable ISA while evolving the underlying organization and microarchitecture.** Correct. Freeze the interface, evolve the implementation. That is the whole trick of backward compatibility.
- ✗ **Replacing the ISA every processor generation.** Wrong, and it is the opposite policy. A new ISA every generation would break every old program.
- ✗ **Increasing clock frequency only.** Wrong. A faster clock makes old software run faster, it does not make it run at all.

OBJECTIVE SECTION

Instruction Set Architecture and Microarchitecture

13 questions. Taught in Module One, Unit 4. Cover the answers before you start.

1. **TEST 1 Q22** A game console experiences improved frame rates because the compiler uses efficient memory-access techniques. Which architectural component is most directly involved? (a) Control signals (b) System buses (c) Addressing modes (d) Memory chips
2. **TEST 1 Q39** Which of the following would most likely be visible to a programmer? (a) Memory chip technology (b) Cache implementation details (c) Control signal generation circuitry (d) Instruction formats
3. **TEST 1 Q50** A programmer wants to know whether a processor supports floating-point instructions. This concern relates to: (a) Control signals (b) Memory hierarchy (c) Bus organization (d) Computer architecture
4. **TEST 2 Q5** A compiler attempts to minimize memory accesses by keeping frequently used variables inside processor registers. Which instruction category benefits the most? (a) Register-register arithmetic (b) Privileged instructions (c) Input/output instructions (d) Interrupt instructions
5. **TEST 2 Q6** Which statement best distinguishes ISA from microarchitecture? (a) ISA specifies transistor layout. (b) ISA determines cache replacement policies. (c) ISA specifies the programmer-visible interface. (d) ISA defines how instructions are executed internally.
6. **TEST 2 Q10** Which instruction is typically executed immediately after a successful comparison operation? (a) Divide (b) Conditional Branch (c) STORE (d) LOAD
7. **TEST 2 Q13** Which instruction category is responsible for moving data between CPU registers and memory? (a) Floating-point instructions (b) Data transfer instructions (c) Control instructions (d) Logical instructions
8. **TEST 2 Q14** Which instruction would most likely access the main memory? (a) XOR (b) LOAD (c) SHIFT (d) ADD
9. **TEST 2 Q16** Why are specialized instructions such as SIMD instructions introduced into modern processors? (a) To increase instruction fetch time (b) To simplify cache design (c) To eliminate registers (d) To accelerate data-parallel computations
10. **TEST 2 Q22** A software developer successfully runs the same compiled application on two processors from different manufacturers. This is possible because both processors implement the same: (a) Instruction Set Architecture (b) Microarchitecture (c) Control unit design (d) Cache hierarchy
11. **TEST 2 Q33** Which instruction category primarily changes the sequence of program execution? (a) Arithmetic (b) Control Transfer (c) Data Transfer (d) Logical

12. **TEST 2 Q40** Which of the following is NOT defined by an Instruction Set Architecture?
(a) Internal ALU implementation (b) Addressing modes (c) Instruction formats (d) Programmer-visible registers
13. **TEST 2 Q55** A processor supports arithmetic, logical, branch, load/store, and input/output instructions. These collectively constitute its: (a) Instruction set (b) Address bus (c) Register file (d) Pipeline stages

1. The answer is **Addressing modes**. Addressing modes are the ways an instruction may name its operand, and they are the part of the ISA a compiler actually chooses between when it optimises memory access.

- ✗ **Control signals.** Wrong. Control signals are generated by the hardware itself. A compiler cannot select them.
- ✗ **System buses.** Wrong. Buses are physical organization. The compiler cannot choose a different bus.
- ✓ **Addressing modes.** Correct. The stem says the COMPILER did it, and addressing modes are the memory-access choice available to a compiler.
- ✗ **Memory chips.** Wrong. Memory chips are hardware fitted to the board, invisible to and unselectable by the compiler.

2. The answer is **Instruction formats**. Instruction formats are part of the ISA, which is by definition the programmer-visible interface. Everything else listed sits below that line, in the organization.

- ✗ **Memory chip technology.** Wrong. A program behaves the same whichever memory technology is fitted, so the technology is invisible to it.
- ✗ **Cache implementation details.** Wrong. Cache is deliberately transparent: it changes timing, never results.
- ✗ **Control signal generation circuitry.** Wrong. Control circuitry is deep microarchitecture, generated inside the processor and never named by a program.
- ✓ **Instruction formats.** Correct. A programmer or compiler must know the exact bit layout to encode an instruction at all.

3. The answer is **Computer architecture**. Which instructions EXIST is an architecture question, settled by the ISA. How fast they run would be an organization question.

- ✗ **Control signals.** Wrong. Control signals implement instructions that already exist, they do not decide which exist.
- ✗ **Memory hierarchy.** Wrong. The memory hierarchy affects how quickly operands arrive, not whether an instruction is supported.
- ✗ **Bus organization.** Wrong. Bus organization is about moving data between units, again a speed matter.
- ✓ **Computer architecture.** Correct. The set of supported instructions is exactly what the architecture defines.

4. The answer is **Register-register arithmetic**. Register-register arithmetic takes both operands from registers and writes the result back to a register, so it never touches memory at all.

- ✓ **Register-register arithmetic.** Correct. Operands already in registers is precisely the condition this category needs.
- ✗ **Privileged instructions.** Wrong. Privileged instructions are about protection and operating-system mode, not operand location.
- ✗ **Input/output instructions.** Wrong. Input/output instructions talk to devices, so keeping variables in registers does not help them.
- ✗ **Interrupt instructions.** Wrong. Interrupt instructions handle external events. They are unaffected by where a variable lives.

5. The answer is **ISA specifies the programmer-visible interface**. The ISA is WHAT the machine does, the interface a programmer codes against. The microarchitecture is HOW it is done inside: transistors, caches, pipelines.

- ✗ **ISA specifies transistor layout.** Wrong. Transistor layout is physical implementation, the far end of microarchitecture.

- ✗ **ISA determines cache replacement policies.** Wrong. Cache replacement, whether LRU or random, is invisible to a program and is a microarchitecture policy.
- ✓ **ISA specifies the programmer-visible interface.** Correct, and this is the definition to memorise.
- ✗ **ISA defines how instructions are executed internally.** Wrong. "How instructions are executed internally" is the definition of MICROARCHITECTURE, offered here as the near miss.

6. The answer is **Conditional Branch**. A compare sets the condition flags. The instruction that reads those flags and decides where to go next is the conditional branch.

- ✗ **Divide.** Wrong. Divide is arithmetic. It neither reads nor needs the condition flags.
- ✓ **Conditional Branch.** Correct. Compare then branch is the standard pair, which is how an if statement is built.
- ✗ **STORE.** Wrong. STORE writes a register to memory and ignores the flags.
- ✗ **LOAD.** Wrong. LOAD reads memory into a register and also ignores the flags.

7. The answer is **Data transfer instructions**. Data transfer is the category that only moves values around: LOAD, STORE, MOVE. It changes location, never value.

- ✗ **Floating-point instructions.** Wrong. Floating-point instructions compute on real numbers, they do not exist to move data.
- ✓ **Data transfer instructions.** Correct. LOAD, STORE and MOVE are its members.
- ✗ **Control instructions.** Wrong. Control instructions change which instruction runs next, not where data sits.
- ✗ **Logical instructions.** Wrong. Logical instructions such as AND, OR and NOT compute on values already held.

8. The answer is **LOAD**. LOAD fetches an operand from main memory into a register. The others work on values already held in registers.

- ✗ **XOR.** Wrong. XOR is a logical operation between register values.
- ✓ **LOAD.** Correct. Reaching into memory for an operand is the entire job of a LOAD.
- ✗ **SHIFT.** Wrong. SHIFT moves bits within a single register.
- ✗ **ADD.** Wrong. ADD sums register values, which is why compilers work so hard to keep operands in registers.

9. The answer is **To accelerate data-parallel computations**. SIMD is Single Instruction, Multiple Data: one instruction applies the same operation to many values at once, which is exactly what graphics and matrix work need.

- ✗ **To increase instruction fetch time.** Wrong, and nobody adds a feature to make fetching SLOWER.
- ✗ **To simplify cache design.** Wrong. SIMD makes the datapath wider and the design more complex, not the cache simpler.
- ✗ **To eliminate registers.** Wrong, and backwards. SIMD needs wide vector registers, so it adds register state rather than removing it.
- ✓ **To accelerate data-parallel computations.** Correct. One instruction, many data elements, which is where the name comes from.

10. The answer is **Instruction Set Architecture**. A compiled binary is a stream of ISA instructions. Any processor implementing that ISA can execute it, whoever built it and however differently it is built inside.

- ✓ **Instruction Set Architecture.** Correct. The ISA is the contract a binary is written against.
- ✗ **Microarchitecture.** Wrong, and the point of the question is that the two microarchitectures are almost certainly different.
- ✗ **Control unit design.** Wrong. Control unit design is internal and varies freely between manufacturers.
- ✗ **Cache hierarchy.** Wrong. Cache sizes and levels differ between vendors and are invisible to the binary anyway.

11. The answer is **Control Transfer**. Control transfer instructions, the jumps, branches, calls and returns, write a new value into the program counter and so redirect the flow.

- ✗ **Arithmetic.** Wrong. Arithmetic computes values, then execution simply falls through to the next instruction.
- ✓ **Control Transfer.** Correct. Jump, branch, call and return are its members.
- ✗ **Data Transfer.** Wrong. Data transfer moves values between registers and memory, leaving the flow untouched.
- ✗ **Logical.** Wrong. Logical instructions compute a result. Only a following branch changes the sequence.

12. The answer is **Internal ALU implementation**. The ISA defines the instructions, their formats, their addressing modes and the registers you can name. How the ALU is built to carry out an ADD is microarchitecture.

- ✓ **Internal ALU implementation.** Correct, and remember the question asks which is NOT. Two chips with the same ISA may use completely different adder designs, and no program can tell.
- ✗ **Addressing modes.** Wrong choice. Addressing modes ARE in the ISA, since an instruction must state how it names its operand.
- ✗ **Instruction formats.** Wrong choice. Instruction formats ARE in the ISA, since the bit layout must be fixed for code to be encoded.
- ✗ **Programmer-visible registers.** Wrong choice. Programmer-visible registers ARE in the ISA, since instructions name them directly.

13. The answer is **Instruction set**. Every instruction a processor understands, taken together, is its instruction set. The five categories listed are the standard grouping of that set.

- ✓ **Instruction set.** Correct. The word "collectively" is the clue: all the instructions together are the set.
- ✗ **Address bus.** Wrong. The address bus is a set of wires carrying a location, not a collection of instructions.
- ✗ **Register file.** Wrong. The register file is storage inside the CPU, not the operations it can perform.
- ✗ **Pipeline stages.** Wrong. Pipeline stages are how one instruction is processed in steps, unrelated to how many instructions exist.

OBJECTIVE SECTION

Number Systems and Codes

16 questions. Taught in Foundations F.1 and F.2, and Module One, Unit 2. Cover the answers before you start.

1. **TEST 1 Q1** BCD stands for: (a) Binary Control Digit (b) Base Conversion Decimal (c) Binary Coded Decimal (d) Binary Character Decimal
2. **TEST 1 Q8** The decimal value 255 represented in base 16 is: (a) FE (b) FF (c) AE (d) F0
3. **TEST 1 Q10** Why is Excess-3 considered self-complementing? (a) Its 1's complement represents the 9's complement of the original digit (b) It eliminates overflow (c) It uses 3-bit encoding (d) It automatically detects errors
4. **TEST 1 Q11** Why is hexadecimal preferred for displaying binary information? (a) It increases precision (b) It groups bits conveniently into sets of four (c) It reduces storage requirements (d) It speeds processor execution

5. TEST 1 Q12 Unicode was introduced primarily to: (a) Replace binary arithmetic (b) Support multiple world languages (c) Improve floating-point performance (d) Reduce storage
6. TEST 1 Q13 A company logo needs to be enlarged to billboard size without losing quality. Which format is most appropriate? (a) ASCII (b) Bitmap (c) Vector (d) BCD
7. TEST 1 Q14 If a hexadecimal value is shifted right by one bit, the operation is equivalent to: (a) Multiplication by 2 (b) Division by 16 (c) Division by 2 (d) Multiplication by 16
8. TEST 1 Q17 The 8421 BCD representation of decimal 59 is: (a) 00110101 (b) 00111011 (c) 01011100 (d) 01011001
9. TEST 1 Q27 A binary number occupies 20 bits. How many hexadecimal digits are needed to represent it exactly? (a) 4 (b) 6 (c) 8 (d) 5
10. TEST 1 Q28 A processor receives the hexadecimal value 7D3A. What is its decimal equivalent? (a) 32090 (b) 32058 (c) 32042 (d) 32122
11. TEST 1 Q30 A forensic analyst discovers that a malware author stored the value 111010111011 (binary). What is its hexadecimal equivalent? (a) EBB (b) EBB16 (c) EAB (d) EBB and EBB16 are equivalent
12. TEST 1 Q36 Which code was specifically designed to simplify decimal arithmetic operations? (a) Unicode (b) ASCII (c) EBCDIC (d) Excess-3
13. TEST 1 Q42 The Excess-3 code for decimal 9 is: (a) 1001 (b) 1010 (c) 1100 (d) 1111
14. TEST 1 Q43 The hexadecimal value 2A7F corresponds to which binary representation? (a) 0010101001111111 (b) 1010101111111111 (c) 1010100011111111 (d) 0010101001111110
15. TEST 1 Q57 Excess-3 code is: (a) Non-weighted (b) Weighted (c) ASCII-based (d) Floating-point
16. TEST 1 Q59 The Excess-3 code 0110 corresponds to decimal: (a) 6 (b) 9 (c) 12 (d) 3

1. The answer is **Binary Coded Decimal**. Binary Coded Decimal. Each decimal digit is coded separately in four bits, which is why 59 becomes 0101 1001 rather than the pure binary 111011.

- ✗ **Binary Control Digit**. Wrong. No such term. "Control" belongs to the control unit, not to a numeric code.
- ✗ **Base Conversion Decimal**. Wrong. Base conversion is an operation you perform, not the name of a code.
- ✓ **Binary Coded Decimal**. Correct. Binary Coded Decimal: decimal digits, each coded in binary.
- ✗ **Binary Character Decimal**. Wrong. "Character" points at text codes such as ASCII, while BCD codes numbers.

2. The answer is **FF**. $255 = 15 \times 16 + 15$, and 15 is F, so the answer is FF. Worth memorising: 255 is FF, is 11111111, and is the largest value eight unsigned bits can hold.

- ✗ **FE**. Wrong. FE is $15 \times 16 + 14 = 254$, one short.
- ✓ **FF**. Correct. Both digits are F, that is $15 \times 16 + 15 = 255$.
- ✗ **AE**. Wrong. AE is $10 \times 16 + 14 = 174$.
- ✗ **F0**. Wrong. F0 is $15 \times 16 + 0 = 240$.

3. The answer is **Its 1's complement represents the 9's complement of the original digit**. Take 3, whose Excess-3 code is 0110. Invert every bit and you get 1001, the Excess-3 code for 6, and 6 is the 9's complement of 3. Inverting the bits performs the decimal 9's complement for free.

- ✓ **Its 1's complement represents the 9's complement of the original digit**. Correct. Bit inversion in the code equals 9's complement in decimal, which is what self-complementing means.
- ✗ **It eliminates overflow**. Wrong. No code eliminates overflow. Overflow is a range problem, not an encoding problem.
- ✗ **It uses 3-bit encoding**. Wrong. Excess-3 uses FOUR bits per digit. The 3 in its name is the excess added to the digit, not a bit count.
- ✗ **It automatically detects errors**. Wrong. Error detection needs a parity or checksum scheme. Excess-3 carries no check bit.

4. The answer is **It groups bits conveniently into sets of four**. One hex digit is exactly four bits, because 16 is 2 to the power 4. The bits are simply easier for a human to read.

- ✗ **It increases precision**. Wrong. Changing the display base cannot change the value or its precision.
- ✓ **It groups bits conveniently into sets of four**. Correct. Four bits per digit means a 32-bit word reads as eight tidy characters.
- ✗ **It reduces storage requirements**. Wrong. The stored bits are identical. Only the way they are printed changes.
- ✗ **It speeds processor execution**. Wrong. Hexadecimal is a notation for humans. The processor works in binary either way.

5. The answer is **Support multiple world languages**. ASCII in seven bits covers 128 characters, enough for English alone. Unicode was created to give every script in the world a code point.

- ✗ **Replace binary arithmetic**. Wrong. Unicode is a character code. It has nothing to do with arithmetic.
- ✓ **Support multiple world languages**. Correct. One code space covering the world's writing systems.
- ✗ **Improve floating-point performance**. Wrong. Floating point is a numeric format, untouched by a text standard.

- ✗ **Reduce storage.** Wrong, and backwards. Unicode uses more bits per character than ASCII, so it costs storage rather than saving it.
6. The answer is **Vector**. A vector image stores shapes as mathematical descriptions, so it is redrawn at any size. A bitmap stores a fixed grid of pixels, so enlarging it enlarges the pixels.
- ✗ **ASCII.** Wrong. ASCII encodes text characters, not images.
- ✗ **Bitmap.** Wrong, and this is the trap. A bitmap has a fixed pixel grid, so blowing it up to billboard size shows the blocks.
- ✓ **Vector.** Correct. Shapes are stored as maths and redrawn cleanly at any scale.
- ✗ **BCD.** Wrong. BCD is a numeric code for decimal digits, with no image capability.
7. The answer is **Division by 2**. Shifting is a BIT operation, so the base you display the number in is irrelevant. One bit right halves the value.
- ✗ **Multiplication by 2.** Wrong direction. Multiplying by 2 is a shift LEFT.
- ✗ **Division by 16.** Wrong. Dividing by 16 needs a shift of four bits, or one whole hex digit. The stem says one BIT.
- ✓ **Division by 2.** Correct. Each bit position right is one halving.
- ✗ **Multiplication by 16.** Wrong on both counts, wrong direction and wrong factor.
8. The answer is **01011001**. Code each digit on its own in four bits: 5 is 0101 and 9 is 1001, so 59 is 0101 1001.
- ✗ **00110101.** Wrong. 0011 0101 is BCD for 35, the digits reversed.
- ✗ **00111011.** Wrong, and this is the trap. 00111011 is the PURE BINARY 59, not its BCD coding.
- ✗ **01011100.** Wrong. The first nibble 0101 is right for 5, but 1100 is 12, which is not a valid BCD digit at all.
- ✓ **01011001.** Correct. 0101 for 5 and 1001 for 9.
9. The answer is **5**. Four bits per hex digit, so 20 divided by 4 is 5. Run the rule backwards and a 5-digit hex value needs 20 bits.
- ✗ **4.** Wrong. 4 hex digits hold only 16 bits.
- ✗ **6.** Wrong. 6 hex digits hold 24 bits, more than asked for, and the question says exactly.
- ✗ **8.** Wrong. 8 hex digits hold 32 bits, the size of a full word.
- ✓ **5.** Correct. 20 divided by 4 is 5.
10. The answer is **32058**. Weight each digit by its power of 16: $7 \times 4096 = 28672$, D is 13 so $13 \times 256 = 3328$, $3 \times 16 = 48$, and A is 10. Adding gives $28672 + 3328 + 48 + 10 = 32058$.
- ✗ **32090.** Wrong, 32 too high. It is what you get by mis-weighting the third digit.
- ✓ **32058.** Correct. $28672 + 3328 + 48 + 10 = 32058$.
- ✗ **32042.** Wrong, 16 too low, the result of reading the 3 as a 2.
- ✗ **32122.** Wrong, 64 too high. Check the $3 \times 16 = 48$ term.
11. The answer is **EBB and EBB16 are equivalent**. Group the bits in fours from the right: 1110 1011 1011, which is E, B, B.
- ✗ **EBB.** Right value, but incomplete. The paper also offers the same value written with its base, so a is not the fullest answer.
- ✗ **EBB16.** Also right in value. EBB16 is EBB with the base 16 written as a subscript.
- ✗ **EAB.** Wrong. EAB would need the middle group to be 1010, but it is 1011.
- ✓ **EBB and EBB16 are equivalent.** Correct, and this is why the paper set the question. The subscript states the base, it does not change the value, so a and b name the same number.
12. The answer is **Excess-3**. Excess-3 is a numeric code, built self-complementing so decimal subtraction can be done by complementing and adding.
- ✗ **Unicode.** Wrong. Unicode is a character code for the world's languages.

- ✗ **ASCII.** Wrong. ASCII is a character code for text.
- ✗ **EBCDIC.** Wrong. EBCDIC is another character code, IBM's rival to ASCII.
- ✓ **Excess-3.** Correct, and it is the only NUMERIC code in the list. The other three all encode text.

13. The answer is **1100**. Add the excess of 3 first: $9 + 3 = 12$, and 12 in four bits is 1100.

- ✗ **1001.** Wrong, and the classic trap. 1001 is plain BCD for 9, with the excess not yet added.
- ✗ **1010.** Wrong. 1010 is 10, which would be the Excess-3 code for 7.
- ✓ **1100.** Correct. $9 + 3 = 12 = 1100$.
- ✗ **1111.** Wrong. 1111 is 15, which would encode the digit 12, and no such decimal digit exists.

14. The answer is **0010101001111111**. Expand each digit into four bits: 2 is 0010, A is 1010, 7 is 0111, F is 1111.

- ✓ **0010101001111111.** Correct. 0010 1010 0111 1111, read straight off the four digits.
- ✗ **1010101111111111.** Wrong. It starts 1010, which is A, so this decodes as ABFF and not 2A7F.
- ✗ **1010100011111111.** Wrong. Its first two groups give A8, not 2A.
- ✗ **0010101001111110.** Wrong by exactly one bit. It ends 1110, which is E, so the final F is lost. Always check the last nibble.

15. The answer is **Non-weighted**. In a weighted code such as 8421 BCD each bit position carries a fixed value you can add up. Excess-3 shifts every code by 3 first, so the positions no longer carry those weights.

- ✓ **Non-weighted.** Correct. No set of positional weights reproduces the digit, because of the added 3.
- ✗ **Weighted.** Wrong. That describes 8421 BCD, where the bits are worth 8, 4, 2 and 1.
- ✗ **ASCII-based.** Wrong. ASCII is a character code and has no bearing on Excess-3.
- ✗ **Floating-point.** Wrong. Excess-3 codes a single decimal digit, and has no sign, exponent or mantissa.

16. The answer is **3**. Decode by subtracting the excess: 0110 is 6, and 6 minus 3 is 3.

- ✗ **6.** Wrong, and the trap. 6 is the raw binary value before the excess of 3 is removed.
- ✗ **9.** Wrong. That is 6 plus 3, so it adds the excess when decoding instead of subtracting it.
- ✗ **12.** Wrong. 12 is the code you would WRITE for the digit 9, not a digit itself.
- ✓ **3.** Correct. 6 minus 3 is 3.

OBJECTIVE SECTION

Signed Numbers and Complements

14 questions. Taught in Module One, Unit 2. Cover the answers before you start.

1. **TEST 1 Q2** What is the range of values representable by a 10-bit unsigned integer? (a) -1023 to 1023 (b) 0 to 1023 (c) -512 to 511 (d) 0 to 1024
2. **TEST 1 Q3** The 1's complement representation 11100011 corresponds to: (a) -29 (b) -28 (c) -31 (d) -30
3. **TEST 1 Q6** Which statement about one's complement is TRUE? (a) It has one representation of zero (b) Overflow never occurs (c) It has positive and negative zero (d) It has no negative numbers
4. **TEST 1 Q20** In one's complement representation, the additive inverse of a number is obtained by: (a) Shifting right (b) Flipping all bits (c) Adding 1 (d) Shifting left
5. **TEST 1 Q21** For an 8-bit sign-magnitude system, the decimal value -0 is represented by: (a) 11111111 (b) 00000000 (c) 01111111 (d) 10000000

6. TEST 1 Q26 An 8-bit unsigned register stores 11111111. What decimal value does it represent? (a) 127 (b) 255 (c) 254 (d) 128
7. TEST 1 Q32 An 8-bit register contains 10000000 in two's complement. What decimal value does it represent? (a) -127 (b) +128 (c) -128 (d) 0
8. TEST 1 Q45 How many representations of zero exist in sign-magnitude representation? (a) 2 (b) Depends on word length (c) 1 (d) 4
9. TEST 1 Q47 The two's complement of 00101100 is: (a) 10110100 (b) 11110011 (c) 11010011 (d) 11010100
10. TEST 1 Q49 Overflow occurs when: (a) Result exceeds representable range (b) Division by zero occurs (c) Sign changes unexpectedly (d) Bits are shifted
11. TEST 1 Q53 Which representation requires an end-around carry during addition? (a) Excess-3 (b) Sign Magnitude (c) One's Complement (d) Two's Complement
12. TEST 1 Q55 In sign-magnitude representation, the 8-bit value 10101010 represents: (a) +170 (b) -170 (c) +42 (d) -42
13. TEST 1 Q56 Why is two's complement widely used in modern computers? (a) Easier decimal conversion (b) Eliminates overflow (c) Greater storage efficiency (d) Faster and simpler hardware implementation
14. TEST 1 Q60 The major hardware disadvantage of sign-magnitude representation is: (a) Inability to represent positive numbers (b) Slow and complex arithmetic circuitry (c) Large memory usage (d) Limited range

1. The answer is **0 to 1023**. An n-bit unsigned range is 0 to 2 to the power n, minus 1. Here 2 to the power 10 is 1024, so the range is 0 to 1023.

✗ **-1023 to 1023**. Wrong. UNSIGNED means no negative values at all, so the range cannot start below zero.

✓ **0 to 1023**. Correct. 1024 distinct patterns, counting from 0, so the top value is 1023.

✗ **-512 to 511**. Wrong. That is the 10-bit TWO'S COMPLEMENT range, offered to catch a reader who skimmed past the word unsigned.

✗ **0 to 1024**. Wrong by one. There are 1024 patterns, but they run 0 to 1023, because 0 uses one of them.

2. The answer is **-28**. The leading bit is 1, so the value is negative. Invert every bit to recover the magnitude: 11100011 becomes 00011100, which is $16 + 8 + 4 = 28$.

✗ **-29**. Wrong, and the trap. -29 is what this pattern means in TWO'S complement, where you invert and add 1.

✓ **-28**. Correct. Invert only, giving 28, and apply the negative sign for -28.

✗ **-31**. Wrong. 31 would be 00011111 after inversion, so the original would have been 11100000.

✗ **-30**. Wrong. 30 would be 00011110 after inversion, so the original would have been 11100001.

3. The answer is **It has positive and negative zero**. 00000000 is +0 and 11111111 is -0, so one's complement carries two zeros, just as sign-magnitude does. Only two's complement has a single zero.

✗ **It has one representation of zero**. Wrong. A single zero is the TWO'S complement property, and it is one of the reasons two's complement won.

✗ **Overflow never occurs**. Wrong. Overflow depends on the width of the word, not on the negation scheme, so it can occur in any of them.

✓ **It has positive and negative zero**. Correct. 00000000 and 11111111 are both zero, which is the scheme's central weakness.

✗ **It has no negative numbers**. Wrong. Representing negatives is the entire purpose of a complement scheme.

4. The answer is **Flipping all bits**. One's complement negation is a straight bit inversion, nothing more.

✗ **Shifting right**. Wrong. Shifting right divides by 2, it does not change the sign.

✓ **Flipping all bits**. Correct. Invert every bit and you have the additive inverse.

✗ **Adding 1**. Wrong on its own, and this is the near miss. Invert AND add 1 gives the TWO'S complement. Adding 1 by itself simply counts up.

✗ **Shifting left**. Wrong. Shifting left multiplies by 2.

5. The answer is **10000000**. Sign-magnitude splits the word into a sign bit and a magnitude. Negative zero is sign bit 1 with a magnitude of zero.

✗ **11111111**. Wrong. 11111111 is negative zero in ONE'S complement, and in sign-magnitude it reads as -127.

✗ **00000000**. Wrong. That is POSITIVE zero, sign bit 0 with zero magnitude.

✗ **01111111**. Wrong. Sign bit 0 with magnitude 1111111 is +127.

✓ **10000000**. Correct. Sign bit 1, magnitude 0000000, so minus nothing.

6. The answer is **255**. Eight ones is 2 to the power 8, minus 1, which is 255. The word UNSIGNED means there is no sign bit to set aside.

✗ **127**. Wrong. 127 is 01111111, the largest value in SEVEN bits, which is what you get if you wrongly reserve a sign bit.

✓ **255**. Correct. $128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 = 255$.

✗ **254**. Wrong by one. 254 is 11111110.

✗ **128.** Wrong. 128 is 10000000, just the top bit.

7. The answer is **-128**. Two's complement is asymmetric: 8 bits run from -128 to +127, and 10000000 is that most negative value. Weight the sign bit negatively and you get $-128 + 0 = -128$.

✗ **-127.** Wrong. -127 is 10000001. Invert and add 1 to check: 10000001 gives 01111111, which is 127.

✗ **+128.** Wrong, and impossible. +128 does not fit in 8-bit two's complement, where the largest positive is +127.

✓ **-128.** Correct. The one negative value with no positive partner.

✗ **0.** Wrong. Zero is 00000000, and two's complement has only that one zero. Reading this pattern as -0 is the sign-magnitude habit.

8. The answer is **2**. There are always exactly two, +0 and -0, because the sign bit is free to be either value while the magnitude is zero.

✓ **2.** Correct. One sign bit, two settings, both with an all-zero magnitude.

✗ **Depends on word length.** Wrong. A longer word adds more magnitude bits, but there is still only ever one sign bit, so the count stays at 2.

✗ **1.** Wrong. A single zero is the TWO'S complement property.

✗ **4.** Wrong. Four would need two sign bits, which no scheme uses.

9. The answer is **11010100**. Two steps, in order. Invert 00101100 to get 11010011, then add 1 to get 11010100. Check it: the original is 44, and the answer should read as -44.

✗ **10110100.** Wrong. This matches neither step. Inverting gives 11010011, and adding 1 gives 11010100.

✗ **11110011.** Wrong. 11110011 is not the inversion of 00101100. Compare them bit by bit.

✗ **11010011.** Wrong, and the classic near miss. 11010011 is the ONE'S complement, the result of stopping before you add 1.

✓ **11010100.** Correct. Inverted, then incremented.

10. The answer is **Result exceeds representable range**. Overflow is a result too large for the word to hold.

✓ **Result exceeds representable range.** Correct, and this is the definition. The word has a fixed width, and the answer did not fit.

✗ **Division by zero occurs.** Wrong. Division by zero is a separate arithmetic exception with its own trap.

✗ **Sign changes unexpectedly.** Wrong, but tempting: an unexpected sign flip is how you DETECT two's complement overflow. It is the symptom, not the definition.

✗ **Bits are shifted.** Wrong. Shifting is a normal operation, though shifting bits out of the top can itself cause a loss of value.

11. The answer is **One's Complement**. In one's complement, a carry out of the most significant bit must be added back into the least significant bit.

✗ **Excess-3.** Wrong. Excess-3 is a digit code, not a signed number scheme, so end-around carry does not arise.

✗ **Sign Magnitude.** Wrong. Sign-magnitude addition compares magnitudes and decides the sign. There is no carry to wrap.

✓ **One's Complement.** Correct. The carry out is fed back into the low end, and the fix exists because of the two zeros.

✗ **Two's Complement.** Wrong. Two's complement simply DISCARDS the carry out, which is one of the reasons its hardware is simpler.

12. The answer is **-42**. Split off the sign bit: 1 means negative, and the remaining seven bits 0101010 are $32 + 8 + 2 = 42$.

- ✗ **+170.** Wrong twice. It reads all eight bits as the magnitude, giving 170, and then ignores the sign bit.
- ✗ **-170.** Wrong once. The sign is right, but the magnitude wrongly includes the sign bit, giving 170 instead of 42.
- ✗ **+42.** Wrong once. The magnitude is right, but the leading 1 was ignored, so the sign is lost.
- ✓ **-42.** Correct. Negative, magnitude 42.

13. The answer is **Faster and simpler hardware implementation.** One adder handles both addition and subtraction, there is a single zero, and no end-around carry is needed.

- ✗ **Easier decimal conversion.** Wrong. Converting a two's complement number to decimal by hand is arguably the fiddliest of the three schemes.
- ✗ **Eliminates overflow.** Wrong, and false on its face. Overflow is a consequence of a fixed word width and no scheme removes it.
- ✗ **Greater storage efficiency.** Wrong. All three schemes use exactly the same number of bits for the same width.
- ✓ **Faster and simpler hardware implementation.** Correct. One adder for both operations, one zero, no carry to wrap around.

14. The answer is **Slow and complex arithmetic circuitry.** The circuit must inspect both sign bits, decide whether to add or subtract, compare magnitudes and work out the sign of the result. Two's complement just adds.

- ✗ **Inability to represent positive numbers.** Wrong, and plainly false. Sign bit 0 gives every positive value.
- ✓ **Slow and complex arithmetic circuitry.** Correct. Signs must be compared, magnitudes ordered and the result's sign derived, all in hardware.
- ✗ **Large memory usage.** Wrong. An 8-bit sign-magnitude value occupies 8 bits, the same as any other scheme.
- ✗ **Limited range.** Wrong. Its range, -127 to +127, is essentially the same as two's complement, losing only the extra negative value.

OBJECTIVE SECTION

Floating Point and IEEE 754

10 questions. Taught in Module One, Unit 3. Cover the answers before you start.

1. **TEST 1 Q5** Which strategy helps reduce overflow/underflow occurrences? (a) Using ASCII (b) Using double precision arithmetic (c) Using fewer bits (d) Using BCD
2. **TEST 1 Q7** A banking application depends heavily on accurate decimal calculations. Which architectural feature directly influences calculation accuracy? (a) Floating-point representation (b) Cache hierarchy (c) Control signals (d) Bus width
3. **TEST 1 Q9** Approximately what numeric range can IEEE Double Precision represent? (a) $10^{\pm 38}$ (b) $10^{\pm 128}$ (c) $10^{\pm 308}$ (d) $10^{\pm 256}$
4. **TEST 1 Q15** IEEE Double Precision uses: (a) 64 bits (b) 32 bits (c) 48 bits (d) 128 bits
5. **TEST 1 Q19** A floating-point number is generally stored as: (a) Exponent, Sign, Mantissa (b) Mantissa, Exponent, Sign (c) ASCII digits (d) Sign, Mantissa, Exponent
6. **TEST 1 Q23** What is the principal advantage of floating-point representation? (a) Exact decimal storage (b) Faster arithmetic (c) Elimination of overflow (d) Representation of very large and very small values

7. TEST 1 Q29 A weather simulation program requires highly accurate numerical computations. Which architectural component is most critical? (a) Floating-point representation (b) Cache replacement policy (c) Character encoding (d) Control bus design
8. TEST 1 Q51 Approximately what numeric range can IEEE Single Precision represent? (a) $10^{\pm 308}$ (b) $10^{\pm 38}$ (c) $10^{\pm 20}$ (d) $10^{\pm 10}$
9. TEST 1 Q54 IEEE Single Precision allocates how many bits to the exponent? (a) 8 (b) 23 (c) 7 (d) 11
10. TEST 1 Q58 The exponent bias used in IEEE Double Precision is: (a) 127 (b) 255 (c) 511 (d) 1023

1. The answer is **Using double precision arithmetic**. Double precision widens the exponent from 8 bits to 11, which stretches the range from about 10 to the power plus or minus 38 out to about 10 to the power plus or minus 308.

- ✗ **Using ASCII**. Wrong. ASCII is a character code and performs no arithmetic at all.
- ✓ **Using double precision arithmetic**. Correct. More exponent bits means more range, so fewer results fall off either end.
- ✗ **Using fewer bits**. Wrong, and exactly backwards. Fewer bits shrink the range and cause MORE overflow.
- ✗ **Using BCD**. Wrong. BCD codes decimal digits and is less compact than binary, so it does not extend numeric range.

2. The answer is **Floating-point representation**. Accuracy is decided by how a number is REPRESENTED: how many mantissa bits hold the significant digits.

- ✓ **Floating-point representation**. Correct. The mantissa width sets how many significant digits survive, which is precision itself.
- ✗ **Cache hierarchy**. Wrong. Cache changes how quickly data arrives, never what the computed value is.
- ✗ **Control signals**. Wrong. Control signals sequence the hardware. They do not alter numeric precision.
- ✗ **Bus width**. Wrong. A wider bus moves more bits per transfer, a speed matter and not an accuracy one.

3. The answer is **$10^{\pm 308}$** . Double precision reaches roughly 10 to the power plus or minus 308. Learn it as a pair with single precision at roughly 10 to the power plus or minus 38, because this paper sets both.

- ✗ **$10^{\pm 38}$** . Wrong. That is the SINGLE precision figure, the other half of the pair.
- ✗ **$10^{\pm 128}$** . Wrong. 128 is close to the single precision exponent bias of 127, not a decimal range.
- ✓ **$10^{\pm 308}$** . Correct. About 10 to the power plus or minus 308.
- ✗ **$10^{\pm 256}$** . Wrong. 256 is a familiar power of two, put here to look plausible. It is not the figure.

4. The answer is **64 bits**. Double precision is 64 bits: 1 sign, 11 exponent, 52 mantissa. Single precision is 32 bits: 1 sign, 8 exponent, 23 mantissa. Those two lines answer most floating-point questions on this paper.

- ✓ **64 bits**. Correct. $1 + 11 + 52 = 64$.
- ✗ **32 bits**. Wrong. 32 bits is SINGLE precision, and the name double says twice that.
- ✗ **48 bits**. Wrong. 48 bits is not an IEEE 754 basic format.
- ✗ **128 bits**. Wrong. 128 bits is quadruple precision, which the course does not set.

5. The answer is **Sign, Mantissa, Exponent**. Sign, Mantissa, Exponent is what the exam keyed, and it is the only option offered that puts the sign first. Know the truth as well: real IEEE 754 hardware lays the fields out as Sign, EXPONENT, Mantissa, and that ordering is not offered here. Take the keyed answer in this test, and write Sign, Exponent, Mantissa in a theory answer.

- ✗ **Exponent, Sign, Mantissa**. Wrong. The sign is always the leading bit, so it cannot come second.
- ✗ **Mantissa, Exponent, Sign**. Wrong. This puts the sign last, when it is in fact the first bit of the word.
- ✗ **ASCII digits**. Wrong. ASCII digits would store the number as text, which is not a floating-point format at all.
- ✓ **Sign, Mantissa, Exponent**. Correct as the paper keyed it, and the only answer offered that puts the sign first. Be aware that the true IEEE 754 order is Sign, Exponent, Mantissa, which this paper does not list.

6. The answer is **Representation of very large and very small values**. A movable binary point buys enormous dynamic range in a fixed number of bits.

- ✗ **Exact decimal storage.** Wrong, and the opposite of the truth. Floating point is approximate: 0.1 cannot be stored exactly in binary.
- ✗ **Faster arithmetic.** Wrong. Floating-point arithmetic is generally SLOWER than integer arithmetic, since it must align exponents and normalise.
- ✗ **Elimination of overflow.** Wrong. The range is larger, not infinite, so overflow still occurs at the ends.
- ✓ **Representation of very large and very small values.** Correct. Huge and tiny magnitudes in the same fixed-width word, which is what dynamic range means.

7. The answer is **Floating-point representation**. Same rule as the banking question: when a stem says ACCURATE or PRECISE numerical work, the answer is floating-point representation. When it says FASTER, look to cache and organization instead.

- ✓ **Floating-point representation.** Correct. Precision comes from the number format, and simulations accumulate error over millions of steps.
- ✗ **Cache replacement policy.** Wrong. A replacement policy affects hit rate and therefore speed, not the computed values.
- ✗ **Character encoding.** Wrong. Character encoding is for text, and a simulation computes on numbers.
- ✗ **Control bus design.** Wrong. The control bus carries command signals, which has no bearing on numerical accuracy.

8. The answer is **$10^{\pm 38}$** . Single precision reaches roughly 10 to the power plus or minus 38.

- ✗ **$10^{\pm 308}$.** Wrong. That is the DOUBLE precision figure, offered to catch a reader who learned only one of the pair.
- ✓ **$10^{\pm 38}$.** Correct. About 10 to the power plus or minus 38 from 32 bits with an 8-bit exponent.
- ✗ **$10^{\pm 20}$.** Wrong, far too small. 8 exponent bits reach well past 10 to the power 20.
- ✗ **$10^{\pm 10}$.** Wrong, smaller still. That is roughly the range of a 32-bit integer, not of a float.

9. The answer is **8**. Single precision is $1 + 8 + 23 = 32$ bits, so the exponent takes 8.

- ✓ **8.** Correct. 8 exponent bits, which is where the plus or minus 38 range comes from.
- ✗ **23.** Wrong. 23 is the MANTISSA width of the same format.
- ✗ **7.** Wrong. 7 bits is the width of an ASCII character, not of this exponent.
- ✗ **11.** Wrong. 11 is the exponent width of DOUBLE precision.

10. The answer is **1023**. The bias is 2 to the power (e minus 1), minus 1, where e is the exponent width. Double has 11 exponent bits, so the bias is 1024 minus $1 = 1023$.

- ✗ **127.** Wrong. 127 is the SINGLE precision bias, from 8 exponent bits.
- ✗ **255.** Wrong. 255 is the largest value in 8 bits, not a bias.
- ✗ **511.** Wrong. 511 would be the bias for a 10-bit exponent, and double uses 11.
- ✓ **1023.** Correct. 2 to the power 10, which is 1024, minus 1.

OBJECTIVE SECTION

Instruction Types and Formats

8 questions. Taught in Module One, Unit 4. Cover the answers before you start.

1. **TEST 2 Q17** An Instruction format defines: (a) Layout of bits within an instruction (b) Cache organization (c) Internal processor wiring (d) Operating system memory allocation
2. **TEST 2 Q18** In a typical instruction format, the opcode field specifies: (a) Memory address (b) Operation to perform (c) Register contents (d) Interrupt priority

3. **TEST 2 Q20** A processor designer wants to increase the number of available opcodes without changing instruction length. Which modification is most appropriate? (a) Increase RAM capacity (b) Increase opcode bits while reducing operand bits (c) Add pipeline stages (d) Increase cache size
4. **TEST 2 Q24** Which instruction format generally results in fewer memory accesses? (a) Two-address (b) Three-address (c) Zero-address (d) One-address
5. **TEST 2 Q30** A stack machine executes arithmetic operations without explicitly specifying operand addresses because operands are: (a) Stored in cache (b) Stored in virtual memory (c) Stored in ROM (d) Located at the top of the stack
6. **TEST 2 Q39** A processor employs fixed-length 32-bit instructions. Which advantage is most likely achieved? (a) Faster instruction decoding (b) Elimination of registers (c) Unlimited opcode space (d) Smaller program size
7. **TEST 2 Q44** An instruction contains the following fields: Opcode 8 bits, Register field 6 bits, Address field 18 bits. What is the total instruction length? (a) 40 bits (b) 32 bits (c) 36 bits (d) 30 bits
8. **TEST 2 Q60** Which instruction format is most commonly associated with stack-based architectures? (a) Three-address (b) Zero-address (c) Two-address (d) One-address

1. The answer is **Layout of bits within an instruction**. A format is a map of the instruction word: which bits are the opcode, which name registers, which carry an address or a constant.

- ✓ **Layout of bits within an instruction.** Correct. It is the bit layout, and the decoder reads the word according to it.
- ✗ **Cache organization.** Wrong. Cache organization is about memory levels and mapping, not the shape of an instruction.
- ✗ **Internal processor wiring.** Wrong. Internal wiring is microarchitecture, hidden below the format.
- ✗ **Operating system memory allocation.** Wrong. Memory allocation is a software concern, handled by the operating system at run time.

2. The answer is **Operation to perform**. Opcode is short for operation code. It answers WHAT to do, while the operand fields answer WHAT TO DO IT TO.

- ✗ **Memory address.** Wrong. An address lives in the ADDRESS field, a separate part of the word.
- ✓ **Operation to perform.** Correct. ADD, LOAD, JUMP and the rest are named by the opcode.
- ✗ **Register contents.** Wrong. Register CONTENTS are data held in the register file. The instruction can only name the register, never carry its contents.
- ✗ **Interrupt priority.** Wrong. Interrupt priority is set by the interrupt controller, not encoded in every instruction.

3. The answer is **Increase opcode bits while reducing operand bits**. The instruction word is a fixed budget of bits. If the total length may not grow, the only way to widen one field is to narrow another. An extra opcode bit doubles the opcode count, since k bits give 2 to the power k of them.

- ✗ **Increase RAM capacity.** Wrong. RAM capacity is outside the processor and has no effect on how an instruction word is divided.
- ✓ **Increase opcode bits while reducing operand bits.** Correct. A fixed word, so one field grows only at another's expense.
- ✗ **Add pipeline stages.** Wrong. Pipeline stages change how instructions are PROCESSED, not how many distinct ones can be encoded.
- ✗ **Increase cache size.** Wrong. Cache size affects speed, not the encoding space of the instruction set.

4. The answer is **Three-address**. A three-address instruction names both sources and the destination, so a whole operation is done in one go. The shorter formats need extra loads, stores and pushes to move operands into place, and each of those is another access.

- ✗ **Two-address.** Wrong. Two-address must overwrite one source, so a value that is needed again has to be copied first.
- ✓ **Three-address.** Correct. Everything is named at once, so the expression completes in the fewest instructions.
- ✗ **Zero-address.** Wrong, and the furthest from it. A stack machine must PUSH each operand and POP the result, which is the most traffic of the four.
- ✗ **One-address.** Wrong. One-address routes everything through the accumulator, so each extra operand costs a separate load or store.

5. The answer is **Located at the top of the stack**. The operands are implied: an ADD pops the top two items, adds them and pushes the result back. Because their location is fixed by the rule, no address field is needed.

- ✗ **Stored in cache.** Wrong. Cache may well hold the stack, but cache is transparent and never removes the need for an address.

- ✗ **Stored in virtual memory.** Wrong. Virtual memory is an address translation scheme, so it still requires addresses.
- ✗ **Stored in ROM.** Wrong. ROM holds firmware, and operands there could not be pushed and popped.
- ✓ **Located at the top of the stack.** Correct. The top of the stack is an implied location, which is exactly why the format is zero-address.

6. The answer is **Faster instruction decoding**. Every instruction starts at a predictable boundary and every field sits in a known place, so decoding is simple and pipelining is easy.

- ✓ **Faster instruction decoding.** Correct. No need to decode part of an instruction to learn where the next one begins.
- ✗ **Elimination of registers.** Wrong. Instruction length has nothing to do with whether registers exist, and RISC machines with fixed lengths have many.
- ✗ **Unlimited opcode space.** Wrong, and impossible. A fixed word actually LIMITS opcode space, since the field cannot grow.
- ✗ **Smaller program size.** Wrong, and this is the trap. Smaller programs are the VARIABLE-length advantage, which is why CISC uses variable length.

7. The answer is **32 bits**. The fields simply add up: $8 + 6 + 18 = 32$ bits.

- ✗ **40 bits.** Wrong, 8 too many. Check that the opcode was not counted twice.
- ✓ **32 bits.** Correct. $8 + 6 + 18 = 32$, which is also the usual word size, a useful sanity check.
- ✗ **36 bits.** Wrong, 4 too many.
- ✗ **30 bits.** Wrong, 2 too few. Re-add the register field of 6.

8. The answer is **Zero-address**. Stack machines need no address fields at all because the operands are implied by the stack, so their arithmetic instructions are zero-address.

- ✗ **Three-address.** Wrong. Three-address names every operand explicitly, the opposite of the stack idea.
- ✓ **Zero-address.** Correct. Zero addresses, because the top of the stack is implied. Note that PUSH and POP still carry one address.
- ✗ **Two-address.** Wrong. Two-address is the classic register or memory format, as in ADD A, B.
- ✗ **One-address.** Wrong. One-address is the ACCUMULATOR machine, where the accumulator is implied but the operand is still named.

OBJECTIVE SECTION

Addressing Modes

9 questions. Taught in Module One, Unit 4. Cover the answers before you start.

1. **TEST 1 Q38** A compiler embeds a constant value directly within an instruction instead of storing it in memory. Which addressing mode is being used? (a) Immediate Addressing (b) Direct Addressing (c) Indirect Addressing (d) Register Addressing
2. **TEST 2 Q8** Assume $R1 = 2000$ and $\text{Memory}[2000] = 500$. Using Register Indirect Addressing, the operand is: (a) Memory address only (b) 2000 (c) 500 (d) R1
3. **TEST 2 Q9** Suppose the instruction address = 500 and displacement = +30. Using PC-relative addressing, the effective address becomes: (a) 30 (b) 500 (c) 470 (d) 530
4. **TEST 2 Q19** Which addressing mode generally requires the fewest memory accesses during instruction execution? (a) Immediate (b) Indexed (c) Direct (d) Indirect
5. **TEST 2 Q35** Consider $R2 = 300$, $\text{Memory}[320] = 900$, and instruction displacement = 20. Using Indexed Addressing with R2 as the index register, the effective address is: (a) 300 (b) 320 (c) 280 (d) 900

6. **TEST 2 Q37** Given that the instruction specifies address = 700, Memory[700] = 1500, and Memory[1500] = 60. Under Indirect Addressing, the operand equals: (a) 700 (b) 60 (c) Undefined (d) 1500
7. **TEST 2 Q41** An instruction contains the operand itself within the instruction. Which addressing mode is used? (a) Indexed (b) Register (c) Immediate (d) Direct
8. **TEST 2 Q47** Suppose the base register = 4000 and displacement = 120. Using Base Register Addressing, the effective address is: (a) 3880 (b) 120 (c) 4000 (d) 4120
9. **TEST 2 Q53** A processor designer aims to improve the efficiency of accessing dynamically allocated data structures such as linked lists. Which addressing mode is generally the most suitable? (a) Immediate addressing (b) Implied addressing (c) Register addressing (d) Indirect addressing

1. The answer is **Immediate Addressing**. Immediate means the operand itself is inside the instruction word, so no memory access is needed to fetch it.

- ✓ **Immediate Addressing**. Correct. The value travels with the instruction, which is what "embedded directly" says.
- ✗ **Direct Addressing**. Wrong. Direct puts the ADDRESS of the operand in the instruction, so one memory read is still required.
- ✗ **Indirect Addressing**. Wrong. Indirect puts the address OF the address in the instruction, needing two reads.
- ✗ **Register Addressing**. Wrong. Register addressing names a register that holds the operand, so the value is in the register file and not in the instruction.

2. The answer is **500**. In register indirect, the register holds an ADDRESS, not the value. R1 gives 2000, and the operand is the contents of location 2000, which is 500.

- ✗ **Memory address only**. Wrong. The question asks for the operand, the value being worked on, and "memory address only" names no value.
- ✗ **2000**. Wrong. 2000 is the effective ADDRESS, one step short of the answer. This is the commonest slip on this question.
- ✓ **500**. Correct. Follow the pointer once: R1 gives 2000, and Memory[2000] gives 500.
- ✗ **R1**. Wrong. R1 is the register's name, not its contents nor the value it points at.

3. The answer is **530**. PC-relative adds the displacement to the current instruction address: $500 + 30 = 530$. This is how branches are encoded, and why code can be relocated freely.

- ✗ **30**. Wrong. 30 is the displacement alone, with the base forgotten.
- ✗ **500**. Wrong. 500 is the base alone, with the displacement forgotten.
- ✗ **470**. Wrong sign. 500 minus 30 gives 470, but the displacement is stated as +30.
- ✓ **530**. Correct. $500 + 30 = 530$.

4. The answer is **Immediate**. Count the trips to memory after the instruction is fetched. Immediate needs zero, direct and indexed need one, indirect needs two.

- ✓ **Immediate**. Correct. Zero further accesses, because the operand arrived inside the instruction.
- ✗ **Indexed**. Wrong. Indexed computes an address, then makes one access to fetch the operand.
- ✗ **Direct**. Wrong. Direct also makes one access, using the address in the instruction.
- ✗ **Indirect**. Wrong, and the most expensive of the four. Indirect reads the pointer, then reads the operand, so two accesses.

5. The answer is **320**. Indexed addressing adds the index register to the displacement: $300 + 20 = 320$. The question asks for the ADDRESS, so stop there.

- ✗ **300**. Wrong. 300 is the index register alone, with the displacement not yet added.
- ✓ **320**. Correct. $300 + 20 = 320$.
- ✗ **280**. Wrong sign. That subtracts the displacement instead of adding it.
- ✗ **900**. Wrong, and the trap the stem was built for. 900 is the CONTENTS of address 320, the operand, but the question asked for the effective address.

6. The answer is **60**. Indirect means two hops. The instruction gives 700, location 700 holds the real address 1500, and location 1500 holds the operand 60.

- ✗ **700**. Wrong. 700 is the address written in the instruction, which is only the start of the chain.
- ✓ **60**. Correct. Follow both hops: 700 gives 1500, and 1500 gives 60.
- ✗ **Undefined**. Wrong. Every value needed is supplied, so nothing here is undefined.
- ✗ **1500**. Wrong, and the commonest error. 1500 is the EFFECTIVE ADDRESS, one hop short of the operand. Under DIRECT addressing 1500 would have been the answer.

7. The answer is **Immediate**. The operand itself, not its address, sits in the instruction. That is the definition of immediate addressing.

✗ **Indexed**. Wrong. Indexed carries a displacement to be added to an index register, not the operand.

✗ **Register**. Wrong. Register addressing carries a register NAME, and the operand lives in that register.

✓ **Immediate**. Correct. The value is in the instruction word itself.

✗ **Direct**. Wrong. Direct carries the operand's ADDRESS. The distinction between direct and immediate is exactly what this question tests.

8. The answer is **4120**. Base plus displacement: $4000 + 120 = 4120$. Base register addressing is how a program is relocated, since moving the base moves everything at once.

✗ **3880**. Wrong sign. 4000 minus 120 gives 3880, but the displacement is added.

✗ **120**. Wrong. 120 is the displacement alone, with no base.

✗ **4000**. Wrong. 4000 is the base alone, with no displacement.

✓ **4120**. Correct. $4000 + 120 = 4120$.

9. The answer is **Indirect addressing**. A linked list IS a chain of pointers, and indirect addressing is the mode that follows a pointer to reach the data.

✗ **Immediate addressing**. Wrong. Immediate holds a constant fixed at compile time, and a dynamically allocated address is not known then.

✗ **Implied addressing**. Wrong. Implied names no operand at all, as in a stack machine, so it cannot chase a pointer.

✗ **Register addressing**. Wrong. Register addressing holds the VALUE in a register. It is a useful helper, but it does not itself follow a pointer to memory.

✓ **Indirect addressing**. Correct. Following the address held at a location is exactly what traversing a linked list requires.

OBJECTIVE SECTION

Memory, Buses and the Processor

26 questions. Taught in Module Two Unit 1, and Module Three. Cover the answers before you start.

1. **TEST 1 Q4** A processor's control unit is responsible for: (a) Performing arithmetic calculations (b) Storing application files (c) Coordinating instruction execution through control signals (d) Managing peripheral power supply
2. **TEST 1 Q25** A software engineer optimizes a program by storing frequently used variables in CPU registers instead of RAM. The primary benefit is: (a) Improved memory capacity (b) Lower bus traffic only (c) Reduced storage cost (d) Faster execution speed
3. **TEST 1 Q40** Which bus carries information identifying memory locations and I/O device locations? (a) Control Bus (b) Data Bus (c) Memory Bus (d) Address Bus
4. **TEST 1 Q48** An autonomous robot continuously performs sensor processing and actuator control. Which organizational component primarily coordinates these activities? (a) Control Unit (b) ALU (c) Register Set (d) Cache Memory
5. **TEST 1 Q52** A motherboard manufacturer introduces a faster bus system, enabling quicker communication among components. This enhancement belongs to: (a) Computer Architecture (b) Instruction Design (c) Data Representation (d) Computer Organization

6. **TEST 2 Q1** Which statement regarding ROM is correct? (a) ROM is mainly used for temporary program execution. (b) ROM stores firmware permanently. (c) ROM requires periodic refreshing. (d) ROM loses contents after power failure.
7. **TEST 2 Q3** Why is cache memory generally constructed using SRAM rather than DRAM? (a) SRAM offers faster access without refresh operations. (b) SRAM is cheaper. (c) SRAM is non-volatile. (d) SRAM has larger storage capacity.
8. **TEST 2 Q7** Which architectural feature most directly contributes to reducing the average instruction execution time without increasing clock frequency? (a) Increasing monitor resolution (b) Increasing hard disk capacity (c) Expanding ROM size (d) Increasing cache hit rate
9. **TEST 2 Q12** Which of the following correctly orders memory from fastest to slowest? (a) Cache → Registers → SSD → RAM (b) Registers → Cache → Main Memory → Secondary Storage (c) Registers → Main Memory → Cache → SSD (d) Cache → Registers → Main Memory → Hard Disk
10. **TEST 2 Q15** Which combination correctly matches memory type with volatility? (a) Flash Memory, Non-volatile (b) DRAM, Non-volatile (c) SRAM, Non-volatile (d) Cache Memory, Non-volatile
11. **TEST 2 Q21** Which statement correctly explains memory hierarchy? (a) Main memory is faster than registers. (b) Every memory level has identical speed. (c) Higher levels are faster, smaller, and more expensive per bit. (d) Lower levels are faster than upper levels.
12. **TEST 2 Q25** A computer architect argues that registers should be larger than cache memory. Why is this argument incorrect? (a) Registers are virtual memory components. (b) Registers are intended for only the most immediate processor operations. (c) Registers belong to secondary storage. (d) Registers are slower.
13. **TEST 2 Q28** Which memory characteristic is most critical for real-time control systems? (a) Low manufacturing cost (b) Predictable low access time (c) Storage capacity (d) High density
14. **TEST 2 Q29** Why is secondary storage unsuitable for direct CPU execution of instructions? (a) It stores binary data. (b) It has significantly higher access latency than primary memory. (c) It is non-volatile. (d) It uses logical addressing.
15. **TEST 2 Q32** Which statement best summarizes the fundamental trade-off underlying modern computer memory hierarchy? (a) All memory technologies have similar speed and cost. (b) Slower memories always consume more power than faster memories. (c) Faster memories generally provide larger capacities at lower cost. (d) Faster memories are typically smaller and more expensive per bit, while slower memories provide larger capacities at lower cost.
16. **TEST 2 Q34** Which statement best distinguishes SRAM from DRAM? (a) SRAM stores data using flip-flops whereas DRAM stores data using capacitors. (b) DRAM is faster than SRAM. (c) SRAM requires periodic refreshing while DRAM does not. (d) DRAM is non-volatile.
17. **TEST 2 Q45** A computer engineer claims that increasing cache memory always eliminates processor-memory performance bottlenecks. Which statement best evaluates this claim? (a) It is correct only when DRAM is removed. (b) It is incorrect because cache cannot eliminate all memory access delays. (c) It is correct because cache replaces main memory. (d) It is always correct because cache operates at processor speed.

18. TEST 2 Q48 A processor spends considerable time waiting for data despite having a large cache. Which phenomenon is primarily responsible? (a) Address aliasing (b) Register spilling (c) Memory wall (d) Bus inversion
19. TEST 2 Q49 Which component stores the address of the memory location currently being accessed? (a) Memory Address Register (b) Memory Data Register (c) Program Counter (d) Instruction Register
20. TEST 2 Q50 A processor repeatedly accesses the same instruction during program execution. The improved execution speed primarily results from which memory hierarchy principle? (a) Spatial locality only (b) Temporal locality (c) Virtual addressing (d) Sequential consistency
21. TEST 2 Q51 A memory technology provides higher density but lower speed than SRAM. Which technology is being described? (a) ROM (b) EEPROM (c) Register memory (d) DRAM
22. TEST 2 Q52 Which memory level has the highest cost per bit? (a) SSD (b) Registers (c) Main memory (d) Hard disk
23. TEST 2 Q56 Which statement correctly describes virtual memory? (a) It replaces cache memory. (b) It physically increases RAM capacity. (c) It eliminates page faults. (d) It allows programs to execute using an address space larger than physical memory.
24. TEST 2 Q57 Which memory characteristic primarily determines how much information can be stored in a memory device? (a) Access time (b) Capacity (c) Transfer rate (d) Volatility
25. TEST 2 Q58 A memory module has an access time of 60 ns and a cycle time of 90 ns. Which interpretation is correct? (a) Access time is longer than cycle time. (b) Cycle time has no practical significance. (c) Consecutive accesses must be separated by at least 90 ns. (d) A new memory operation can begin every 60 ns.
26. TEST 2 Q59 A workload frequently accesses neighboring memory locations after accessing a particular location. Which locality principle is being demonstrated? (a) Virtual locality (b) Random locality (c) Temporal locality (d) Spatial locality

1. The answer is **Coordinating instruction execution through control signals**. The control unit is the conductor. It decodes each instruction and issues the control signals that tell every other unit what to do and when.

- ✗ **Performing arithmetic calculations.** Wrong. Arithmetic is the ALU's job. Control unit and ALU are the two halves of the CPU, and the exam separates them constantly.
- ✗ **Storing application files.** Wrong. Files are held in secondary storage, managed by the operating system.
- ✓ **Coordinating instruction execution through control signals.** Correct. Decode, then sequence the rest of the machine through control signals.
- ✗ **Managing peripheral power supply.** Wrong. Power management belongs to the power supply and chipset, not to the instruction control path.

2. The answer is **Faster execution speed**. Registers sit inside the CPU and are reached in a single cycle, while RAM costs tens or hundreds of cycles.

- ✗ **Improved memory capacity.** Wrong, and backwards. There are only a few dozen registers, so capacity falls rather than rises.
- ✗ **Lower bus traffic only.** Wrong because of the word ONLY. Less bus traffic is a real side effect, but the PRIMARY benefit is speed.
- ✗ **Reduced storage cost.** Wrong. Registers are the most expensive storage per bit in the whole machine.
- ✓ **Faster execution speed.** Correct. The variable is already where the ALU needs it, so no memory trip is made.

3. The answer is **Address Bus**. Three buses, three jobs: the address bus says WHERE, the data bus carries WHAT, and the control bus says WHEN and WHICH WAY.

- ✗ **Control Bus.** Wrong. The control bus carries timing and command signals such as read and write.
- ✗ **Data Bus.** Wrong. The data bus carries the value itself, once the location has been named.
- ✗ **Memory Bus.** Wrong. "Memory bus" is a loose collective term for the whole path, not the line that carries locations.
- ✓ **Address Bus.** Correct. Location information travels on the address bus, and its width sets how much memory can be addressed.

4. The answer is **Control Unit**. The keyword is COORDINATES. Whenever a stem asks which component sequences or directs activity, the answer is the control unit.

- ✓ **Control Unit.** Correct. Coordination and sequencing is the control unit's defining role.
- ✗ **ALU.** Wrong. The ALU computes results when told to, it does not decide the order of work.
- ✗ **Register Set.** Wrong. Registers hold operands and results. Storage is not coordination.
- ✗ **Cache Memory.** Wrong. Cache speeds up access to recently used data, which is a performance aid and not a controller.

5. The answer is **Computer Organization**. A faster bus changes how components are interconnected and how quickly they talk. No instruction changes, so nothing programmer-visible changes.

- ✗ **Computer Architecture.** Wrong. Architecture would mean a change to the instruction set or the programmer-visible model, and there is none here.
- ✗ **Instruction Design.** Wrong. No instruction was added, removed or re-encoded.
- ✗ **Data Representation.** Wrong. Data representation is about encoding numbers and characters, untouched by bus speed.
- ✓ **Computer Organization.** Correct. Interconnection and speed are organization.

6. The answer is **ROM stores firmware permanently**. ROM is read-only and non-volatile, which is why the boot firmware lives there and survives a power cut.

- ✗ **ROM is mainly used for temporary program execution.** Wrong. Temporary working storage is RAM's job. ROM holds code that must persist.
- ✓ **ROM stores firmware permanently.** Correct. Firmware such as the BIOS is held permanently in ROM.
- ✗ **ROM requires periodic refreshing.** Wrong. Refreshing is a DRAM requirement, because DRAM stores charge in capacitors that leak.
- ✗ **ROM loses contents after power failure.** Wrong, and the direct opposite. ROM is non-volatile, so its contents survive power loss.

7. The answer is **SRAM offers faster access without refresh operations**. SRAM holds each bit in a flip-flop, so it is fast and needs no refresh. Cache exists to be fast, so it pays SRAM's price.

- ✓ **SRAM offers faster access without refresh operations.** Correct. Speed with no refresh cycles stealing access time.
- ✗ **SRAM is cheaper.** Wrong, and backwards. SRAM uses about six transistors per bit against DRAM's one transistor and one capacitor, so it costs far more.
- ✗ **SRAM is non-volatile.** Wrong. SRAM is volatile, like DRAM. It loses everything when power goes.
- ✗ **SRAM has larger storage capacity.** Wrong. DRAM is the denser technology, which is exactly why main memory is DRAM.

8. The answer is **Increasing cache hit rate**. Average access time is hit time plus miss rate times miss penalty. Raise the hit rate and the average falls, with the clock untouched.

- ✗ **Increasing monitor resolution.** Wrong. Monitor resolution is a display property and does not touch instruction timing.
- ✗ **Increasing hard disk capacity.** Wrong. Disk capacity is about how much you can store, not how fast instructions execute.
- ✗ **Expanding ROM size.** Wrong. A bigger ROM holds more firmware. It does not speed up running code.
- ✓ **Increasing cache hit rate.** Correct. Fewer misses means fewer long trips to main memory.

9. The answer is **Registers → Cache → Main Memory → Secondary Storage**. Registers, cache, main memory, secondary storage. Learn this ladder once and it answers a whole family of questions: speed and cost per bit fall together as capacity rises.

- ✗ **Cache → Registers → SSD → RAM.** Wrong twice. Registers must come before cache, and RAM is faster than an SSD, not slower.
- ✓ **Registers → Cache → Main Memory → Secondary Storage.** Correct. The standard hierarchy, top to bottom.
- ✗ **Registers → Main Memory → Cache → SSD.** Wrong. Cache sits between registers and main memory, not below main memory.
- ✗ **Cache → Registers → Main Memory → Hard Disk.** Wrong. The last two are in order, but registers are faster than cache, so the first pair is swapped.

10. The answer is **Flash Memory, Non-volatile**. Only flash keeps its contents without power. DRAM, SRAM and the cache built from SRAM are all volatile.

- ✓ **Flash Memory, Non-volatile.** Correct. Flash holds charge on a floating gate, so a USB drive still has your files tomorrow.
- ✗ **DRAM, Non-volatile.** Wrong. DRAM is volatile, and it does not even hold data for a full second without refreshing.
- ✗ **SRAM, Non-volatile.** Wrong. SRAM is volatile too. It needs no refresh, but it does need power.
- ✗ **Cache Memory, Non-volatile.** Wrong. Cache is built from SRAM, so it is volatile as well.

11. The answer is **Higher levels are faster, smaller, and more expensive per bit**. Climbing the hierarchy, speed rises, size falls and cost per bit rises. All three move to-

gether, which is why the pyramid has its shape.

- ✗ **Main memory is faster than registers.** Wrong, and inverted. Registers are the fastest storage in the machine.
- ✗ **Every memory level has identical speed.** Wrong. If every level were equally fast there would be no reason to have a hierarchy at all.
- ✓ **Higher levels are faster, smaller, and more expensive per bit.** Correct, and all three properties are stated together.
- ✗ **Lower levels are faster than upper levels.** Wrong, and inverted again. Lower means larger, cheaper and slower.

12. The answer is **Registers are intended for only the most immediate processor operations.** Registers exist to hold the handful of values in use RIGHT NOW. Adding more would need wider instruction fields to name them and would slow the register file down.

- ✗ **Registers are virtual memory components.** Wrong. Virtual memory is a main memory and disk mechanism. Registers are physical CPU storage.
- ✓ **Registers are intended for only the most immediate processor operations.** Correct. Their job is the immediate working set, so a small, very fast file is the right design.
- ✗ **Registers belong to secondary storage.** Wrong. Secondary storage is disk and SSD, at the very bottom of the hierarchy.
- ✗ **Registers are slower.** Wrong, and plainly false. Registers are the FASTEST storage there is.

13. The answer is **Predictable low access time.** Real-time means a deadline must be met EVERY time. Predictability matters as much as raw speed, since a fast average with an occasional long stall still misses deadlines.

- ✗ **Low manufacturing cost.** Wrong. Cost is a business constraint, not a timing guarantee.
- ✓ **Predictable low access time.** Correct. Note the word PREDICTABLE. This is why some real-time designs prefer tightly coupled memory over an unpredictable cache.
- ✗ **Storage capacity.** Wrong. A control system's working set is usually small. Capacity is not the binding constraint.
- ✗ **High density.** Wrong. Density is a manufacturing property, and it says nothing about worst-case timing.

14. The answer is **It has significantly higher access latency than primary memory.** Disk latency is measured in milliseconds against nanoseconds for RAM, a gap of about a million to one. That is why programs are loaded into main memory before they run.

- ✗ **It stores binary data.** Wrong. Every level of the hierarchy stores binary data, so this cannot be the distinguishing reason.
- ✓ **It has significantly higher access latency than primary memory.** Correct. The latency gap is the whole reason for loading before executing.
- ✗ **It is non-volatile.** Wrong. Non-volatility is an ADVANTAGE of secondary storage, not an obstacle to execution.
- ✗ **It uses logical addressing.** Wrong. Logical addressing is a translation scheme, and it does not prevent execution.

15. The answer is **Faster memories are typically smaller and more expensive per bit, while slower memories provide larger capacities at lower cost.** You cannot have fast, large and cheap at once. The hierarchy exists to buy the ILLUSION of all three by keeping hot data high and cold data low.

- ✗ **All memory technologies have similar speed and cost.** Wrong. If speed and cost were similar everywhere, one flat memory would do and there would be no hierarchy.

- ✗ **Slower memories always consume more power than faster memories.** Wrong. "Always" overstates it, and slower technologies such as disk and DRAM are generally cheaper per bit in power terms too.
- ✗ **Faster memories generally provide larger capacities at lower cost.** Wrong, and exactly inverted. Fast memory is small and expensive.
- ✓ **Faster memories are typically smaller and more expensive per bit, while slower memories provide larger capacities at lower cost.** Correct, and it states both halves of the trade-off.

16. The answer is **SRAM stores data using flip-flops whereas DRAM stores data using capacitors.** One structural fact explains everything else: flip-flops hold their state, so SRAM is fast and needs no refresh but costs many transistors. Capacitors leak, so DRAM must be refreshed but packs far more bits per chip.

- ✓ **SRAM stores data using flip-flops whereas DRAM stores data using capacitors.** Correct, and it is the root fact from which the rest follows.
- ✗ **DRAM is faster than SRAM.** Wrong, and inverted. SRAM is the faster of the two, which is why cache uses it.
- ✗ **SRAM requires periodic refreshing while DRAM does not.** Wrong, and inverted. DRAM is the one that must be refreshed, and the D stands for dynamic.
- ✗ **DRAM is non-volatile.** Wrong. DRAM is volatile, and it loses data in milliseconds without refreshing.

17. The answer is **It is incorrect because cache cannot eliminate all memory access delays.** Compulsory misses still happen, large working sets still overflow, and a bigger cache is itself slower to search. Cache narrows the gap, it never closes it.

- ✗ **It is correct only when DRAM is removed.** Wrong. Removing DRAM would leave nowhere to back the cache from, so the machine could not run at all.
- ✓ **It is incorrect because cache cannot eliminate all memory access delays.** Correct. Misses are inevitable, so some delay always remains.
- ✗ **It is correct because cache replaces main memory.** Wrong. Cache SUPPLEMENTS main memory, holding copies. It does not replace it.
- ✗ **It is always correct because cache operates at processor speed.** Wrong. Cache is fast, but the claim fails on "always": a miss still costs a full trip to main memory.

18. The answer is **Memory wall.** Processor speed has grown far faster than memory speed for decades. That widening gap is the memory wall, and cache only softens it.

- ✗ **Address aliasing.** Wrong. Address aliasing is two addresses referring to one location, a correctness issue rather than a speed one.
- ✗ **Register spilling.** Wrong. Register spilling is the compiler running out of registers and pushing values to memory. It is a contributor at most, not the phenomenon named here.
- ✓ **Memory wall.** Correct. The processor-memory speed gap, which is exactly what the stem describes.
- ✗ **Bus inversion.** Wrong. Bus inversion is a signalling trick for cutting switching power, unrelated to waiting for data.

19. The answer is **Memory Address Register.** MAR holds WHERE, MDR holds WHAT. The pair is worth memorising together, because the paper sets both.

- ✓ **Memory Address Register.** Correct. The Memory Address Register drives the address bus.
- ✗ **Memory Data Register.** Wrong. The MDR holds the DATA travelling to or from that address.
- ✗ **Program Counter.** Wrong, and the near miss. The Program Counter holds the address of the NEXT INSTRUCTION, not of the location being accessed right now.
- ✗ **Instruction Register.** Wrong. The Instruction Register holds the instruction currently being decoded and executed.

20. The answer is **Temporal locality.** Temporal is about TIME: the SAME location used again soon. Spatial is about PLACE: NEARBY locations used next. The word "re-

peatedly" settles it.

- ✗ **Spatial locality only.** Wrong. Spatial locality concerns neighbouring addresses, and the word ONLY makes it worse.
- ✓ **Temporal locality.** Correct. The same item reused shortly after, which is a cache hit.
- ✗ **Virtual addressing.** Wrong. Virtual addressing is an address translation mechanism, not a locality principle.
- ✗ **Sequential consistency.** Wrong. Sequential consistency is a memory ordering model for multiprocessors.

21. The answer is **DRAM**. One transistor and one capacitor per bit against SRAM's six transistors. That is why DRAM packs more bits per chip and runs slower.

- ✗ **ROM.** Wrong. ROM is read-only, so it is not the read-write main memory the comparison implies.
- ✗ **EEPROM.** Wrong. EEPROM is non-volatile and much slower to write, used for firmware rather than main memory.
- ✗ **Register memory.** Wrong. Registers are the fastest and least dense storage in the machine, the opposite of this description.
- ✓ **DRAM.** Correct. Denser and slower than SRAM, which is why main memory is DRAM and cache is SRAM.

22. The answer is **Registers**. Cost per bit rises as you climb the hierarchy, so the top level is the dearest. Registers are built from the fastest logic and sit inside the processor die.

- ✗ **SSD.** Wrong. An SSD is near the bottom, cheap per bit and large.
- ✓ **Registers.** Correct. Highest in the hierarchy, therefore highest cost per bit.
- ✗ **Main memory.** Wrong. Main memory is DRAM, far cheaper per bit than the SRAM and logic above it.
- ✗ **Hard disk.** Wrong. Hard disk is the cheapest per bit of the four.

23. The answer is **It allows programs to execute using an address space larger than physical memory**. Virtual memory gives each program a large logical address space, with pages moved between RAM and disk as needed.

- ✗ **It replaces cache memory.** Wrong. Cache and virtual memory sit at different levels and coexist.
- ✗ **It physically increases RAM capacity.** Wrong. No RAM is added. The illusion of more is created using disk, and the word PHYSICALLY gives this away.
- ✗ **It eliminates page faults.** Wrong, and inverted. Page faults are the MECHANISM of virtual memory, not something it removes.
- ✓ **It allows programs to execute using an address space larger than physical memory.** Correct. An address space larger than the physical memory installed.

24. The answer is **Capacity**. Capacity is HOW MUCH. Access time and transfer rate are HOW FAST, and volatility is WHETHER IT SURVIVES power loss.

- ✗ **Access time.** Wrong. Access time is the delay before data arrives, a speed measure.
- ✓ **Capacity.** Correct. Capacity is the total amount that can be held.
- ✗ **Transfer rate.** Wrong. Transfer rate is bits per second, another speed measure.
- ✗ **Volatility.** Wrong. Volatility says whether contents survive without power, not how much fits.

25. The answer is **Consecutive accesses must be separated by at least 90 ns**.

Access time is how long until the data arrives, 60 ns here. Cycle time is how long until the memory can START AGAIN, 90 ns here, because it needs 30 ns of recovery. Cycle time is always the larger of the two.

- ✗ **Access time is longer than cycle time.** Wrong, and contradicted by the figures given: 60 is less than 90.
- ✗ **Cycle time has no practical significance.** Wrong. Cycle time sets the maximum sustainable rate, which is very practical indeed.

- ✓ **Consecutive accesses must be separated by at least 90 ns.** Correct. The next access can only begin after the full 90 ns cycle.
- ✗ **A new memory operation can begin every 60 ns.** Wrong, and the trap. 60 ns is when the DATA ARRIVES, not when the next operation may start.

26. The answer is **Spatial locality**. NEIGHBOURING locations means space, so this is spatial locality. It is why cache fetches a whole block rather than a single word, and why array traversal is fast.

- ✗ **Virtual locality.** Wrong. There is no such principle. It is invented to sound like virtual memory.
- ✗ **Random locality.** Wrong. Random access is the ABSENCE of locality, which is the worst case for a cache.
- ✗ **Temporal locality.** Wrong, and the near miss. Temporal locality is the same location reused over TIME, while this stem says neighbouring.
- ✓ **Spatial locality.** Correct. Nearby addresses accessed next.

OBJECTIVE SECTION

Performance, RISC and CISC

7 questions. Taught in Module Two Unit 2, and Module Four Unit 1. Cover the answers before you start.

1. **TEST 2 Q4** A computer architect argues that processor performance should no longer be evaluated solely by clock frequency. Which of the following provides the strongest justification for this argument? (a) Modern processors execute exactly one instruction per clock cycle. (b) Processor performance also depends on CPI (Cycles Per Instruction), Instruction-Level Parallelism (ILP), cache performance, branch prediction accuracy, and memory latency. (c) Clock frequency completely determines instruction execution time. (d) Higher clock frequency eliminates cache misses.
2. **TEST 2 Q11** A processor executes a benchmark requiring 12 billion instructions, operating at 4 GHz, with an average CPI of 1.5. What is the approximate execution time? (a) 2.5 seconds (b) 4.5 seconds (c) 5.5 seconds (d) 3.5 seconds
3. **TEST 2 Q43** Suppose Processor X executes fewer instructions than Processor Y for the same program, but Processor Y completes execution faster due to better pipelining. Which statement is most accurate? (a) Processor X has a superior ISA. (b) Microarchitecture significantly affects performance. (c) ISA alone determines performance. (d) Instruction count is irrelevant.
4. **TEST 2 Q2** A processor implements speculative execution and branch prediction. The primary objective of these techniques is to: (a) Increase storage capacity. (b) Reduce pipeline stalls caused by control hazards. (c) Reduce transistor count. (d) Replace cache memory.
5. **TEST 2 Q23** Variable-length instruction formats are commonly associated with: (a) CISC architectures (b) SIMD processors (c) RISC architectures (d) Harvard architecture
6. **TEST 2 Q36** An embedded processor minimizes silicon area and power consumption by reducing instruction complexity and relying heavily on compiler optimization. This design reflects: (a) SIMD architecture (b) RISC philosophy (c) VLIW architecture (d) CISC philosophy
7. **TEST 2 Q42** A computer architect wants to reduce compiler complexity by designing instructions that closely resemble high-level programming constructs. Which ISA philos-

ophy best supports this objective? (a) Stack-based ISA (b) Harvard Architecture (c) RISC (d) CISC

1. The answer is **Processor performance also depends on CPI (Cycles Per Instruction), Instruction-Level Parallelism (ILP), cache performance, branch prediction accuracy, and memory latency**. Execution time is instruction count times CPI divided by clock rate. Frequency is one term of three, and CPI itself depends on cache, branches and parallelism.

✗ **Modern processors execute exactly one instruction per clock cycle**. Wrong. Superscalar processors issue several instructions per cycle, and stalls can push it below one. "Exactly one" is false.

✓ **Processor performance also depends on CPI (Cycles Per Instruction), Instruction-Level Parallelism (ILP), cache performance, branch prediction accuracy, and memory latency**. Correct. It names the other terms in the performance equation and the factors that drive CPI.

✗ **Clock frequency completely determines instruction execution time**. Wrong, and it argues the OPPOSITE of the architect's case.

✗ **Higher clock frequency eliminates cache misses**. Wrong. A faster clock does not change how often data is found in cache. If anything a miss costs MORE cycles at a higher frequency.

2. The answer is **4.5 seconds**. CPU time = (instruction count x CPI) / clock rate. Cycles needed = $12 \times 10^9 \times 1.5 = 18 \times 10^9$. Divide by 4×10^9 cycles per second to get 4.5 seconds.

✗ **2.5 seconds**. Wrong. No correct combination of the three figures gives 2.5.

✓ **4.5 seconds**. Correct. 18 billion cycles at 4 billion cycles per second is 4.5 seconds.

✗ **5.5 seconds**. Wrong. Too high. Recheck the division by 4, not by 3.

✗ **3.5 seconds**. Wrong. Note that dropping the CPI gives $12 / 4 = 3$ seconds, which is not offered, so this option catches a half-remembered formula rather than that specific slip.

3. The answer is **Microarchitecture significantly affects performance**. Fewer instructions did not win, because Y's better pipelining cut its CPI. Instruction count and CPI trade against each other, and this is the classic RISC against CISC comparison.

✗ **Processor X has a superior ISA**. Wrong. A superior ISA that finishes LAST is a strange claim, and instruction count alone does not make an ISA better.

✓ **Microarchitecture significantly affects performance**. Correct. Pipelining is microarchitecture, and here it outweighed the instruction count advantage.

✗ **ISA alone determines performance**. Wrong, and directly contradicted: the two differ in pipelining, which is not the ISA.

✗ **Instruction count is irrelevant**. Wrong, too strong. Instruction count is a real term in the performance equation, it simply was not decisive here.

4. The answer is **Reduce pipeline stalls caused by control hazards**. A branch is a control hazard: the pipeline does not know which instruction follows until the branch resolves. Predicting and running ahead speculatively keeps the pipeline full.

✗ **Increase storage capacity**. Wrong. Neither technique stores anything for the program. They are about timing, not capacity.

✓ **Reduce pipeline stalls caused by control hazards**. Correct, and note the exact phrase CONTROL HAZARD, which is the hazard branches cause.

✗ **Reduce transistor count**. Wrong, and backwards. Prediction tables and speculation logic ADD transistors.

✗ **Replace cache memory**. Wrong. Cache addresses data delay, branch prediction addresses control delay. They are different problems.

5. The answer is **CISC architectures**. CISC packs many operations of differing complexity into one instruction set, so instructions vary in length. RISC fixes the length to make decoding and pipelining simple.

✓ **CISC architectures.** Correct. Variable length is a defining CISC trait, and x86 is the standing example.

✗ **SIMD processors.** Wrong. SIMD is about operating on many data elements at once, which is independent of instruction length.

✗ **RISC architectures.** Wrong, and the direct opposite. RISC uses FIXED-length instructions.

✗ **Harvard architecture.** Wrong. Harvard is about separate instruction and data memories, not about encoding.

6. The answer is **RISC philosophy**. Simple instructions, simple hardware, and complexity pushed into the compiler. That trade is the RISC philosophy in one sentence.

✗ **SIMD architecture.** Wrong. SIMD adds wide vector units for data parallelism, which costs area rather than saving it.

✓ **RISC philosophy.** Correct. Reduced instruction complexity plus compiler reliance is RISC by definition.

✗ **VLW architecture.** Wrong, though it is the closest decoy: VLW also leans on the compiler, but it does so to issue several operations at once, not to minimise area and power.

✗ **CISC philosophy.** Wrong, and the opposite. CISC puts the complexity in the HARDWARE to make the compiler's job easier.

7. The answer is **CISC**. Closing the semantic gap between high-level languages and machine instructions was the original CISC goal, so one instruction could do the work of a whole statement.

✗ **Stack-based ISA.** Wrong. A stack ISA simplifies operand addressing, but it does not aim to mirror high-level constructs.

✗ **Harvard Architecture.** Wrong. Harvard describes the memory organization, not the instruction philosophy.

✗ **RISC.** Wrong, and the opposite. RISC deliberately keeps instructions simple and makes the COMPILER work harder.

✓ **CISC.** Correct. Complex instructions resembling high-level constructs, which is where the semantic gap argument comes from.

OBJECTIVE SECTION

Von Neumann, Harvard and Image Data

5 questions. Taught in Module One Unit 1, and Module One Unit 2. Cover the answers before you start.

1. **TEST 1 Q18** For a monochrome bitmap image, each pixel requires: (a) 2 bits (b) 8 bits (c) 1 bit (d) 4 bits
2. **TEST 1 Q31** Photographs are generally better represented using: (a) Vector graphics (b) ASCII encoding (c) Excess-3 encoding (d) Bitmap graphics
3. **TEST 1 Q37** A processor fetches an instruction while simultaneously reading data from memory. Which architecture makes this possible? (a) Stack Architecture (b) CISC Architecture (c) Von Neumann Architecture (d) Harvard Architecture
4. **TEST 1 Q44** An embedded system uses separate memory modules for instructions and data to improve throughput. Which architectural design is being used? (a) Von Neumann Architecture (b) Stack Architecture (c) Pipeline Architecture (d) Harvard Architecture
5. **TEST 1 Q46** Which characteristic is a major advantage of the Harvard Architecture? (a) Single memory space for all data (b) Simpler hardware implementation (c) Simultaneous access to instructions and data (d) Reduced memory requirements

1. The answer is **1 bit**. Monochrome means two states, black or white, and one bit gives exactly two states. The general rule is that k bits per pixel give 2^k colours.

✗ **2 bits**. Wrong. 2 bits give 4 levels, which is a four-shade image rather than a monochrome one.

✗ **8 bits**. Wrong. 8 bits give 256 levels, the usual greyscale depth.

✓ **1 bit**. Correct. One bit, two states, so image size in bits equals the number of pixels.

✗ **4 bits**. Wrong. 4 bits give 16 levels, the old 16-colour standard.

2. The answer is **Bitmap graphics**. A photograph is continuous tone with no clean edges to describe as shapes, so it is stored as a grid of pixel values. Pair this with the logo question, where the answer was vector.

✗ **Vector graphics**. Wrong. Vectors describe shapes with equations, which suits logos and diagrams, not the texture of a real scene.

✗ **ASCII encoding**. Wrong. ASCII encodes text characters.

✗ **Excess-3 encoding**. Wrong. Excess-3 encodes single decimal digits.

✓ **Bitmap graphics**. Correct. A pixel grid captures continuous tone, which is what a photograph is.

3. The answer is **Harvard Architecture**. Harvard keeps instructions and data in separate memories with separate buses, so both can be read in the same cycle.

✗ **Stack Architecture**. Wrong. Stack architecture describes how operands are addressed, not how memory is wired.

✗ **CISC Architecture**. Wrong. CISC is an instruction set philosophy, unrelated to the number of memory paths.

✗ **Von Neumann Architecture**. Wrong, and this is the contrast being tested. Von Neumann shares ONE memory and one bus for both, which forces the two accesses to take turns. That queue is the Von Neumann bottleneck.

✓ **Harvard Architecture**. Correct. Separate paths mean genuinely simultaneous access.

4. The answer is **Harvard Architecture**. The phrase SEPARATE MEMORY for instructions and data is the definition of Harvard. It is common in embedded and signal processing designs for exactly the throughput reason given.

✗ **Von Neumann Architecture**. Wrong. Von Neumann is the SINGLE shared memory design, the opposite of what the stem describes.

✗ **Stack Architecture**. Wrong. Stack architecture is about operand addressing, not memory separation.

✗ **Pipeline Architecture**. Wrong. Pipelining overlaps the stages of instruction processing. It is a technique, not a memory arrangement, though Harvard helps it along.

✓ **Harvard Architecture**. Correct. Separate instruction and data memories.

5. The answer is **Simultaneous access to instructions and data**. Two memories with two buses means an instruction fetch and a data read can happen at once, which raises throughput and removes the Von Neumann bottleneck.

✗ **Single memory space for all data**. Wrong, and it describes VON NEUMANN. Harvard deliberately splits the memory space in two.

✗ **Simpler hardware implementation**. Wrong, and backwards. Two memory systems and two bus sets are MORE hardware, which is the price Harvard pays.

✓ **Simultaneous access to instructions and data**. Correct. Both accesses in the same cycle.

✗ **Reduced memory requirements**. Wrong. Splitting memory does not reduce how much is needed, and fixed separate pools can waste space when one fills before the other.

MOCK EXAMINATION ONE

Sit This Paper Under Exam Conditions

Two hours, no notes, one sheet of paper. This paper is weighted towards the 2024/25 and 2025/26 style: performance, pipelining, cache and floating point. Answer Question One and any other two. Full worked answers follow, so attempt everything before you turn to them.



2 hours
70 marks

Q1 (30) compulsory
+ any two of 20

Question Paper

Time: 2 hours. Answer **Question One (compulsory, 30 marks)** and **any other two questions (20 marks each)**. Total 70 marks.

These questions were written for this manual. They are not a real paper. Every number in them is new, so none of them carries an **As printed** bar: that bar is reserved for the examiner's own words, and nothing on this page is the examiner's. What this paper copies is the *shape* of the questions that have actually been set, chiefly the 2024/25 and 2025/26 papers: a compulsory Question One mixing architecture, performance, representation and cache, then four optional questions of 20 marks. When you want the real wording, go to the Past Papers part, where both of those papers are quoted in full and answered.

QUESTION ONE

COMPULSORY · 30 MARKS

- (a)** (i) Differentiate computer architecture from computer organization. (2) (ii) Compare the Von Neumann and Harvard architectures, with a diagram for each. (3)
- (b)** A processor runs 2 million instructions with this mix and cycle cost: ALU 45% at 1 cycle, Load 25% at 4 cycles, Store 15% at 2 cycles, Branch 15% at 3 cycles. (i) Find the average CPI. (ii) Find the total clock cycles. (iii) If Load is optimised to 2 cycles, find the new CPI. (iv) Find the percentage performance improvement. (6)
- (c)** Using a **12-bit** register, represent **150** and **−95** in unsigned, sign-magnitude, one's complement and two's complement. (6)
- (d)** (i) Represent **39.5** in IEEE-754 single precision, giving the hex. (ii) Find the decimal value of **42480000H** under IEEE-754. (9)
- (e)** A 32-bit machine has a direct-mapped cache of 1024 lines with 32-byte blocks. Give the number of tag, index and offset bits. (4)

QUESTION TWO

20 MARKS

- (a)** A 5-stage pipeline has stage times 20, 15, 25, 20 and 10 ns with a 5 ns latch delay. For 1000 tasks find (i) the cycle time, (ii) the non-pipeline time for one task, (iii) the pipeline time for 1000 tasks, (iv) the sequential time for 1000 tasks, (v) the speedup. (10)
- (b)** Explain the three kinds of pipeline hazard, and say how forwarding and a single stall handle a load-use data hazard. (6)
- (c)** Why is the speedup of a real pipeline always less than the number of stages? (4)

QUESTION THREE**20 MARKS**

- (a)** A system has 256 MB main memory, a 64 KB cache with 32-byte blocks, using 4-way set associative mapping. Find (i) the number of cache blocks, (ii) the number of sets, (iii) the tag, set and offset field widths. (8)
- (b)** The cache hit ratio is 90%, cache access is 3 ns and main memory is 60 ns. Find the average memory access time for simultaneous and for hierarchical organisation. (6)
- (c)** Explain locality of reference, distinguishing temporal from spatial locality, and say how each shapes cache design. (6)

QUESTION FOUR**20 MARKS**

- (a)** Write $Y = (A + B) \times (C - D)$ in the 3-address, 2-address, 1-address and 0-address instruction formats. (10)
- (b)** Given PC = 4000, R1 = 2500, an address field of 3200, and memory where M[1500] = 700, M[2500] = 900, M[3200] = 1500 and M[7200] = 350, give the effective address and operand for direct, indirect, register-indirect (R1) and relative addressing. (6)
- (c)** State four differences between RISC and CISC, with one example processor of each. (4)

QUESTION FIVE**20 MARKS**

- (a)** State Amdahl's law. A program has 90% of its work parallelised to run 8 times faster. Find the overall speedup. (6)
- (b)** State four differences between DRAM and SRAM, and say which is used for cache and which for main memory, and why. (8)
- (c)** Describe the four memory access methods, with one example device each. (6)

MOCK EXAMINATION ONE

Answers

Full worked solutions. Each cross-refers to the unit that teaches it, so a slip points you straight to the revision you need.

(A) ARCHITECTURE VS ORGANIZATION; VON NEUMANN VS HARVARD (SEE M1 U1)

Architecture is the programmer-visible design (instruction set, registers, addressing modes): what the machine does. Organization is how that is built in hardware (control signals, buses, memory technology): how it does it. Von Neumann uses one memory and one bus for instructions and data (simple, but the bus is a bottleneck); Harvard uses separate memories and buses (simultaneous access, faster, costlier). Draw both diagrams.

(B) CPI (SEE M2 U2)

(i) $CPI = 0.45(1) + 0.25(4) + 0.15(2) + 0.15(3) = 0.45 + 1.00 + 0.30 + 0.45 = \mathbf{2.20}$. (ii) Total cycles = $2\,000\,000 \times 2.20 = \mathbf{4\,400\,000}$. (iii) With Load at 2 cycles: $CPI = 0.45 + 0.25(2) + 0.30 + 0.45 = \mathbf{1.70}$. (iv) Improvement = $(2.20 - 1.70) / 2.20 = \mathbf{22.73\%}$.

(C) 12-BIT REPRESENTATIONS (SEE M1 U2)

150 = 0000 1001 0110 (positive, so identical in all four schemes). For -95: +95 = 0000 0101 1111.

VALUE	UNSIGNED	SIGN-MAGNITUDE	ONE'S COMPLEMENT	TWO'S COMPLEMENT
150	0000 1001 0110	0000 1001 0110	0000 1001 0110	0000 1001 0110
-95	not representable	1000 0101 1111	1111 1010 0000	1111 1010 0001

(D) IEEE-754 (SEE M1 U3)

(i) $39.5 = 100111.1 = 1.001111 \times 2^5$. Exponent $5 + 127 = 132 = 1000\,0100$. Sign 0. Mantissa 001111 padded to 23 bits. Word = 0 10000100 001111000000000000000000 = **421E0000H**. (ii) 42480000H = 0100 0010 0100 1000...: sign 0, exponent 1000 0100 = 132 so $e = 5$, mantissa 1001..., value = $1.1001 \times 2^5 = 110010 = \mathbf{50.0}$.

(E) CACHE FIELDS (SEE M3 U3)

Offset = $\log_2 32 = 5$ bits; index = $\log_2 1024 = 10$ bits; tag = $32 - 10 - 5 = \mathbf{17\ bits}$.

(A) PIPELINE TIMING (SEE M2 U3)

(i) Cycle time = slowest stage + latch = $25 + 5 = 30$ ns. (ii) Non-pipeline one task = $20 + 15 + 25 + 20 + 10 = 90$ ns. (iii) Pipeline for 1000 tasks = $(5 + 999) \times 30 = 1004 \times 30 = 30\ 120$ ns. (iv) Sequential for 1000 = $1000 \times 90 = 90\ 000$ ns. (v) Speedup = $90\ 000 / 30\ 120 = 2.99$.

(B) HAZARDS (SEE M2 U3, M4 U2)

Structural (two instructions need the same hardware: duplicate it), data (a value is needed before it is written back: use forwarding, and one stall for a load-use case), control (a branch changes the next instruction: resolve early or flush). Forwarding feeds a result straight from where it is produced; a load value is not ready until the memory stage, so the very next instruction still needs one stall.

(C) WHY SPEEDUP < STAGES

The pipeline must first fill (the first task still takes k cycles), and hazards insert stalls, so the ideal ceiling of k is never quite reached, especially for few tasks.

(A) CACHE CONFIGURATION (SEE M3 U3)

(i) Blocks = cache / block = $64\text{ KB} / 32\text{ B} = 2^{16} / 2^5 = 2048$ blocks. (ii) Sets = $2048 / 4 = 512$ sets. (iii) Address = $256\text{ MB} = 2^{28}$, so 28 bits. Offset = $\log_2 32 = 5$, set = $\log_2 512 = 9$, tag = $28 - 9 - 5 = 14$ bits. Format: tag 14, set 9, offset 5.

(B) AMAT (SEE M3 U3)

Simultaneous = $0.9(3) + 0.1(60) = 2.7 + 6 = 8.7$ ns. Hierarchical = $3 + 0.1(60) = 3 + 6 = 9.0$ ns.

(C) LOCALITY (SEE M3 U1)

Locality of reference is the clustering of accesses in a small region over any short time. Temporal: a location used now is likely used again soon (so cache keeps recently used data). Spatial: nearby locations are likely used soon (so cache fetches whole blocks of neighbours).

(A) $Y = (A + B) \times (C - D)$ IN FOUR FORMATS (SEE M1 U4)

3-ADDRESS	2-ADDRESS	1-ADDRESS	0-ADDRESS
ADD T1,A,B	MOV T1,A	LOAD A	PUSH A
SUB T2,C,D	ADD T1,B	ADD B	PUSH B
MUL Y,T1,T2	MOV T2,C	STORE T1	ADD
	SUB T2,D	LOAD C	PUSH C
	MUL T1,T2	SUB D	PUSH D
	MOV Y,T1	MUL T1	SUB
		STORE Y	MUL
			POP Y

(B) ADDRESSING MODES (SEE M1 U4)

MODE	EFFECTIVE ADDRESS	OPERAND
Direct	3200	M[3200] = 1500
Indirect	M[3200] = 1500	M[1500] = 700
Register-indirect (R1)	R1 = 2500	M[2500] = 900
Relative	PC + 3200 = 7200	M[7200] = 350

(C) RISC VS CISC (SEE M4 U1)

RISC: few simple fixed-length one-cycle instructions, load-store, many registers (example ARM or MIPS). CISC: many complex variable-length multi-cycle instructions, direct memory access, fewer registers (example Intel x86 or Motorola 68000).

(A) AMDAHL'S LAW (SEE M2 U2)

Stating it (the question says "state", so a formula alone does not answer it): *the gain in overall performance from improving one part of a system is limited by the fraction of the total time that part accounts for.* If a fraction f of the execution time is made s times faster and the rest is unchanged, the overall speedup is $S = 1 / ((1 - f) + f/s)$; and as s grows without bound S tends to $1 / (1 - f)$, which is the ceiling no improvement to that part can pass.

Applying it. With $f = 0.90$ and $s = 8$: Speedup = $1 / (0.10 + 0.90 / 8) = 1 / (0.10 + 0.1125) = 1 / 0.2125 = 4.71$. Sanity check against the ceiling: $1 / (1 - 0.90) = 10$, and 4.71 is below it, as it must be.

(B) DRAM VS SRAM (SEE M3 U2)

DRAM stores a bit as capacitor charge, needs refresh, is slower, denser and cheaper, and is used for **main memory**. SRAM stores a bit in a transistor latch, needs no refresh, is faster, less dense and dearer, and is used for **cache**. Cache needs speed; main memory needs cheap capacity.

(C) MEMORY ACCESS METHODS (SEE M3 U1)

Sequential (read in order, example magnetic tape), direct (jump near then search, example disk), random (any location in equal time, example RAM), associative (located by content, example a cache tag store).

MOCK EXAMINATION TWO

Sit This One After the First

A second full paper in the same shape, leaning to the 2023/24 and 2024/25 style: number representation, floating point, cache mapping and instruction formats. Two hours. Answer Question One and any other two. Solutions follow.



2 hours
70 marks

Q1 (30) compulsory
+ any two of 20

Question Paper

Time: 2 hours. Answer **Question One (compulsory, 30 marks)** and **any other two questions (20 marks each)**. Total 70 marks.

These questions were written for this manual. They are not a real paper. Every number in them is new, so none of them carries an **As printed** bar: that bar is reserved for the examiner's own words, and nothing on this page is the examiner's. What this paper copies is the *shape* of questions that have actually been set, leaning to the 2023/24 and 2024/25 papers: signed representation tables, an IEEE-754 conversion each way, cache field widths and the instruction formats. When you want the real wording, go to the Past Papers part for 2024/25 in full, or to the unit that teaches each topic, where the 2023/24 questions are quoted where they are taught.

QUESTION ONE

COMPULSORY · 30 MARKS

- (a)** (i) State the Von Neumann concept and two of its significant aspects. (3) (ii) What is the Von Neumann bottleneck, and give two ways to reduce it? (2)
- (b)** A 16-bit two's complement processor stores 300, -200 and -5000. (i) Give the representable range and confirm all three fit. (ii) Represent each in binary and hexadecimal. (iii) Compute $300 + (-200)$ in two's complement. (9)
- (c)** (i) Represent **85.75** in IEEE-754 single precision (hex). (ii) Find the decimal value of **C2960000H** under IEEE-754. (8)
- (d)** Write $Z = (A + B) \times (C + D) + E$ in the 3-address and 0-address formats, and count the bytes of each assuming 1 byte per opcode and 2 bytes per address. (8)

QUESTION TWO

20 MARKS

- (a)** A 32-bit machine has a direct-mapped cache in which each block holds 8 words and there are 512 blocks. (i) Give the offset, index and tag field widths. (ii) In which cache block do the addresses ABCD1234H and 5678FEDCH fall? (10)
- (b)** Explain the three cache mapping functions (direct, fully associative, set associative), with one advantage and one disadvantage of each. (10)

QUESTION THREE

20 MARKS

- (a)** Using a **10-bit** register, complete the unsigned, sign-magnitude, one's complement and two's complement forms of **200** and **-75**. (8)
- (b)** Using end-around carry in one's complement, compute $200 - 75$ in a 10-bit register. (6)
- (c)** Find the memory needed for a black-and-white image of 8 inches by 10 inches at 200 dpi. Give the answer in megabytes. (6)

QUESTION FOUR**20 MARKS**

- (a)** Find the decimal value of **43A00000H** under IEEE-754 single precision. (6)
- (b)** Name the three categories of instructions, with two examples of each. (6)
- (c)** Describe the immediate, direct, indirect and register-indirect addressing modes, saying where the operand is in each. (8)

QUESTION FIVE**20 MARKS**

- (a)** A 4-stage pipeline has stage times 40, 30, 50 and 35 ns with a 5 ns latch. For 200 tasks, find the cycle time, the pipeline time, the sequential time and the speedup. (8)
- (b)** Draw and describe the memory hierarchy, from registers to secondary storage, saying how speed, size and cost change down it. (6)
- (c)** A cache has an 85% hit ratio, 4 ns cache access and 70 ns main memory access. Find the average memory access time for a hierarchical organisation. (6)

MOCK EXAMINATION TWO

Answers

Full worked solutions, each cross-referenced to the unit that teaches it.

(A) VON NEUMANN (SEE M1 U1)

(i) The Von Neumann concept: a stored-program computer keeps instructions and data in the same memory and executes instructions sequentially. Two aspects: a single shared memory for code and data; sequential fetch-execute of a stored program. (ii) The bottleneck: the single shared bus lets only instructions or data move at a time, so the CPU waits on memory. Reduce it with cache and with separate instruction and data caches (or wider buses).

(B) 16-BIT TWO'S COMPLEMENT (SEE M1 U2)

(i) Range = -2^{15} to $+(2^{15}-1) = -32768$ to 32767 ; 300, -200 and -5000 all fit.

VALUE	BINARY (16-BIT TWO'S COMPLEMENT)	HEX
300	0000 0001 0010 1100	012C
-200	1111 1111 0011 1000	FF38
-5000	1110 1100 0111 1000	EC78

(iii) $300 + (-200)$: add the patterns $012C + FF38 = 1\ 0064$, discard the carry out of bit 16, leaving $0064H = \mathbf{100}$. Check: $300 - 200 = 100$.

(C) IEEE-754 (SEE M1 U3)

(i) $85.75 = 1010101.11 = 1.01010111 \times 2^6$. Exponent $6 + 127 = 133 = 1000\ 0101$. Mantissa 01010111 to 23 bits. Word = $0\ 10000101\ 010101110000000000000000 = \mathbf{42AB8000H}$. (ii) $C2960000H$: sign 1, exponent $1000\ 0101 = 133$ so $e = 6$, mantissa 00101110 , value = $-1.0010110 \times 2^6 = -1001011 = \mathbf{-75.0}$.

(D) INSTRUCTION FORMATS AND BYTES (SEE M1 U4)

3-address (opcode 1, three addresses 2 each = 7 bytes each):

ADD T1, A, B	ADD T2, C, D	MUL T1, T1, T2	ADD Z, T1, E
--------------	--------------	----------------	--------------

4 instructions $\times 7 = \mathbf{28\ bytes}$. 0-address (postfix $A\ B + C\ D + \times E +$): PUSH A, PUSH B, ADD, PUSH C, PUSH D, ADD, MUL, PUSH E, ADD, POP Z = 10 instructions = 5 pushes $\times 3 + 1$ pop $\times 3 + 4$ ops $\times 1 = \mathbf{22\ bytes}$.

(A) CACHE FIELDS AND BLOCK LOOKUP (SEE M3 U3)

(i) Offset = $\log_2 8 = 3$, index = $\log_2 512 = 9$, tag = $32 - 9 - 3 = \mathbf{20}$. (ii) Block = (address with the low 3 offset bits removed) mod 512, that is (address $\div 8$) mod 512. ABCD1234H gives block **70**; 5678FEDCH gives block **475**.

(B) THE THREE MAPPING FUNCTIONS (SEE M3 U3)

Direct: one line per block (fast and simple, but conflict misses). Fully associative: a block may go anywhere (no conflict misses, but every tag must be searched, costly). Set associative: a block maps to one set of k lines (the compromise, fewer conflicts than direct at less cost than fully associative).

ANSWER

QUESTION THREE

(A) 10-BIT REPRESENTATIONS (SEE M1 U2)

200 = 0011001000 (positive, same in all four). For -75: +75 = 0001001011.

VALUE	UNSIGNED	SIGN-MAGNITUDE	ONE'S COMPLEMENT	TWO'S COMPLEMENT
200	0011001000	0011001000	0011001000	0011001000
-75	not representable	1001001011	1110110100	1110110101

(B) END-AROUND CARRY (SEE M1 U2)

$200 - 75 = 200 + (-75)$. 200 = 0011001000; -75 in one's complement = 1110110100. Add: 0011001000 + 1110110100 = 1 0001111100, a carry out of bit 10. End-around: drop it and add 1 to the low bit: 0001111100 + 1 = 0001111101 = **125**. Check: $200 - 75 = 125$.

(C) IMAGE MEMORY (SEE M1 U2)

Pixels = $(8 \times 200) \times (10 \times 200) = 1600 \times 2000 = 3\,200\,000$. At 1 bit per pixel that is 3 200 000 bits = 400 000 bytes = $400\,000 / 1\,048\,576 = \mathbf{0.381\ MB}$ (about 0.38 MB).

ANSWER

QUESTION FOUR

(A) IEEE-754 REVERSE (SEE M1 U3)

43A00000H = 0100 0011 1010 0000...: sign 0, exponent 1000 0111 = 135 so $e = 8$, mantissa 0100000. Value = $1.0100000 \times 2^8 = 101000000 = \mathbf{320.0}$.

(B) INSTRUCTION CATEGORIES (SEE M1 U4)

Data transfer (LOAD, STORE, MOVE, PUSH, POP), arithmetic and logic (ADD, SUB, MUL, AND, OR), control or test-and-branch (JUMP, BEQ, CALL, RETURN).

(C) ADDRESSING MODES (SEE M1 U4)

Immediate: the operand is the address field itself (a constant). Direct: the operand is $M[\text{address field}]$. Indirect: the field points to a location that holds the address, so the operand is $M[M[\text{address field}]]$. Register-indirect: a register holds the address, so the operand is $M[\text{register}]$.

ANSWER

QUESTION FIVE

(A) PIPELINE TIMING (SEE M2 U3)

Cycle time = slowest stage + latch = $50 + 5 = \mathbf{55\ ns}$. Pipeline time for 200 tasks = $(4 + 199) \times 55 = 203 \times 55 = \mathbf{11\,165\ ns}$. Sequential time = $200 \times (40 + 30 + 50 + 35) = 200 \times 155 = \mathbf{31\,000\ ns}$. Speedup = $31\,000 / 11\,165 = \mathbf{2.78}$.

(B) MEMORY HIERARCHY (SEE M3 U1)

Registers, then cache (L1, L2, L3), then main memory (RAM), then secondary storage (disk, SSD), then tertiary (tape). Going down: bigger, slower and cheaper per byte. Going up: smaller, faster and dearer.

(C) AMAT (SEE M3 U3)

Hierarchical: $AMAT = T_{\text{cache}} + (1 - h) \times T_{\text{memory}} = 4 + 0.15 \times 70 = 4 + 10.5 = \mathbf{14.5\ ns}$.

MOCK EXAMINATION THREE

The Hardest of the Three

The last paper in the book, and deliberately the most demanding: every numeric type on the syllabus appears, with numbers you have not seen before. Answer Question One and any other two. If you can score well here under the clock, the real paper will hold no surprises.



2 hours

70 marks

Q1 (30) compulsory

+ any two of 20

fresh numbers throughout

Question Paper

Time: 2 hours. Answer **Question One (compulsory, 30 marks)** and **any other two** questions (20 marks each). Total 70 marks.

These questions were written for this manual. They are not a real paper. Every number in them is new and deliberately harder than anything that has been set, so none of them carries an **As printed** bar: that bar is reserved for the examiner's own words, and nothing on this page is the examiner's. What this paper copies is the *shape* of the real papers, and it goes one step past them on purpose, so that the genuine article feels easier than your last rehearsal. When you want the real wording, go to the Past Papers part, where 2024/25 and 2025/26 are quoted in full and answered.

Close the book on the answers. Use the Reference part only after you have finished, when you are marking yourself.

EXAM QUESTION

QUESTION ONE (COMPULSORY, 30 MARKS)

(a) Differentiate between computer architecture and computer organization, and give one example that shows the difference. **(2 marks)**

(b) Using **11 bits**, represent **178** and **−92** in unsigned, sign-magnitude, one's complement and two's complement. State the range of 11-bit two's complement and say whether both values fit. **(6 marks)**

(c) Represent **92.375** in IEEE-754 single precision. Give the answer in binary and in hexadecimal. **(5 marks)**

(d) A processor executes **2 500 000** instructions with the mix below. Find (i) the average CPI, (ii) the total clock cycles, (iii) the new average CPI if the 4-cycle class is optimised to 2 cycles, and (iv) the percentage improvement. **(6 marks)**

CLASS	A	B	C	D	E
SHARE	35%	25%	20%	12%	8%
CYCLES	1	2	3	4	5

(e) A machine has **24-bit** addresses, a block size of **32 bytes** and a direct-mapped cache of **512 lines**. Give the address format, then state the cache line each of these addresses maps to: **A5C3E7**, **7F0044**, **C0FFEE**. **(6 marks)**

(f) Given PC = 4000, R2 = 2600, address field = 900, and the memory table below, find the effective address and the operand under immediate, direct, indirect, register indirect (R2) and relative addressing. **(5 marks)**

ADDRESS	900	2600	3100	4900	4000
VALUE	3100	4900	750	88	900

EXAM QUESTION**QUESTION TWO (20 MARKS)**

- (a) A 5-stage pipeline has stage times **40, 55, 45, 60 and 50 ns** with a **5 ns** latch delay. Find (i) the cycle time, (ii) the non-pipelined time for one task, (iii) the ideal speedup, (iv) the pipelined and sequential times for **800** tasks, and (v) the steady-state throughput. **(10 marks)**
- (b) Name and explain the three classes of pipeline hazard, and state which one forwarding cannot remove. **(4 marks)**
- (c) A compiler optimisation speeds up **75%** of a program by a factor of **8**. Find the overall speedup by Amdahl's law, and explain why it is not 8. **(6 marks)**

EXAM QUESTION**QUESTION THREE (20 MARKS)**

- (a) A cache of **32 KB** with **64-byte** blocks backs a **16 MB** main memory, **8-way** set associative. Find the number of lines, the number of sets, the address format, and the size of the tag directory. **(10 marks)**
- (b) The cache has a hit ratio of **92%**, an access time of **3 ns**, and main memory takes **70 ns**. Find the average memory access time under hierarchical and under simultaneous organisation. **(6 marks)**
- (c) Explain locality of reference, and distinguish temporal from spatial locality with one example of each. **(4 marks)**

EXAM QUESTION**QUESTION FOUR (20 MARKS)**

- (a) Use end-around carry in one's complement to compute **245 – 178** in a **10-bit** register. Show every step. **(6 marks)**
- (b) (i) Encode **507** in Excess-3. (ii) Decode **0111 1001 0110** from Excess-3. (iii) State the property that makes Excess-3 self-complementing. **(4 marks)**
- (c) Write $Y = (P + Q) \times (R - S) / T$ in the 3, 2, 1 and 0 address formats, and count the bytes of each, taking **1 byte per opcode** and **2 bytes per address**. **(10 marks)**

EXAM QUESTION**QUESTION FIVE (20 MARKS)**

- (a) Find the decimal value of **C2C60000H** under IEEE-754 single precision. **(6 marks)**
- (b) A computer uses **21 address lines** and a word size of **4 bytes**, word-addressable. Find the address space, the total memory in MB, and the width of the program counter. **(5 marks)**
- (c) Find the memory needed for a black-and-white image **8.5 by 11 inches** at **200 dpi**, in KB. **(4 marks)**
- (d) State four differences between RISC and CISC, and name two example processors of each. **(5 marks)**

Answers

Full worked solutions, every number recomputed. Each part names the unit that teaches it, so a slip points you straight at the revision you need.

(A) ARCHITECTURE VERSUS ORGANIZATION (2 MARKS) [M1 U1]

Architecture is the set of attributes visible to the programmer (instruction set, registers, addressing modes): **what** the machine does. Organization is how those are realised in hardware (control signals, buses, memory technology): **how** it does it.

Example: the architecture says a MULTIPLY instruction exists; the organization decides whether a dedicated multiplier or repeated addition carries it out.

(B) 178 AND -92 IN 11 BITS (6 MARKS) [M1 U2]

GIVEN	values 178 and $-92 \cdot n = 11$ bits $\cdot$ four schemes
FIND	each value in each scheme, plus the two's complement range
FORMULA	sign-magnitude = sign bit + magnitude $\cdot$ one's complement = invert all n bits $\cdot$ two's complement = invert then add 1 $\cdot$ range = -2^{n-1} to $+(2^{n-1}-1)$

- 1 178 = 128+32+16+2, so in 11 bits 178 = 000 1011 0010. Positive, so identical in all four schemes.
- 2 For -92 , first write $+92 = 64+16+8+4 = 000 0101 1100$.
- 3 Sign-magnitude: set the sign bit, keep the magnitude: 100 0101 1100.
- 4 One's complement: invert all 11 bits of $+92$: 111 1010 0011.
- 5 Two's complement: add 1 to that: 111 1010 0100.

VALUE	UNSIGNED	SIGN-MAGNITUDE	ONE'S COMPLEMENT	TWO'S COMPLEMENT
178	000 1011 0010	000 1011 0010	000 1011 0010	000 1011 0010
-92	not representable	100 0101 1100	111 1010 0011	111 1010 0100

11-bit two's complement range = -2^{10} to $+(2^{10}-1) = -1024$ to 1023 . Both 178 and -92 fall well inside it, so both are representable.

(C) 92.375 IN IEEE-754 SINGLE PRECISION (5 MARKS) [M1 U3]

GIVEN	value = 92.375 $\cdot$ IEEE-754 single precision $\cdot$ bias 127
FIND	the 32-bit pattern, in binary and hex
FORMULA	normalise to $1.m \times 2^e$, then $E = e + 127$

- 1 Sign: positive, so **0**.
- 2 Binary: $92 = 1011100$ and $0.375 = 0.011$, so $92.375 = 1011100.011$.
- 3 Normalise: move the point 6 places left, 1.011100011×2^6 , so $e = 6$.
This one terminates, so it is exact.
- 4 Exponent field: $6 + 127 = 133 = 1000 0101$.
- 5 Mantissa: 011100011 padded to 23 bits = 0111 0001 1000 0000 0000 000.

$92.375 = 0 \ 10000101 \ 01110001100000000000000 = 0100 \ 0010 \ 1011 \ 1000 \ 1100 \ 0000 \ 0000 \ 0000 = 42B8C000H$. Converting back gives 92.375 exactly, which is the check to write down.

(D) CPI OVER 2 500 000 INSTRUCTIONS (6 MARKS) [M2 U2]

GIVEN	IC = 2 500 000 · A 35% at 1, B 25% at 2, C 20% at 3, D 12% at 4, E 8% at 5 · for (iii), class D falls to 2 cycles
FIND	average CPI, total cycles, new CPI, percentage improvement
FORMULA	$CPI = \sum(\text{fraction}_i \times CPI_i) \cdot \text{cycles} = IC \times CPI \cdot \text{improvement} = (\text{old} - \text{new})/\text{old}$

- 1 CPI = $0.35(1) + 0.25(2) + 0.20(3) + 0.12(4) + 0.08(5) = 0.35 + 0.50 + 0.60 + 0.48 + 0.40 = \mathbf{2.33}$.
- 2 Total cycles = $2\,500\,000 \times 2.33 = \mathbf{5\,825\,000}$.
- 3 Class D from 4 to 2 cycles: $CPI = 2.33 - 0.12(4) + 0.12(2) = 2.33 - 0.48 + 0.24 = \mathbf{2.09}$.
- 4 Improvement = $(2.33 - 2.09) / 2.33 = 0.24 / 2.33 = \mathbf{10.30\%}$.

CPI 2.33; 5 825 000 cycles; new CPI 2.09; improvement 10.30%. Note the improvement is measured against the **old** CPI.

(E) 24-BIT ADDRESS, 32-BYTE BLOCK, 512-LINE DIRECT-MAPPED CACHE (6 MARKS) [M3 U3]

GIVEN	address = 24 bits · block = 32 bytes · 512 lines · direct mapped
FIND	the address format and the line each address maps to
FORMULA	$\text{offset} = \log_2(\text{block}) \cdot \text{index} = \log_2(\text{lines}) \cdot \text{tag} = \text{address} - \text{index} - \text{offset} \cdot \text{the index field is the line number}$

- 1 Offset = $\log_2 32 = \mathbf{5}$ bits. Index = $\log_2 512 = \mathbf{9}$ bits. Tag = $24 - 9 - 5 = \mathbf{10}$ bits.
- 2 Line = $(\text{address} \div 32) \bmod 512$, that is shift right 5 and keep the low 9 bits.
- 3 A5C3E7 = 10 863 591. Shift right 5: 339 487. mod 512 = **31**.
- 4 7F0044 = 8 323 140. Shift right 5: 260 098. mod 512 = **2**.
- 5 C0FFEE = 12 648 430. Shift right 5: 395 263. mod 512 = **511**.

Format: **tag 10 / line 9 / offset 5**. Lines: A5C3E7 → **31**, 7F0044 → **2**, C0FFEE → **511**. Here the fields do not fall on hex-digit boundaries (9 index bits is not a whole number of hex digits), so convert to decimal and divide rather than reading digits off, which is the trap this part sets.

(F) EFFECTIVE ADDRESSES (5 MARKS) [M1 U4]

GIVEN	PC = 4000 · R2 = 2600 · address field = 900 · M[900] = 3100, M[2600] = 4900, M[3100] = 750, M[4900] = 88
FIND	the effective address and operand in each mode
FORMULA	immediate: operand = the field · direct: EA = field · indirect: EA = M[field] · register indirect: EA = register · relative: EA = PC + field

MODE	EFFECTIVE ADDRESS	OPERAND
Immediate	none	900, the field itself
Direct	900	$M[900] = 3100$
Indirect	$M[900] = 3100$	$M[3100] = 750$
Register indirect (R2)	$R2 = 2600$	$M[2600] = 4900$
Relative	$PC + 900 = 4900$	$M[4900] = 88$

(A) PIPELINE TIMING (10 MARKS) [M2 U3]

GIVEN	$k = 5$ stages · stage times 40, 55, 45, 60, 50 ns · latch = 5 ns · $n = 800$ tasks
FIND	cycle time, one-task non-pipelined time, ideal speedup, 800-task pipelined and sequential times, throughput
FORMULA	cycle = slowest stage + latch · pipelined = $(k + n - 1) \times \text{cycle}$ · sequential = $n \times \sum(\text{stage times})$ · speedup = sequential / pipelined · throughput = $1 / \text{cycle}$ in the steady state

- 1 Cycle time = slowest stage + latch = $60 + 5 = 65$ ns.
- 2 Non-pipelined, one task = $40+55+45+60+50 = 250$ ns.
- 3 Ideal speedup = $250 / 65 = 3.85$ (its ceiling is $k = 5$; the latch and the uneven stages cost the rest).
- 4 800 tasks pipelined = $(5 + 800 - 1) \times 65 = 804 \times 65 = 52\,260$ ns.
- 5 800 tasks sequentially = $800 \times 250 = 200\,000$ ns, so the realised speedup is $200\,000 / 52\,260 = 3.83$.
- 6 Throughput = $800 / 52\,260$ ns = **15.31 million tasks per second**; in the steady state this is one task per cycle, $1 / 65$ ns = 15.38 million per second.

Cycle 65 ns; one task 250 ns; ideal speedup 3.85; 800 tasks 52 260 ns pipelined against 200 000 ns sequential, a speedup of 3.83; throughput about 15.31 million tasks per second.

(B) THE THREE HAZARDS (4 MARKS) [M2 U3]

Structural: two instructions need the same hardware in the same cycle (one memory serving a fetch and a data access); cured by duplicating the resource, as with separate instruction and data memories. **Data:** an instruction needs a result the one ahead has not produced yet (read after write); mostly cured by **forwarding**. **Control:** a branch changes what runs next after the pipeline has already fetched the following instructions; cured by resolving the branch early, predicting it, or flushing. Forwarding **cannot** remove the **load-use** data hazard: the loaded value is still being fetched from memory, so exactly one stall remains.

(C) AMDAHL'S LAW (6 MARKS) [M2 U2]

GIVEN	$f = 0.75 \cdot s = 8$
FIND	the overall speedup, and why it is not 8
FORMULA	speedup = $1 / ((1 - f) + f/s)$

- 1 Untouched part: $1 - f = 0.25$, still at full cost.
- 2 Sped-up part: $f / s = 0.75 / 8 = 0.09375$.
- 3 Speedup = $1 / (0.25 + 0.09375) = 1 / 0.34375 = 2.91$.

Overall speedup **2.91**, not 8. The 25% that was not improved still takes its original time, and it alone caps the total gain at $1/0.25 = 4$ even if the other 75% were made infinitely fast. That ceiling is the whole content of Amdahl's law.

(A) 8-WAY SET-ASSOCIATIVE CACHE DESIGN (10 MARKS) [M3 U3]

GIVEN cache = 32 KB · block = 64 bytes · main memory = 16 MB · k = 8 ways
 FIND lines, sets, address format, tag directory size
 FORMULA address bits = $\log_2(\text{memory})$ · lines = cache / block · sets = lines / k · offset = $\log_2(\text{block})$ · tag = address – set – offset · tag directory = lines × tag bits

- 1 Address: 16 MB = 2^{24} , so **24-bit** addresses.
- 2 Lines = 32 KB / 64 B = 32768 / 64 = **512**.
- 3 Sets = 512 / 8 = **64**.
- 4 Offset = $\log_2 64$ = **6**. Set field = $\log_2 64$ = **6**. Tag = 24 – 6 – 6 = **12**.
- 5 Tag directory = 512 lines × 12 bits = 6144 bits = **768 bytes**.

512 lines in 64 sets of 8. Format **tag 12 / set 6 / offset 6**. Tag directory **768 bytes**, which is the storage overhead the cache pays on top of its 32 KB of data.

(B) AMAT BOTH WAYS (6 MARKS) [M3 U3]

GIVEN $h = 0.92$ · $T_c = 3$ ns · $T_m = 70$ ns
 FIND AMAT under hierarchical and simultaneous organisation
 FORMULA hierarchical = $T_c + (1-h) \times T_m$ · simultaneous = $h \times T_c + (1-h) \times T_m$

- 1 Miss ratio = $1 - 0.92 = 0.08$.
- 2 Hierarchical = $3 + 0.08(70) = 3 + 5.6 =$ **8.6 ns**.
- 3 Simultaneous = $0.92(3) + 0.08(70) = 2.76 + 5.6 =$ **8.36 ns**.

Hierarchical **8.6 ns**, simultaneous **8.36 ns**. The difference is only whether the cache time is paid on every access or only on hits; say which organisation you assumed and you cannot lose the mark.

(C) LOCALITY OF REFERENCE (4 MARKS) [M3 U1]

Locality of reference is the tendency of a program to concentrate its memory accesses in a small, predictable set of locations during any short period. **Temporal locality:** a location used now is likely to be used again soon, for example a loop counter read on every iteration, which is why cache keeps recently used data. **Spatial locality:** locations near one just used are likely to be used soon, for example successive elements of an array, which is why cache fetches a whole block of neighbouring words rather than a single one.

(A) 245 – 178 BY END-AROUND CARRY (6 MARKS) [M1 U2]

GIVEN	$245 - 178 \cdot n = 10 \text{ bits} \cdot \text{one's complement}$
FIND	the difference as a pattern and in decimal
FORMULA	$a - b = a + (-b)$ with $-b$ the bitwise inversion; a carry out of the top is added back into the lowest bit

- 1 245 in 10 bits: $245 = 128+64+32+16+4+1 = 00 \ 1111 \ 0101$.
- 2 178 in 10 bits: $178 = 128+32+16+2 = 00 \ 1011 \ 0010$.
- 3 -178 in one's complement: invert every bit: $11 \ 0100 \ 1101$.
- 4 Add: $00 \ 1111 \ 0101 + 11 \ 0100 \ 1101 = 1 \ 00 \ 0100 \ 0010$.
A 1 has carried out of the 10th bit.
- 5 End-around carry: drop it and add it into the lowest bit: $00 \ 0100 \ 0010 + 1 = 00 \ 0100 \ 0011$.
- 6 Read back: $00 \ 0100 \ 0011 = 64 + 2 + 1 = \mathbf{67}$, and $245 - 178 = 67$. Correct.

$245 - 178 = \mathbf{00 \ 0100 \ 0011} = \mathbf{67}$. The signature of one's complement is that the carry never vanishes, it comes back in at the bottom.

(B) EXCESS-3 (4 MARKS) [M1 U2]

GIVEN	(i) 507 to encode, (ii) 0111 1001 0110 to decode · code = Excess-3, 4 bits per digit
FIND	the code, the decimal, and the self-complementing property
FORMULA	encode: digit + 3 in 4 bits · decode: nibble value – 3

- 1 507: $5+3 = 8 = 1000$; $0+3 = 3 = 0011$; $7+3 = 10 = 1010$. So $507 = \mathbf{1000 \ 0011 \ 1010}$.
- 2 Decode 0111 1001 0110: $0111 = 7$, minus 3 = 4; $1001 = 9$, minus 3 = 6; $0110 = 6$, minus 3 = 3. So it reads **463**.

$507 \rightarrow \mathbf{1000 \ 0011 \ 1010}$; $0111 \ 1001 \ 0110 \rightarrow \mathbf{463}$. (iii) Excess-3 is **self-complementing**: the code of $(9 - d)$ is the bit-by-bit complement of the code of d . For example 4 is 0111 and 5 is 1000, which are exact complements, so taking the nine's complement of a digit is just inverting its bits. That is what made the code attractive for decimal subtraction hardware.

(C) $Y = (P + Q) \times (R - S) / T$ IN FOUR FORMATS (10 MARKS) [M1 U4]

GIVEN	$Y = (P + Q) \times (R - S) / T$ · opcode = 1 byte · each address = 2 bytes
FIND	the four programs and their byte totals
FORMULA	bytes per instruction = opcode bytes + (address fields $\times$ bytes per address); total = instruction count $\times$ that

3-ADDRESS OP D,S1,S2	2-ADDRESS OP D,S	1-ADDRESS ACCUMULATOR	0-ADDRESS STACK
ADD T1,P,Q SUB T2,R,S MUL T1,T1,T2 DIV Y,T1,T	MOV T1,P ADD T1,Q MOV T2,R SUB T2,S MUL T1,T2 DIV T1,T MOV Y,T1	LOAD P ADD Q STORE T1 LOAD R SUB S MUL T1 DIV T STORE Y	PUSH P PUSH Q ADD PUSH R PUSH S SUB MUL PUSH T DIV POP Y

4 instructions

7 instructions

8 instructions

10 instructions

FORMAT	BYTES PER INSTRUCTION	TOTAL
3-address	$1 + 3 \times 2 = 7$	$4 \times 7 = \mathbf{28}$
2-address	$1 + 2 \times 2 = 5$	$7 \times 5 = \mathbf{35}$
1-address	$1 + 1 \times 2 = 3$	$8 \times 3 = \mathbf{24}$
0-address	3 for PUSH/POP, 1 for an operator	$5 \times 3 + 1 \times 3 + 4 \times 1 = \mathbf{22}$

Totals: **28, 35, 24 and 22 bytes** for 3, 2, 1 and 0 address. The postfix form for the stack machine is **P Q + R S - × T /**. Note the 0-address machine wins here on bytes: its five PUSHes and one POP carry addresses, but its four operators carry none.

(A) C2C60000H IN IEEE-754 SINGLE PRECISION (6 MARKS) [M1 U3]

GIVEN pattern = C2C60000H · IEEE-754 single precision · bias 127
 FIND the decimal value
 FORMULA value = $\pm 1.M \times 2^{(E - 127)}$

- 1 Expand: C2C60000 = 1100 0010 1100 0110 0000 0000 0000 0000.
- 2 Split 1 / 8 / 23: sign = 1; E = 1000 0101; M = 100 0110 0000... (rest zeros).
- 3 Sign 1 means negative.
- 4 E = 1000 0101 = 133, so e = 133 - 127 = 6.
- 5 Restore the hidden 1: 1.1000110×2^6 . Move the point 6 places right: 1100011.0.
- 6 Evaluate $1100011 = 64 + 32 + 2 + 1 = 99$.

C2C60000H = **-99**. The two marks most often dropped here are forgetting the hidden 1 and forgetting to subtract the bias. Do both, every time.

(B) 21 ADDRESS LINES, 4-BYTE WORDS (5 MARKS) [M2 U1]

GIVEN k = 21 address lines · word = 4 bytes · word-addressable
 FIND address space, total memory in MB, PC width
 FORMULA address space = 2^k · capacity = $2^k \times$ bytes per location · PC width = k

- 1 Address space = $2^{21} = \mathbf{2\ 097\ 152}$ locations (words).
- 2 Each word is 4 bytes, so capacity = $2\ 097\ 152 \times 4 = 8\ 388\ 608$ bytes.
- 3 $8\ 388\ 608 \div 1024 \div 1024 = \mathbf{8\ MB}$.
- 4 The PC must hold any address, so it is **21 bits** wide.

2 097 152 words, **8 MB** in total, PC **21 bits**. Because the machine is word-addressable, each address names a 4-byte word, so the byte capacity is four times the number of locations.

(C) IMAGE SIZE (4 MARKS) [M1 U2]

GIVEN 8.5 in × 11 in · 200 dpi · black and white, 1 bit per pixel
 FIND the memory required, in KB
 FORMULA bits = (w×dpi) × (h×dpi) × bpp, then ÷8 for bytes and ÷1024 for KB

- 1 Pixels across: $8.5 \times 200 = 1700$. Pixels down: $11 \times 200 = 2200$.
- 2 Total pixels = $1700 \times 2200 = \mathbf{3\ 740\ 000}$.
- 3 At 1 bit per pixel: $3\ 740\ 000 \text{ bits} \div 8 = \mathbf{467\ 500}$ bytes.
- 4 $467\ 500 \div 1024 = \mathbf{456.54\ KB}$.

3 740 000 pixels = 467 500 bytes ≈ **456.54 KB**. Had a colour depth been given, say 24-bit, you would multiply the pixel count by 24 before dividing by 8.

(D) RISC VERSUS CISC (5 MARKS) [M4 U1]

RISC	CISC
Few, simple, fixed-length instructions	Many, complex, variable-length instructions
Mostly one cycle per instruction	An instruction may take many cycles
Load/store: only those instructions touch memory	Arithmetic may act directly on memory
Many registers, hardwired control, easy to pipeline	Fewer registers, microcoded control, harder to pipeline
Examples: RISC, ARM and MIPS (also SPARC, RISC-V). CISC, Intel x86 and Motorola 68000 (also VAX). The compiler does more work on a RISC machine; the hardware does more on a CISC one.	

RECALL AND CHECK

In Question One (e) you could not read the cache line off the hex digits, though you could in the past papers. Why, and what is the general rule?

ANSWER

Reading fields off hex digits only works when each field is a whole number of hex digits, that is a multiple of **4 bits**. There the offset was 5 bits and the index 9, neither a multiple of 4, so the fields straddle hex-digit boundaries. The general rule: convert to decimal (or to full binary) and use $\text{line} = (\text{address} \div \text{block size}) \bmod \text{number of lines}$. Use the hex shortcut only when the offset and index are multiples of 4 bits, as they were with a 16-word block and 256 lines.